ANALYSIS

PROBLEM DEFINITION AND STAKEHOLDERS

My math tutor Matt, who has taught me since I was young, has asked me to create a website that gives users mental math questions including basic arithmetic and algebraic problems.

Matt is currently teaching 15 students ranging from 13 to 14 years old, who are all working towards doing their GCSE exams at the age of 15.

My math tutor Matt is my client and stakeholder, and the students described prior are also my stakeholders.

According to Matt his pupils are all pretty incompetent at mental math wasting valuable time in exams by typing everything into their calculators. During lessons Matt also says that they struggle to understand him due to doing calculations in his head and them not being able to follow him.

Therefore, he wants a website that practices mental math basics to improve his students' mental math abilities. This is the use of the website.

His first requirement is to **see every students' results and stats** that only, as a teacher, he can see in order to track their progress. This is otherwise known as an **admin interface**.

How to make the mental math questions Matt is unsure about, so I have suggested to him making it in the form of a **game where, as you progress the difficulty increases**. I have also suggested **making different games** to make it more interesting. The game can be made using JavaScript, and the website will use HTML and CSS. Like on other websites, Matt wants the games to be repeatable and quick to complete. The questions will be random, and of random types. i.e., there will be a mixture between addition, subtraction, multiplication or division. You will also have the option to choose between game types. For instance, you can choose between only addition and subtraction or multiplication and division or both mixed together. As you get further on in the game the questions gradually become more challenging.

An example Matt gave is of multiple math questions appearing on the screen which you then have to type the answer to. If you don't get the answer in time you lose a life. You have 3 chances until the game ends. Your score depends on the number of correct answers and how quickly you answered them. Wrong answers add a penalty to your score.

Further-more, Matt says he want to be able to **see all the scores that each student has set in descending order** and therefore being able to tell who has and has not practised.

Matt would like **separate logins for each student**, so they feel more independent. This would **require a database** storing Username, password, email and high score made using MongoDB.

About the structure of the website Matt doesn't care how it looks, as long as it is functional, but I suggested to him that it **should look aesthetically pleasing and appropriate for its function**, to make it less boring for students. I also suggested the game itself should be good looking, and with some fun animations and engaging background music.

The website should always be open for use by the students for practice.

COMPUTATIONAL METHODS

Since I require a way to make an accessible place for students to access from anywhere in the world, I must make use of websites. Websites are very useful since they can be accessed wherever as long as you have internet, and it allows for the function of storing data on a server that can be shared with others, such as game scores on a leader board.

Of course, this can function is best executed by use of Computational Methods, which allows for powerful and efficient databases to store data, unlike a pen and paper.

Thinking ahead, I know that I will be making the website using HTML and CSS. For the database I will be using MongoDB, and for the back end I will be using Node JS which is a runtime environment where I can create a highly efficient server. The particular server I will be making is called a REST API. Node JS is modern and especially good at quick real time response between client and server.

Thinking procedurally with decomposition, I know I must split the problem into chunks that I will tackle on an individual basis. I will then further break it into smaller parts such as modules, in order to make debugging and readability much easier. This allows me to be more efficient with time. I will tackle the website design first and then move on to the games.

Through thinking logically, it is clear to me that some features such as a homework system, or chat capabilities are unreasonably complicated, and are unnecessary for the project to do its function. Having a simple leader board, that other students can see, as well as storing the logins of all users, and of an admin interface is more than enough to accomplish the task at hand. I also consider hashing passwords and adding salt to make a more secure database.

Some things, such as validation of inputs, checking the passwords and username, must happen at the same time, in quick succession. Thinking concurrently, to do this multitasking within code, I plan to use asynchronous JavaScript which allows multiple tasks to run at the same moment.

RESEARCH

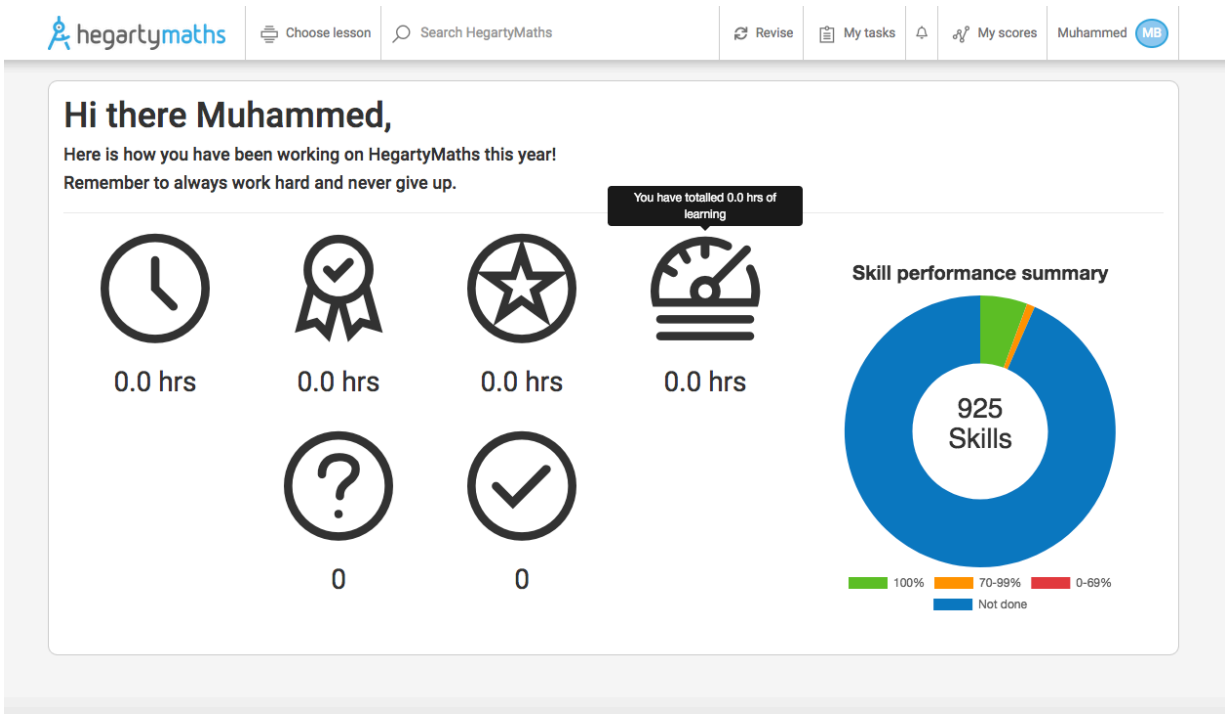
Methods of Analysis

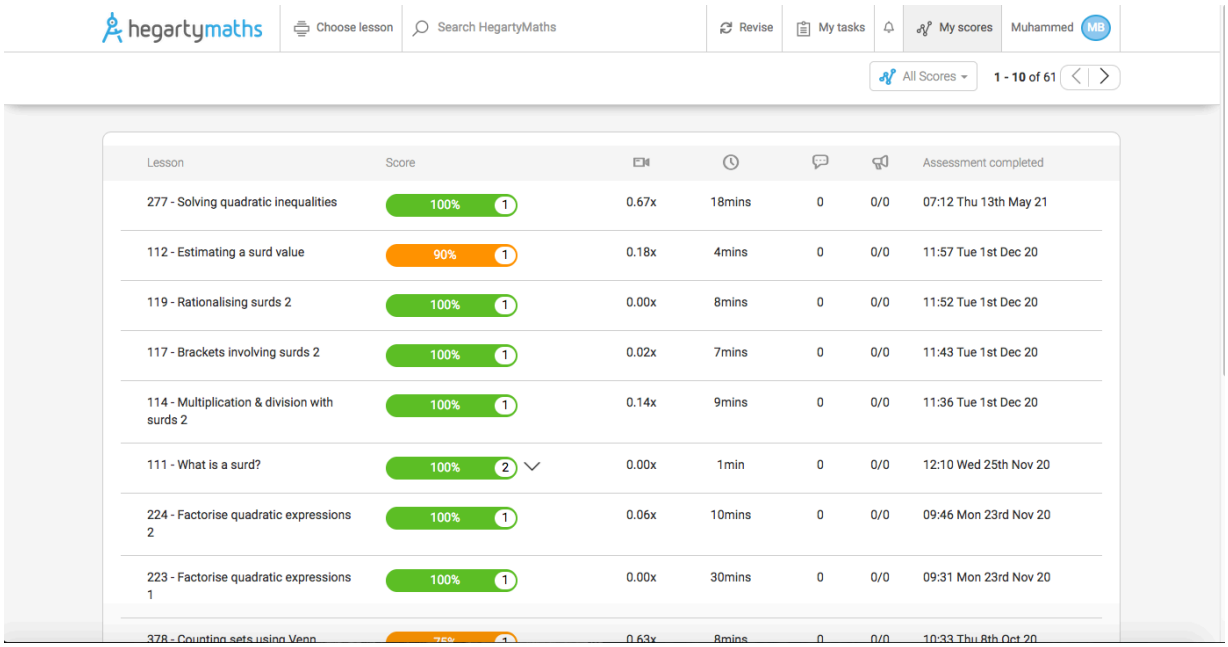
This is process of my research:

1. I will analyse a number of different math question related websites noting the key features and function i.e., Layout, aesthetic, presentation, and include them in my website. I will also note what I do not want to include.
2. I will further my understanding of Matt's vision by interviewing him. Some questions will be informed by the websites I researched.
3. I will ask my users about their preferences. Note any opinions can be overturned by Matt.
4. I will research and learn the optimum approach to creating this website.

Website 1

- https://hegartymaths.com
- A math learning and practice website. Teachers can set homework to students on the many tasks on GCSE Math subjects within the actual website. Students can also choose to do tasks of the choice for practice. On each task a teaching video is linked created by Mr Hegarty himself.



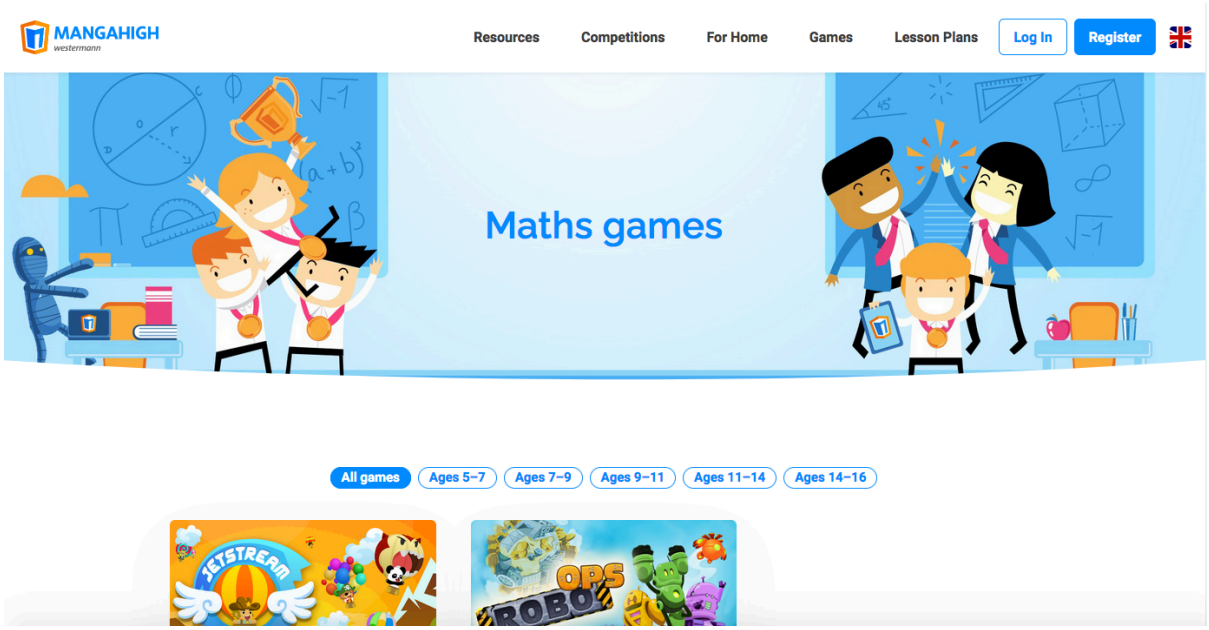


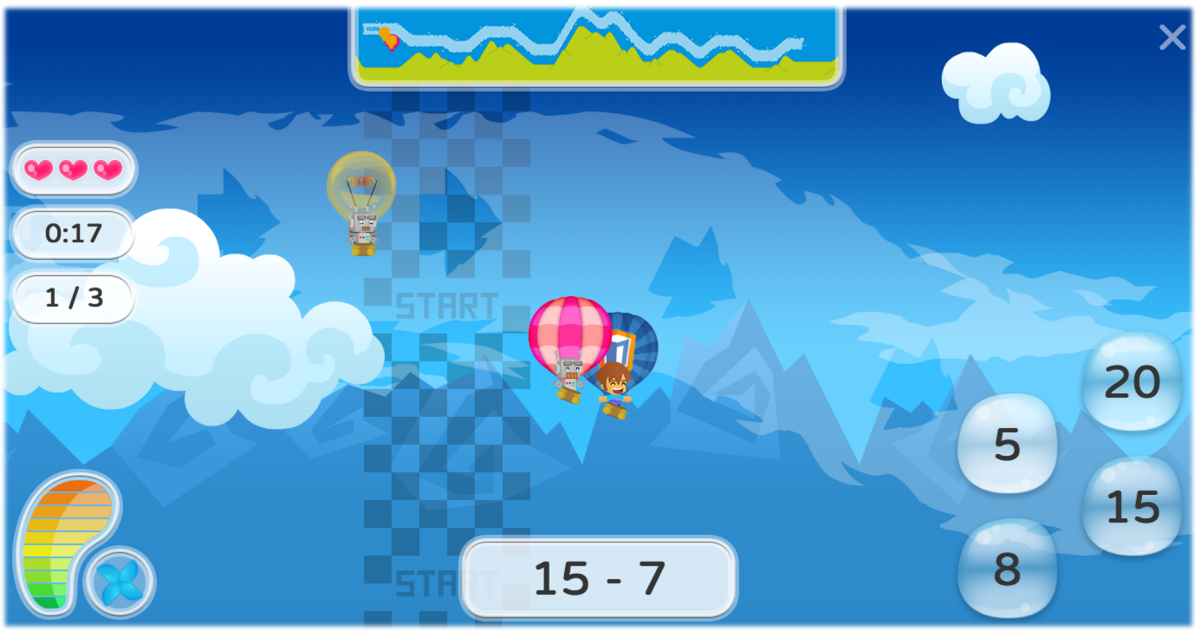
Website 1 Conclusions

Things I like/might include	**Things I don't like/will not include**
The website is easy to navigate	The tasks that you can set are purely math questions with no game aspects, such as, against the clock or a life system.
There are analytics of the individual user for them to track their own progress.	
The users can see their track record of how they have done on previous tasks with a score and time record.	
The colour scheme is blue and white. Attractive and simple.	
If you leave the task is saves your progress and allows you to continue where you left off.	
There is an admin interface for teachers per class.	
There is a profile section.	

Website 2

- https://www.mangahigh.com/en-gb/games
- A math practice website that uses games. The games are different and have many extra but simple mechanics. Each game is unique and practices a different part of mental math. Teachers can host competitions within their classes for students to compete against each other in different games.



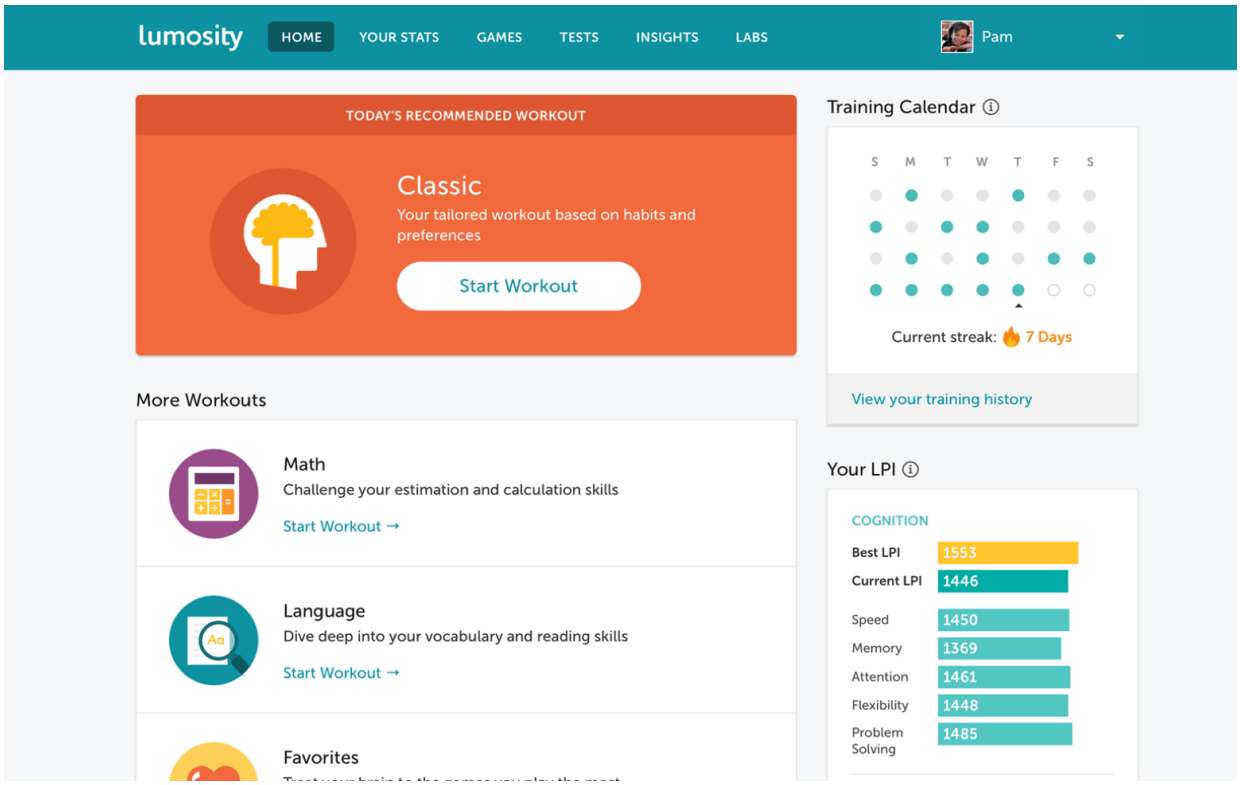


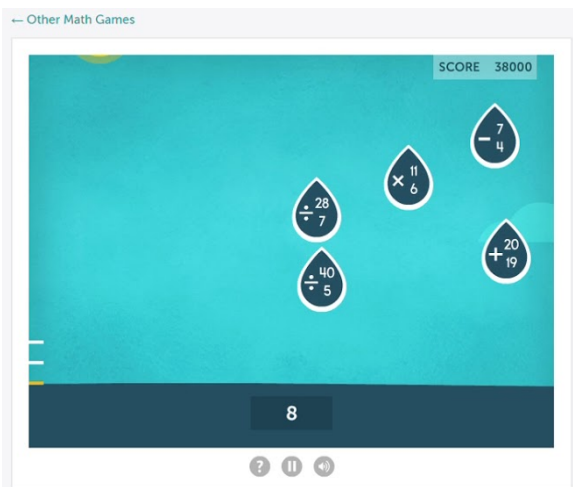
Website 2 Conclusions

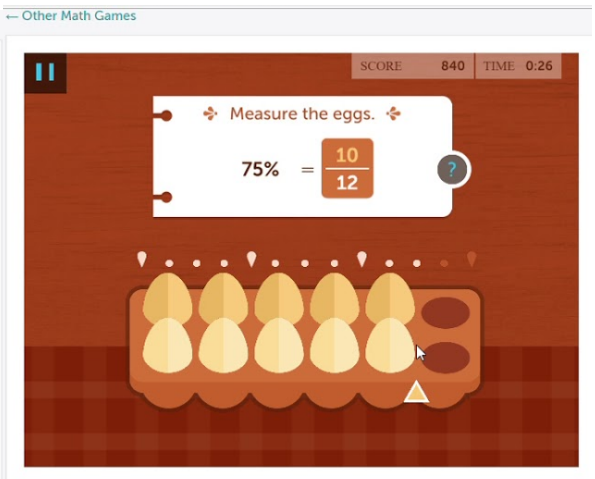
Things I like/might include	**Things I don't like/will not include**
There is a profile section.	The design of the website is much to childish for my stakeholder's preferences. I will ask them about this.
The games are intuitive and engaging. They look nice and have some really cool animations.	There is a level system within some of the games, which is not I am aiming for.
There are many different games working different areas of math.	
The nav bar is simplistic.	
You can either register as a student or a teacher freely.	
The login and register pages are simple only requiring user ID as a prerequisite.	

Website 3

- https://www.lumosity.com/en/
- A brain training website that uses games to exercise different cognitive attributes. Clicking on the games page you are open to choose from a plethora of different games training specific cognitive attributes. There is a daily workout that takes place allowing you to track your progress.







Website 3 Conclusions

Things I like/might include	**Things I don't like/will not include**
The look is very clean and well ordered.	Many of the games on the website are unrelated to math.
There are personal analytics for you to gage your personal progress.	The website tends to spam your inbox with emails.
The games are time based.ie the point system is based on the time taken or in other ways.	
The games are simple and fun to play and very quickly repeatable.	
A display of high scores for that game are displayed after you finish the game.	
There is a sense of progress within the games like levels and so on.	

Website 4

- https://www.bbc.co.uk/bitesize
- A learning and practice website for ages 5 to 16 with everything from KS1 to GCSE. The website gives you options to practice different areas of math which correspond to a different game.
- The website shown is of an early version of BBC bitesize which I like more than the new website. There is no longer any access to it.

Website 4 Conclusions

Things I like/might include	Things I don't like/will not include
There are a range of games exploring many areas of mental arithmetic.	The games are quite childish and not appropriate for my audience.
There are different games that you can play depending on your level of learning, which you can select. This picture is of the website for KS1.	There is no login or sign-up implementations.
The games are entertaining with cool animations.	
The navigation of the website is easy.	

CLIENT INTERVIEW

In order to get a better idea of my clients vision I put together some questions that I wanted to ask him. Most of these questions stemmed from the research I recently did.

Questions:

1. How many different types of games do you want to be on the website? *"One or two games would be enough. As long as there repeatable and engaging and their function is correct, I have no qualms."*
2. Would you like each user to have a personalize able profile section? *"I think that would be a good idea. It would involve my students more into the process."*
3. Do you want the games to have timed or under timed conditions aspect, or what do you want otherwise? *"Having the game be timed would allow my student and me to better understand their progress, so therefore, yes, I do want the games to have 'under timed conditions' aspects."*
4. Would you like a form of analytics to be displaced for the user, like a pie chart or something saying how long you have spent playing which game? *"Yes, I think that would be a good idea. It will involve the student more."*
5. What colour scheme for the website would you prefer: blue and white or aqua, white and orange buttons? *"Aqua, white and orange buttons would be nice."*
6. Do you want reminders to be sent to your students' email to do the homework if they haven't done it? *"I think the user should have the option to not allow reminder emails to be sent to their email. Otherwise, I think it is a good option."*
7. Would you like there to be an option to toggle light or dark mode in a drop-down settings panel? *"Yes, that would improve my students' experience."*

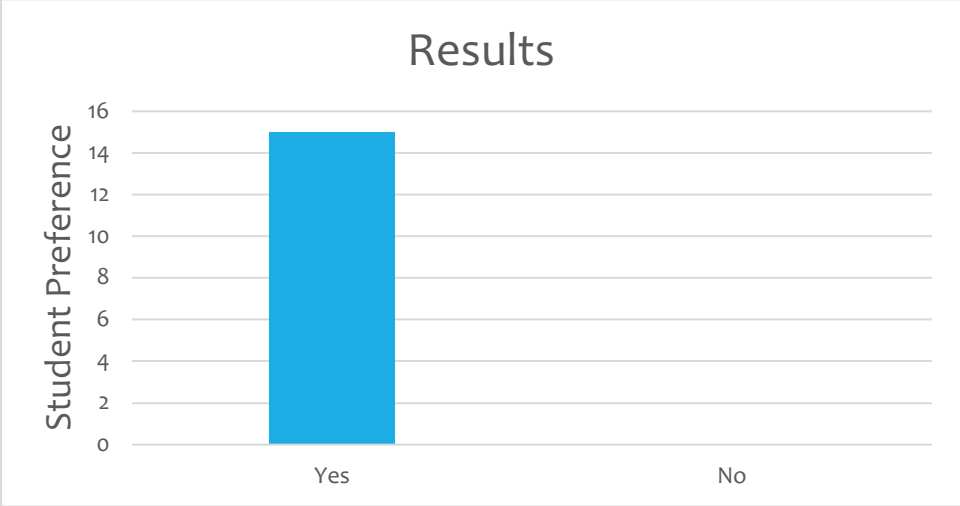
Conclusion:

- One or two games.
- A custom profile page.
- The games must have 'under-timed conditions' aspects.
- Analytics for users to see.
- A colour theme of aqua, white and orange buttons.
- Automatic emails to be sent to the students' email as a reminder. Option to turn this function of.
- Option to toggle light or dark mode.

STAKEHOLDER QUESTIONNAIRE

I asked my stakeholders a few questions to understand their majority preferences about some aspects of the website.

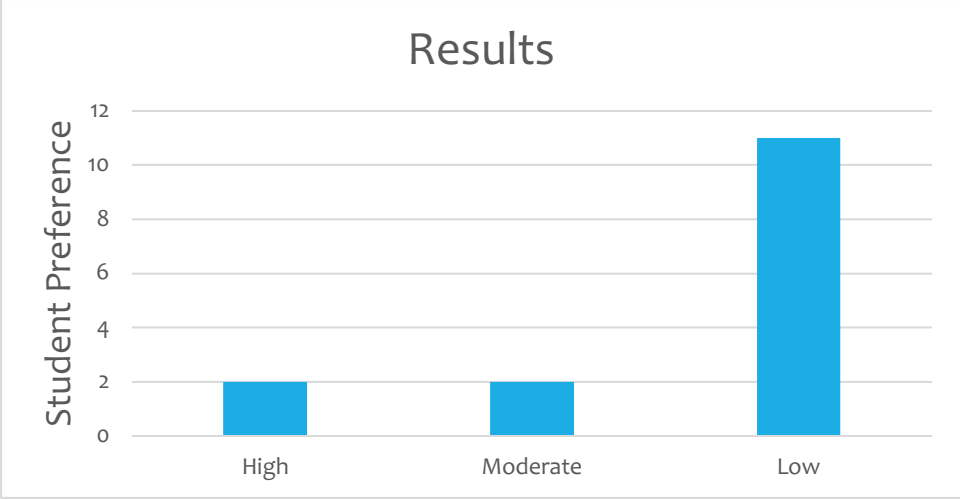
1. Is a personalized profile important?



Yes – 15
No – 0

Conclusion: from these results I know I need to add a separate tab or page for the profile with the appropriate features such as a personalized profile picture or a short blog that other users will be able to see upon clicking your profile or statistics for your high scores against time and other such things.

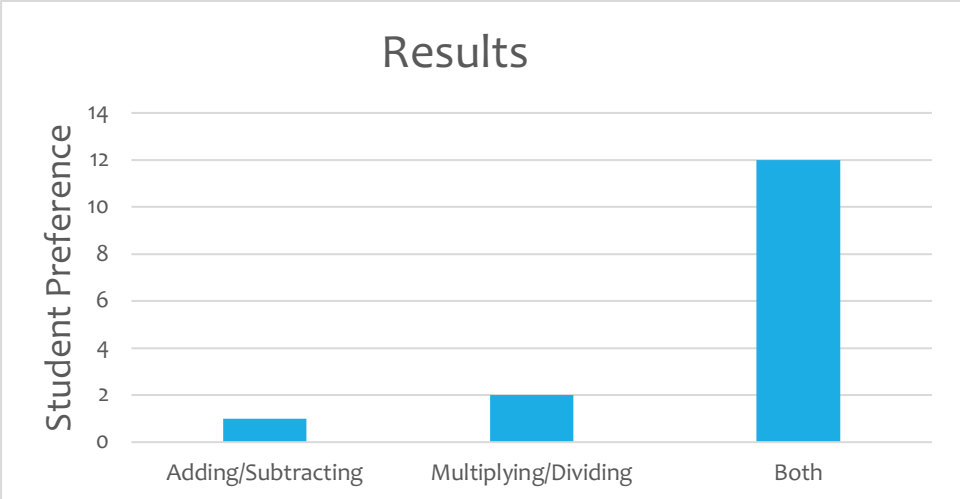
2. If you were to play a math game, how much complexity in terms of extra thinking/strategy would you prefer?



High – 2
Moderate – 2
Low – 11

Conclusion: From this result players of the game evidently want a game more math based with less extra mechanics that might not accurately represent their actual mathematical abilities. Therefore, I will endeavour to create a game that has low extra thinking from other features and is more math oriented.

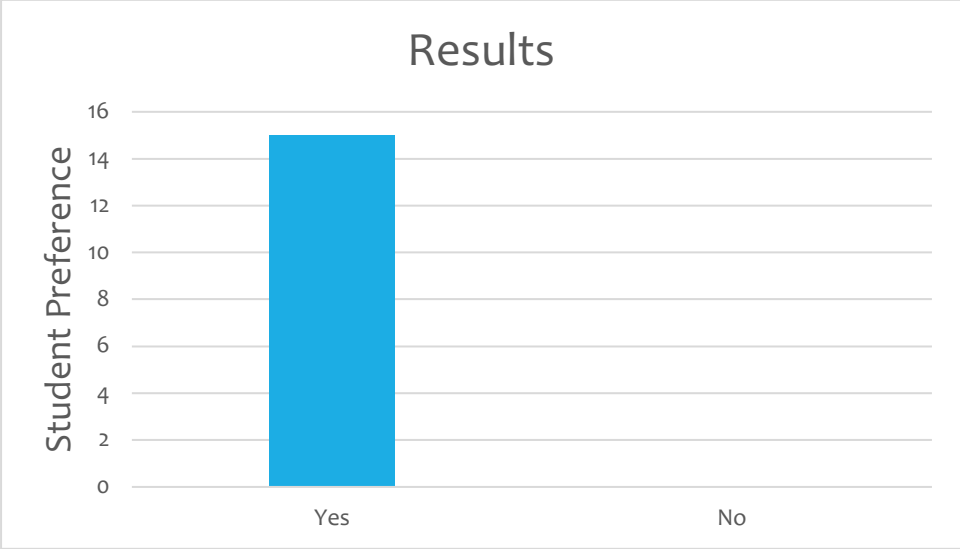
3. What part of mental arithmetic do you struggle with most?



Adding/Subtracting – 1
Multiplying/Dividing – 2
Both – 12

Conclusion: From this set of results, I know I will have to design a game that mixes both adding/subtracting and multiplying/dividing.

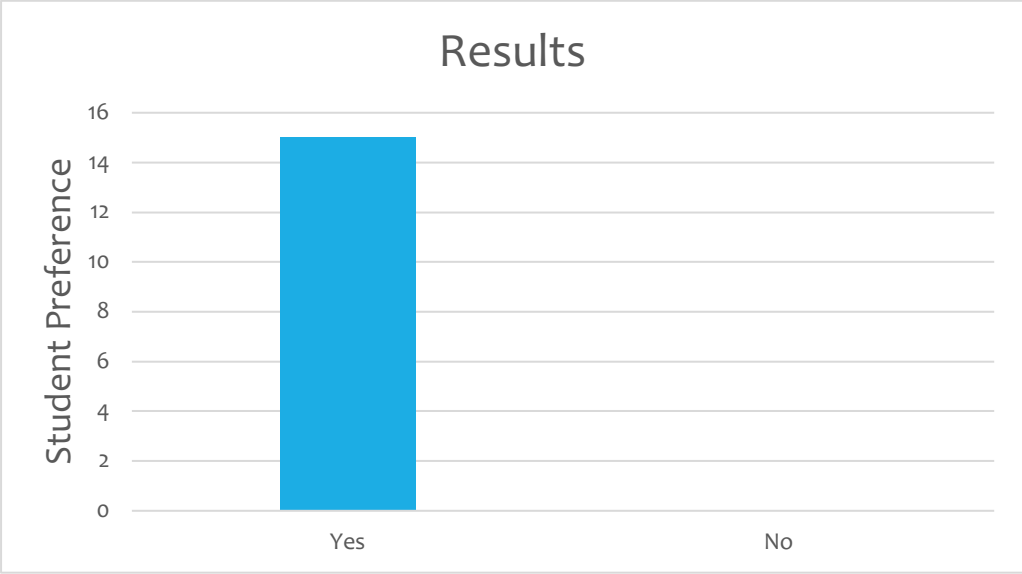
4. Is being able to see your recent scores in an ascending table important?



Yes – 15
No – 0

Conclusion: Here I can infer that users seeing their scores is important and is a vital feature to the website.

5. If you forgot your password, would you like to able to reset it?



Yes – 15
No – 0

Conclusion: The extra feature of resetting your password is, from this data, must be a feature included with possibly a button beneath the log in page to reset password.

6. Are the bitesize and manga-high websites too childish for your liking?

Yes – 12
No – 3

Conclusion: From the preference of the opinions of my stakeholders, it is evident that the design of the game should stay far away from childish i.e., baby colours, bubble font, cartoons etc. It should have a modern clean look.

QUESTIONNAIRE CONTINUATION

Stakeholder questionnaire summary:

1. To have a customizable profile which has other features such as: A PFP, a blog, a recent high-score board, and so on.
2. The games should be heavily math orientated with little extra mechanics incorporating all parts of mental arithmetic: adding, subtracting, multiplying, dividing etc. The games also should not have a childish aesthetic.
3. An option to reset password on login page.
4. The actual website should not look childish, but instead should be modern and clean.

FEATURES

The following table shows a detailed summary of all the features I would like to include in my website:

Index	Feature	Source	Details
1	Navigation Bar	Website 1, 2, 3, 4	A quick and simple way to navigate the website.
2	Stats screen on a different window	Website 1 & client interview	Pie charts, bar charts high-score table line graph etc.
3	Profile Page	Website 1, 2 & client/ stakeh-interview	Shows all the users details, scores, and allows for a personal blog to be written.
4	Admin interface	Website 1 & client interview	Admin screen for teacher to monitor student progress.
5	Login and sign-up window	Website 1, 2, 3	Self-explanatory.
6	One or two math games and its many aspects.	All research	Increasing difficulty with progress. Appropriate aesthetic. Repeatable and fun. Heavily math orientated. Practices all parts of mental math.
7	Change password option	Stakeholder interview	Option in the profile page to change your password.
8	Aqua, white accents with orange buttons	Client interview	Self-explanatory.
9	Appropriate font and style	Website 3	
10	Option to toggle light and dark mode	Client interview	Option in drop-down settings panel.
11	Automated email capabilities	Client interview	Reminds students to do practice on website. Option to disable this function included.

CHOOSING TECHNOLOGIES

The required technology depends on the main requirements that Matt needs which come down to:

- Online logins stored in a database.
- A game/games integrated into the website.
- An admin interface for Matt.
- Storing high scores of the users.
- Autonomous systems such as reminder emails

This project, therefore, will need the following technologies:

- A database
- A server
- A web formatting and styling facility
- A game coding and website functionality facility.

For the database and server, I plan to use Node JS, which is a fairly modern way of making web servers and then connect to a database created with MongoDB which will be used to perform all CRUD capabilities. For web formatting and styling, I plan to use HTML and CSS and JavaScript. For programming the game, I will use JavaScript, although I realise it will be quite tedious.

LIMITATIONS

In this stage of development, I will have to make use of a local server, since an online server has not been provided by my client. He wants the idea to work before he invests any money.

The amount of time does not allow me to make anything too complicated, such as the game. The game, therefore, will have to be simple since otherwise it will be too time consuming.

Any complex functions or features of a language may limit me by how difficult it is to learn and implement.

The website may not work on other devices such as on phones or iPads. A way to tackle this, however, is to use rem measurements so that the styling matches the individual device.

The laptop I am using, since it is an old model of mac book, may not support certain downloads and libraries very well.

Some browsers disallow JavaScript to run. Therefore, some users may not have access to the website's capabilities.

HARDWARE AND SOFTWARE

Software:

VS Code:

To code the HTML, CSS, JavaScript. VS Code also allows for extensions that make coding easier such as Liver Server and Prettier. It also has an inbuilt terminal where libraries and dependencies can be downloaded.

Postman:

Postman is a website that acts as the front end of an application. This means it provides simulated CRUD requests to your back-end server allowing such requests to be tested.

Firefox:

Firefox for hosting and testing the website. The inbuilt Firefox development tools are extremely useful to inspect code and bug test code.

Node JS:

Node JS is a runtime environment that allows for JavaScript to be run server side rather than on a browser. I will use this to make a RESTful API (Representational State Transfer) (Application Programming Interface).

NPM:

NPM is a huge JavaScript library that I will be using to download packages from.

MongoDB:

MongoDB is a non-SQL database. I will be using MongoDB Compass to inspect my database where I can manually perform CRUD operations. I can use this for testing.

Hardware:

Device: macOS High Sierra 10.13

RAM: 4GB 1333Hz

Processor: 2GHz Intel core i7

SUCCESS CRITERIA

#	**Feature**	**Source**	**Success Criteria**
1	Navigation bar	Website 1, 2, 3, 4	Navigation bar that allows quick and easy access to different parts of the website. For example: button to profile page, games page, stats page, home page.
2	Home page	Website 1, 2, 3, 4	Home page that welcomes the user and gives directories to different pages on the website.
3	Stats page	Website 1 & client interview	Stats page that shows detailed statistics of your progress, recent high scores in a line graph and so on.
4	Game page	Website 2, 3, 4	Game page that shows a list of the games on the website
5	Profile page	Website 1, 2 & client/ stakeholder interview	Profile page that displays all of the user's details as well as the user's top ten scores. There is also a field that allows users to write a personal blog.
6	Admin interface	Website 1 & client interview	Admin page for teacher to see all the details of students like their First and last name, email, scores and high scores.
7	Login and sign-up window	Website 1, 2, 3	Login and sign-up page that is secure and simple. For example: First and last name, email, password. Each input should be fully validated. Admin register should have a field for a code that allows for an admin account to be created.
8	**Game** {One or two math games	All research	One or two math games. Minimum of one.
9	Increasing difficulty		Game/games that in increase in difficulty as your score increases

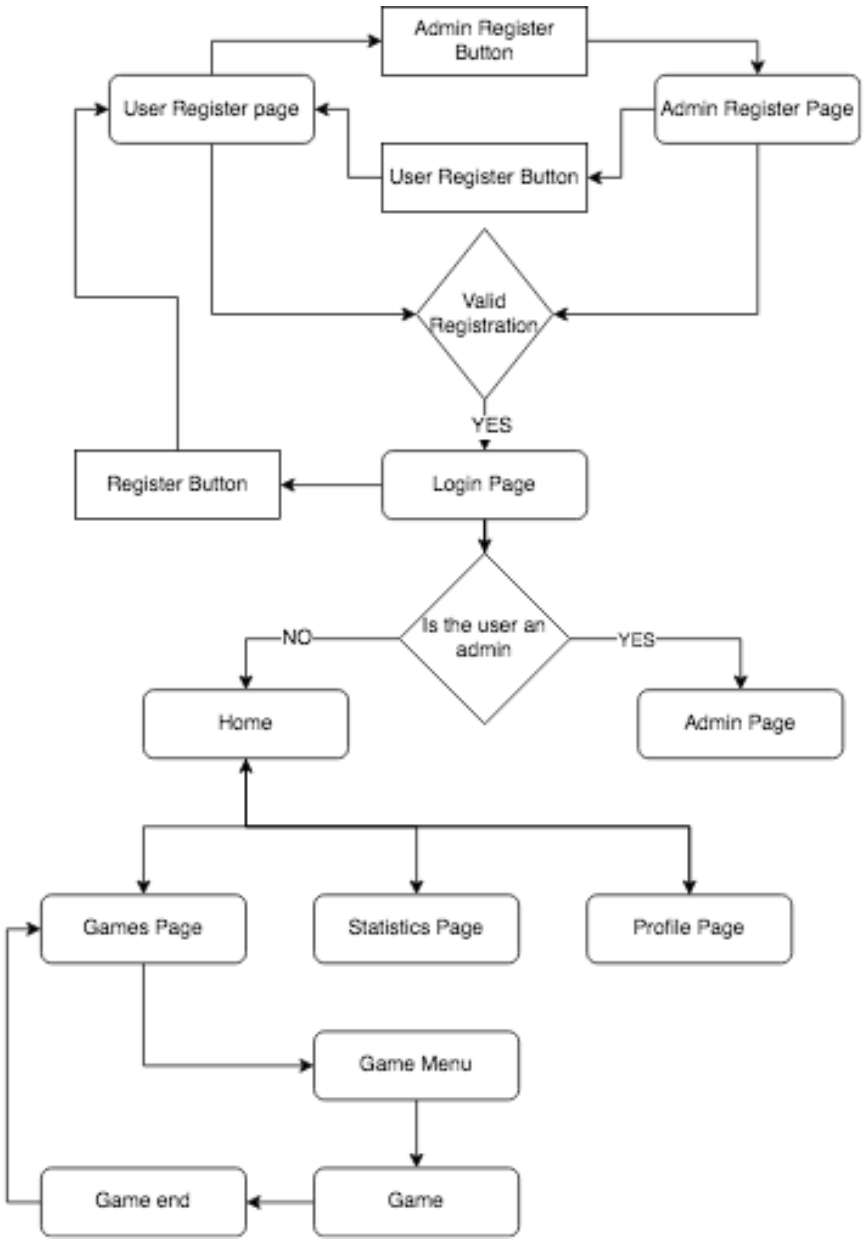
#	Feature	Source	Success Criteria
10	Different Game Types	All research	Players have the option to choose between having only addition and subtraction questions, multiplication or division questions or a mix of both types
11	Heavily math orientated		There will be little extra thinking with mechanics not involved with the math part of the game.
12	Practices all parts of mental math		Addition, subtraction, dividing, multiplying, fractions, algebra. The last two may be limited by time
13	Appropriate aesthetic		Rounded edges, nice and satisfying animations, bold colours.
14	Repeatable and fun}		Game is easy to play, easy to access and repeat many times over.
15	Change password option	Stakeholder interview	Users have the ability to change their password when- ever they want through the profile page
16	Aqua, white accents with orange buttons	Client interview	Aqua, white accents with orange buttons for the whole webpage.
17	Appropriate font and style	Website 3	Appropriate font and style for the website.
18	Option to toggle light and dark mode	Client interview	Option to toggle light and dark mode that changes the styling depending on the mode selected.
19	Automated email capabilities	Client interview	Automated email capabilities that will send reminder emails to the users email for them to spend such and such time on one of the math games. This feature can be turned off.

DESIGN

WEBSITE PAGE DESIGN

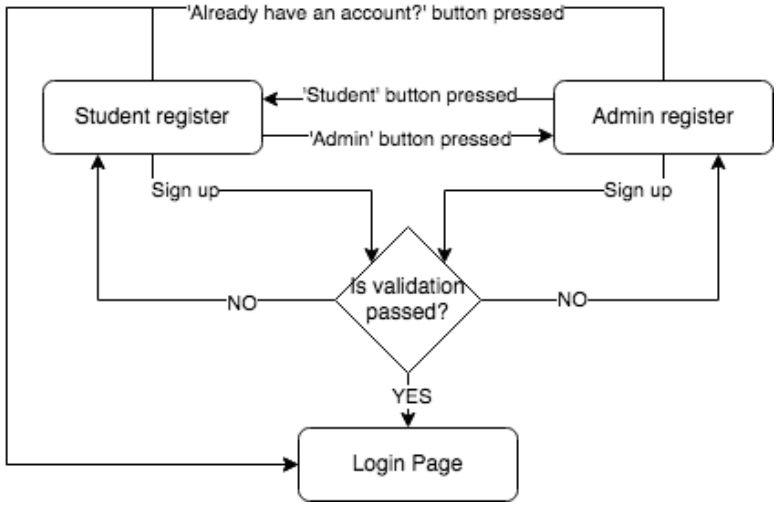
Overall Design of the Website

The below diagram represents the overall website design. The arrows represent the routes from one page of the website to another. The user starts at 'User Register page'.



Student and Admin Register Page Zoom In

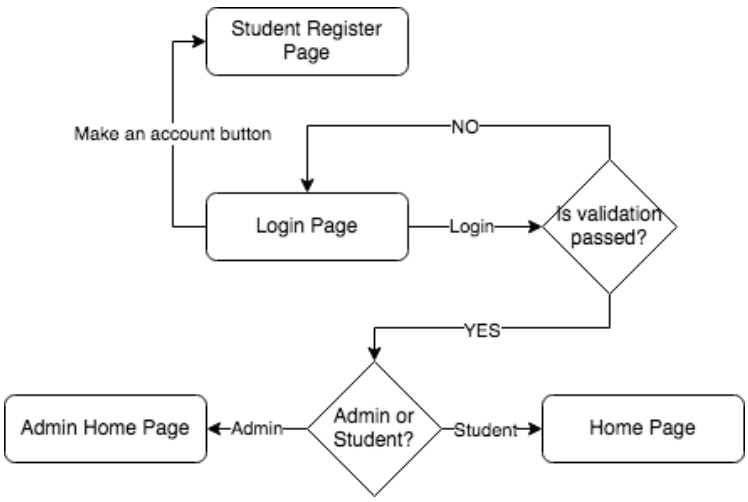
Below is the more detailed design of the user/student register page:



- "Student" and "Admin" buttons which allow the user to go between the Admin and Student register pages.
- "Already have an account?" on both pages that reroutes the user to the Login page.
- "Sign up" button which sends the data in the HTML form to the server side to be validated. If the response from the server in negative, the relevant error message is shown. If the response is positive, the user is added to the database and rerouted to the Login page.

Login Page Zoom In

Below is the more detailed design of the login page:

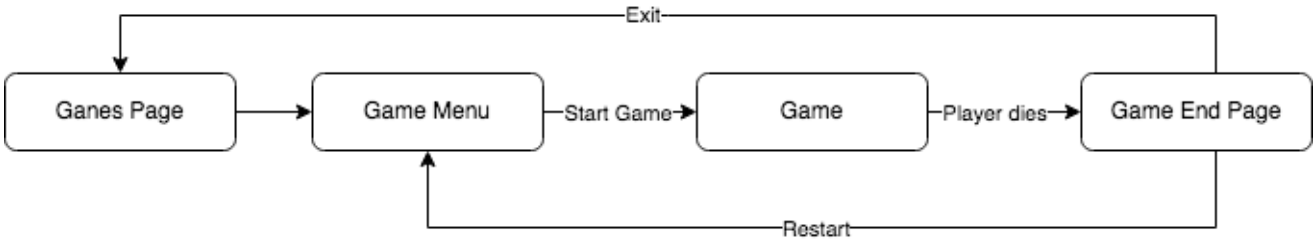


- "Make an account" button that takes the user to the Student register page.

- The "Login" button that sends the data from the HTML to be compared against the data in the database. If the data sent in is correct, then the response is positive, and the user is sent to the Home page or the Admin page if the user is an admin. If the response is negative, the relevant error is sent back.

Game Loop Zoom In

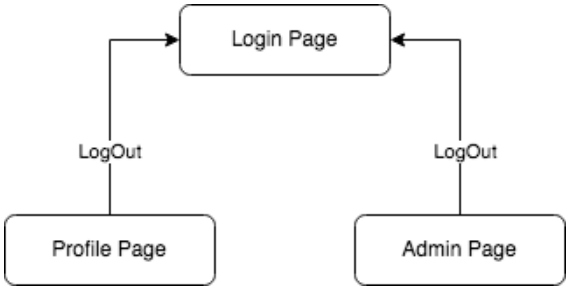
Below is the more detailed design of the game loop:



- From the game page, the user can select the game.
- Selecting the game takes the user to the Game menu page where the user will select the game mode.
- Clicking the "Start" button takes the user to play the actual game.
- When the live count goes to zero, the user is taken to the game end page where the player's final score will be shown. The "Exit" button on this page takes the user to the Games page and the "Restart" button takes the user to the Game menu.

Profile and Admin Page Zoom In

Below is the more detailed design of the profile and admin pages:



- Logout buttons on the Admin and Profile pages which take the user to the Login page.

ALGORITHMS

The algorithms detailed below are the most essential algorithms to the website that provide its functionality as a whole.

Fetch API

Code:

Fetch("endpoint") {method: "POST or GET or DELETE or PUT", body: {first name, last name, email, password etc}}

After operation complete THEN (Handle response from server)

This general form of algorithm will access the server side through a pre-established endpoint on the browser. An example would be http://localhost:3000/register. The server then handles the request and sends back a response which this algorithm then handles appropriately.

Setting Up an Endpoint

Code:

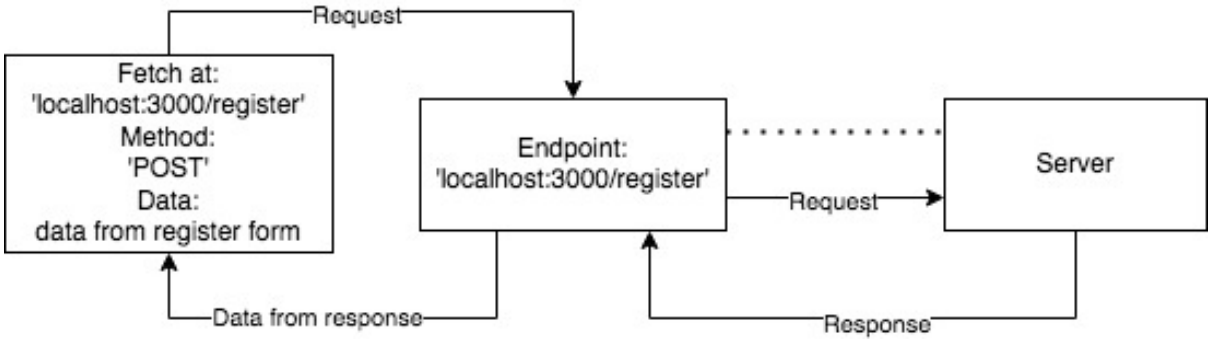
Route. post/get/put/delete ("endpoint", (request, response) {Handle request and send a response})

This general form of algorithm sets up an endpoint in the form of a URL. This means that the server is monitoring this URL and watching for it being called. If it is called this sends a request to the server. The request can be of the common forms of GET, POST, PUT or DELETE. GET is READ, POST is CREATE, PUT is UPDATE and DELETE is DELETE as from the CRUD model. After handling the request, the server sends back a response to the same endpoint.

These two algorithms form the bases of how my code will be working. Setting up an endpoint on the server side that does a specific job and then using fetch API to call that endpoint and get back and handle the response from the server.

General Algorithm Visualization

Here is a visualization of what the algorithms are doing together.



The **dotted line** represents the server watching for the endpoint being called. When it is called the sever takes in the request from the endpoint.

Why this algorithm?

This type of algorithm is very modular and repeatable. You can easily add endpoints with different functions and on the frontend JavaScript you can access these endpoints with fetch. Therefore, a website using these algorithms is extremely expandable and great for big projects being worked on by many people.

There is also a clear distinction between the backend and frontend making it easy to bug fix and edit.

For my project, I have a decent number of pages and functions and since it will be a big system as a whole, I would prefer if the code is sorted nicely which these algorithms solve by being split in two parts. So therefore, I think it **justified** to use these algorithms.

MongoDB Queries

Code:

Variable User = instance of User database

User. Find/ findAndUpdate/ findAndDelete({filter}). THEN (handle data from database)

This algorithm does a certain operation on the database depending on the filter given in. A filter might look like *'email: math@teacher.com'* or *'id: 24'*. This algorithm is essential for interaction with my database. I will be able to find, delete and update records.

There is also code to add data to the database.

Code:

Variable data = {first name, last name, email etc}

Variable user = new instance of User (data)

user. Save ()

Here you simply make a new instance of the user model and save it too the database with the inputted data.

Register Algorithm

Code:

Server side:

```
Route. post ("http://localhost:3000/register", (request, response) {
    Variables (first name, last name, email, password) from request. body
    Variable validation = function Validate Data (first name, last name, email, password)
    If (validation = False) THEN:
        send response {error: True, message: validation message}
    Else THEN:
        Variable User = instance of User database
        User. Find ({email: email}). THEN (
            If (No results from database) THEN:
                Variable data = {first name, last name, email etc}
                Variable user = new instance of User (data)
                user. Save ()
                send response {error: False, message: 'user created'}
            Else THEN:
                send response {error: True, message: 'This user already exists'}
        ) })
```

On the server side an endpoint with method POST is set up. The data from the request body is fetched and then validated. If the data is correct, then a query is run to find a user with the email from the data. If query returns empty, then a user with this email does not exist and a user is created, and a positive response is sent back. Otherwise, an error is sent saying: *"This user already exists".* If the validation in incorrect an error is also sent back with specific validation error.

Client side:

Variable user Data = {first name, last name, email, password} from Register Form

Fetch("http://localhost:3000/register") {method: "POST", body: user Data}}

After operation complete THEN (

If (response. error = True) THEN:

Display response. message on screen

Else THEN:

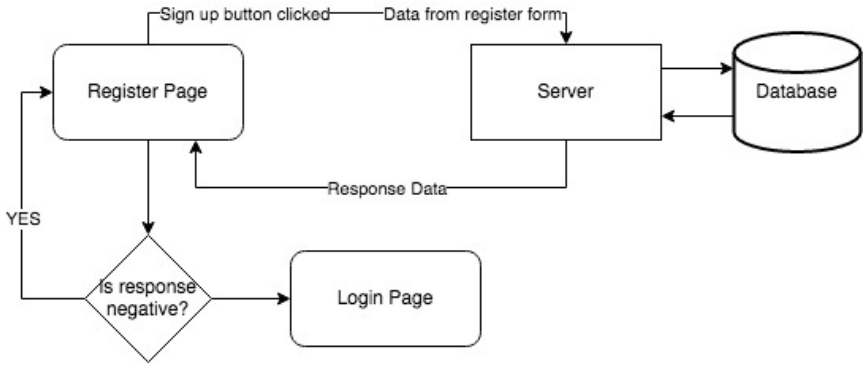
Go to Login Page

)

On the client side, the fetch API algorithm calls the *'http://localhost:3000/register'* endpoint with the POST method which allows data to be passed to the endpoint as the request body. First name, last name, email and password are taken from the HTML form input elements and are put in the request body. After the response is returned, if an error is returned then an error message will be show to the client about the relevant validation error. Otherwise, the user will be sent to the login page.

Algorithm Visualization

Whole process:



Login Algorithm

Code:

Server side:

```
Route. post ("http://localhost:3000/login", (request, response) {
    Variables (email, password) from request. body
    Variable User = instance of User database
    User. Find ({email: email}). THEN (
        If (No results from database) THEN:
            send response {error: True, message: 'This user does not exist'}
        Else THEN:
            If (password = password from database) THEN:
                send response {error: False, message: 'You logged in'}
            Else THEN:
                send response {error: True, message: 'Wrong password'}
                    )
    })
```

On the server side an endpoint with method POST is set up. The data from the request body is fetched. The fact that a wrong email will not work means that no validation is needed. A query is run to find a user with the email from the data. If query returns empty, then a user with this email does not exist and an error saying: *"This user does not exist"* is sent back. Otherwise, the password from the query return data is matched to the request body password. If they match, then the user is sends backs a positive response. Otherwise, an error is sent back saying: *"Wrong password".*

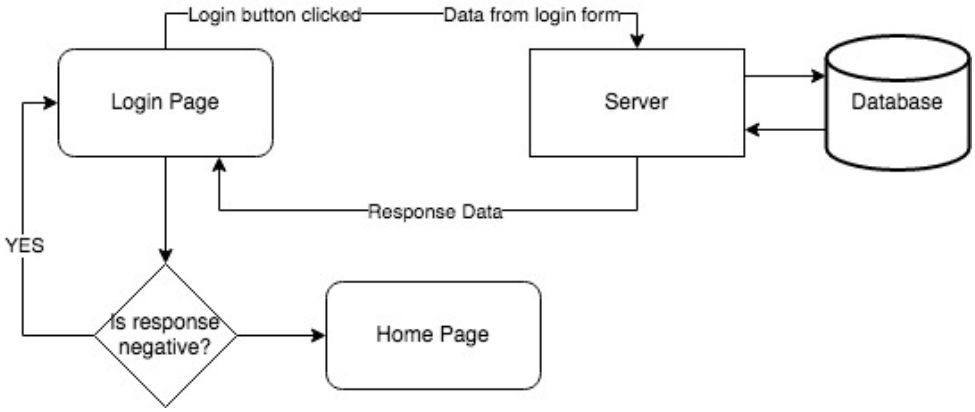
Client side:

```
Variable user Data = {email, password} from Login Form
Fetch("http://localhost:3000/login") {method: "POST", body: user Data}}
After operation complete THEN (
If (response. error = True) THEN:
    Display response. message on screen
Else THEN:
    Go to Home Page)
```

On the client side, the fetch API algorithm calls the *'http://localhost:3000/login'* endpoint with the POST method which allows data to be passed to the endpoint as the request body. Email and password are taken from the HTML form input elements and are put in the request body. After the response is returned, if an error is returned then an error message will be show to the client about the relevant validation error. Otherwise, the user will be sent to the home page

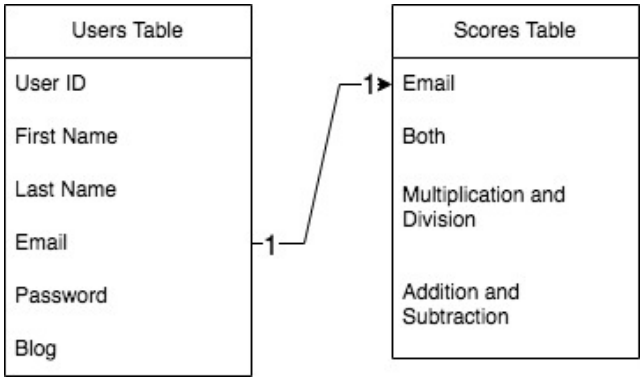
Algorithm Visualization

Whole process:



The Database

Here is a representation of my database in MongoDB.



Email is the **Primary Key** for both tables**.** The use of the 1 to 1 relationship is to separate the data more neatly. Technically I could have both of the tables together, but it would make my code less efficient when getting data from the database due to the potentially large amount of data being received all at once. By separating the scores from the user details, it also makes it easier to manage in the code. I used the email as the Primary Key because I did not actually create a relational database. Therefore, I can't get the same userID in the Scores table as the Users table automatically. Therefore, I made the email the Primary Key. This does mean though if a user's email is changed it would have to be changed in both tables.

VARIABLES

Below are detailed the main variables per page on the website and where necessary the validation for each variable will be shown.

Student Register Page

Variable	Validation	Variable Type
First Name	Length check, capitalize function, required	String
Last Name	Length check, capitalize function, required	String
Email	Email format check, lowercase function, required	String
Password	Length check, required	String

Admin Register Page

Variable	Validation	Variable Type
As above	As above	As above
Admin Code	Required	String

Login Page

Variable	Validation	Variable Type
Email	Email format check, lowercase function, required	String
Password	Length check, required	String

Game

Variable	Validation	Variable Type
Score	None	Integer
Lives	None	Integer

Statistics Page

Variable	Validation	Variable Type
Game Scores: All game mode lists.	None	List

Profile Page

Variable	Validation	Variable Type
Personal Blog	Length check	String
New Password	Length check, required	String
User Details: first name, last name, email, password.	None	List
Game Scores: All game mode lists.	None	List

Admin Page

Variable	Validation	Variable Type
Personal Blog	None	String
User Details: first name, last name, email.	None	List
Game Scores: All game mode lists.	None	List

Validation

Required: This variable cannot be empty.

Reason: This stops empty data from being put into the database and causing errors. When logging in, the database is checked to find matching emails. With empty space being present, errors may occur and may give unauthorised users access to the website

Email format check: The email must be of the conventional format. This includes the following:

An email is split in two parts. The stuff before the @ sign is the local part and the stuff after the @ is the domain. The local part may be up to 18 characters long and contain:

- uppercase and lowercase letters
- numerical values
- special characters, such as #! % $ ‘ & + * – / = ? ^ _ `. { | } ~

A domain name consists of one or more parts separated by a dot, such as *example.com.* Each part must be no longer than 18 characters and can contain:

- uppercase and lowercase letters
- numerical values, on the condition that the domain can’t all be numeric.
- a hyphen (-), on condition that it is not placed at the beginning or end of the domain.

The check can be done using the match function in JavaScript. The algorithm looks like such:

```
function validateEmail(email) {
  let mailformat = /^\w+([\.-]?\w+)*@\w+([\.-]?\w+)*(\.\w{2,3})+$/;
  if (email.match(mailformat)) {
    return true;
  } else {
    return false;
  }
}
```

Reason: This means that emails sent to that email address will actually work.

Lowercase function: The variable is automatically made lowercase. This can be done with the toLowerCase function in JavaScript.

Reason: This is for convention's sake. Data will be standardized throughout the database.

Capitalize function: The variable is automatically capitalized so only the first letter is uppercase. This can be done with the code below:

```
function capitalizeWord(word) {
  word = word.toLowerCase();
  const firstLetter = word.slice(0, 1);
  word = word.replace(firstLetter, firstLetter.toUpperCase());
  return word;
}
```

Reason: Names and such are normally of this convention. Data will be standardized throughout the database.

Length check: A limit is put on the max and minimum number of characters that can be in that variable.

Reason: This means that space is saved in the database.

THE WEBSITE

In this section I will list all the pages of the website. For each page I will detail the page's design, functionality, useability features and the necessary tests that must be done to decern whether the page is properly functional.

Website Useability Features

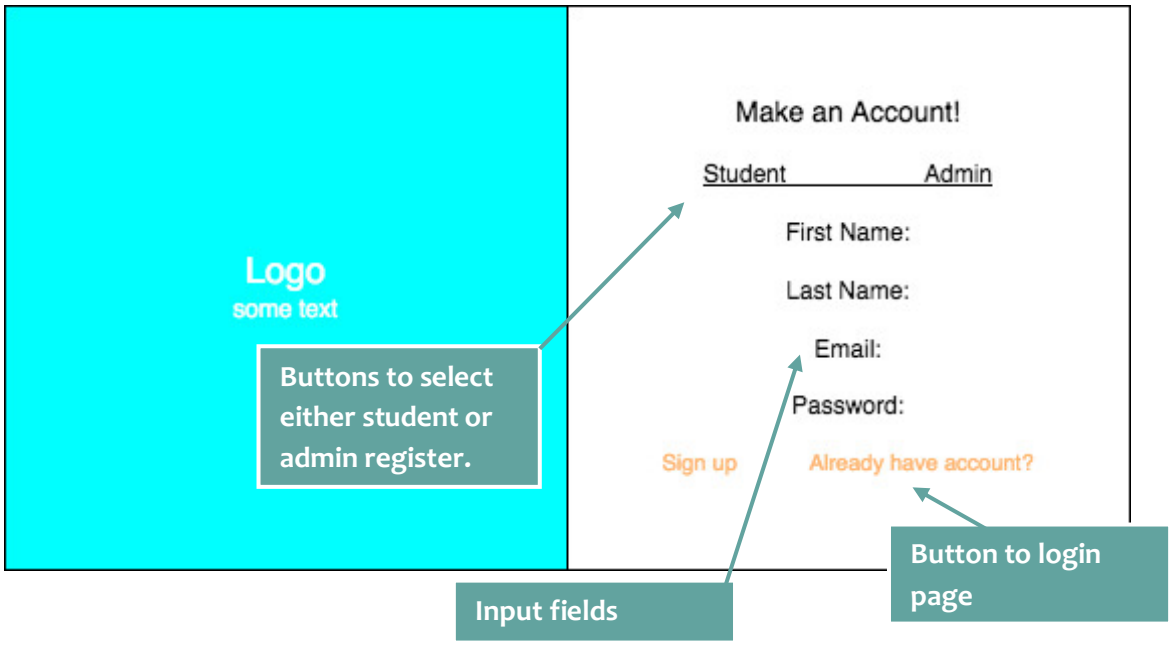
- All buttons are the colour orange. **Why have this feature?** The user will more easily be able to identify clickable objects.
- Buttons will decrease in opacity when hovered over and the cursor will become a pointer. **Why have this feature?** This allows to user to better know what they are clicking on.
- When on the website the navigation bar will be above every page apart from in the actual games. **Why have this feature?** This allows the users to easily navigate from one page to another.
- The pages are simple with little text and clear appropriate titles. **Why have this feature?** This is so users better understand the function of the many pages on the website.

Tests

Test ID	Test Input	Expected Outcome
W1	Try accessing the website when the user is not logged in.	The user should be rerouted to the login page.
W2	A logged in user tries to access an endpoint that doesn't exit.	The user is sent to the 404 page.
W3	A logged in user tries to access the admin page or admin page related endpoints.	The user should be sent an access denied message

Student/Admin Register Page

Design



Function

This page allows users to make an account. Enter the required details and make a password. To register click the 'Sign-up' button.

This page acts as the first page of the website.

If you already have an account, you can go to the login page by clicking the 'Already have account' button

If you want to be an admin you can click the admin button on the top. On the admin register page there is an extra field called 'Admin Code'. To become and admin you must have a secret code.

Useability Features

- Below the password field there is a check box called 'Show password' that toggles password visibility. **Why have this feature?** This makes it easier for users to see if they typed the right password in case, they made a mistake.
- When validation error happens, the error appears in red underneath the appropriate input field. **Why have this feature?** The colour red stands out to the

user, so they know there is an error. The error appearing under the appropriate field allows users to quickly identify and change their input.

- The errors appear systematically if there are multiple errors starting from the first input field. **Why have this feature?** So that users do not overwhelmed by the possible slew of errors.

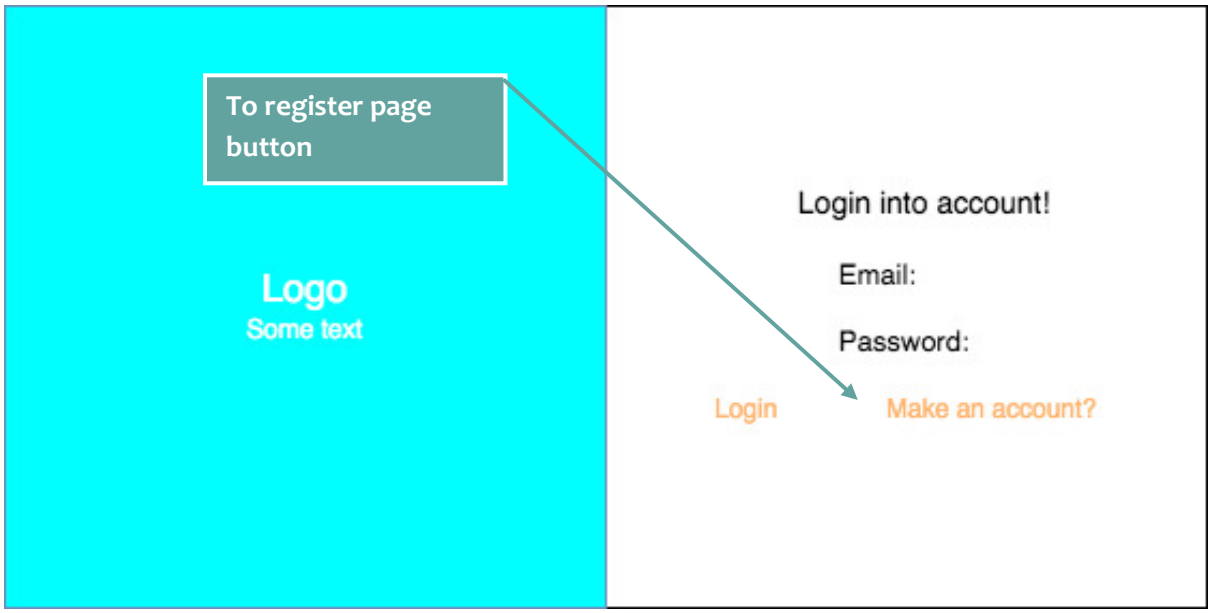
Tests

Test ID	Test Input	Expected Outcome
R1	First Name: John, Last Name: Smith Email: doctor@who.com Password: Tardis12	Positive. User added to database. User redirected to login page.
R2	Any input that warrants an error message.	Negative. User not added to database. Error message sent.
R3	First Name: Jo, Last Name: Smith Email: doctor@who.com Password: Tardis12	Error message displayed under first name field: "First name must be 3 chars or greater"
R4	First Name: (name with 20 chars), Last Name: Smith Email: doctor@who.com Password: Tardis12	Error message displayed under first name field: "First name must be less than 20 chars"
R5	The same for R2 and R3 but for the 'Last Name' field.	The same for R2 and R3 but for the 'Last Name' field.
R6	Email field with the wrong format.	Error message displayed under email field: "Email is of the wrong format"
R7	Password with less than 6 characters	Error message displayed under password field: "Password must be greater than 6 chars"
R8	All fields left empty.	Error message displayed under first name field: "First name is required"

Test ID	Test Input	Expected Outcome
R9	Filling in the field one by one and each time clicking the 'Sign-up' button.	Error message displayed under each field saying that this field is required.
R10	Click the 'Already have account?', 'Student' and 'Admin' buttons.	'Already have account?' button redirects to Login page, 'Student' redirects to basic register page, and 'Admin' button redirects to admin register page.
R11	Click the 'show password' check box.	The password not visible when not checked and visible when checked.
R12	In the admin register, enter the details with the correct admin code and click sign-up.	Positive. User added to database with the role of an admin. User redirected to login page.
R13	In the admin register, enter the details with the wrong admin code and click sign-up.	Error message displayed under the admin code field saying: "This code is invalid".

Login Page

Design



Function

This page allows users to login into an account and get access to the main website.

If you don't have an account, you can go to the register page by clicking the 'Make an account' button.

Both students and admins login through this page.

Useability Features

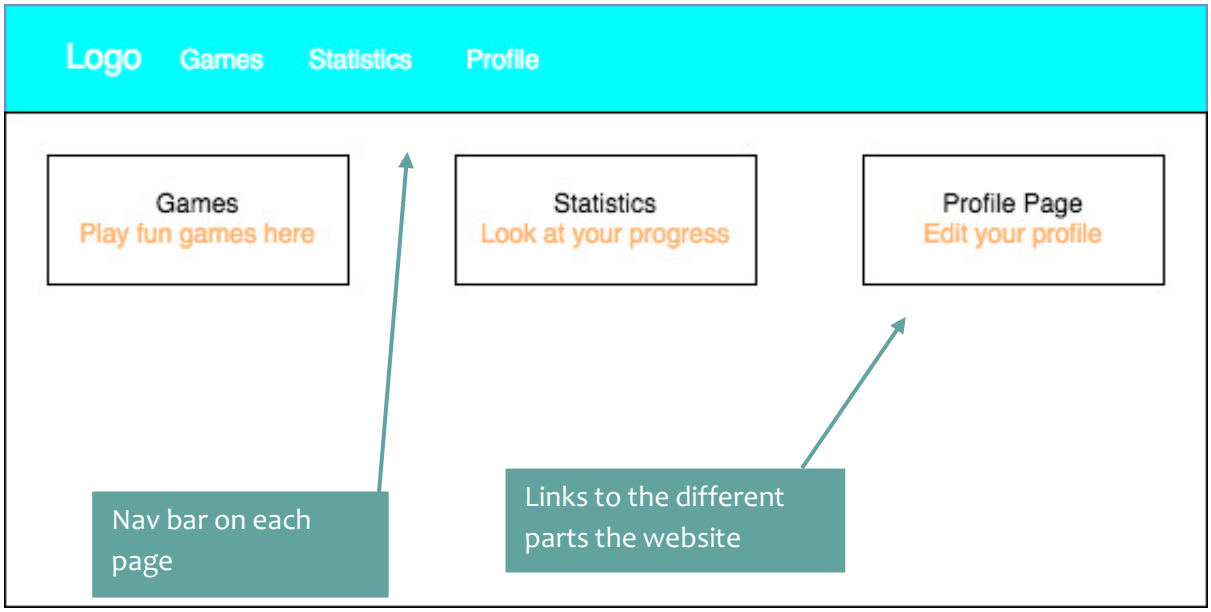
- Below the password field there is a check box called 'Show password' that toggles password visibility. **Why have this feature?** This makes it easier for users to see if they typed the right password in case, they made a mistake.
- When validation error happens, the error appears in red underneath the appropriate input field. **Why have this feature?** The colour red stands out to the user, so they know there is an error. The error appearing under the appropriate field allows users to quickly identify and change their input.
- The errors appear systematically if there are multiple errors starting from the first input field. **Why have this feature?** So that users do not overwhelmed by the possible slew of errors.

Tests

Test ID	Test Input	Expected Outcome
L1	Email: doctor@who.com Password: Tardis12	Positive. User redirected to home page.
L2	Any input that warrants an error message.	Negative. Error message sent.
L3	Email: doctor@whoops.com Password: Tardis12	Error message beneath email field: "This user does not exist".
L4	Email: doctor@who.com Password: Tardis123	Error message beneath password field: "Wrong password".
L5	Format test the email field.	Error message beneath the email field: "Wrong email format".
L6	Email: (nothing) Password: (nothing)	Error message beneath email field: "Both email and password are required".
L7	Email: doctor@who.com Password: (nothing)	Error message beneath email field: "Both email and password are required".
L8	Email: (nothing) Password: Tardis12	Error message beneath email field: "Both email and password are required".
L9	Click the 'show password' check box.	The password not visible when not checked and visible when checked.
L10	Click the "Make an account" button.	User taken to the basic register page.

Home Page

Design



Function

This page introduces the user to each part of the website and their function.

The parts in orange are links to the different parts of the website.

Useability Features

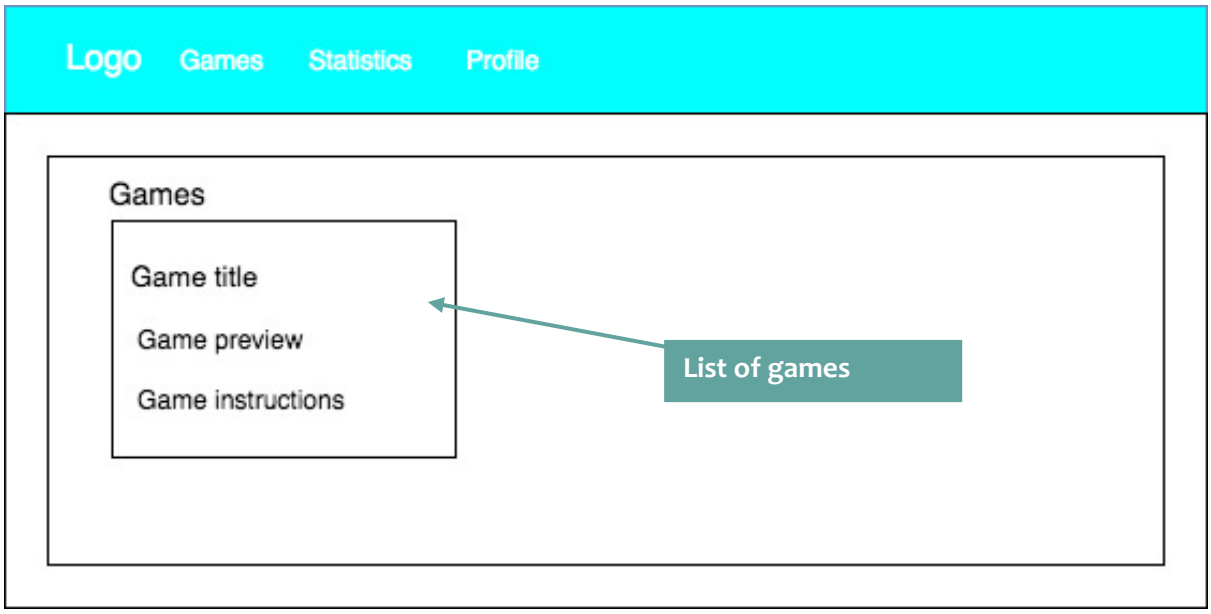
- There is a short description of the function of each page of the website.

Tests

Test ID	Test Input	Expected Outcome
H1	Click the button for Game on the nav bar and underneath the 'Games' title	User redirected to games page.
H2	Click the button for Statistics on the nav bar and underneath the 'Statistics' title	User redirected to stats page.
H3	Click the button for Profile on the nav bar and underneath the 'Profile' title	User redirected to profile page.

Games Page

Design



Function

Shows the user a list of games with its title and instructions on how to play.

Useability Features

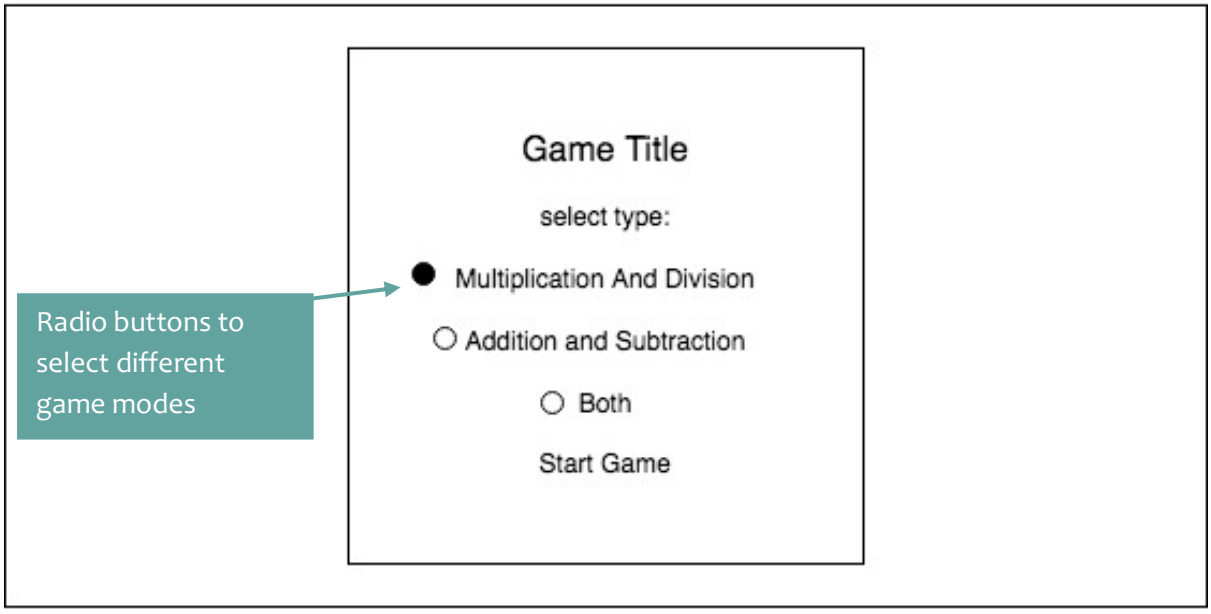
- The game preview is a screen shot of actual game gameplay. **Why have this feature?** This allows the user to get feel for the type of game the game is.
- The game preview acts as the button to enter the game. **Why have this feature?** This makes sure the user understands they will enter the game if they click the picture.

Tests

Test ID	Test Input	Expected Outcome
G1	Click the game preview picture.	User redirected to game menu page.
G2	Hover cursor over the game preview.	The border changes colour to orange.

Game Menu

Design



Function

This pages as the starting screen for the game where the player can choose their game mode.

Useability Features

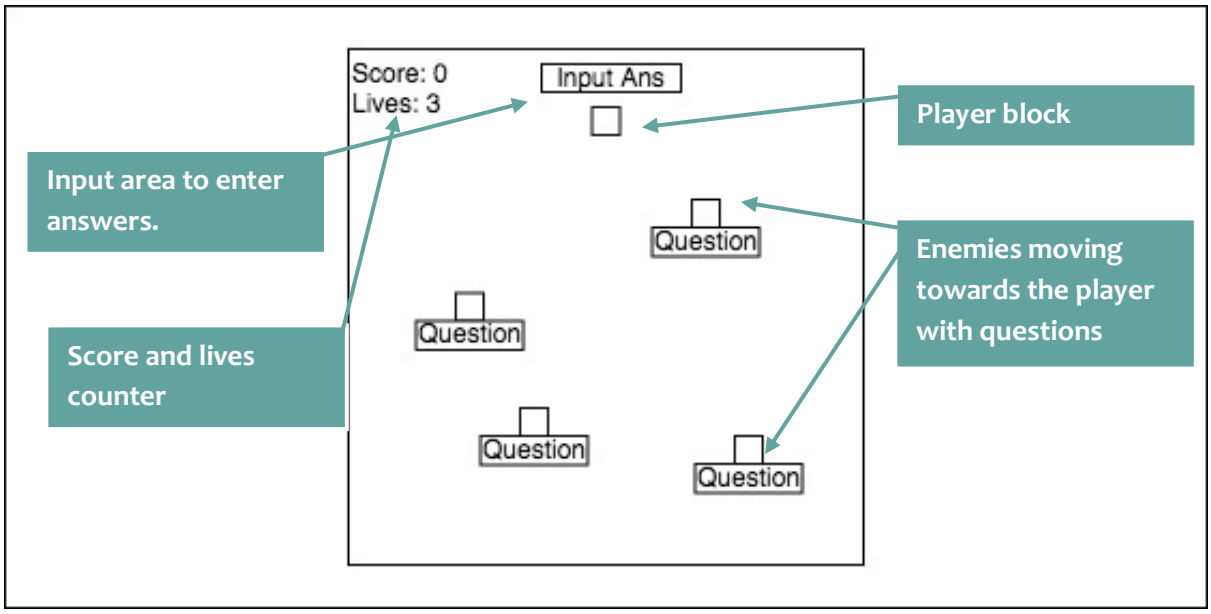
- When entering the game menu, the game mode you previously played is automatically selected. **Why have this feature?** Players can quickly restart the game without having to reselect the game mode.
- The game selection is made using radio buttons. **Why have this feature?** So, its quick and easy for players to select the game mode.

Tests

Test ID	Test Input	Expected Outcome
GM1	Select each radio button one at a time and play the game.	The questions are correct for the game mode selected.
GM2	Observe the game mode selected after restarting.	The game mode should be the same as the game mode previously selected.

Actual Game

Design



Function

Enemies with questions under them, which randomly spawn, are moving towards the player. The player to enter to questions beneath the enemies to 'kill' them.

For each enemy 'killed' the score increments be 1.

If the enemy reaches the player block the lives counter decreases by 1. The player has 3 lives. When it reaches zero the game ends.

As the players score gets greater, the difficulty of the game increases.

Useability Features

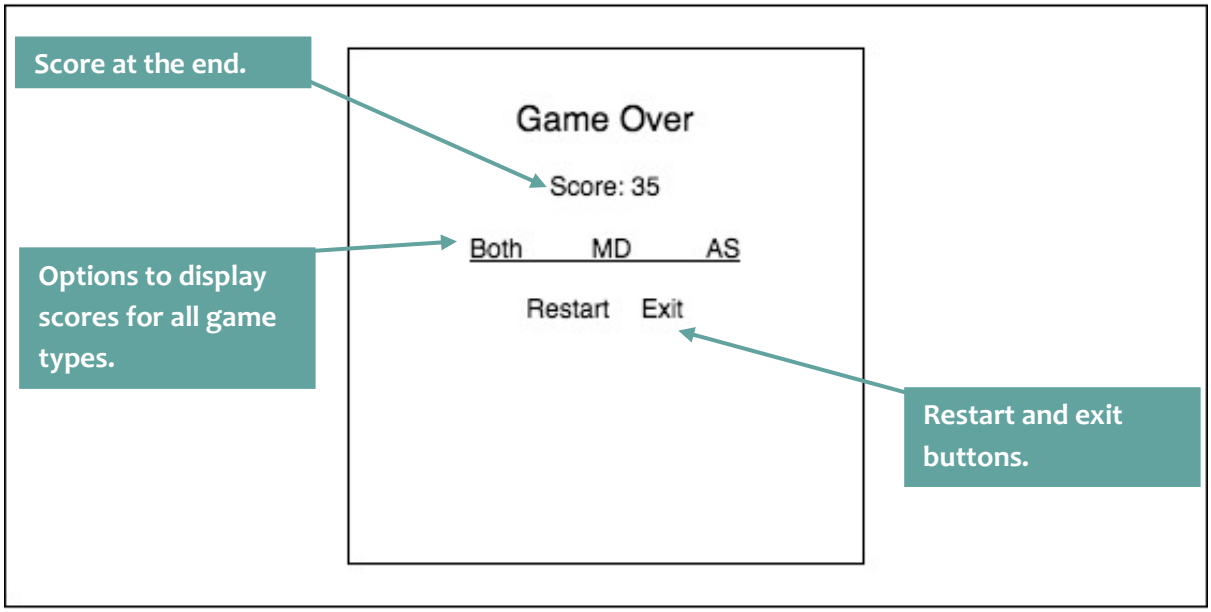
- When starting the game, the input area is auto selected. **Why have this feature?** So, the player can start typing straight away.
- The enemies are a different colour from the player. **Why have this feature?** So, the player can easily differentiate between the two.
- The questions have a white background. **Why have this feature?** So, when multiple enemies spawn in, the questions don't overlap each other making it difficult to read.

Tests

Test ID	Test Input	Expected Outcome
AG1	Enter an answer into the input area.	The enemy with the questions to that answer should disappear from the DOM.
AG2	Enter a wrong answer into the input area.	Nothing should happen.
AG3	Observe the lives and score counters.	Score should increase by 1 when answer is correct, and lives should decrease by one when an enemy reaches the player.
AG4	Play the game.	As the game progresses the enemies should start to move faster, and the questions should become more challenging. All the other outcomes expected from playing the game.
AG5	Observe when lives go down to 0.	User redirected to Game End page.
Test ID	Test Input	Expected Outcome
GE1	Click all the game mode buttons.	A top 15 list in descending order should appear for the relevant game mode.
GE2	Observe the current game mode score board	The score just attained should be on the score board if it is great enough.
GE3	Click the 'Restart' and 'Exit' buttons.	The restart button should take the user to the game menu, and the exit button takes you to the games page.

Game End Page

Design



Function

Shows the user score for the current game just played and also their place on their personal leader board.

There is also a display for their top 15 scores on all the game modes. They can select which score board is displayed.

You can restart and exit the game from this page.

Useability Features

- The relevant score board is shown for the game mode just played. **Why have this feature?** So, the user can quickly know what their place on the score board is.
- The selected score board changes boarder colour. **Why have this feature?** So that the user can better know what list of scores they are looking at.

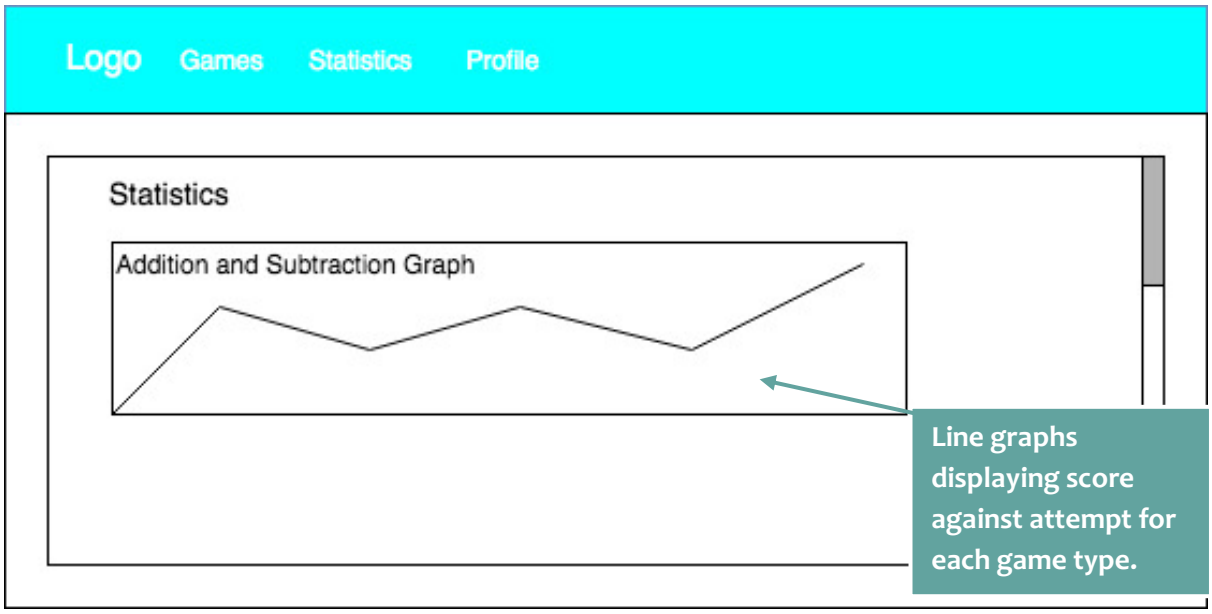
Tests

Test ID	Test Input	Expected Outcome
GE1	Click all the game mode buttons.	A top 15 list in descending order should appear for the relevant game mode.

Test ID	Test Input	Expected Outcome
GE2	Observe the current game mode score board	The score just attained should be on the score board if it is great enough.
GE3	Click the 'Restart' and 'Exit' buttons.	The restart button should take the user to the game menu, and the exit button takes you to the games page.

Statistics Page

Design



Function

For each game mode a line graph with score number against attempt is displayed.

Useability Features

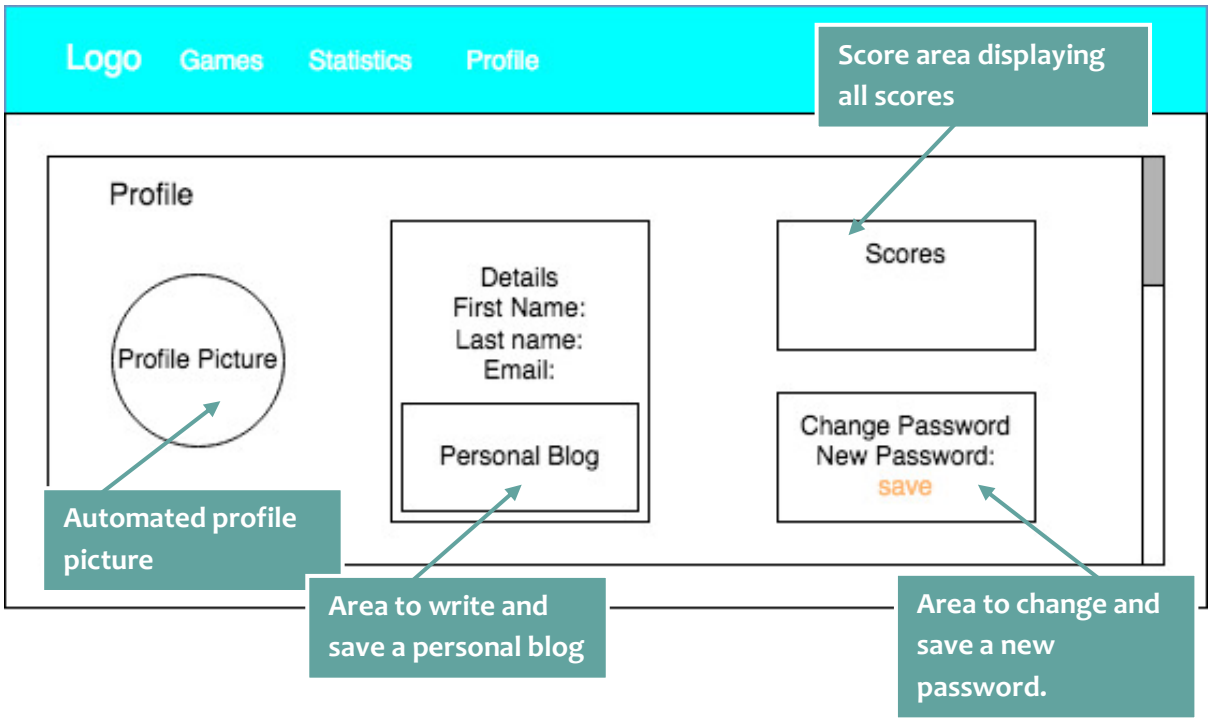
- The data points are plotted in a different colour than the line. **Why have this feature?** To highlight the data point so users can visualize their data better.

Tests

Test ID	Test Input	Expected Outcome
S1	Play a game and come back to the stats page.	The new data point with the correct score and attempt number should appear.
S2	Observe the graphs.	The data should all be correctly placed.

Profile Page

Design



Function

This page is to show the user a full display of all their details and scores.

On this page the user can log out, change their password or edit their personal blog.

Useability Features

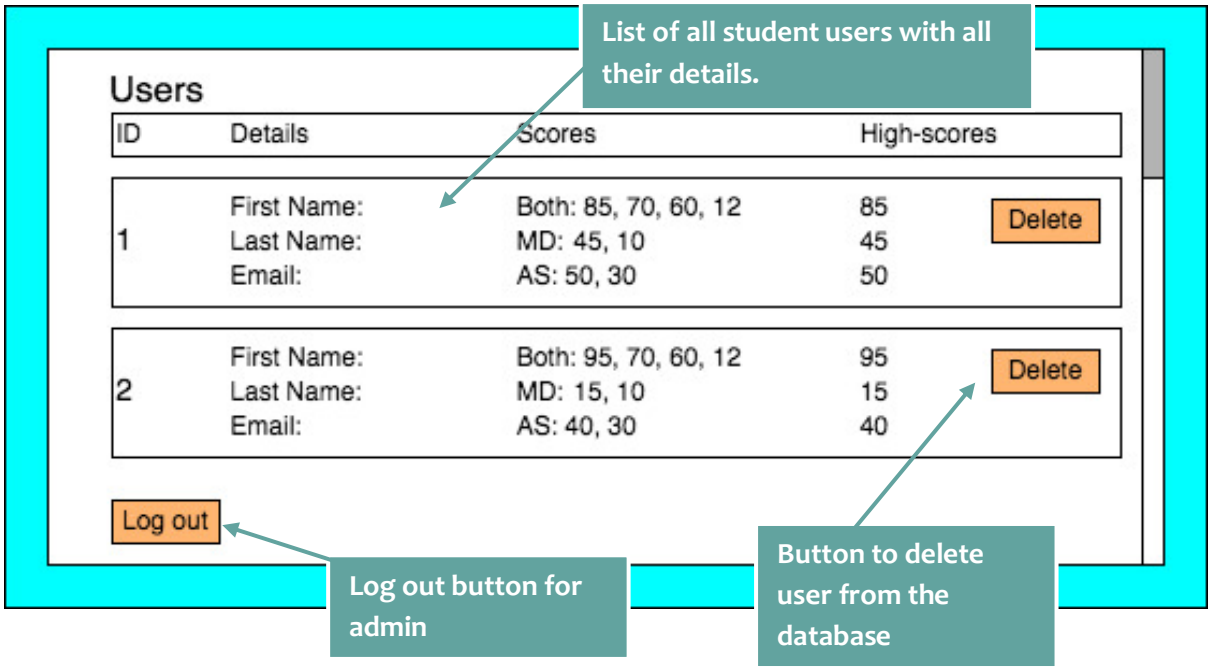
- Below the password field there is a check box called 'Show password' that toggles password visibility. **Why have this feature?** This makes it easier for users to see if they typed the right password in case, they made a mistake.
- The error message for the password appears below the input field in red. **Why have this feature?** The message stands out so, the user can more clearly see the error.
- The button to save blog and save password word change their text when clicked from "Save blog" to "Blog saved", and "Change Password" to "Password Changed". **Why have this feature?** So that the user knows the process of saving the blog and password is complete.

Tests

Test ID	Test Input	Expected Outcome
P1	Observe the data on screen.	All the relevant data should match the data in the database. For instance: First Name should match, the list of scores should match and so on.
P2	Type in a password with less than 6 chars into the new password field and click 'Save Password'.	Error message should be return saying: "Password must be greater than 6 chars".
P3	Type in a password with greater than 6 chars into the new password field and click 'Save Password'.	The text on the button should change to "Password Saved". The new password should appear in the database.
P4	Type anything into the blog text area and click 'Save blog'.	The text of the button should change to "Blog Saved". The blog should appear inside of the database.
P5	Type greater than 100 characters into the blog text area and click 'Save blog'.	The text of the button should not change, and the text should not be saved the database. Error msg sent.

Admin Page

Design



Function

The admin is able to see all the users details and scores and high scores. This allows the admin, who is likely to be a teacher, to see who is making progress.

The admin can also delete users.

Useability Features

- If the list of scores is greater than five, clicking a button below the list called 'Show more' displays a drop-down list that sits on top of everything else. **Why have this feature?** So, the admin can easily see all the scores without cluttering the screen.
- The button to delete users only appears when hovering over that particular record. **Why have this feature?** So, that the admin doesn't accidently delete the wrong record.

Tests

Test ID	Test Input	Expected Outcome
A1	Observe the data for each record on the screen.	All the relevant data should match the data in the database. For instance: First Name should match, the list of scores should match and so on.
A2	Click delete button for a record.	The record should be removed from the DOM. The record should be removed from the database.

POST DEVELOPMENT TEST PLAN

Below are detailed the tests that must be done after the development stage.

Test ID	Feature	Success Criteria	The Function in Question	Expected Outcome	User Opinion Feature successful: **Yes/No. If No, why?**
1	Navigation bar	Navigation bar that allows quick and easy access to different parts of the website	Nav bar buttons	Users quickly able to navigate the website without getting confused. The nav buttons work as intended.	
2	Home page	Home page that welcomes the user and gives directories to different pages on the website.	Home page navigation buttons	The home buttons take the user to the correct pages.	
3	Stats page	Stats page that shows all scores on a line graph and so on.	Line graphs	The correct data is shown on the line graphs.	
4	Game page	Game page that shows a list of the games on the website	Game preview	Clicking the game preview take the users to the game menu.	
5	Profile page	Profile page that displays all of the user's details as well as the user's top ten scores.	User details and scores	Correct user details and scores displayed.	
		A field that allows users to write a personal blog.	Blog area	Validation on blog works. Saving the blog works.	
		Area for user to change their password.	Change password area	Validation on new password works. Saving the new password works.	

Test ID	Feature	Success Criteria	The Function in Question	Expected Outcome	User Opinion Feature successful: **Yes/No. If No, why?**
6	Admin interface	Admin page for teacher to see all the details of students.	All user details	All correct data displayed.	
7	Login and sign-up window	Login and sign-up page that is simple.	To login and register buttons.	Clicking the buttons take the user from the login to register page and vice versa.	
		Each input should be fully validated.	Login Button	If the user input is correct, they are taken to the home page. If not, then an error message is shown.	
			Register Button	If the user input is correct, they are taken to the login page and their details are added to the database. If not, then an error message is shown.	
		Admin register should have a field for a code that allows for an admin account to be created.	Admin code field	Only users with the special key can become an admin.	
8	**Game** {One or two math games	One or two math games. Minimum of one.	The game being present	A playable game.	
9	Increasing difficulty	Game/games that in increase in difficulty as your score increases	Increasing difficulty through progression	Questions getting harder and enemy speed increasing.	

Test ID	Feature	Success Criteria	The Function in Question	Expected Outcome	User Opinion Feature successful: **Yes/No. If No, why?**
10	Different Game Types	Players have the option to choose between having only addition and subtraction questions, multiplication or division questions or a mix of both types	Game mode options	The different game modes work as intended.	
11	Heavily math orientated	There will be little extra thinking with mechanics not involved with the math part of the game.	The game functionality	Up to user opinion.	
12	Practices all parts of mental math	Addition, subtraction, dividing, multiplying, fractions, algebra. The last two may be limited by time	The math questions	Up to user opinion.	
13	Appropriate aesthetic	Satisfying animations and bold colours.	Game styles	Up to user opinion.	
14	Repeatable and fun}	Game is easy to play, easy to access and repeat many times over.	Game styles	Up to user opinion.	
15	Change password option	Users have the ability to change their password when- ever they want through the profile page	Change password area	Validation on new password works. Saving the new password works.	
16	Aqua, white accents with orange buttons	Aqua, white accents with orange buttons for the whole webpage.	Website styles	Up to user opinion.	

Test ID	Feature	Success Criteria	The Function in Question	Expected Outcome	User Opinion Feature successful: **Yes/No. If No, why?**
17	Appropriate font and style	Appropriate font and style for the website.	Website styles	Up to user opinion.	

Test Data

Function to be Tested	Test Data
Blog area	100 characters: "0123456789012345678901234567890123456789012345678901234567890123456789012345678901234567890123456789" 101 characters: "01234567890123456789012345678901234567890123456789012345678901234567890123456789012345678901234567890"
Change password area	Under 7 characters: "Vroom!" Over 20 characters: "Tweeeeeeeeeeeeeeeeee" In the acceptable range: "Tardis96"
Register form validation	Correct data: First Name: "John", Last Name: "Smith", Email: "doctor@who.com", Password: "Tardis12" Wrong data: First Name: "Jo/Waaaaaaaaaaaaaaaade", Last Name: "Sm/Smooooooooooooooooth", Email: (Wrong formatted email), Password: (Same as in Change password area Test Data) Wrong data for any field: (empty)
Login form validation	Correct data: Email: doctor@who.com, Password: "Tardis12" Wrong data: Email: doctor@whoops.com / (empty), Password: "Tardis123/(empty)"

DEVELOPMENT

ORDER OF DEVELOPMENT

My development will be taken in stages. **After each stage, the function will be tested.**

Stage Num.	Stage Name	Stage Description
1	Setting up the project	• Download Node JS, VS code, and NPM • Set up the project with NPM • Set up a simple server • Test if the different route methods work using postman
2	Setting up the Database	• Set up the database in MongoDB • Connect the database to the server • Test the database connection is working
3	Making the models	• Create the schemas for the User and Score tables. • Test if the adding a record to database works.
4	Setting up the routes	• Creating the 'api' and 'routes' folders • Set up the prototype routes to the pages on the website and test if they work using postman • Set up error catching on the server side • Test if the error catching works

Stage Num.	Stage Name	Stage Description
5	Login/register pages	• Create the html folder • Make the login and register page prototypes • Add the necessary endpoints • Add the validation • Add the admin register • Style the pages • Carry out the tests for these pages
6	Making the navbar	• Make a functioning navbar • Create a simple page for each route • Test if the routing is working
7	Making the Game and game end/menu pages.	• Make the game menu • Make the game container • Create the enemies and their moving • Make the questions and put on the enemies • Set up the input area • Implement the increasing level of difficulty • Make the game end page • Carry out the tests for this game and pages
8	Setting up the pages	• Make each page • Make each one functional • Carry out the tests for these pages

Stage Num.	Stage Name	Stage Description
9	Making the admin page	• Make the page and make it functional • Carry out the tests for this page
10	Protecting the routes	• Make the userAuth middleware to validate the user • Test if it is functioning
11	Post development testing	• Go through the post development test plan check list

SETTING UP THE PROJECT

Download Node JS & NPM

Download Node JS and NPM from the internet. Since my laptop is quite old, newer versions of Node are not supported. Therefore, I will have to download an older version.

To check Node JS is now on the computer do the following in the command terminal:

```
Macbookpros-MacBook-Pro:Final_Coursework macbookpro$ node -v
v14.21.0
```

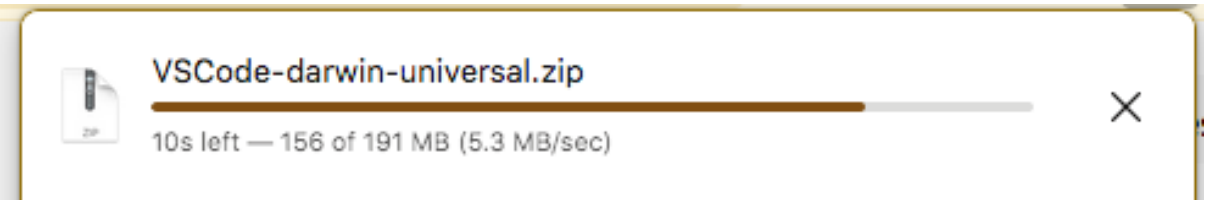
Do the same for NPM:

```
Macbookpros-MacBook-Pro:Final_Coursework macbookpro$ npm -v
6.14.17
```

As is it shown Node JS and NPM are now present on the computer.

Download VS Code

Download VS code from the internet.



Download the Essentials

- Download the Prettier extension to format my code.
- Use NPM to install nodemon, morgan and express JS.
    - Nodemon allows me to restart the server more easily.
    - Morgan shows the requests sent to the server and can display other information.
    - Express JS is a framework that makes it simpler to set up a server.

Setup the Project

After typing 'npm init' into the command prompt, 3 new items have appeared in the VS code explorer:

In the '*node modules*' folder, all the node packages are stored that are available in the project.

The '*package.json*' file contains the following:

```
{
  "name": "final_coursework",
  "version": "1.0.0",
  "description": "Final Coursework",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "author": "Muhammed Bachelani",
  "license": "ISC",
  "dependencies": {
    "express": "^4.18.2",
    "morgan": "^1.10.0",
    "nodemon": "^2.0.22"
  }
}
```

In this file, the project dependencies and other details are shown.

I will add two scripts into the "scripts" object:

1. "start": "node server.js"
2. "dev": "nodemon server.js"

Now whenever 'npm run (script name)' is typed into the terminal, the corresponding script will be run.

The first script is used for general use of the server.

The second script is for editing and making changes on the server and quickly being able to see the effects.

For this development process I will be using the "dev" script and for the testing I will be using the "start" script.

The '*package-lock.json*' file contains all the information that sets up the project. All of it is unimportant to completing this project.

Creating a Simple Server

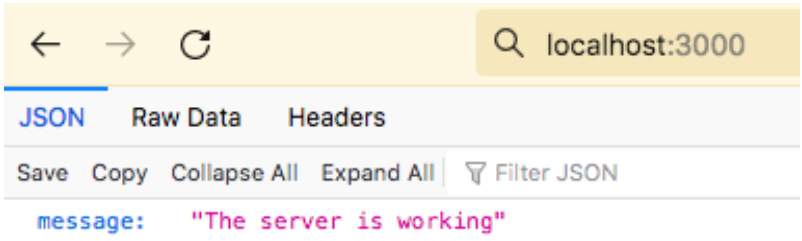
Here is the code to set up the server:

```
1  //The express package is imported here
2  const express = require("express");
3
4  //Creates a express driven server
5  const app = express();
6  const Port = 3000;
7  //the server is no hosted locally on the port 3000
8  app.listen(Port, () => {
9    console.log("Server listening on port", Port);
10 });
11
12 //Creates an endpoint at "localhost:3000/"
13 app.get("/", (req, res) => {
14     //Sends to the endpoint a json response
15   res.json({
16     message: "The server is working",
17   });
18 });
```

Start the server with 'npm run start'.

```
Macbookpros-MacBook-Pro:Final_Coursework macbookpro$ npm run start

> final_coursework@1.0.0 start /Users/macbookpro/Desktop/Projects/Final_Coursework
> node server.js

Server listening on port 3000
```

When calling the endpoint, the following is seen:

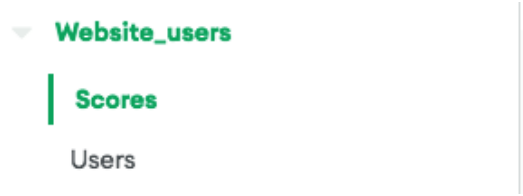


TEST SUCCESS! The server is working!

SETTING UP THE DATABASE

Make a MongoDB Database

In MongoDB Compass, I created a database called 'Website_users' with two tables, on called **'Scores'** and the other called **'Users'**:



Connecting it to the Server

Here I create a new file called '*db.js*'. In here I will make a module to connect to the database.

First, I must download the Mongoose package. Mongoose makes interacting with a MongoDB database easier.

```
15   "mongoose": "^7.0.3",
```

Mongoose is now installed.

Below is the code to connect to the database:

Importing the module inside the server:

```
//Imports the mongoose package
const Mongoose = require("mongoose");
//This is the connection url
const dbURI =

function connectDB() {
  //Makes the connection to the database
  Mongoose.set("strictQuery", true);
  Mongoose.connect(dbURI, { useNewUrlParser: true, useUnifiedTopology: true })
    .then(() => {
      //If the process is successful, run this code
      console.log("Database connected");
    })
    .catch((err) => {
      //If the process is unsuccessful, run this code
      console.log(err);
    });
}
//Exports the function to be used else where
module.exports = connectDB();
```

```
3  //Imports the module from db.js
4  const connectDB = require("./db");
5
6  //Connects to the databse
7  connectDB;
```

Test if the database is working by running the server:

```
Macbookpros-MacBook-Pro:Final_Coursework macbookpro$ npm run start

> final_coursework@1.0.0 start /Users/macbookpro/Desktop/Projects/Final_Coursework
> node server.js

Server listening on port 3000
Database connected
```

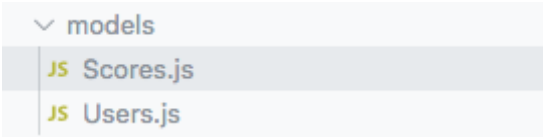
TEST SUCCESS! Database connected appeared in the console therefore the database is working!

MAKING THE MODELS

The mongoose allows you to make a model for each of your tables your database. A model is the format of the information in the database.

Inside of a model you can creates schemas which act describe the format of information in the database.

Make a new folder called 'model' and create two files inside: *'Users.js'* and *'Scores.js'*.



Inside the *'Users.js'* file I code the schema for the **'Users'** table in the database:

```
//Imports the mongoose package
const Mongoose = require("mongoose");

//Creates a new schema
const userSchema = new Mongoose.Schema({
  firstname: {
    type: String,
  },
  lastname: {
    type: String,
  },
  email: {
    type: String,
  },
  password: {
    type: String,
  },
  blog: {
    type: String,
  },
  role: {
    type: String,
  },
});
//Initiates the model
const User = Mongoose.model("Users", userSchema);

//Exports the model
module.exports = User;
```

I will do the same in the *'Scores.js'* file:

```
//Imports the mongoose package
const Mongoose = require("mongoose");
//Creates a new schema
const scoreSchema = new Mongoose.Schema({
  email: {
    type: String,
  },
  typeBoth: {
    type: [Number],
  },
  typeMD: {
    type: [Number],
  },
  typeAS: {
    type: [Number],
  },
  role: {
    type: String,
  },
});
//Initiates the model
const Score = Mongoose.model("Scores", scoreSchema);
//Exports the Score model
module.exports = Score;
```

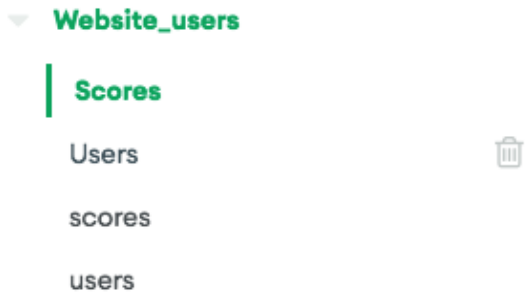
Now I can test if this worked by importing these modules into the server and trying to save a score and user record to the database:

```
//Imports the module from db.js
const connectDB = require("./db");
const Mongoose = require("mongoose");
//Imports the models
const User = require("./models/Users");
const Score = require("./models/Scores");

//Connects to the databse
connectDB;

//Creates a new instance of the User model
const newUser = new User({
  email: "doctor@who.com",
  firstname: "John",
  lastname: "Smith",
  password: "Tardis12",
  blog: "",
  role: "basic"
})
//The instance is saved to the database
newUser.save()
.then((res) => {
  console.log(res)
})
.catch ((err) => {
  console.log(err)
})
```

Run the server and check if the user has been added to the **'Users'** table in the database.

```
Macbookpros-MacBook-Pro:Final_Coursework macbookpro$ npm run start

> final_coursework@1.0.0 start /Users/macbookpro/Desktop/Projects/Final_Coursework
> node server.js

Server listening on port 3000
Database connected
{
  firstname: 'John',
  lastname: 'Smith',
  email: 'doctor@who.com',
  password: 'Tardis12',
  blog: '',
  role: 'basic',
  _id: new ObjectId("6436b874131b6043aab89972"),
  __v: 0
}
```

The save function completed successfully. Now to check the database:



TEST FAILED! It looks like two new tables have been created which should not happen:

This was the problem:

```
22  const Score = Mongoose.model("Scores", scoreSchema);
```

```
26  const User = Mongoose.model("Users", userSchema);
```

In mongoose the model function's first argument is the name of the model which corresponds to the table name, but mongoose automatically looks for the **plural, lowercased version of your model name.**

Therefore, mongoose was not finding a Table named 'scores' or 'score', or 'users' or 'user', because the names of my tables are **'Scores'** and **'Users'**. Therefore, it created two new tables.

To fix this I changed the table names to **'users'** and **'scores'**, and changed the code:

```
26  const User = Mongoose.model("user", userSchema);
```

```
22  const Score = Mongoose.model("score", scoreSchema);
```

Now restarting the server and checking the database:

No new tables. Now inside the **'users'** table:

```
_id: ObjectId('6436b874131b6043aab89972')
firstname: "John"
lastname: "Smith"
email: "doctor@who.com"
password: "Tardis12"
blog: ""
role: "basic"
__v: 0
```

TEST SUCCESS! A new record has been added to the **'users'** table.

Test if trying to make a new record with the wrong format works:

```
13  //Creates a new instance of the User model
14  const newUser = new User({
15    email: "doctor@who.com",
16    firstname: "John",
17    secondname: "Smith",
18    password: "Tardis12",
19    blog: "",
20    role: "basic",
21  });
```

I changed the 'lastname' field to 'secondname'. Now restarting the server and looking in the table **'users'**:

```
_id: ObjectId('6437fef0ad185782b4fa01b7')
firstname: "John"
email: "doctor@who.com"
password: "Tardis12"
blog: ""
role: "basic"
__v: 0
```

TEST SUCCESS! The field 'secondname' was not saved to the database. Therefore, the model works.

Now to do the same with adding a record to the **'scores'** table:

Here is the record to be saved:

```
const newScore = new Score({
  typeBoth: [1, 3, 4],
  typeMD: [2, 5, 7],
  wrongField: [23, 2],
  role: "basic",
});
//The instance is saved to the database
newScore
  .save()
  .then((res) => {
    //If the opertion is successful, execute the following
    console.log(res);
  })
  .catch((err) => {
    //If the opertion is unsuccessful, execute the following
    console.log(err);
  });
```

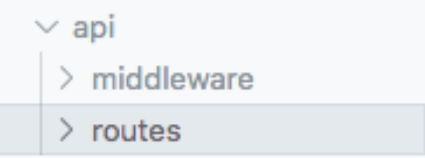
Restarting the server and looking in the **'scores'** table:

```
  _id: ObjectId('64383473f4dde5c8ce3f9340')
▾ typeBoth: Array
    0: 1
    1: 3
    2: 4
▾ typeMD: Array
    0: 2
    1: 5
    2: 7
▾ typeAS: Array
  role: "basic"
  __v: 0
```

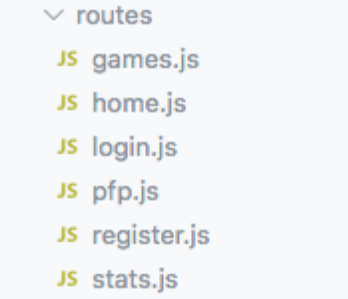
TEST SUCCESS! The record was saved to the **'scores'** table and the field 'wrongField' was not saved to the database. Therefore, the model works.

SETTING UP THE ROUTES

Create the '*api*' folder and inside make the '*routes*' and '*middleware*' folders:



Create separate JavaScript files to harbour the coding of endpoints for those routes:



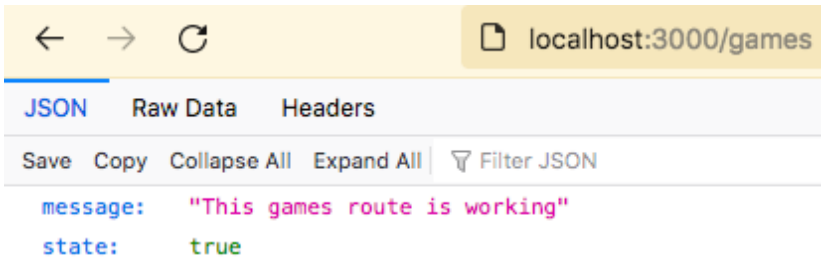
Here is the code to be added in each file:

```
//Imports
const express = require("express");
const route = express.Router();
const path = require("path");

function jsonResponce(res, text, code, state = true) {
  //Sends a JSON responce to the client
  res.status(code).json({
    message: text,
    state: state,
  });
}

route.get("/", (req, res, next) => {
  jsonResponce(res, "This (route name) route is working", 200);
});

module.exports = route;
```

And the code added inside the 'server.js' file:

```
49  //this gives me the access to the code in the routes
50  const registerPage = require("./api/routes/register");
51  const gamesPage = require("./api/routes/game");
52  const profilePage = require("./api/routes/pfp");
53  const homePage = require("./api/routes/home");
54  const statsPage = require("./api/routes/stats");
55  const loginPage = require("./api/routes/login");
56
57  //this sends the argument "/smth" to the paticular route's file
58  app.use("/register", registerPage);
59  app.use("/login", loginPage);
60  app.use("/games", gamesPage);
61  app.use("/pfp", profilePage);
62  app.use("/home", homePage);
63  app.use("/stats", statsPage);
```

Test whether the routes are working. Restart the server and enter 'localhost:3000/games' into URL. The following is shown:

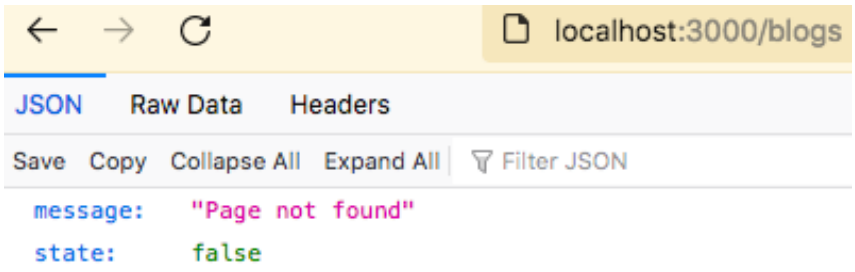


TEST SUCCESS! Calling the route worked. Since all the routes are the same, I will assume they all work. The state parameter will be used later for validation purpose

Next, I will set up the error catching for the whole page in the '*server.js*' file:

```
63  app.use("/stats", statsPage);
64
65
66  //if no page is found then an error is thrown in the json
67  //format saying "Page not found"
68  app.use((req, res, next) => {
69    const error = new Error("Page not found");
70    error.status = 404;
71
72    //this next function says the next function must be called which is the other error handler
73    next(error);
74  });
75
76  //if there is an error that is from anywhere in the code it will be caught here and
77  //returned in json format
78  app.use((error, req, res, next) => {
79    console.log(error.message);
80    res.status(error.status || 500);
81    res.json({
82      message: error.message,
83      state: false,
84    });
85  });
```

Test whether this works by trying to access an endpoint that does not exit:

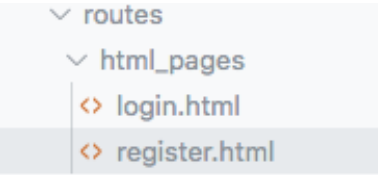


TEST SUCCESS! Since the endpoint 'blogs' does not exist, the error 'Page not found' is thrown.

The other error catching capabilities will be tested in the future.

LOGIN/REGISTER PAGES

Here I create a folder called 'html_pages' in the 'routes' folder. Inside this folder I added two HTML files called 'login.html' and 'register.html':



In these pages I add the following html:

In the 'login.html' file:

```
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta http-equiv="X-UA-Compatible" content="IE=edge" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Alien Arithmetic | Login</title>
  </head>
  <body>
    <style>
      * {
        margin: 0;
        padding: 0;
      }
      body {
        font-family: "Josefin Sans", serif;
        margin: 0 auto;
      }
    </style>

    <div class="landing-body">
      <div class="title">
        <div class="description">
          <h1>Alien Arithmetic</h1>
          <!-- Logo and a short description -->
          <p>Fun games to practice basic arithemtic with aliens</p>
        </div>
      </div>
      <div class="login">
        <form class="login-form">
          <!-- the form tag that contains the input fields -->
          <h2>Log into to play!</h2>
          <label for="email">Email:</label><br />
          <input type="email" id="email" name="email" /><br />
          <p class="error" id="email-error"></p>
          <br />
          <label for="password">Password:</label><br />
          <input type="password" id="password" name="password" /><br />
```

HTML continues:

```
            <!-- Show password button -->
            <input type="checkbox" onclick="showPassword()" id="show" />
            <label for="show">Show password</label>
            <br /><br />
            <p class="error" id="password-error"></p>
            <br />
          </form>
          <div class="buttons">
            <!-- button to login -->
            <button id="login">Login</button>
            <!-- button to go to the sign up page -->
            <button id="to-signup">
              <a href="#">Make an account</a>
            </button>
          </div>
        </div>
      </div>
    </body>
</html>
```

In the 'register.html' file:

```
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta http-equiv="X-UA-Compatible" content="IE=edge" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Alien Arithmetic | Sign Up</title>
  </head>
  <body>
    <style>
      * {
        margin: 0;
        padding: 0;
      }
      body {
        font-family: "Josefin Sans", serif;
        margin: 0 auto;
      }
    </style>
    <div class="landing-body">
      <div class="title">
        <div class="description">
          <h1>Alien Arithmetic</h1>
          <p>Fun games to practice basic arithemtic with aliens</p>
          <!-- Logo and a short description -->
        </div>
      </div>
      <div class="sign-in">
        <form class="sign-up-form">
            <!-- the form tag that contains the input fields -->
          <h2>Make an account to play!</h2>
          <div class="type">
            <nav>
              <ul>
                <li class="student"><a href="#">Student</a></li>
                <li class="admin"><a href="#">Admin</a></li>
              </ul>
            </nav>
```

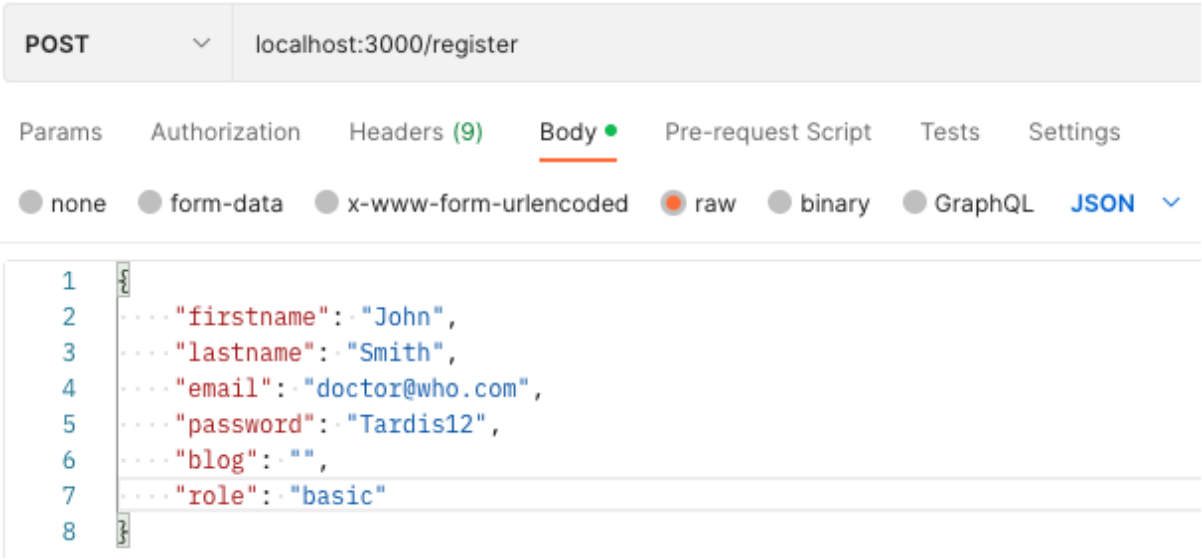
```
            </div>
            <label for="firstName">First Name:</label><br />
            <input type="text" id="firstName" name="firstName" />
            <br />
            <p class="error" id="firstnameError"></p>
            <br />
            <label for="lastName">Second Name:</label><br />
            <input type="text" id="lastName" name="secondName" />
            <br />
            <p class="error" id="lastnameError"></p>
            <br />

            <label for="email">Email:</label><br />
            <input type="email" id="email" name="email" />

            <br />
            <p class="error" id="emailError"></p>
            <br />
            <label for="password">Password:</label><br />
            <input type="password" id="password" name="password" />
            <br />
            <input type="checkbox" onclick="showPassword()" id="show" />
            <label for="show">Show password</label>
            <br /><br />
            <p class="error" id="passwordError"></p>
            <br />
          </form>
          <div class="buttons">
              <!-- button to sign up -->
            <button id="sign-in">Sign Up</button>
            <!-- button to go to the login page -->
            <button id="to-login">
              <a href="#">Already have an account</a>
            </button>
          </div>
        </div>
      </div>
    </body>
  </html>
```

Here I add the endpoints for the login and sign-up page:

In the 'register.js' file here is the new code:

```
18  //Takes in a POST/CREATE request from the front-end to register a new user
19  route.post("/", (req, res, next) => {
20    const data = {
21      //Gets the data from the request body
22      firstname: req.body.firstname,
23      lastname: req.body.lastname,
24      email: req.body.email,
25      password: req.body.password,
26      blog: "",
27      role: req.body.role,
28    };
29    res.json(data);
30  });
```

Test whether the endpoint is working using Postman:



Response after clicking Send

```
{
    "message": "Cannot read property 'firstname' of undefined",
    "state": false
}
```

TEST FAILED! The request body was not able to be read by the server.

The problem is that I forgot to some middleware on the server to convert the request body data into readable JavaScript.

Here is what I added to the 'server.js' file:

```
//This converts JSON in the request body into readable JavaScript
app.use(express.json({ extended: false }));
```

I also added a package to show the requests being sent to the server:

```
//importing a helpful debug libary
const morgan = require("morgan");
app.use(morgan("dev"));
```

Now restarting the server again:

In the console:

```
POST /register 200 146.121 ms - 111
```

The request is shown as a POST request to the /register endpoint.

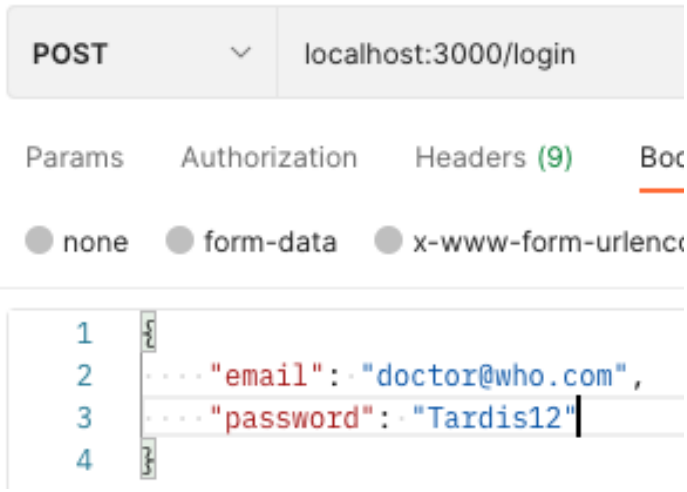
In the request body in postman:

```
{
    "firstname": "John",
    "lastname": "Smith",
    "email": "doctor@who.com",
    "password": "Tardis12",
    "blog": "",
    "role": "basic"
}
```

TEST SUCCESS! The server sent back the inputted data as expected!

Do the same for the /login endpoint:

In the request body in postman:



Response:

```
{
    "email": "doctor@who.com",
    "password": "Tardis12"
}
```

TEST SUCCESS! The server sent back the inputted data as expected!

Now I add the validation.

In the /register endpoint:

```
function capitalizeWord(word) {
  //Capatilizes the first letter of a word
  //and makes the ofthe word lower case
  word = word.toLowerCase();
  const firstLetter = word.slice(0, 1);
  word = word.replace(firstLetter, firstLetter.toUpperCase());
  return word;
}

//Sets up an endpoint with the POST method on /register
route.post("/", (req, res, next) => {
  const data = {
    //Gets the data from the request body
    firstname: capitalizeWord(req.body.firstname),
    lastname: capitalizeWord(req.body.lastname),
    email: req.body.email.toLowerCase(),
    password: req.body.password,
    blog: "",
    role: "admin",
  };
  res.json(data);
});
```

This makes sure the first and last name are capitalized, and the email is in lower case.
Now test is works:

In request body in postman:

```
{
    "firstname": "jOhN",
    "lastname": "sMith",
    "email": "DOCTOR@who.com",
```

Response:

```
{
    "firstname": "John",
    "lastname": "Smith",
    "email": "doctor@who.com",
```

TEST SUCCESS! The first and last names and email are now properly formatted.

Here is the validation email format validation code:

```
function validateEmail(email) {
  //Validates an emails format
  let mailformat = /^\w+([\.-]?\w+)*@\w+([\.-]?\w+)*(\.\w{2,3})+$/;
  if (email.match(mailformat)) {
    return true;
  } else {
    return false;
  }
}
```

Test if this works:

Inside the /register POST endpoint:

```
if (validateEmail(data.email)) {
    jsonResponce(res, "Email has the correct format", 200);
  } else {
    jsonResponce(res, "Email has the wrong format", 200, false);
  }
```

In the request body in postman:

Format tests:

1. No @ symbol:

   ```
   "email": "DOCTORwho.com",
   ```

   Response:

   ```
   {
       "message": "Email has the wrong format",
       "state": false
   }
   ```

2. No full stop:

   ```
   "email": "DOCTOR@whocom",
   ```

   Response:

   ```
   {
       "message": "Email has the wrong format",
       "state": false
   }
   ```

3. More than 63 letters in the local part:

   ```
   "email": "tttttt
   ```

TEST FAILED! The request seems to not be working and is stuck on sending the request. It is broken for all requests no matter the data in the request body.

The problem is that if an email with over the max number of letters is entered and then sent, the validate email function breaks for some reason rendering all use from the function useless.

To solve this issue, I made a separate function to validate email length and limited the length of the local and domain parts to 18 characters for simplicities sake:

```
74  if (validateEmailLength(data.email)) {
75      if (validateEmail(data.email)) {
76        jsonResponce(res, "Email has the correct format", 200);
77      } else {
78        jsonResponce(res, "Email has the wrong format", 200, false);
79      }
80    } else {
81      jsonResponce(res, "Email has the wrong format", 200, false);
82    }
```

```
29  function validateEmailLength(email) {
30    //Validates the length of the local and domain lengths
31    const letters = email.split("");
32    let atCounter = 0;
33    for (let i = 0; i < letters.length; i++) {
34      if (letters[i] === "@") {
35        atCounter++;
36      }
37    }
38    if (atCounter !== 1) {
39      return false;
40    } else {
41      email = email.split("@");
42      if (18 < email[0].length || email[0].length < 1) {
43        return false;
44      } else {
45        if (18 < email[1].length || email[1].length < 1) {
46          return false;
47        } else {
48          return true;
49        }
50      }
51    }
52  }
```

When putting in the same data:

```
4   "email": "ttttttttttttttttttttttttttttttttttttttttttttttttttttttttttttttttttttqqqq@whocom",
```

Response:

```
{
    "message": "Email has the wrong format",
    "state": false
}
```

After checking all the different combinations of incorrect email formats, I can now conclude that the email validation works. **TESTS SUCCESS!**

Now adding the validation for the rest of the fields in the Mongoose schemas:

```
//Creates a new schema
const userSchema = new Mongoose.Schema({
  firstname: {
    //Inside the object is where the field is validated
    type: String,
    required: [true, "First Name is required"],
    minLength: [3, "Name must be greater than 2 chars"],
    maxLength: [20, "Must be less than 21 characters"]
  },
  lastname: {
    type: String,
    required: [true, "Last Name is required"],
    minLength: [3, "Name must be greater than 2 chars"],
    maxLength: [20, "Must be less than 21 characters"]
  },
  email: {
    type: String,
    //Emails must be unique. This is not validation though.
    unique: true,
    required: [true, "Email is required"],
  },
  password: {
    type: String,
    required: [true, "Password is required"],
    minLength: [7, "Name must be greater than 6 chars"],
    maxLength: [20, "Must be less than 21 characters"]
  },
  blog: {
    type: String,
    maxLength: [100, "Must be less than 101 characters"]
  },
  role: {
    type: String,
  },
});
```

There is no validation for the Score schema.

Beneath is the code to check whether the entered email is unique:

```
const User = require("../models/Users");
const Score = require("../models/Scores");
const mongoose = require("mongoose");
```

```
//Checks if the email exists
  User.find({ email: data.email })
    .exec()
    .then((result) => {
      //If the email does not exist
      if (result.length === 0) {
        const user = new User(data);
        user
          .save()
          .then(() => {
            //Creates the scores record for the user
            const score = new Score({
              typeBoth: [],
              typeAS: [],
              typeMD: [],
              email: data.email,
              role: "basic",
            });
            score
              .save()
              .then(() => {
                jsonResponce(res, "Successful register", 201);
              })
              .catch((err) => {
                next(err);
              });
          })
          .catch((err) => {
            next(err);
          });
      } else {
        //If the email exists then do this
        jsonResponce(
          res,
          "user validation failed: email: This email already exists",
          200,
          false
        );
      }
    })
```

Here I put the email validation in the Users schema to clean up the code and to make the order of errors more intuitive:

```
email: {
    type: String,
    required: [true, "Email is required"],

    validate: {
      validator: function validateEmail(email) {
        //Validates the length of the local and domain lengths
        const letters = email.split("");
        let atCounter = 0;
        for (let i = 0; i < letters.length; i++) {
          if (letters[i] === "@") {
            atCounter++;
          }
        }
        if (atCounter !== 1) {
          return false;
        } else {
          splitEmail = email.split("@");
          if (18 < splitEmail[0].length || splitEmail[0].length < 1) {
            return false;
          } else {
            if (18 < splitEmail[1].length || splitEmail[1].length < 1) {
              return false;
            } else {
              //Validates an emails format
              let mailformat = /^\w+([\.-]?\w+)*@\w+([\.-]?\w+)*(\.\w{2,3})+$/;
              if (email.match(mailformat)) {
                return true;
              } else {
                return false;
              }
            }
          }
        }
      },
      message: "Wrong email format",
    },
    //Emails must be unique. This is not validation though.
    unique: true,
  },
```

Test some data:

Data	`{ "firstname": "", "lastname": "", "email": "", "password": "" }`
Response	`{ "message": "user validation failed: firstname: First Name is required, lastname: Last Name is required, email: Email is required, password: Password is required", "state": false }`
Data	`{ "firstname": "John", "lastname": "Smith", "email": "", "password": "" }`
Response	`{ "message": "user validation failed: email: Email is required, password: Password is required", "state": false }`
Data	`{ "firstname": "John", "lastname": "Smith", "email": "doctor@who.com", "password": "Tardis12" }`

Response	`{ "message": "Successful register", "state": true }`
	In the database:
	`_id: ObjectId('643e7d3536'` `firstname: "John"` `lastname: "Smith"` `email: "doctor@who.com"` `password: "Tardis12"` `blog: ""` `role: "basic"` `__v: 0`
	`_id: ObjectId('643e7d353` `email: "doctor@who.com"` `typeBoth: Array` `typeMD: Array` `typeAS: Array` `role: "basic"` `__v: 0`

Data	{
	"firstname": "Jo",
	"lastname": "Sm",
	"email": "doctorwho.com",
	"password": "Tar12"
	}

Response	{
	"message": "user validation failed: firstname: Name must be greater than 2 chars, lastname: Name must be greater than 2 chars, email: Wrong email format, password: Name must be greater than 6 chars",
	"state": false
	}

Data	{ "firstname": "anananananananananananw", "lastname": "anananananananananananw",

Response	{
	"message": "user validation failed: firstname: Must be less than 21 characters, lastname: Must be less than 21 characters",
	"state": false
	}

Data	"email": "doctor@who.com"

Response	{
	"message": "user validation failed: email: This email already exists",
	"state": false
	}

TESTS SUCCESS! Everything worked as intended!

Here I have made the separate endpoints for admin and basic users (students):

```
//Code to allow users to become admins
const secretCode = "ZzaR0sHoVG";
```

```
//Sets up an endpoint with the POST method on /register/admin
route.post("/admin", (req, res, next) => {
  const data = {
    //Gets the data from the request body
    firstname: capitalizeWord(req.body.firstname),
    lastname: capitalizeWord(req.body.lastname),
    email: req.body.email,
    password: req.body.password,
    blog: "",
    role: "admin",
    code: req.body.code,
  };
  //If the code entered is correct do the below
  if (data.code === secretCode) {
```

```
98  } else {
99      //If the code is incorrect do the below
100     jsonResponce(
101       res,
102       "user validation failed: code: This code is invalid",
103       false,
104       200
105     );
106   }
```

This is the endpoint for basic users:

```
85  //Sets up an endpoint with the POST method on /register/basic
86  route.post("/basic", (req, res, next) => {
87    const data = {
88      //Gets the data from the request body
89      firstname: capitalizeWord(req.body.firstname),
90      lastname: capitalizeWord(req.body.lastname),
91      email: req.body.email,
92      password: req.body.password,
93      blog: "",
94      role: "basic",
95    };
```

The rest of the contents of the endpoints are the same.

Here I finish the login endpoint with validation and what not:

```
5   const User = require("../models/Users");
6   const mongoose = require("mongoose");
```

```
22  //Sets up an endpoint with the POST method on /login
23  route.post("/", (req, res, next) => {
24    //Gets the values stored in the email and password from the body
25    const { email, password } = req.body;
26    if (!email || !password) {
27      jsonResponce(
28        res,
29        "password: Both email and password are required",
30        false,
31        200
32      );
33    } else {
34      //If the fields are both filled do the below
35      User.find({ email: req.body.email.toLowerCase() })
36        .exec()
37        .then((user) => {
38          const userContent = user[0];
```

```
        //If an account with email exists then do the below
        if (user.length >= 1) {
          //If the password passed in is correct then do the below
          if (userContent.password === req.body.password) {
            res.status(200).json({
              message: "You logged in",
              state: true,
            });
            //If the password is incorrect then do the below
          } else {
            jsonResponce(res, "password: Wrong password", false, 200);
          }
          //If the email does not exist then do the below
        } else {
          jsonResponce(res, "email: This user does not exist", false, 200);
        }
      })
      .catch((err) => {
        next(err);
      });
  }
});
```

Below are the endpoints to send the html pages for the login and register pages on request:

```
//Sets up an endpoint with the GET method on /login
route.get("/", (req, res, next) => {
  //Send a html file to the display
  res.sendFile(path.join(__dirname, "/html_pages/login.html"));
});
```

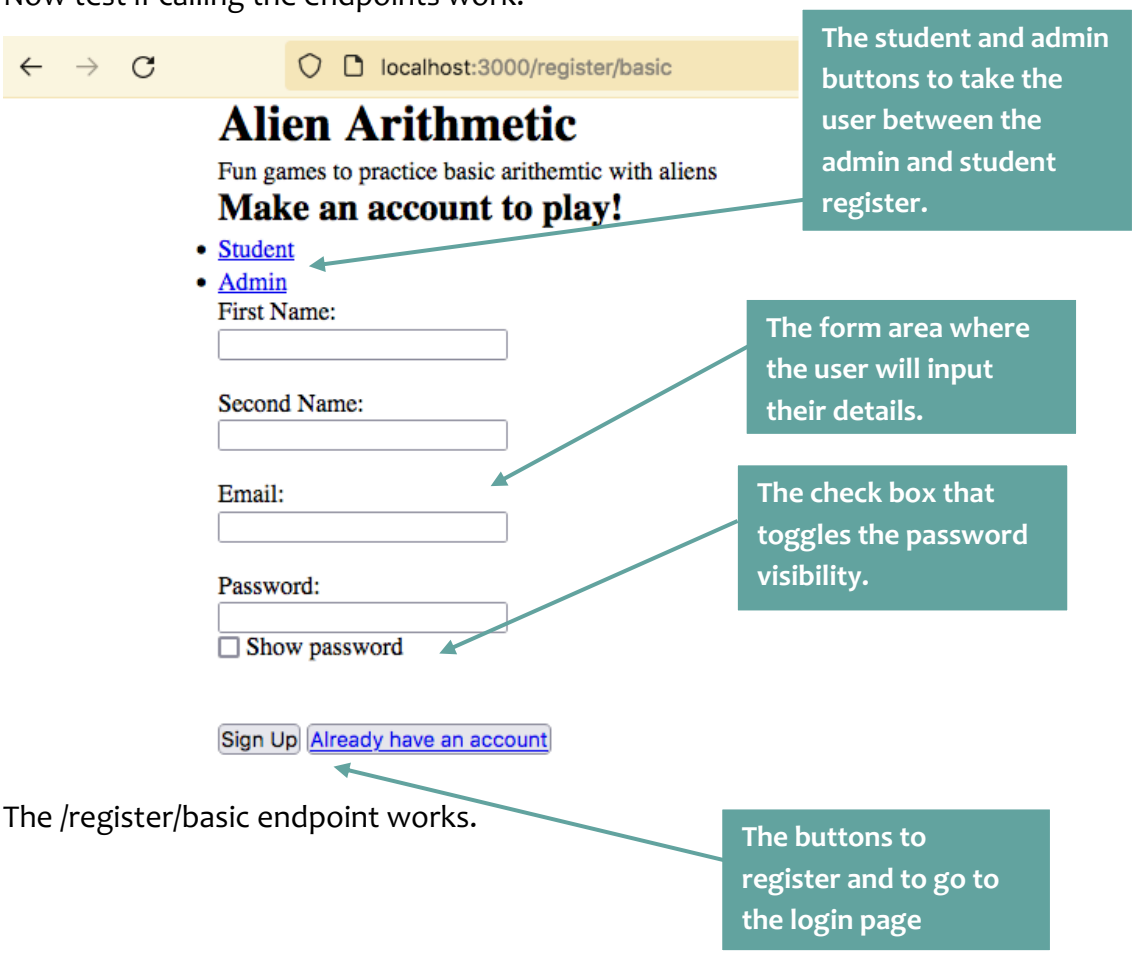
```
//Sets up an endpoint with the GET method on /register
route.get("/", (req, res, next) => {
  res.sendFile(path.join(__dirname, "/html_pages/register.html"));
});

//Sets up an endpoint with the GET method on /register/basic
route.get("/basic", (req, res, next) => {
  res.sendFile(path.join(__dirname, "/html_pages/register.html"));
});

//Sets up an endpoint with the GET method on /register/admin
route.get("/admin", (req, res, next) => {
  res.sendFile(path.join(__dirname, "/html_pages/admin-register.html"));
});
```

I made a new html file in the html folder called '*admin-register.html*':

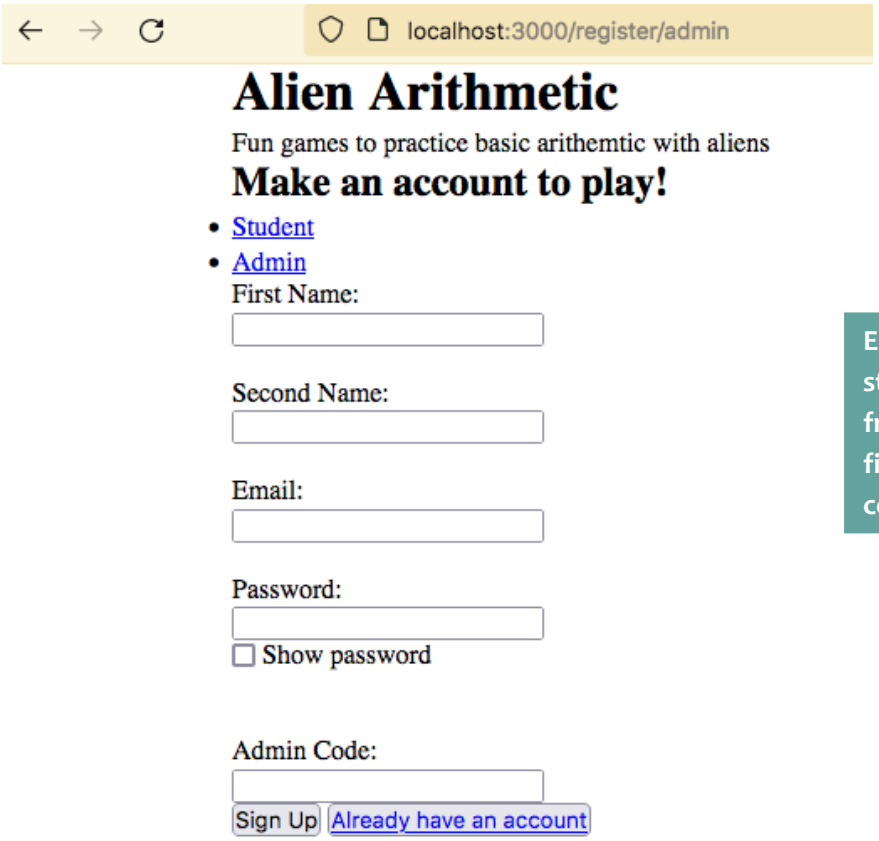
It has exactly the same html as the '*register.html*' file but with an extra field in the form called Admin Code:

```
63  <!-- This field is for to mke sure only users with the code can become admins -->
64            <label for="code">Admin Code:</label><br />
65            <input type="text" id="code" name="code" />
66            <br />
67            <p class="error" id="codeError"></p>
```

Now test if calling the endpoints work:



The /register/basic endpoint works.



Exact same as the student register apart from the new input field for the admin code.

The /register/admin endpoint works.

The email and password inputs to access the website.

The /login endpoint works.

TESTS SUCCESS!

The buttons to login and go to the register page.

At the moment the pages are not functional. Here I make the pages functional.

Register Page:

HTML:

```
<ul>
<!-- Buttons to and from the admin and student register pages -->
<li class="student"><a href="/register/basic">Student</a></li>
<li class="admin"><a href="/register/admin">Admin</a></li>
</ul>
```

```
<div class="buttons">
          <!-- button to sign up -->
          <button id="sign-in" onClick="register()">Sign Up</button>
          <!-- button to go to the login page -->
          <button id="to-login">
            <a href="/login">Already have an account</a>
          </button>
        </div>
```

JavaScript in script tags:

```
<script>
      function showPassword() {
        //Function to change password to toggle visability
        const password = document.getElementById("password");
        if (password.type === "password") {
          password.type = "text";
        } else {
          password.type = "password";
        }
      }

      function getElement(id) {
        //Gets a inputs fields value from the DOM
        const ele = document.getElementById(id);
        return ele.value;
      }

      function cleanForm() {
        //Clears all the text from the error p tags
        const errorTags = document.querySelectorAll(".error");
        errorTags.forEach((ele) => {
          ele.textContent = "";
        });
      }
```

The code continues

```
function valiadateData(data) {
        //Takes in the validation message from the server
        //and displays the messages in the DOM
        const splitData = data.split(" ");
        for (let i = 0; i < 3; i++) {
          splitData.shift();
        }
        const newData = splitData.join(" ").split(", ");

        const errorMsgs = newData.map((error) => {
          error = error.split(":");
          return [error[0], error[1]];
        });

        const element = document.getElementById(errorMsgs[0][0] + "Error");
        element.textContent = errorMsgs[0][1];
      }

      function register() {
        //Sends a POST request to the server in order to create a user
        fetch("http://localhost:3000/register/basic", {
          method: "POST",
          body: JSON.stringify({
            firstname: getElement("firstName"),
            lastname: getElement("lastName"),
            email: getElement("email"),
            password: getElement("password"),
          }),
          headers: {
            "Content-type": "application/json; charset=UTF-8",
          },
        })
          .then((res) => {
            return res.json();
          })
          .then((data) => {
            //When the data is recieved check whether it was succesful or not in creating
            //a user
            if (data.state) {
              location.href = "http://localhost:3000/login";
            } else {
              cleanForm();
              valiadateData(data.message);
            }
          })
          //If there is a network error it is caught here
          .catch((err) => {
            console.log(err.message);
          });
      }
```

Admin register:

HTML:

```
<ul>
<!-- Buttons to and from the admin and student register pages -->
<li class="student"><a href="/register/basic">Student</a></li>
<li class="admin"><a href="/register/admin">Admin</a></li>
</ul>
```

```
<div class="buttons">
          <!-- button to sign up -->
          <button id="sign-in" onClick="register()">Sign Up</button>
          <!-- button to go to the login page -->
          <button id="to-login">
            <a href="/login">Already have an account</a>
          </button>
        </div>
```

JavaScript: Everything is the same apart from the register function:

```
function register() {
        //Sends a POST request to the server in order to create a user
        fetch("http://localhost:3000/register/admin", {
          method: "POST",
          body: JSON.stringify({
            firstname: getElement("firstName"),
            lastname: getElement("lastName"),
            email: getElement("email"),
            password: getElement("password"),
            code: getElement("code"),
          }),
          headers: {
            "Content-type": "application/json; charset=UTF-8",
          },
        })
          .then((res) => {
            return res.json();
          })
          .then((data) => {
            //When the data is recieved check whether it was succesful or not in creating
            //a user
            if (data.state) {
              location.href = "http://localhost:3000/login";
            } else {
              cleanForm();
              valiadateData(data.message);
            }
          })
          //If there is a network error it is caught here
          .catch((err) => {
            console.log(err.message);
          });
      }
    </script>
```

Login Page:

HTML:

```
<div class="buttons">
          <!-- button to login -->
          <button id="login" onclick="login()">Login</button>
          <!-- button to go to the sign up page -->
          <button id="to-signup">
            <a href="/register/basic">Make an account</a>
          </button>
        </div>
```

JavaScript:

```
function login() {
        //Sends a POST request to the server in order to login a user
        fetch("http://localhost:3000/login", {
          method: "POST",
          body: JSON.stringify({
            email: getElement("email"),
            password: getElement("password"),
          }),
          headers: {
            "Content-type": "application/json; charset=UTF-8",
          },
        })
          .then((res) => {
            return res.json();
          })
          .then((data) => {
            if (data.state) {
              location.href = "http://localhost:3000/home";
            } else {
              cleanForm();
              valiadateData(data.message);
            }
          })
          .catch((err) => {
            console.log(err.message);
          });
      }
```

All the rest of the JavaScript is the same as on the register and admin register pages.

Now to style the pages:

Register Page:

```
<style>
      /* Imports a font family form google fonts*/
      @import url("https://fonts.googleapis.com/css2?family=Josefin+Sans&display=swap");
      * {
        margin: 0;
        padding: 0;
      }
      body {
        /* sets the font family on the whole page */
        font-family: "Josefin Sans", serif;
      }
      .sign-up-form {
        /* styles the form */
        margin: 0 auto;
        margin-top: 10.1vh;
        max-width: 400px;
      }
      .sign-up-form input {
        /* styles the inputs int the form*/
        display: block;
        width: 100%;
        margin: 5px 0;
        padding: 5px;
      }

      #show {
        /* styles the show check box */
        display: inline;
        width: 14px;
        margin: 0;
        padding: 0;
      }

      .sign-up-form h2 {
        /* styles the form title*/
        margin-bottom: 30px;
      }

      button#sign-in {
        /* styles the 'Sign up' button*/
        border: 0;
        background-color: orange;
        padding: 8px;
      }

      button#to-login {
        /* styles the 'Already have account?' button*/
        background-color: white;
        border: 0;
        color: orange;
      }

      button:hover {
        /* When hovering over any buttons the opacity
        decreases*/
        opacity: 0.75;
      }
```

CSS continues:

```
      .landing-body {
        /* styles the div that contains everything*/
        display: flex;
        justify-content: center;
        height: 100vh;
      }

      .title {
        /* styles the title*/
        margin: 0 auto;
        text-align: center;
        background-color: rgb(50 216 223);
        color: white;
        width: 100%;
        height: 100%;
      }

      .description {
        margin-top: 46vh;
      }

      .sign-in {
        width: 100%;
      }

      .buttons {
        /* styles the buttons*/
        display: flex;
        justify-content: space-evenly;
        align-items: center;
        width: 100%;
      }

      .error {
        /* styles the error messages*/
        color: red;
      }

      .type {
        margin-bottom: 20px;
      }

      .type ul {
        display: flex;
        align-items: center;
      }
      /* All the below styles the 'Student'
      and 'Admin' buttons*/
      li {
        list-style-type: none;
        width: 50%;
        padding: 5px;
        text-align: center;
        cursor: pointer;
        border: 2px solid white;
        border-right-color: orange;
        border-bottom-color: orange;
      }
      li.admin {
        border-right-color: white;
      }
      li a {
        color: black;
        text-decoration: none;
      }

      li:hover {
        opacity: 0.75;
      }
      a {
        color: orange;
        text-decoration: none;
      }
    </style>
```

Admin Register page:

It has exactly the same CSS as the register page.

Login:

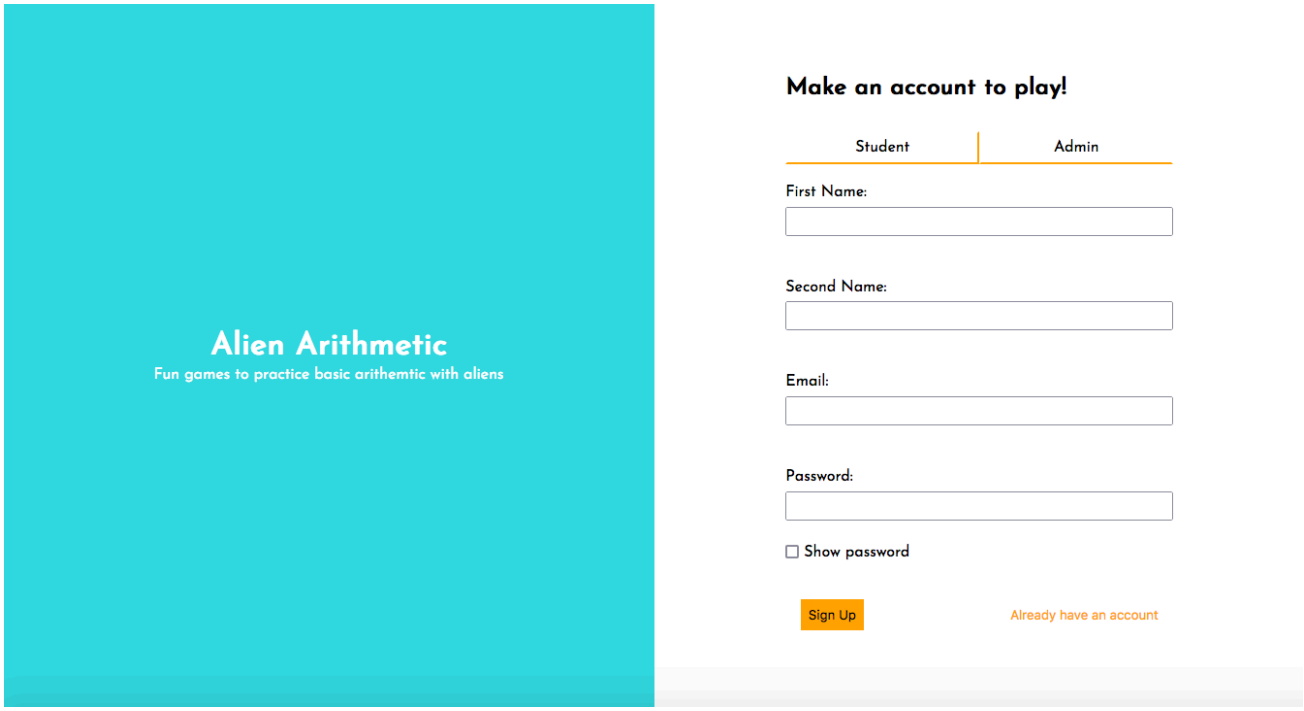
```
<style>
      /* Imports a font family from google fonts */
      @import url("https://fonts.googleapis.com/css2?family=Josefin+Sans&display=swap");
      * {
        margin: 0;
        padding: 0;
      }
      body {
        font-family: "Josefin Sans", serif;
      }

      .login-form {
        /* This styles the form tags */
        margin: 0 auto;
        margin-top: 18%;
        max-width: 400px;
      }
      .login-form #password,
      #email {
        /* This styles the form input fields  */
        display: block;
        width: 100%;
        margin: 5px 0;
        padding: 5px;
      }

      .login-form h2 {
        /* This styles the form title */
        margin-bottom: 30px;
      }

      button#login {
        /* This styles the 'Login' button */
        border: 0;
        background-color: orange;
        padding: 8px;
        font-weight: bold;
      }

      button#to-signup {
        /* This styles the 'Make an account' button */
        background-color: white;
        border: 0;
      }

      a {
        /* This styles the link tags */
        color: orange;
        text-decoration: none;
      }

      .buttons {
        /* This aligns the buttons to login and sign up */
        display: flex;
        justify-content: space-evenly;
        align-items: center;
        width: 100%;
      }
```

CSS continues:

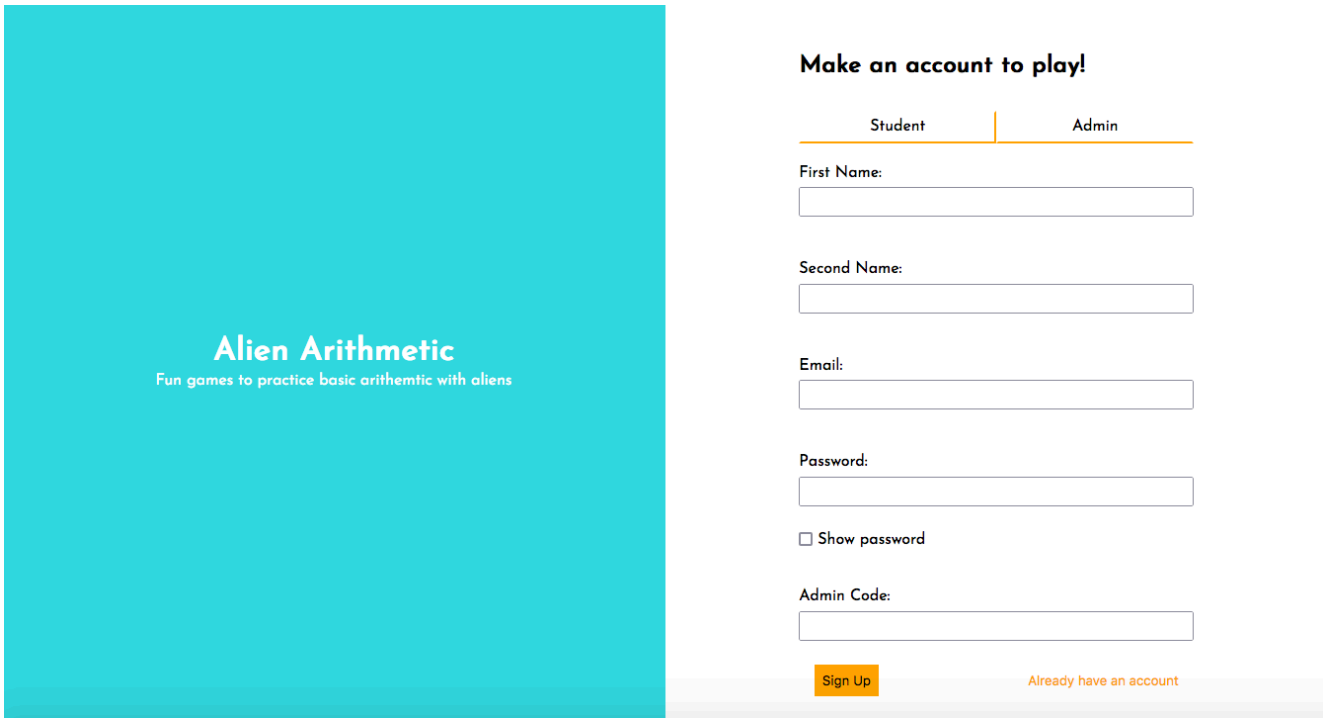
```
      button:hover {
        /* When hovering over a button the
        opacity decreases*/
        opacity: 0.75;
      }

      .landing-body {
        /* This div aligns the children divs to
        be centerd on the y axis */
        display: flex;
        justify-content: center;
        height: 100%;
      }

      .title {
        /* This styles the logo */
        margin: 0 auto;
        text-align: center;
        background-color: rgb(50 216 223);
        color: white;
        width: 100%;
        height: 100%;
      }

      .description {
        margin-top: 46vh;
      }

      .login {
        width: 100%;
      }

      .error {
        /* This styles error messages */
        color: red;
      }
    </style>
```

This is what they look like now:

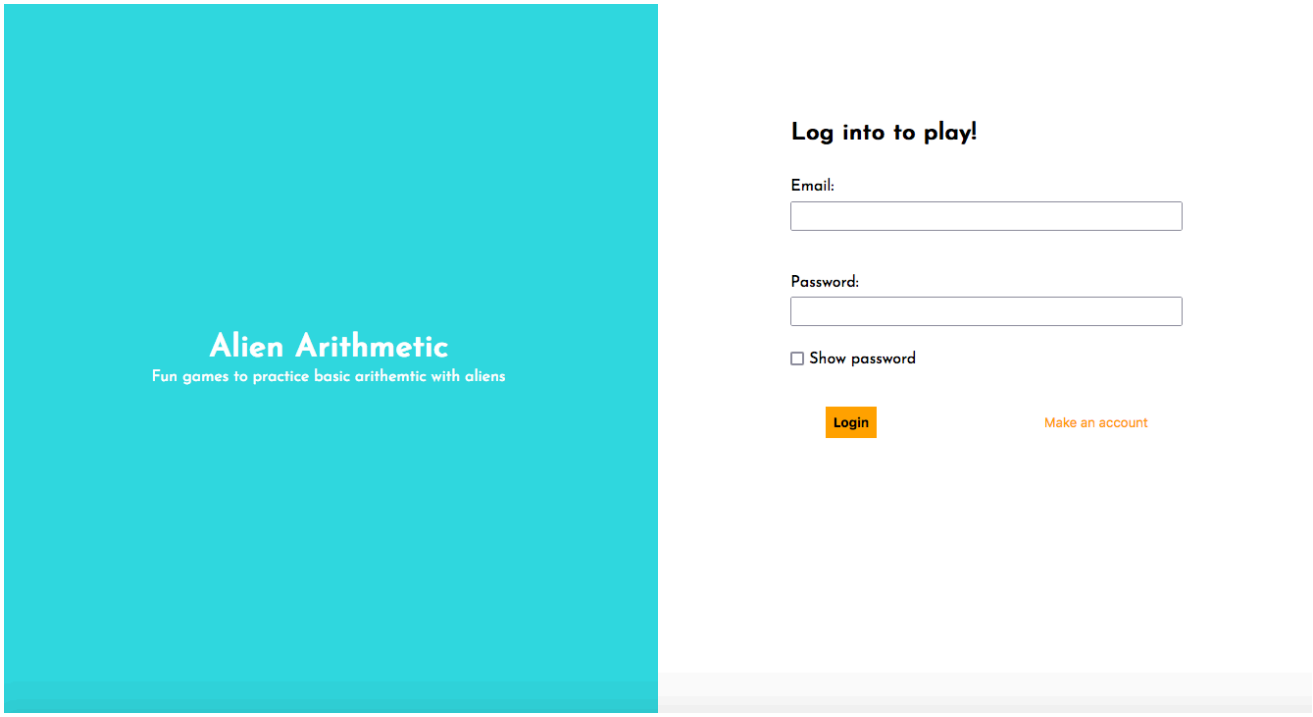
Basic register page:



Admin register page:



Login page:


Alien Arithmetic
Fun games to practice basic arithemtic with aliens
Log into to play!
Email:
Password:
Show password
Login
Make an account

Testing the Register Page

Test ID	Test Input	Expected Outcome
R1	First Name: John, Last Name: Smith Email: doctor@who.com Password: Tardis12	Positive. User added to database. User redirected to login page.
R2	Any input that warrants an error message.	Negative. User not added to database. Error message sent.
R3	First Name: Jo, Last Name: Smith Email: doctor@who.com Password: Tardis12	Error message displayed under first name field: "First name must be greater than 2 chars"
R4	First Name: (name with 21 chars), Last Name: Smith Email: doctor@who.com Password: Tardis12	Error message displayed under first name field: "First name must be less than 21 chars"
R5	The same for R3 and R4 but for the 'Last Name' field.	The same for R2 and R3 but for the 'Last Name' field.
R6	Email field with the wrong format.	Error message displayed under email field: "Email is of the wrong format"
R7	Password with less than 6 characters	Error message displayed under password field: "Password must be greater than 6 chars"
R8	All fields left empty.	Error message displayed under first name field: "First name is required"
R9	Filling in the field one by one and each time clicking the 'Sign-up' button.	Error message displayed under each field saying that this field is required.
R10	Click the 'Already have account?', 'Student' and 'Admin' buttons.	'Already have account?' button redirects to Login page, 'Student' redirects to basic register page, and 'Admin' button redirects to admin register page.
R11	Click the 'show password' check box.	The password not visible when not checked and visible when checked.

Test ID	Test Input	Expected Outcome
R12	In the admin register, enter the details with the correct admin code and click sign-up.	Positive. User added to database with the role of an admin. User redirected to login page.
R13	In the admin register, enter the details with the wrong admin code and click sign-up.	Error message displayed under the admin code field saying: “This code is invalid”.

Test R1

Input:

First Name:

John

Second Name:

Smith

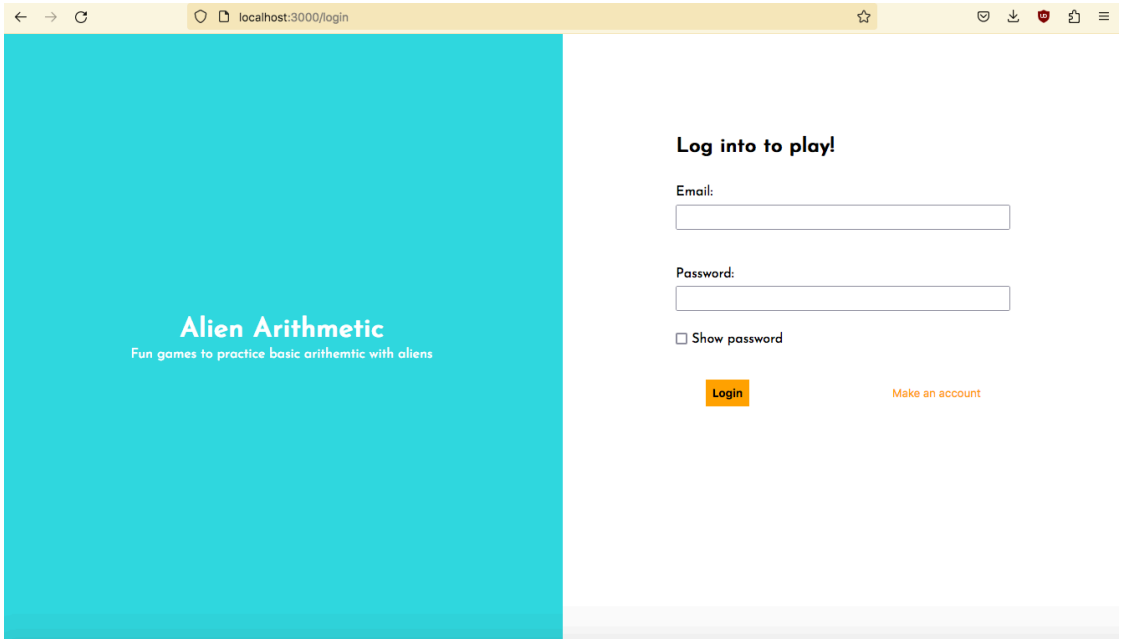
Email:

doctor@who.com

Password:

••••••••

Outcome:



In **users** and **scores** table:

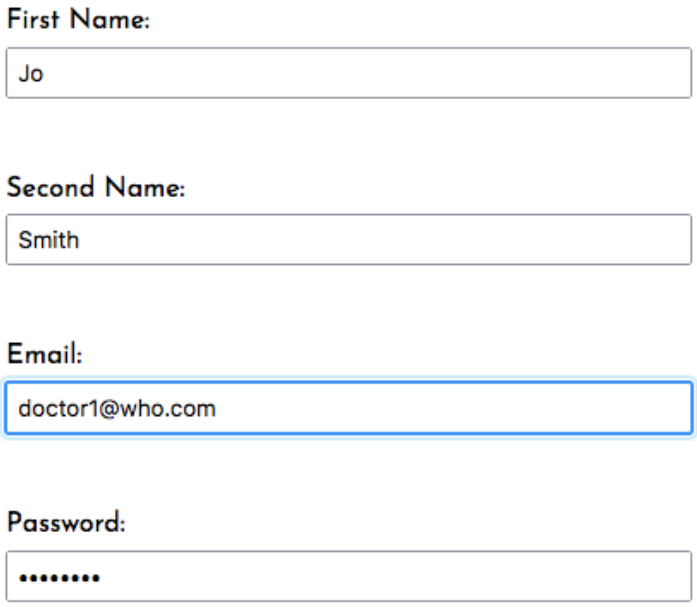

```
_id: ObjectId('643ff65d4
firstname: "John"
lastname: "Smith"
email: "doctor@who.com"
password: "Tardis12"
blog: ""
role: "basic"
__v: 0
```

```
_id: ObjectId('643ff65d
email: "doctor@who.com"
▸ typeBoth: Array
▸ typeMD: Array
▸ typeAS: Array
role: "basic"
__v: 0
```

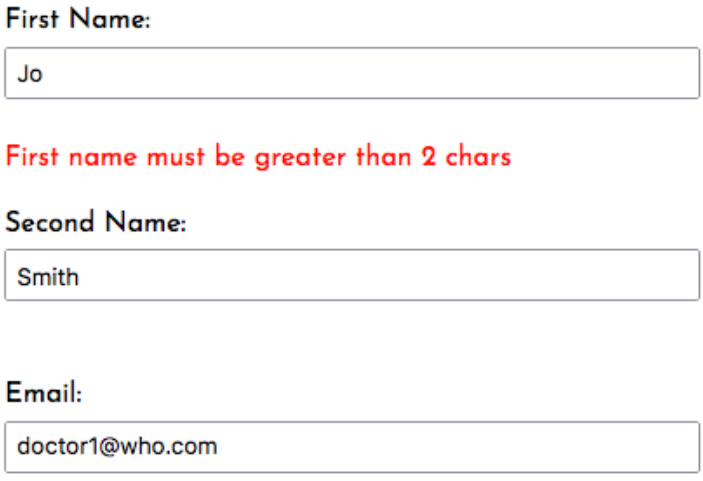

TEST SUCCESS!

Test R3

Input:



Outcome:



TEST SUCCESS!

Test R4

Input:

First Name:

Waaaaaaaaaaaaaaaaade

Second Name:

Smith

Email:

doctor1@who.com

Password:

••••••••

Outcome:

First Name:

Waaaaaaaaaaaaaaaaade

First name must be less than 21 characters

Second Name:

Smith

Email:

doctor1@who.com

Password:

••••••••

TEST SUCCESS!

Test R5

Input:

First Name:

John

Second Name:

Sm

Email:

doctor1@who.com

Password:

••••••••

Outcome:

First Name:

John

Second Name:

Sm

Last name must be greater than 2 chars

Email:

doctor1@who.com

Password:

••••••••

TEST SUCCESS!

Input:

First Name:

John

Second Name:

Smooooooooooooooooth

Email:

doctor1@who.com

Password:

••••••••

☐ Show password

Outcome:

First Name:

John

Second Name:

Smooooooooooooooooth

Last name must be less than 21 characters

Email:

doctor1@who.com

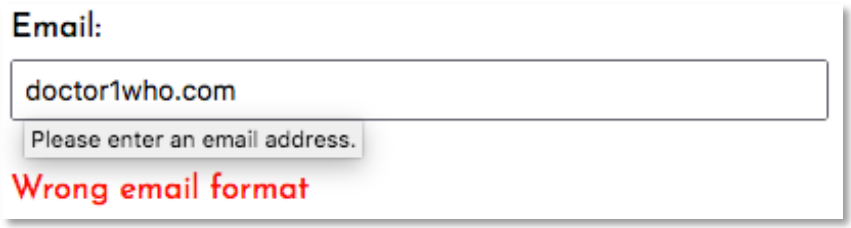
Password:

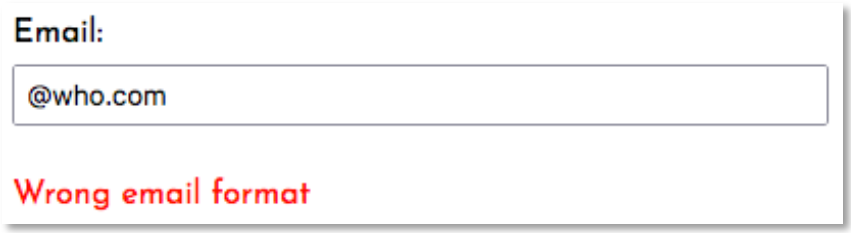
••••••••

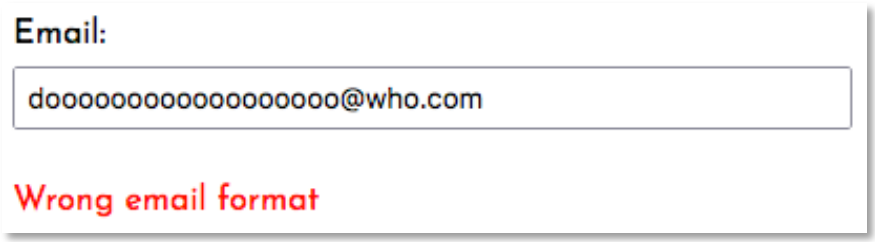
TEST SUCCESS!

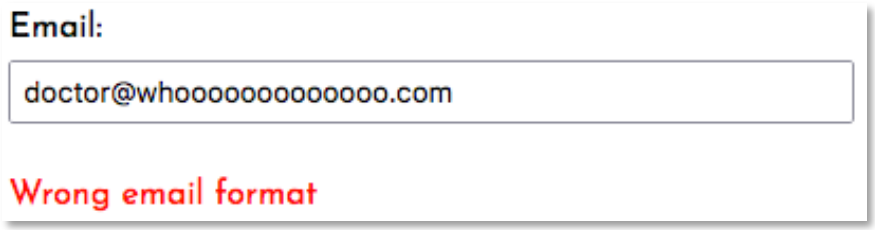
Test R6

Inputs and Outcomes:





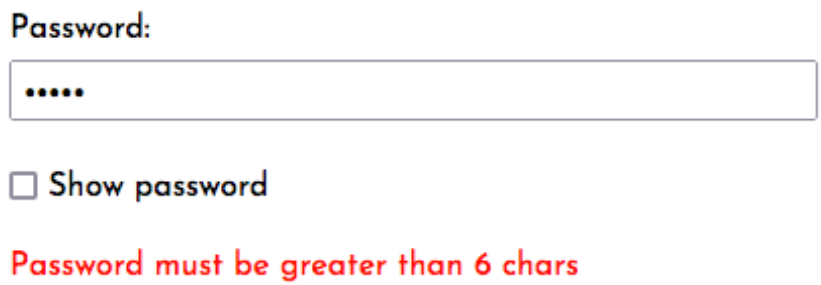




TEST SUCCESS!

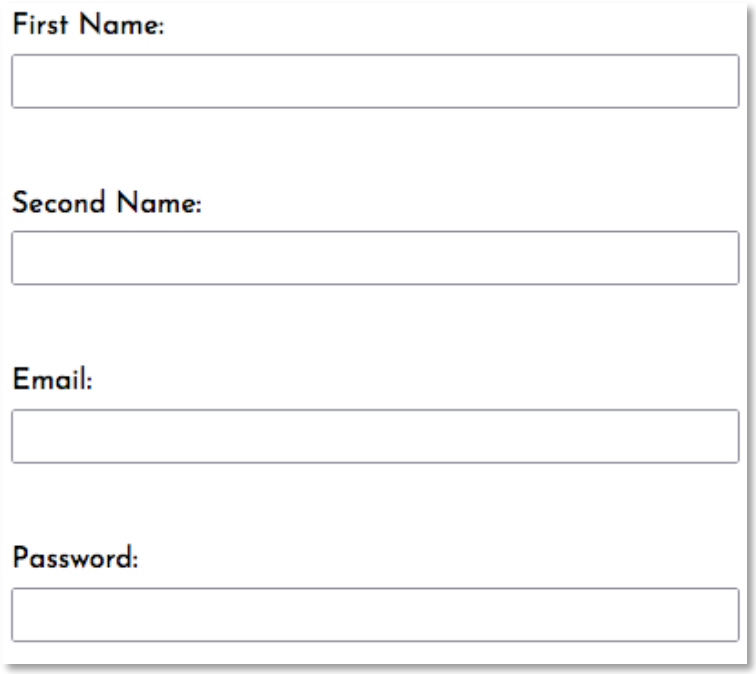
Test R7

Input and Outcome:

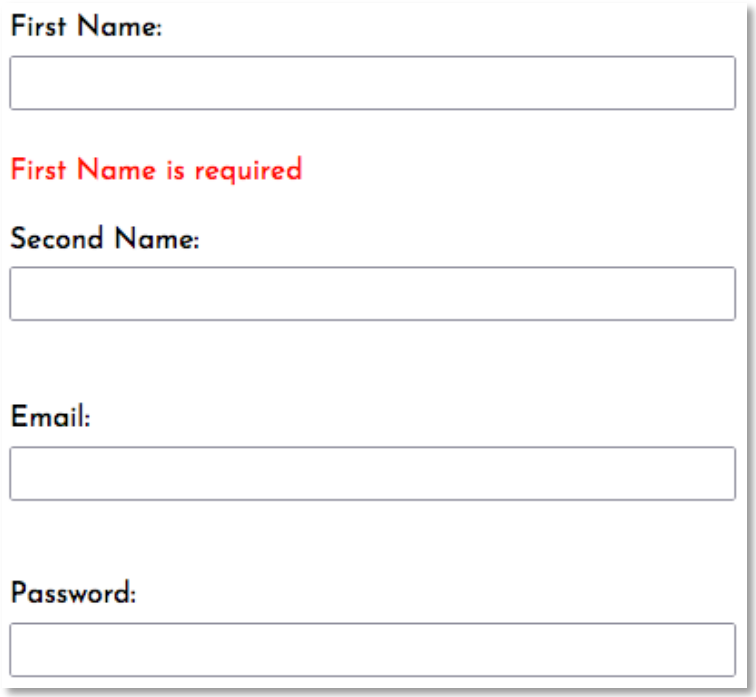


Test R8

Input:



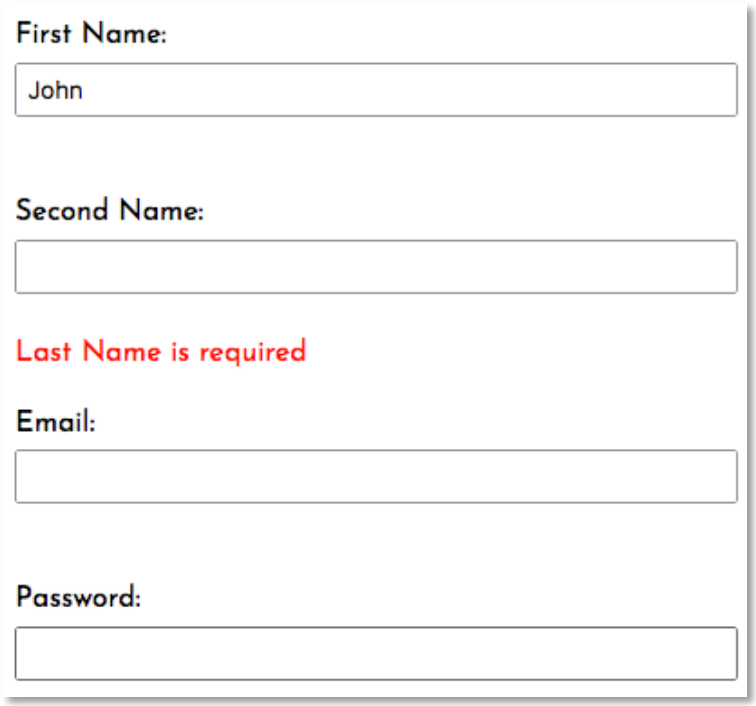
Outcome:

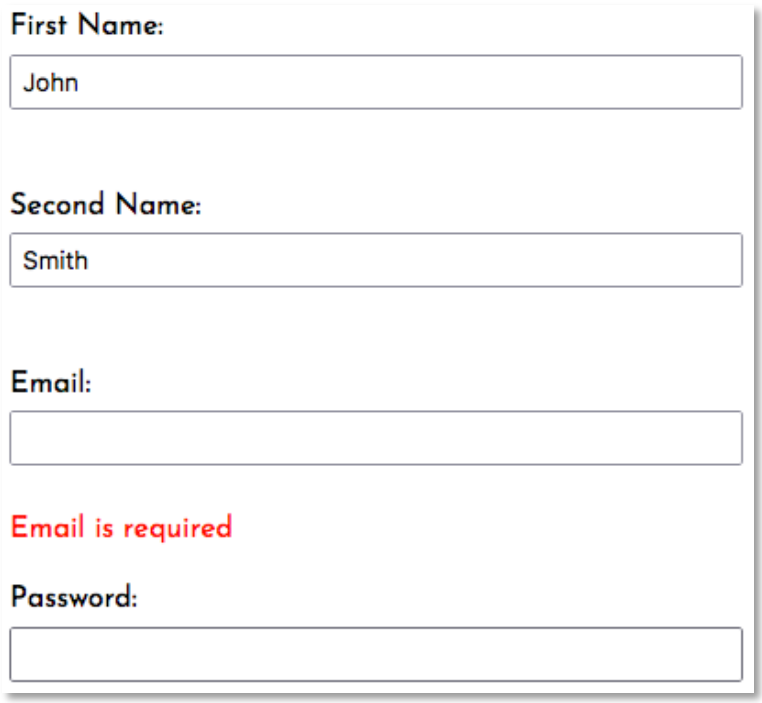


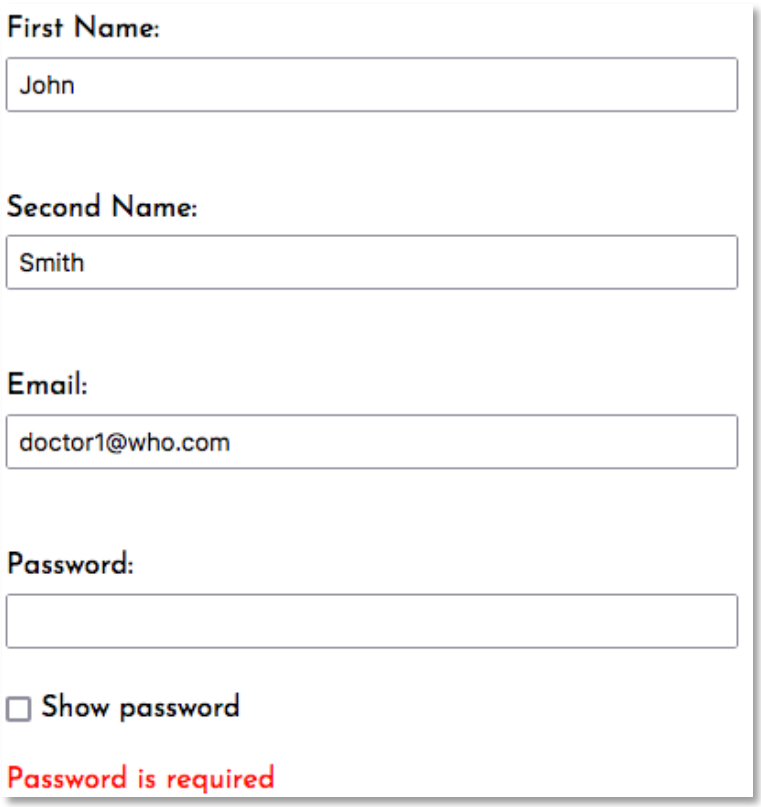
TEST SUCCESS!

Test R9

Inputs and Outcomes:







TEST SUCCESS!

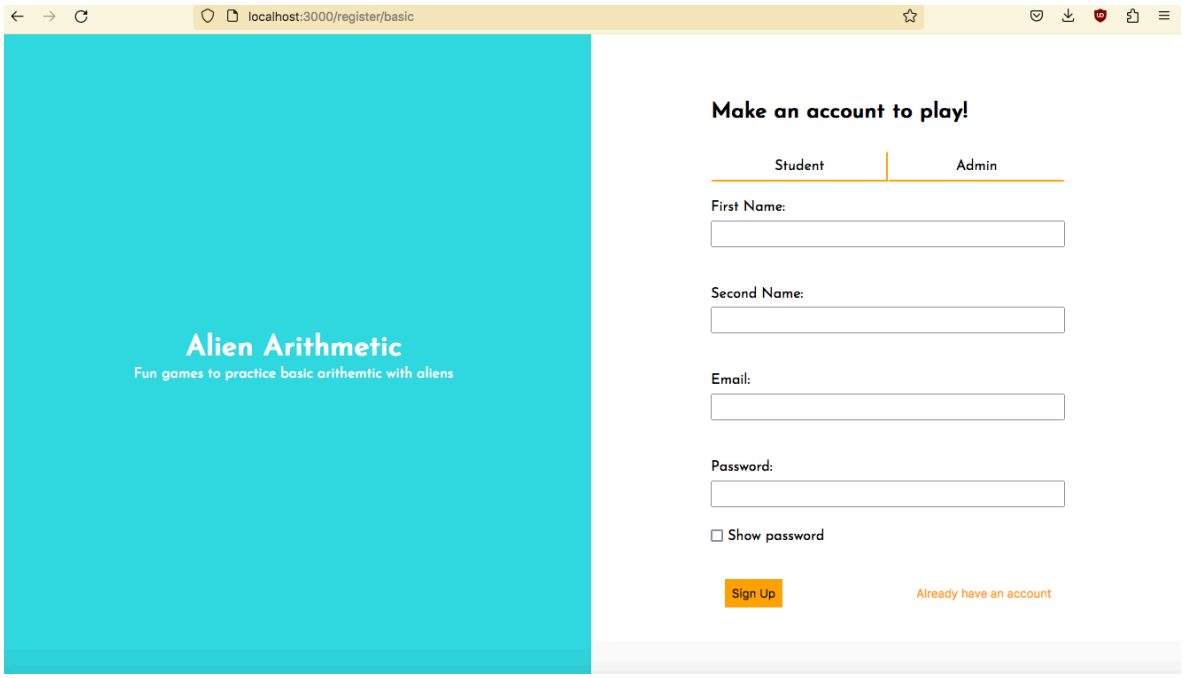
Test R10

Inputs:

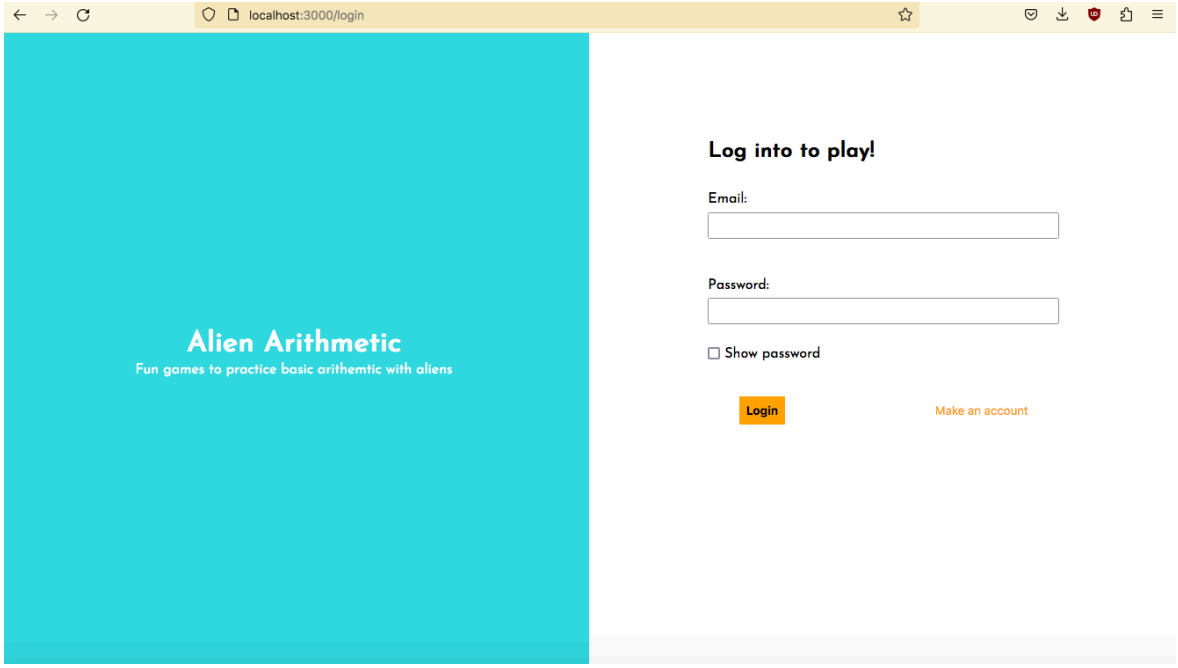
1. Clicking the ‘Admin’ button
2. Clicking the ‘Already have account?’
3. Clicking the ‘Student’ button

Outcomes:

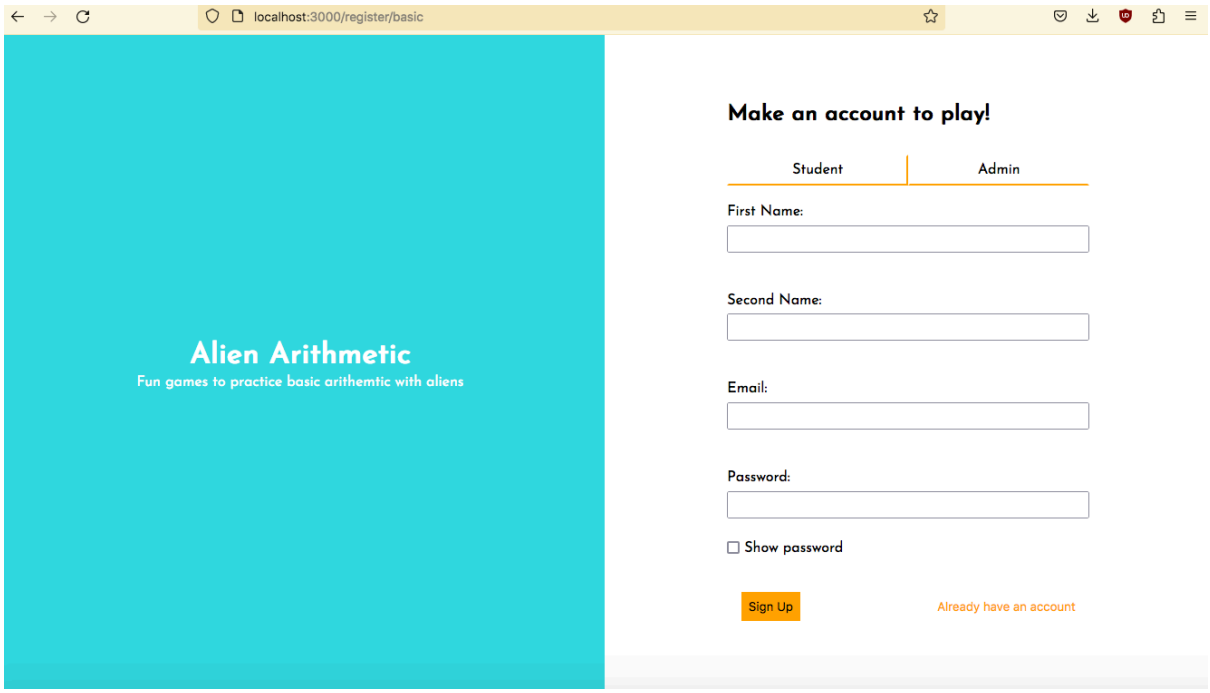
1.



2.



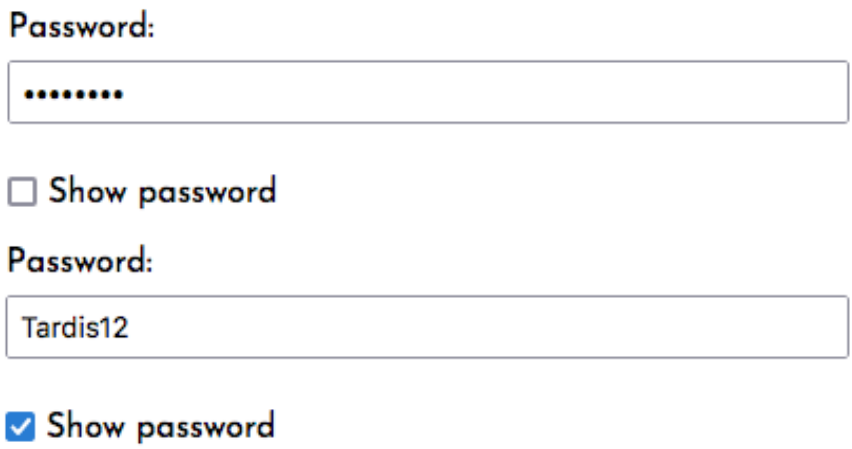
3.



TEST SUCCESS!

Test R11

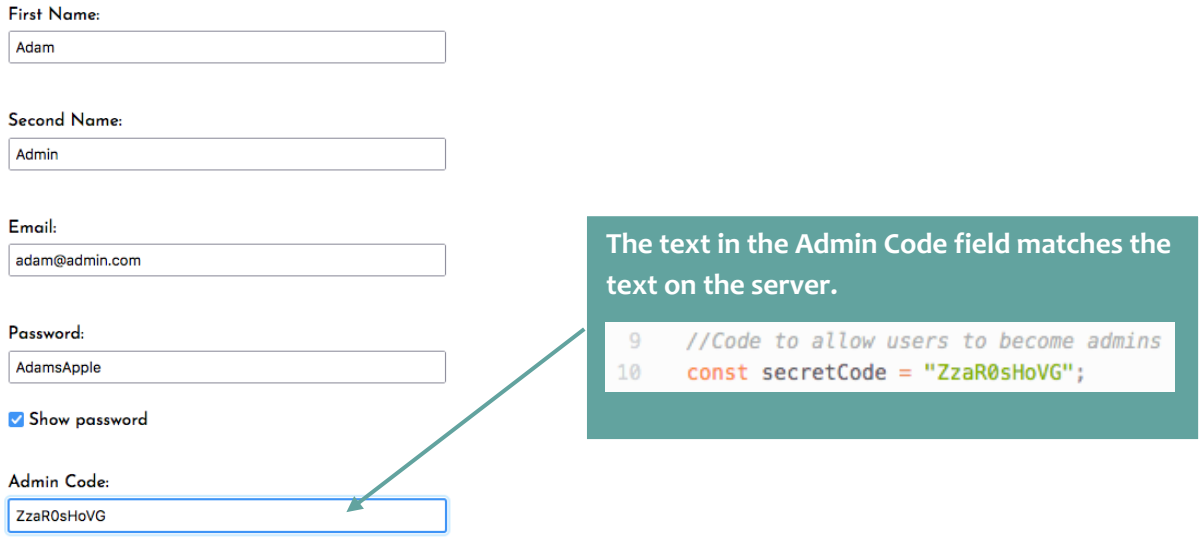
Inputs and Outcomes:



TEST SUCCESS!

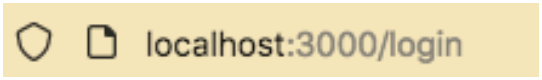
Test R12

Input:



Outcome:

```
_id: ObjectId('64553e25c95
firstname: "Adam"
lastname: "Admin"
email: "adam@admin.com"
password: "AdamsApple"
blog: ""
role: "admin"
```



TEST SUCCESS!

Test R13

Input and Outcome:

TEST SUCCESS!

Test R2

Throughout the testing, whenever there was an error, an error message would be shown, and the database would stay unchanged. Therefore:

TEST SUCCESS!

Testing the Login

Test ID	Test Input	Expected Outcome
L1	Email: doctor@who.com Password: Tardis12	Positive. User redirected to home page.
L2	Any input that warrants an error message.	Negative. Error message sent.
L3	Email: doctor@whoops.com Password: Tardis12	Error message beneath email field: "This user does not exist".
L4	Email: doctor@who.com Password: Tardis123	Error message beneath password field: "Wrong password".
L5	Format test the email field.	Error message beneath the email field: "Wrong email format".
L6	Email: (nothing) Password: (nothing)	Error message beneath email field: "Both email and password are required".
L7	Email: doctor@who.com Password: (nothing)	Error message beneath email field: "Both email and password are required".

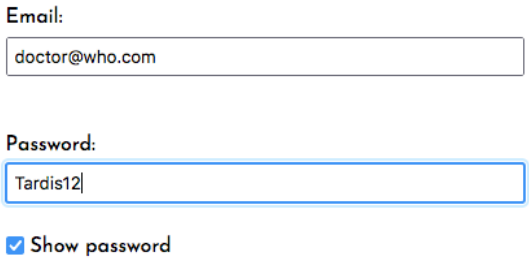
Test ID	Test Input	Expected Outcome
L8	Email: (nothing) Password: Tardis12	Error message beneath email field: "Both email and password are required".
L9	Click the 'show password' check box.	The password not visible when not checked and visible when checked.
L10	Click the "Make an account" button.	User taken to the basic register page.

Test L1

```
email: "doctor@who.com"
password: "Tardis12"
```

The email and password stored in the database.

Input:



The Outcome of all these tests is after the Login button is clicked.

Outcome:

The URL changed to the home URL.

TEST SUCCESS!

Test L3

Input and Outcome:

TEST SUCCESS!

Test L4

Input and Outcome:

TEST SUCCESS!

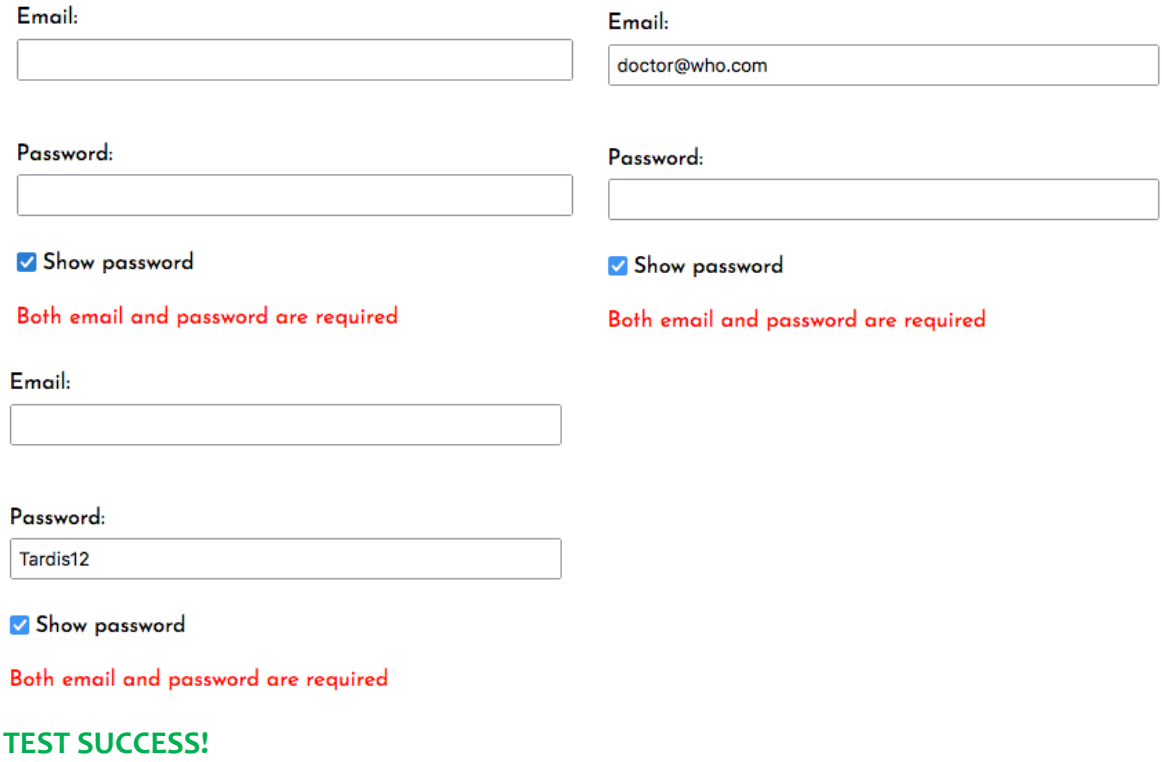
Test L5

This test was done in Test R6.

TEST SUCCESS!

Test L6, 7, 8

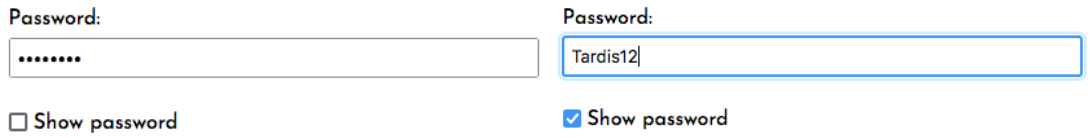
Inputs and Outcomes:



TEST SUCCESS!

Test L9

Inputs and Outcomes:



TEST SUCCESS!

Test L10

Input:

Outcome:

TEST SUCCESS!

Test L2

Throughout the testing, whenever there was an error, an error message would be shown, and the database would stay unchanged. Therefore:

TEST SUCCESS!

Stakeholder Evaluation

My client and the students, all liked the login and register pages. My client especially liked the code that allows people to register as admins. The only addition they would have made to the register and login page, was to have a forgotten password button and feature. This involves sending automatic emails, and it far out of my scope for this stage of development. This feature could maybe be added as a later improvement to the website, post development.

Overall Evaluation: Mostly Positive. No further changes needed.

CREATING THE NAVIGATION BAR

Below is the html for the navigation bar:

```
<nav class="nav-bar">
      <ul class="nav-links">
        <li class="logo"><a href="#">Alien Arithmetic</a></li>
        <li><a href="#">Games</a></li>
        <li><a href="#">Statistics</a></li>
        <li><a href="#">Profile</a></li>
      </ul>
    </nav>
```

Now putting in each of the valid URLS in the href part of the 'a' tags:

```
<nav class="nav-bar">
      <ul class="nav-links">
        <li class="logo"><a href="/home">Alien Arithmetic</a></li>
        <li><a href="/games">Games</a></li>
        <li><a href="/stats">Statistics</a></li>
        <li><a href="/pfp">Profile</a></li>
      </ul>
    </nav>
```

Now when clicking the 'a' tag, the URL will change to the corresponding URL in the href property.

Since I already have endpoints made for each page on the website, I will make a new HTML file called 'home.html' in the 'html_pages' folder and I will put the nav bar at the top of the body:

```html
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta http-equiv="X-UA-Compatible" content="IE=edge" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Alien Arithmetic | Home</title>
  </head>
  <body>
    <nav class="nav-bar">
      <ul class="nav-links">
        <li class="logo"><a href="/home">Alien Arithmetic</a></li>
        <li><a href="/games">Games</a></li>
        <li><a href="/stats">Statistics</a></li>
        <li><a href="/pfp">Profile</a></li>
      </ul>
    </nav>
  </body>
</html>
```

Now, accessing the 'home.html' file by changing the endpoint in the 'home.js' file to the below:

```
//Sets up an endpoint with the GET method on /home
route.get("/", (req, res, next) => {
  res.sendFile(path.join(__dirname, "/html_pages/home.html"));
});
```

Now when starting the server and typing: 'localhost:3000/home' into URL you get this:

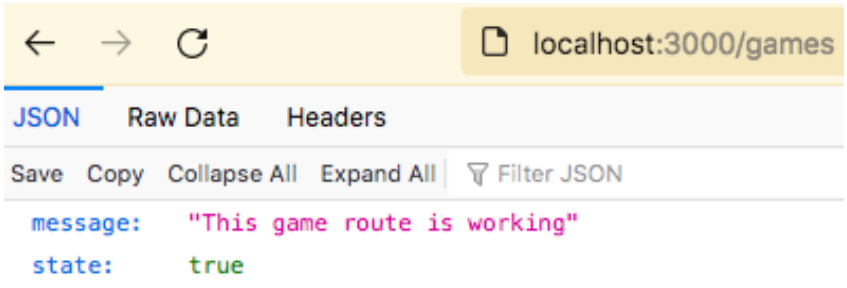
- Alien Arithmetic
- Games
- Statistics
- Profile

Clicking the 'Alien Arithmetic' link which in the future will be the logo:

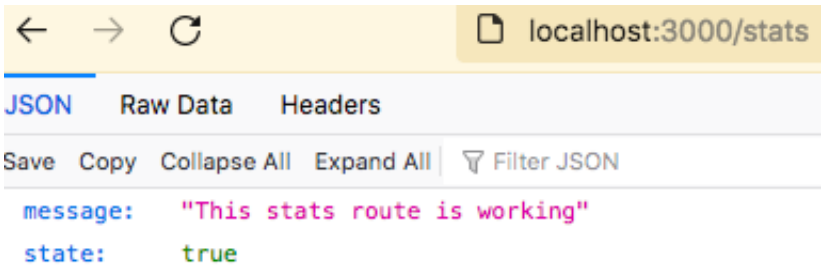
- Alien Arithmetic
- Games
- Statistics
- Profile

You get sent to the home page.

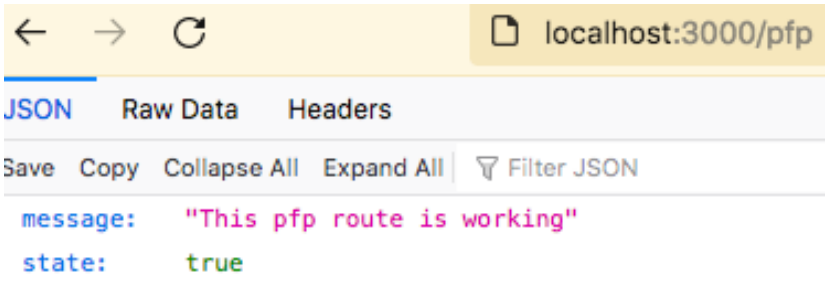
Clicking the ‘Games’ link:



Clicking the ‘Statistics’ link:

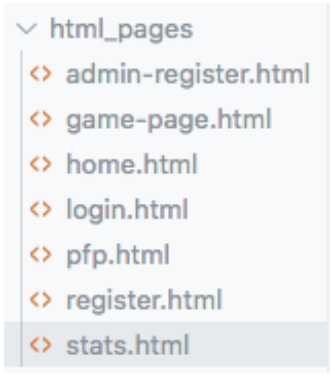


Clicking the ‘Profile’ link:



TEST SUCCESS! All the links are working correctly!

Here I make a html file for each part of the website that has a nav bar:

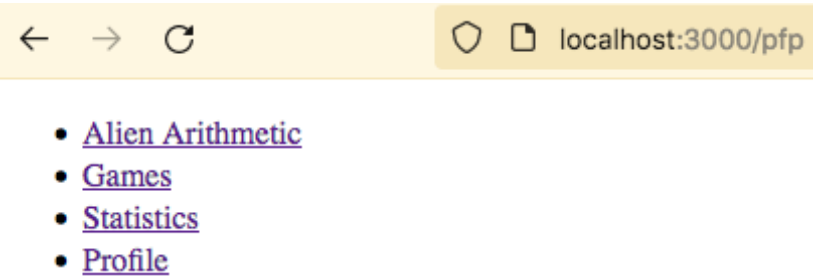


In each I add the same html as in the home page, changing the titles.

After I change the GET method endpoints for each 'website-page.js' file to the general form below:

```
14  //Sets up an endpoint with the GET method on /(website page)
15  route.get("/", (req, res, next) => {
16    res.sendFile(path.join(__dirname, "/html_pages/(website page).html"));
17  });
```

Now clicking through the links you're met will html pages. For example, clicking the 'profile' link in the home page shows this:



Now I style the nav bar. Since the CSS on each page is exactly the same, I will make a 'styles.css' and link to it. Also, in the CSS I will define the general styles for each page:

Here is the CSS:

```css
/*Imports a font family from google fonts */
@import url("https://fonts.googleapis.com/css2?family=Josefin+Sans&display=swap");
* {
  margin: 0;
  padding: 0;
}
body {
  /* This applies sets the default value for the
  font, cursor and color on all the web pages */
  font-family: "Josefin Sans", serif;
  cursor: default;
  color: rgb(66, 66, 66);
}

a,ul,p,h1,h2,h3,h4,h5,h6 {
  /* This defaults all the tags above
  so they are easier to work with*/
  margin: 0;
  padding: 0;
  text-decoration: none;
}

nav.nav-bar {
  /* This styles th nav bar body. The children of
  this container are centered on the y axis */
  padding: 20px 0px 20px 0px;
  display: flex;
  justify-content: space-between;
  background-color: rgb(50 216 223);
  box-shadow: 0 -7px 20px 0 rgba(0, 0, 0, 0.75);
}

ul.nav-links {
  display: flex;
  justify-content: space-between;
  align-items: center;
}

li {
  list-style-type: none;
  margin-left: 20px;
  cursor: pointer;
}
li:hover {
  /* When hovering over the links, the opacity
  decreases */
  opacity: 0.75;
}

.nav-links a {
  color: white;
}

li.logo {
  font-size: 2em;
}
```

My ‘styles.css’ file is in the ‘html_pages’ folder.

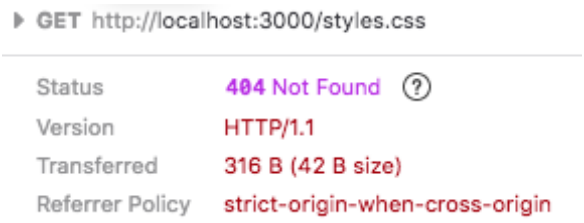
Now I add the following to each header in the html files:

```
6   <link rel="stylesheet" href="./styles.css" />
```

Testing whether the HTML pages are now styled by calling the /home endpoint:

- Alien Arithmetic
- Games
- Statistics
- Profile

TEST FAILED! The HTML pages are not styled! Looking in the console on Firefox, the following error message is shown:



Here is the problem:

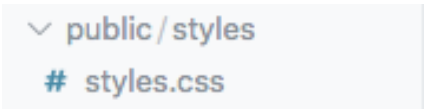
From looking at the error message we can see that the endpoint /styles.css was called as an endpoint and that the status code is 404, meaning: Page not found. It looks like the link tag in the header in the HTML file is calling the endpoint /styles. This is because I am accessing the HTML files through the server so therefore, I it is trying to access the 'styles.css' also through the server. One way of getting around this is by putting the CSS within style tags in the html code, but this will get way too repetitive.

After looking online, this is how I can solve this problem:

```
21  //Sets up static files
22  app.use(express.static(path.join(__dirname, "/public")));
```

Static file are files that are stored on the server. These are most commonly used when using CSS files.

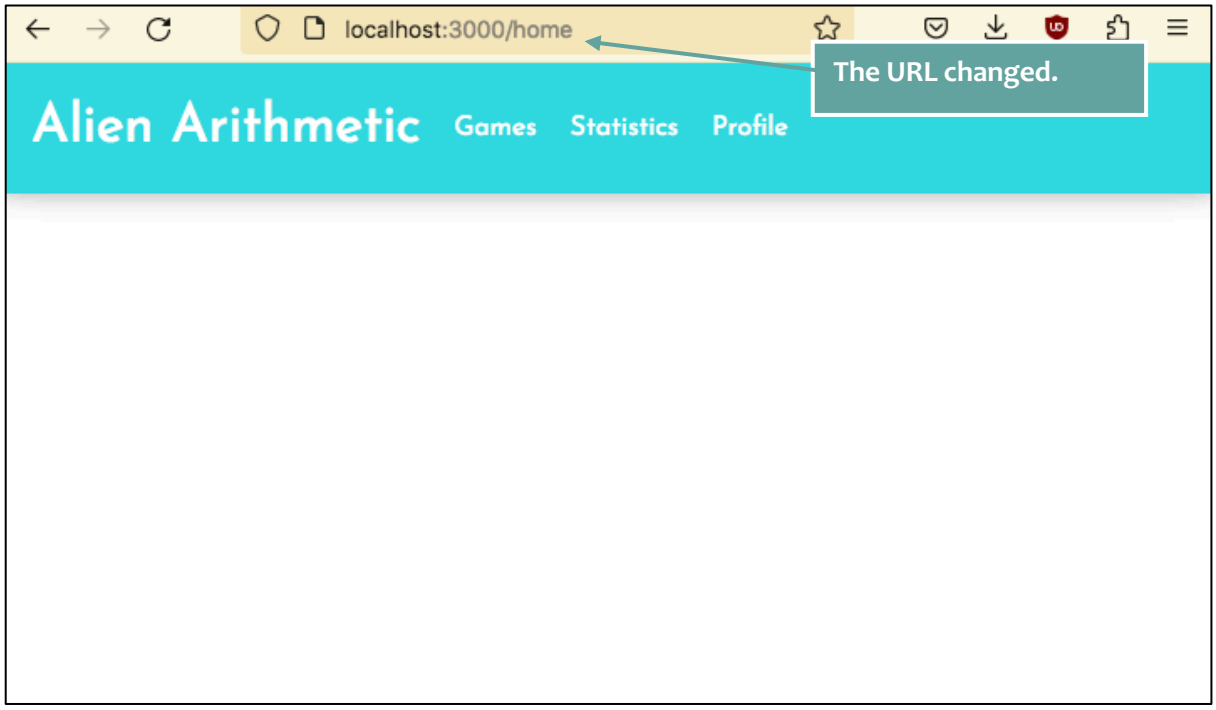
I put this code in the 'server.js' file and will make a new folder called 'public'. Every file in this folder is counted as a static file. In the 'public' folder:



Now each link will be changed to this:

```
6   <link rel="stylesheet" href="http://localhost:3000/styles/styles.css" />
```

Testing whether the HTML pages are now styled by calling the /home endpoint:



Calling the other website endpoints, they all have the same result. Therefore:

TEST SUCCESS! All the HTML pages have a functioning navigation bar!

Stakeholder Evaluation

My client and the students all liked the nav bar. They said it was simple and easy to navigate. The said it might be nice to have the logout button on the nav bar as well, but it would only be a minor inconvenience for not to be there. Therefore, no change in needed.

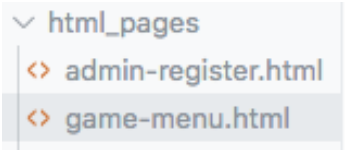
Overall Evaluation: Positive. No further changes needed.

MAKING THE GAME

To access the main body of the game, there must be a game menu where the player can choose the type of game mode.

Here I make the game menu page and make it functional:

I make the 'game-menu.html' file in the 'html_pages' folder:



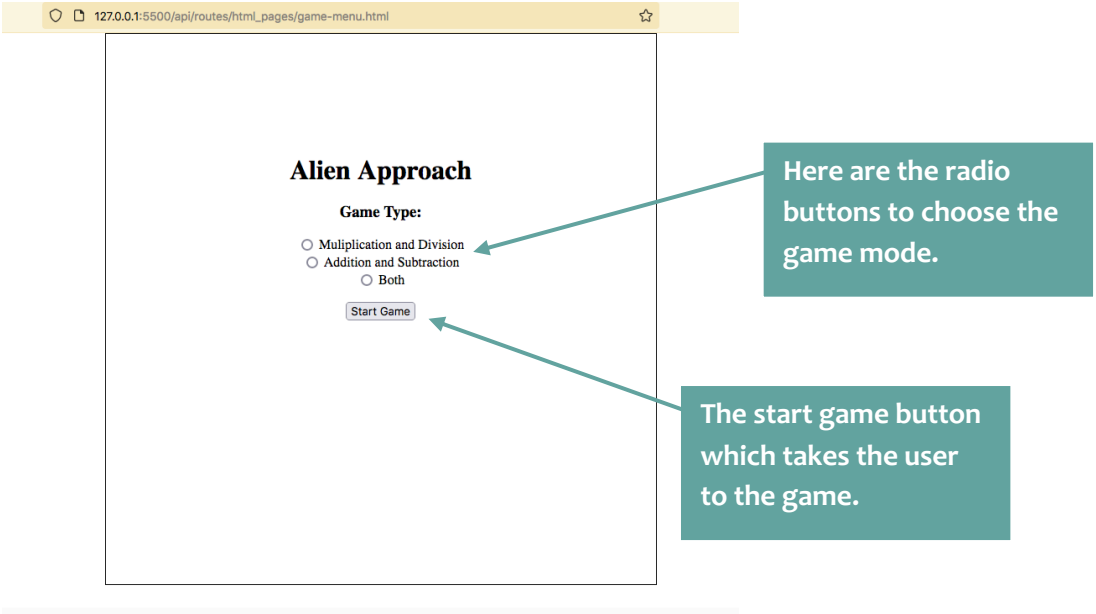
In this file I write the following HTML:

```
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta http-equiv="X-UA-Compatible" content="IE=edge" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Alien Arithmetic | Alien Approach</title>
  </head>
  <body>
    <div class="game-box" id="game-box">
      <div id="menu">
        <!-- The game title -->
        <h1 id="game-title">Alien Approach</h1>
        <br />
        <h3 id="type">Game Type:</h3>
        <form>
          <!-- radio buttons to choose the different game mode -->
          <input type="radio" id="md" name="type" value="md" />
          <label for="md">Muliplication and Division</label><br />
          <input type="radio" id="as" name="type" value="as" />
          <label for="as">Addition and Subtraction</label><br />
          <input type="radio" id="both" name="type" value="both" />
          <label for="both">Both</label>
        </form>
        <!-- button to start the game -->
        <button id="play">Start Game</button>
      </div>
    </div>
  </body>
</html>
```

At the top of the body, I add this CSS:

```
<style>
  body {
    /* Defaults the body tags*/
    margin: 0;
    padding: 0;
  }
  /* This box will harbor the game and game end stuff */
  .game-box {
    border: 1px solid black;
    width: 90vh;
    height: 90vh;
    margin-left: 50vw;
    transform: translateX(-50%);
  }

  a,
  p,
  h1,
  h2,
  h3 {
    /* Defaults these tags*/
    padding: 0;
    margin: 0;
  }
  #menu {
    margin: 0 auto;
    text-align: center;
  }
  #game-title {
    margin-top: 20vh;
  }
  #type {
    margin-bottom: 2.5vh;
  }
</style>
```

Now viewing the page using Live Server (an extension on VS code):



At the moment, this is not functional. Here is the JavaScript in the script tags to make it so:

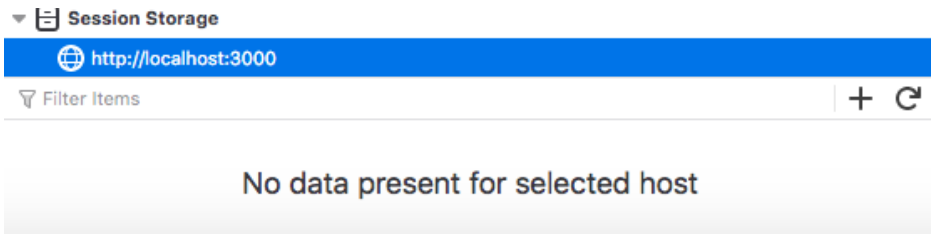
```
<script>
      try {
        //This checks if the type is already selected from
        //a previous game. If yes, then it checked the corresponding
        //radio button.
        let type = sessionStorage.getItem("type");
        const input = document.getElementById(type);
        input.checked = true;
      } catch {
        //If the type is not selected, the Both radio button
        //is checked by default
        const input = document.getElementById("both");
        input.checked = true;
      }

      const startButton = document.querySelector("button");
      startButton.addEventListener("click", () => {
        //When the start button is clicked, store the type of radio button
        //selected and go to the game.
        const inputs = document.querySelectorAll("input");
        inputs.forEach((e) => {
          if (e.checked === true) {
            type = e.value;
            sessionStorage.setItem("type", type);
            location.href = "http://localhost:3000/games/game-one";
          }
        });
      });
    </script>
```

Making a temporary endpoint at /games/game-one for testing purposes in the games.js file:

```
//Sets up an endpoint with the GET method on /games/game-one
route.get("/game-one", (req, res, next) => {
  jsonResponce(res, "Game one page", 200);
});
```

When clicking the start button, the type of game mode should be stored in the session storage.

Starting the server, clicking the start button and using the Firefox dev tools to see the session storage:



TEST FAILED! There is no data in the session storage.

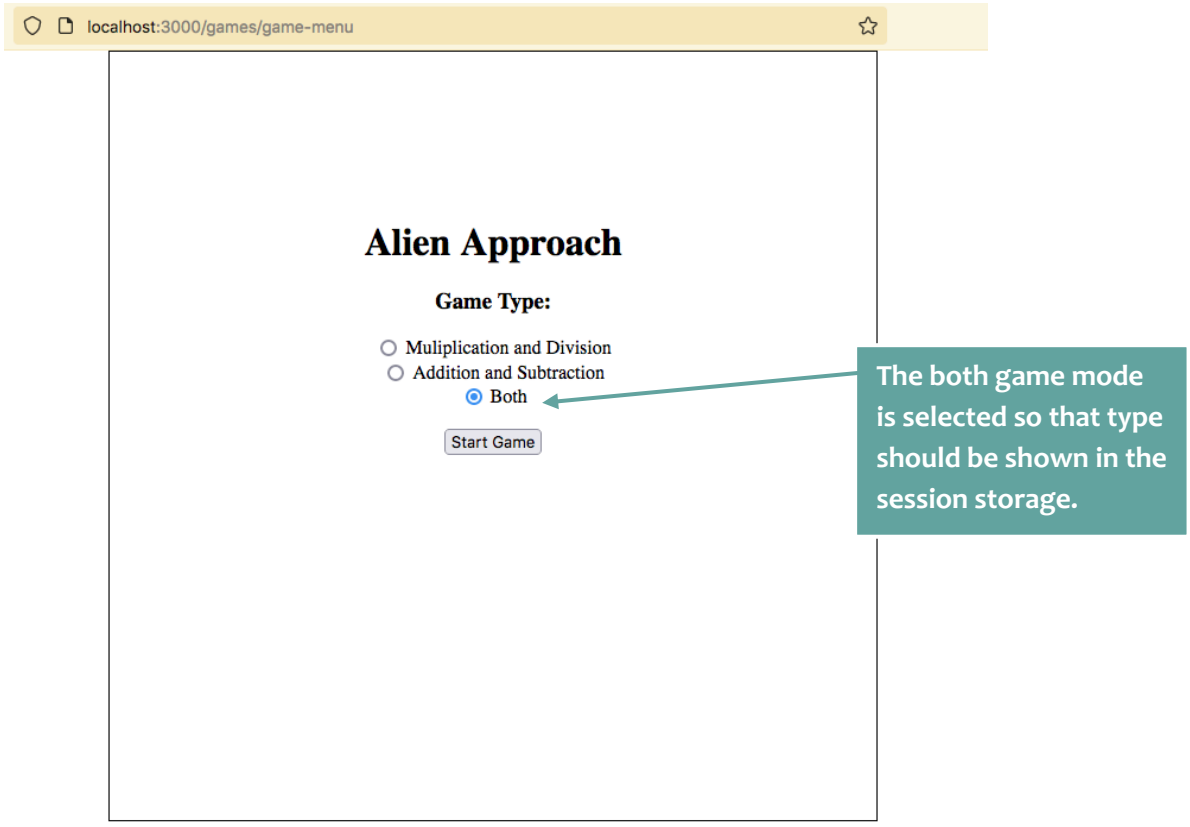
Here is the problem: Because I was using live server to run the game menu page, it acted under a different host more specifically: 'http://127.0.0.1:5500/'. Therefore, the session storage in that domain rather than on 'localhost:3000'.

To fix this I created an endpoint to run the game menu page, so it has a domain of 'localhost:3000':

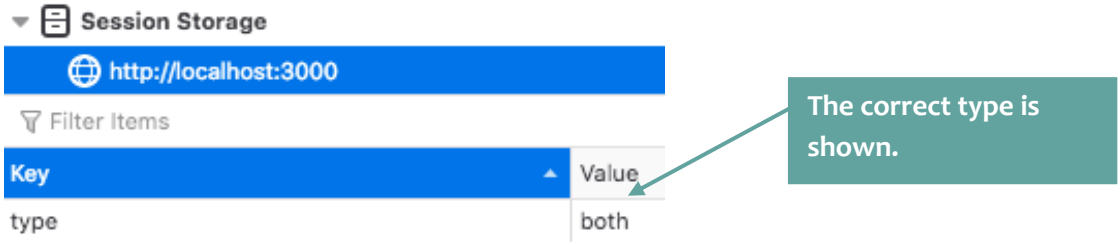
In 'game.js' file:

```
24  //Sets up an endpoint with the GET method on /games/game-menu
25  route.get("/game-menu", (req, res, next) => {
26    res.sendFile(path.join(__dirname, "/html_pages/game-menu.html"));
27  });
```

Calling the endpoint:



Clicking the ‘Start Game’ button and checking the session storage:



TEST SUCCESS! The type I now stored in the session storage and can be accessed elsewhere.

Here I make it so that when the button is clicked, the user is redirected to the actual Game. So, I changed the /games/game-one endpoint to be this:

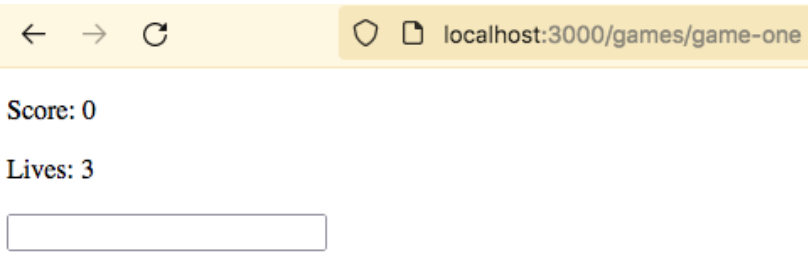
```
//Sets up an endpoint with the GET method on /games/game-one
route.get("/game-one", (req, res, next) => {
  res.sendFile(path.join(__dirname, "/html_pages/game.html"));
});
```

Here I make the ‘game.html’ file in the ‘html_pages’ folder with the following HTML:

```
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta http-equiv="X-UA-Compatible" content="IE=edge" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Alien Arithmetic | Alien Approach</title>
    <!-- connects to the css file on the server -->
    <link rel="stylesheet" href="http://localhost:8000/styles/game.css" />
  </head>
  <body>
    <!-- this div contains the game -->
    <div class="game-box" id="game-box">

      <div class="attributes">
        <!-- The score counter for the player -->
        <p>Score: <span id="score">0</span></p>
        <!-- The lives counter for the player -->
        <p>Lives: <span id="lives">3</span></p>
      </div>

      <input type="text" id="input" />
      <div id="main-char"></div>
    </div>
    <!-- These script tags connect to the js files that are hosted on
    the server -->
    <script
      src="http://localhost:8000/javascript/game.js"
      type="module"
    ></script>
    <script
      src="http://localhost:8000/javascript/enemies.js"
      type="module"
    ></script>
  </body>
</html>
```

I made another CSS file in the static folder 'public' called 'game.css'. I also made two JS files in a new folder called 'javascript' in the 'public' folder, called 'enemies.js' and 'game.js':

Now calling the endpoint /games/game-one the following is shown:



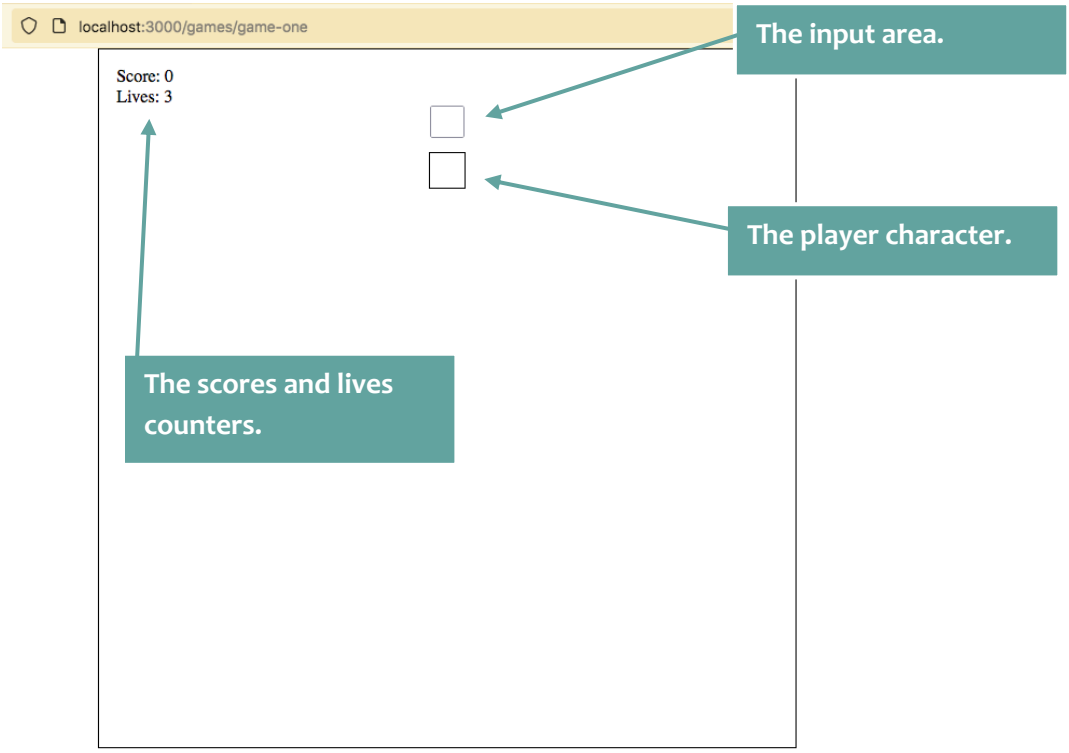
Now I put the following CSS in the game.css file:

```css
body {
  margin: 0;
  padding: 0;
}

.game-box {
  /* This styles the area where the
     game will take place*/
  border: 1px solid black;
  width: 90vh;
  height: 90vh;
  margin-left: 50vw;
  transform: translateX(-50%);
}

input {
  /* This styles the input area */
  position: absolute;
  padding: 5px;
  left: 50%;
  top: 8%;
  transform: translateX(-50%);
  max-width: 5%;
}
```

CSS continues:

```
26  #main-char {
27    /* This styles the player character */
28    position: absolute;
29    left: 50%;
30    bottom: 80%;
31    transform: translateX(-50%);
32    border: 1px solid black;
33    width: 5%;
34    height: 5%;
35  }
36
37  .attributes {
38    /* This styles the lives and
39      scores counters*/
40    margin: 0;
41    padding: 2.5%;
42  }
```

Now the page looks like this:



Now inside the 'game.js' file I use the requestAnimationFrame function to be the framework of how the game will work:

```
function game(time) {
  //runs the game
  console.log(time);
  //This recursively calls the game function
  requestAnimationFrame((timeC) => {
    //timeC is the current time after the start of
    //the animation frames
    game(timeC);
  });
}

function start() {
  //Initiates the start of game
  requestAnimationFrame((timeC) => {
    game(timeC);
  });
}

//start()
```

Testing whether the functions are working as intended by uncommenting the start function and running the code:

In the console:

The functions are working properly.

Here is the code to spawn enemies:

First, I added the code to select a random coordinate in a specified area in the game box:

```
function randomCoords() {
  //produces a random coordinate in the game box
  const choise = rndNum(2);
  let y = rndNum(60) - 1;
  let x = 0;
  if (y > 30) {
    if (choise === 1) {
      x = rndNum(5) - 1;
    } else {
      x = rndNum(5) - 1 + 90;
    }
  } else {
    x = rndNum(95) - 1;
  }
  return [x, y];
}
```

The enemies are defined as below in the CSS within the game.css file. Their position is equal to the x and y variables stored in the CSS:

```
.enemy {
  /* These variables represent the
  enemie's position relative to the game box*/
  --x: 0;
  --y: 0;

  position: absolute;
  /* The number in the variables are
  transferred to percentage*/
  left: calc(var(--x) * 1%);
  bottom: calc(var(--y) * 1%);
  border: 2px solid red;
  width: 5%;
  height: 5%;
  display: flex;
  flex-direction: column;
  align-items: center;
}
```

The enemies are represented by div tags that inherit the size of the game box. Here is the function to add an enemy div to the game box:

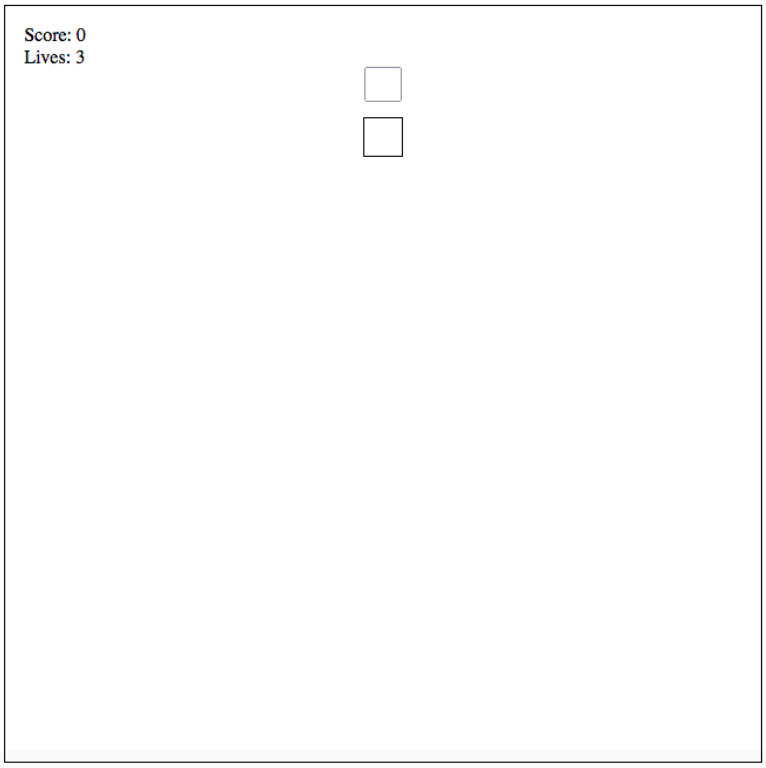
```
function spawnEnemy(id) {
  const gameBox = document.getElementById("game-box");
  //This creates a random set of coords
  const coords = randomCoords();

  //creates the div enemy block
  const enemy = document.createElement("div");
  enemy.setAttribute("class", "enemy");
  enemy.setAttribute("id", `${id}`);
  enemy.style.setProperty("--x", coords[0]);
  enemy.style.setProperty("--y", coords[1]);
  //Adds the enemy to the game box
  gameBox.appendChild(enemy);
}
```

Now I add this function to the game function and added code that only runs at certain intervals:

```
//These variables define the delay between
//enemy spawns
let spawnDelay = 1.75;
let interval = spawnDelay;
let id = 0

function game(timeC) {
  //runs the game
  if (Math.floor(timeC / 10) / 100 > interval) {
    //spawns an enemy every interval
    interval = interval + spawnDelay;
    spawnEnemy(id);
    id++;
  }

  //This recursively calls the game function
  requestAnimationFrame((timeC) => {
    //timeC is the current time after the start of
    //the animation frames
    game(timeC);
  });
}
```

Testing to see if it works by uncommenting the start function and calling the endpoint /games/game-one:

On the screen after 5 seconds:



TEST FAILED! Nothing happened. In the Firefox console the following error message is shown:

```
Uncaught ReferenceError: rndNum is not defined                    game.js:3:18
    randomCoords http://localhost:3000/javascript/game.js:3
    spawnEnemy   http://localhost:3000/javascript/game.js:21
```

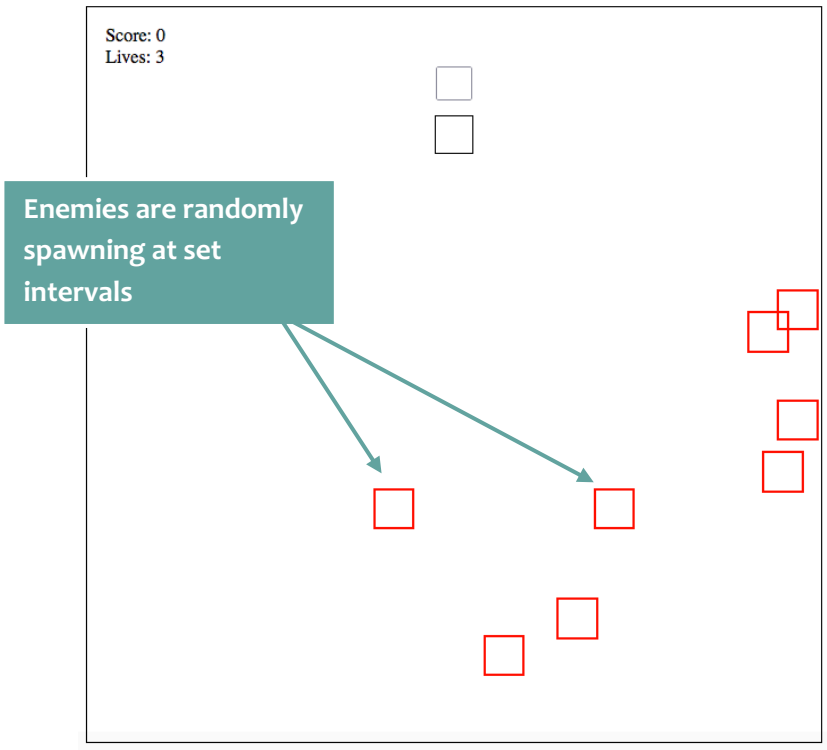
Here was the problem:

The function used in randomCoords is called rndNum is not defined in the code. I simply forgot to add the function in. Here I added the rndNum function:

```
18  function rndNum(num) {
19    //creates a random number that cannot be 0
20    return Math.floor(Math.random() * num) + 1;
21  }
```

Now saving and recalling the endpoint:

On screen after 5 seconds:



Inspecting the game box using the Firefox dev tools:

```
<div id="0" class="enemy" style="--x: 37; --y: 13;"></div> flex
<div id="1" class="enemy" style="--x: 17; --y: 23;"></div> flex
<div id="2" class="enemy" style="--x: 2; --y: 33;"></div> flex
<div id="3" class="enemy" style="--x: 93; --y: 39;"></div> flex
<div id="4" class="enemy" style="--x: 80; --y: 15;"></div> flex
<div id="5" class="enemy" style="--x: 77; --y: 2;"></div> flex
<div id="6" class="enemy" style="--x: 3; --y: 41;"></div> flex
```

TEST SUCCESS! The enemies are being randomly added on the screen at set intervals.

Now I make the functions to create random questions:

First, I need the code to get the game mode from the session storage:

```
23  //gets the type from the browser storage
24  const type = sessionStorage.getItem("type");
```

Now to make the function to create questions based on the game mode:

```
function randomQuestion(asq, mdq, type) {
  //randomizes the type of question created
  let returnQ = "";
  if (type === "both") {
    //depending on the the game mode you will get different
    //questions
    const choise = rndNum(2);
    if (choise === 1) {
      returnQ = randomASQ(asq);
    } else {
      returnQ = randomMDQ(mdq);
    }
  } else if (type === "as") {
    returnQ = randomASQ(asq);
  } else if (type === "md") {
    returnQ = randomMDQ(mdq);
  }
  return returnQ;
}
```

The ‘asq’ and ‘mdq’ parameters are the given ranges of questions that can be created for addition and subtraction, or multiplication and division. This will be used later to implement scaling difficulty of questions.

Now to make the randomASQ and randomMDQ functions:

```
function randomASQ(range) {
  //creates a random addition or subtraction question
  let a = rndNum(range);
  let b = rndNum(range);
  let returnEq = 0;
  let dis = rndNum(2);
  if (dis === 1) {
    //creates random addition equation
    returnEq = evaluateAns(a, "+", b);
  } else {
    //creates random subtraction equation
    if (b > a) {
      //this if statement makes sure that the answer will not be
      //minus number.
      returnEq = evaluateAns(b, "-", a);
    } else {
      returnEq = evaluateAns(a, "-", b);
    }
  }
  return returnEq;
}
```

```
function randomMDQ(range) {
  //creates a random multiplication or division question
  let a = rndNum(range);
  let b = rndNum(range);
  let c = evaluateAns(a, "*", b);
  let returnEq = 0;
  let dis = rndNum(2);
  if (dis === 1) {
    //creates random multiplication equation
    c.que = `${a} X ${b}`;
    returnEq = c;
  } else {
    //creates random division equation
    returnEq = { que: `${c.ans} / ${b}`, ans: a };
  }
  return returnEq;
}
```

Also need the helper function evaluateAns:

```
function evaluateAns(a, sign, b) {
  //calculates the equation
  let c = eval(`${a} ${sign} ${b}`);
  return { que: `${a} ${sign} ${b}`, ans: c };
}
```

Now there needs to be a place for the questions to be displayed under each enemy. For this, in the spawnEnemy function I added a p tag as follows containing the random question:

```
function spawnEnemy(id) {
  //Gets the random question
  const text = randomQuestion(asq, mdq, type);
  const gameBox = document.getElementById("game-box");
  //This creates a random set of coords
  const coords = randomCoords();

  //creates the div enemy block
  const enemy = document.createElement("div");
  enemy.setAttribute("class", "enemy");
  enemy.setAttribute("id", `${id}`);
  //The value in of the enemy div is equal to the
  //equation answer
  enemy.setAttribute("value", `${text.ans}`);
  enemy.style.setProperty("--x", coords[0]);
  enemy.style.setProperty("--y", coords[1]);

  //creates a p tag to harbour a question on the enemy
  const question = document.createElement("p");
  const textnode = document.createTextNode(`${text.que}`);
  question.appendChild(textnode);
  question.setAttribute("class", "question");
  question.setAttribute("id", `${id}`);

  //Adds the p tag to the enemy div
  enemy.appendChild(question);
  //Adds the enemy to the game box
  gameBox.appendChild(enemy);
}
```

To make this displayed properly I added the following CSS to the game.css file to style the p tags:

```
.question {
  margin: 0;
  background-color: white;
  margin-top: 40px;
  padding: 0;
  width: max-content;
  font-weight: bold;
}
```

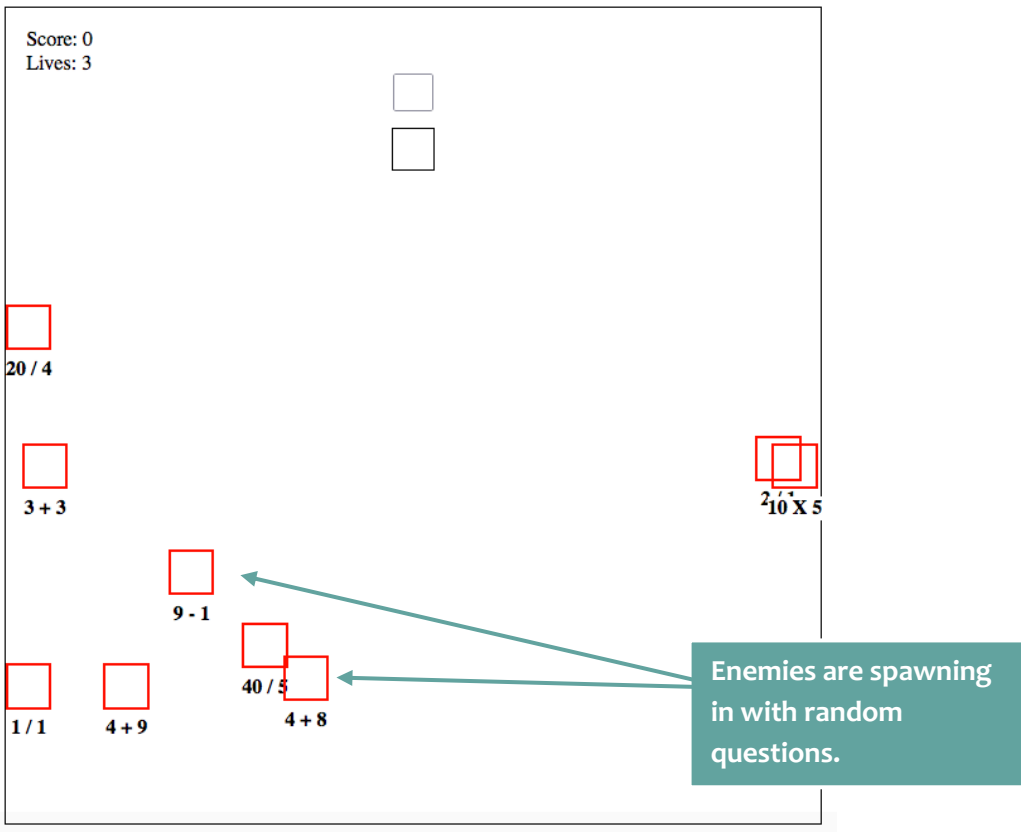
Now editing and putting in the necessary parameters:

```
function spawnEnemy(id, asq, mdq, type) {
```

```
spawnEnemy(id, 10, 10, "both");
```

Testing to see if everything works by saving and calling the endpoint:

The screen after 5 seconds:



Inspecting the game box using the Firefox dev tools:

```
<div id="0" class="enemy" value="5" style="--x: 0; --y: 58;">…</div> flex
<div id="1" class="enemy" value="2" style="--x: 92; --y: 42;">…</div> flex
<div id="2" class="enemy" value="50" style="--x: 94; --y: 41;">…</div>
flex
<div id="3" class="enemy" value="1" style="--x: 0; --y: 14;">…</div> flex
<div id="4" class="enemy" value="8" style="--x: 29; --y: 19;">…</div> flex
<div id="5" class="enemy" value="12" style="--x: 34; --y: 15;">…</div>
```

In each div tag the following is generally seen:

```
<div id="0" class="enemy" value="5" style="--x: 0; --y: 58;"> flex
  <p id="0" class="question">20 / 4</p>
</div>
```

TEST SUCCESS! The random questions are now working!

Now I need to make the enemies move towards the player. To do this, I should start with making a class called Enemy in the enemies.js file. In here will be the functions to make this movement work:

```
1  //Exporting allows this class to be used in other files
2  export default class Enemy {
3      //creates a class called Enemy that defines the enemies
4      constructor(enemyId) {
5        this.element = document.getElementById(`${enemyId}`);
6      }
7
8      get getX() {
9          //This get method gets the enemie's x coords
10       return parseFloat(getComputedStyle(this.element).getPropertyValue("--x"));
11     }
12
13     set changeX(value) {
14         //This set method changes the enemie's x coords
15       this.element.style.setProperty("--x", value);
16     }
17
18     get getY() {
19         //This get method gets the enemie's y coords
20       return parseFloat(getComputedStyle(this.element).getPropertyValue("--y"));
21     }
22
23     set changeY(value) {
24         //This set method changes the enemie's y coords
25       this.element.style.setProperty("--y", value);
26     }
27
28     direction(startP, endP) {
29         //returns the direction of an enemy
30         let dir = { x: endP.x - startP.x, y: endP.y - startP.y };
31         let hyp = Math.sqrt(Math.pow(dir.x, 2) + Math.pow(dir.y, 2));
32         dir = { x: dir.x / hyp, y: dir.y / hyp };
33         return dir;
34       }
```

JavaScript continues:

```
35
36      //-----------CREATE ENEMY-------------
37      createEnemy(enemy, start, aim, speed) {
38        //Creates a moving enemy
39        function update(timeC, speed) {
40          //This is a unit vector towards the player
41          let unitV = direction(start, aim);
42          //This is what moves the enemy
43          enemy.changeX = enemy.getX + unitV.x * speed;
44          enemy.changeY = enemy.getY + unitV.y * speed;
45
46          if (enemy.getY < 80) {
47              //If the enemy has not reached the player
48              //carry on moving towards the player
49            requestAnimationFrame((timeC) => {
50              update(timeC, speed);
51            });
52          } else if (enemy.getY >= 80) {
53              //If the enemy has reached the player stop the
54              //enemie's movement
55            cancelAnimationFrame(timeC);
56          }
57        }
58        requestAnimationFrame((timeC) => {
59          //This initiates the movement
60          update(timeC, speed);
61        });
62      }
63    }
```

In the game.js file, at the top of the code I imported the Enemy class:

```
1  //Imports the Enemy class from the enemies,js file
2  import Enemy from "./enemies.js";
```

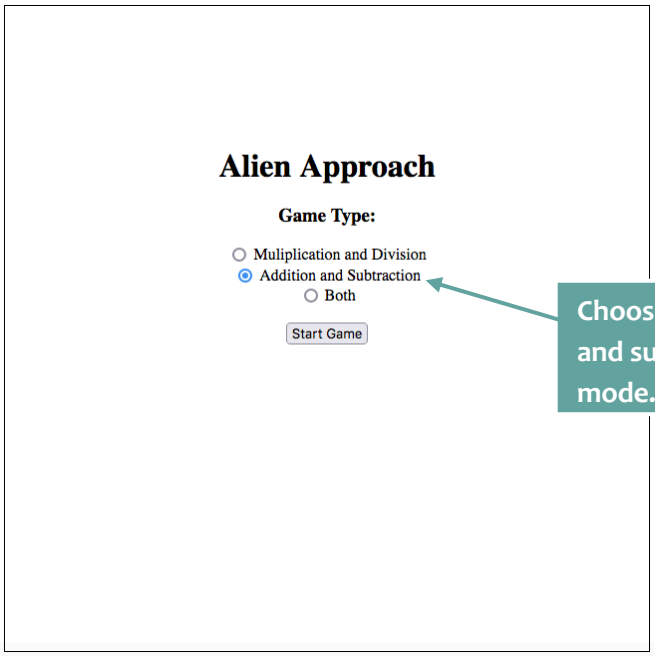
Inside the game function I added the following:

```
136   if (Math.floor(timeC / 10) / 100 > interval) {
137       //spawns an enemy every interval
138       interval = interval + spawnDelay;
139       spawnEnemy(id, asq, mdq, type);
140       //Creates a new instance of the class Enemy
141       let enemy = new Enemy(id);
142       let x = enemy.getX;
143       let y = enemy.getY;
144       enemy.createEnemy(enemy, { x: x, y: y }, { x: 47.5, y: 80 }, movementSpeed);
145       id++;
146     }
```

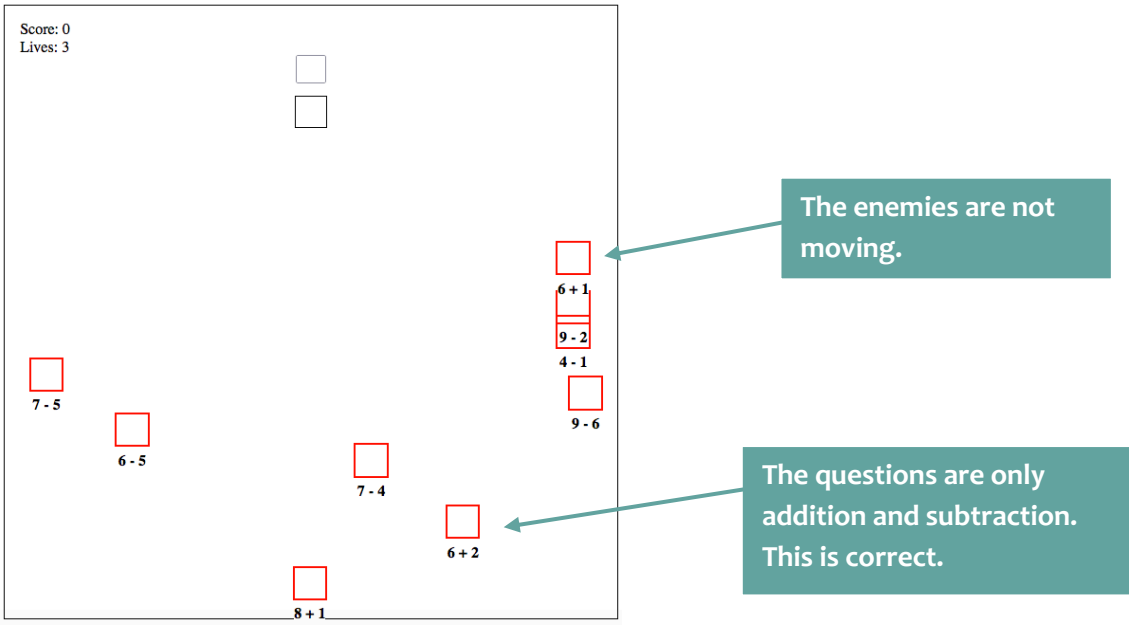
I also took the asq and mdq variables out as global variables as well as defining movement Speed:

```
125  //Defines the variables that change the questions and
126  //attributes of the spawned enemies
127  let spawnDelay = 1.75;
128  let interval = spawnDelay;
129  let id = 0;
130  let asq = 10;
131  let mdq = 10;
132  let movementSpeed = 0.09;
```

To test if this is working, I need to initiate a game mode. Therefore, I must access this game through the game menu page. Saving and calling the endpoint /games/game-menu:



Choosing the Addition and Subtraction game mode and then pressing Start Game:



Enemies are spawning with the correct questions, but they are not towards the player.

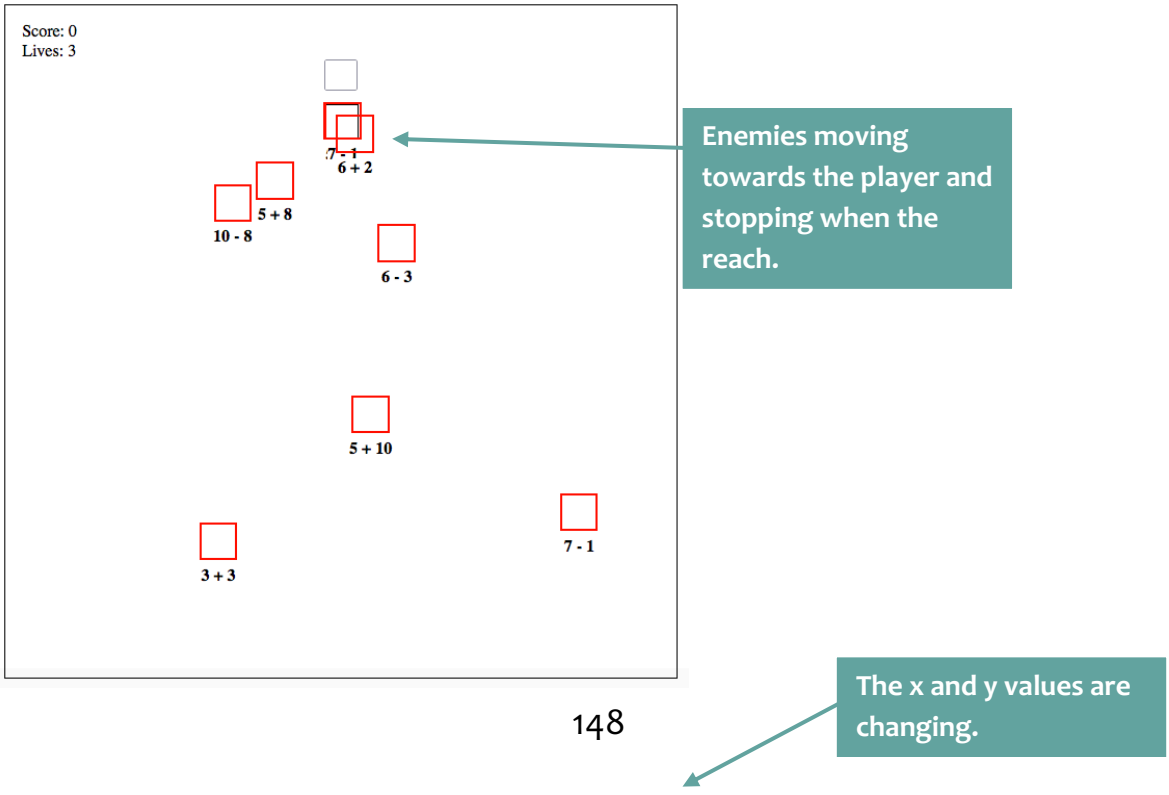
TEST FAILED! Is in the Firefox console the following error message is shown:

```
Uncaught ReferenceError: direction is not defined        13  enemies.js:41:19
    update       …localhost:3000/javascript/enemies.js:41
    createEnemy  …localhost:3000/javascript/enemies.js:60
  [Learn More]
```

Here is the problem:

I called the function direction wrong inside the createEnemy function. To fix this I moved the direction function outside the Enemy class.

Saving and calling the /games/game-one endpoint the following is shown:



And inside the Firefox console:

```
<div id="23" class="enemy" value="4" style="--x: 42.65886305987237; --y:
76.12709044789834;">…</div> flex
<div id="24" class="enemy" value="13" style="--x: 46.121409439204825; --y
57.94255102728432;">…</div> flex
<div id="25" class="enemy" value="7" style="--x: 56.10006982541014; --y:
74.13171706030585;">…</div> flex
<div id="26" class="enemy" value="2" style="--x: 40.10020873487225; --y:
42.17884464490287
```

TEST SUCCESS! The enemies are moving towards the player and then stopping as soon as they reach. The highlighted values mean that the variables are changing. This proves it working function further.

Now I implement the player input:

The following code takes in the value from the input area after pressing enter. If the inputted value equals the value of any enemy on screen, that enemy is deleted:

```
//defines the elements in the html page
const score = document.getElementById("score");
const input = document.querySelector("input");
const body = document.querySelector("body");
let scr = 0;

//This code defaults the input area to be
//already selected
document.querySelector("input").focus();

body.addEventListener("keypress", (event) => {
  //Listens for the enter key press to read the input
  if (event.code === "Enter") {
    //Gets all the enemy divs on screen with the corresponding value
    let find = document.querySelectorAll(`[value="${input.value}"]`);
    if (find) {
      input.value = "";
      //Removes the enemy divs and add 1 to the score counter
      find.forEach((element) => {
        element.remove();
        scr += 1;
        score.innerHTML = scr;
      });
    }
  }
});
```

Test by saving and recalling the endpoint:

In the Firefox inspector in the game box before typing and entering the answer:

```
<div id="0" class="enemy" value="8" style="--x: 40.93770398828891; --y:
72.77410102081068;">…</div> flex
<div id="1" class="enemy" value="4" style="--x: 52.95165137614832; --y:
76.405504587156;">…</div> flex
<div id="2" class="enemy" value="2" style="--x: 52.99169994667566; --y:
45.13964381675887;">…</div> flex
```

After typing and entering the number 8 (the answer to enemy id: 0):

```
<div id="1" class="enemy" value="4" style="--x: 47.46660550458852; --y:
80.02201834862417;">…</div> flex
<div id="2" class="enemy" value="2" style="--x: 47.487592904819095; --y:
80.07875808245979;">…</div> flex
```

The correct enemy div was removed…

Score: 1
Lives: 3

And the Score counter incremented by 1.

TEST SUCCESS! The player can now 'kill' enemies by typing in the answer to the equation below them.

Now when the enemy reaches the player, the enemy should be deleted, and 1 should be subtracted from the lives counter.

I do this by adding the following in the createEnemy function:

```
42  if (enemy.getY < 80) {
43          //If the enemy has not reached the player
44          //carry on moving towards the player
45          requestAnimationFrame((timeC) => {
46            update(timeC, speed);
47          });
48        } else if (enemy.getY >= 80) {
49          //If the enemy has reached the player remove the
50          //enemy and take away a life
51          cancelAnimationFrame(timeC);
52          enemy.element.remove();
53          lives -= 1;
54          live.innerHTML = lives;
55        }
```

At the top the code I added these two variables:

```
1  //Exports the live counter
2  export let lives = 3;
3  const live = document.getElementById("lives");
```

Testing by recalling the /games/game-one endpoint:

Letting the enemies reach the player and inspecting the game box:

The enemies from 0 to 9 have been removed.

```
<div id="main-char"></div>
<div id="10" class="enemy" value="4" style="--x: 47.55598974293617; --y:
77.27516584382838;">…</div> flex
<div id="11" class="enemy" value="10" style="--x: 51.854490655727005; --y:
66.84387929546556;">…</div> flex
<div id="12" class="enemy" value="12" style="--x: 42.34658845172217; --y:
76.12108808194236;">…</div> flex
```

The previous enemies from 0 to 9 have been removed.

Also, the lives counter looks like this:

Score: 0
Lives: -7

It started at 3 and end at -7 for 10 lives lost. Therefore, this is the correct value.

TEST SUCCESS! The lives counter works!

Now here I implement the scaling difficulty. The function below sets the attributes of enemy spawns:

```
161  function setValues(rangeOfAsq, rangeOfMdq, delay, speed) {
162    //changes the attributes of the enemies and the difficulty of the questions
163    asq = rangeOfAsq;
164    mdq = rangeOfMdq;
165    spawnDelay = delay;
166    movementSpeed = speed;
167  }
```

In the game function I added the following code that calls the above function whenever a condition is met therefore, allowing the difficulty and conditions to be changed at will depending on the player's score:

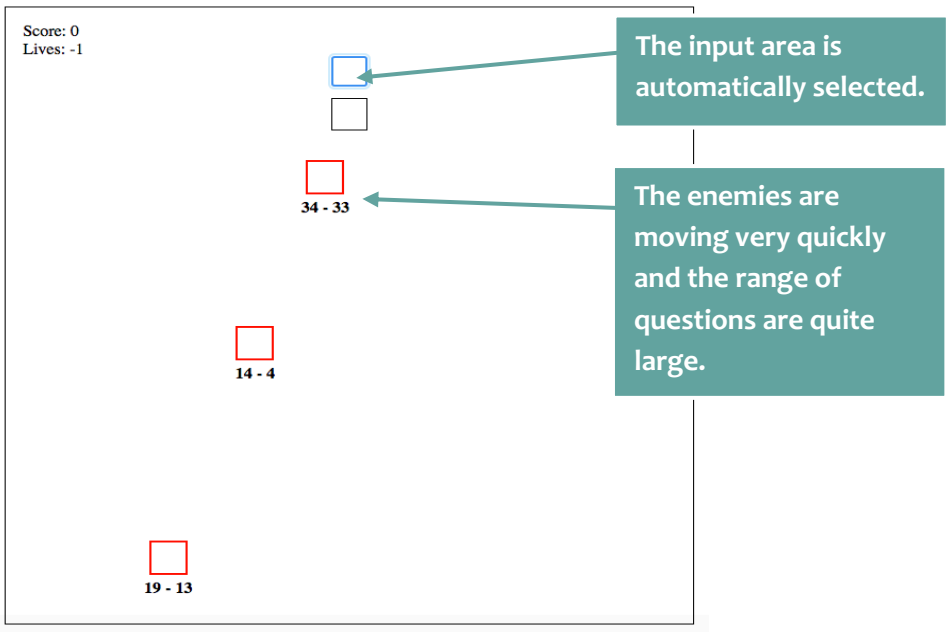
```
function game(timeC) {
  //runs the game
  //depending on the score the attributes and difficulty of
  //questions will change
  if (scr > 90) {
    setValues(35, 14, 1.1, (Math.random() * 12) / 100 + 0.25);
  } else if (scr > 80) {
    setValues(35, 13, 1.1, (Math.random() * 10) / 100 + 0.2);
  } else if (scr > 70) {
    setValues(35, 13, 1.1, (Math.random() * 8) / 100 + 0.18);
  } else if (scr > 50) {
    setValues(30, 12, 1.1, (Math.random() * 5) / 100 + 0.1);
  } else if (scr > 40) {
    setValues(25, 12, 1.2, (Math.random() * 2) / 100 + 0.1);
  } else if (scr > 32) {
    //Here the enemies spawn slowly, but they move very fast
    setValues(12, 12, 4, 0.5);
  } else if (scr > 30) {
    setValues(20, 12, 1.3, 0.1);
  } else if (scr > 20) {
    setValues(15, 12, 1.4, 0.095);
  } else if (scr > 10) {
    setValues(12, 10, 1.5, 0.09);
  }
```

I also added the code to automatically select the input area:

```
//This code defaults the input area to be
//already selected
document.querySelector("input").focus();
```

To test this, I will manually set my score to 91 and show the increase in difficulty:

On the screen after 5 seconds:



In 5 seconds, I have lost 4 lives and the questions being asked are extremely hard. Since the score is going up slowly, so will the difficulty. Therefore:

TEST SUCCESS! The game has increasing difficulty.

Now to make the end function which is called when the lives counter hits zero.

First, import the lives counter form the enemies.js file:

```
1  //Imports the Enemy class and lives from the enemies,js file
2  import Enemy , { lives } from "./enemies.js";
```

Now I make the end condition:

```
204  if (lives > 0) {
205      //checks if the player is still alive
206      //if yes carry on the game
207      requestAnimationFrame((timeC) => {
208        game(timeC);
209      });
210    } else {
211      //stops the game and clears the screen
212      cancelAnimationFrame(timeC);
213      const remainingEnemies = document.querySelectorAll(".enemy");
214      remainingEnemies.forEach((element) => {
215        element.remove();
216      });
217      //Stores the final score in session storage
218      sessionStorage.setItem("score", scr);
219      end();
220    }
221  }
```

Now in the end function I need to store the score in the database. So, first I make an endpoint that does exactly that in the games.js file:

```
//Sets up an endpoint with the PUT method on /games/game-scores
route.put("/game-scores", (req, res) => {
  const email = req.body.email;
  const { score, type } = req.body;
  if (type === "as") {
    Score.findOneAndUpdate({ email: email }, { $push: { typeAS: score } })
      .exec()
      .then((result) => {
        res.json({
          state: true,
        });
      });
  } else if (type === "md") {
    Score.findOneAndUpdate({ email: email }, { $push: { typeMD: score } })
      .exec()
      .then((result) => {
        res.json({
          state: true,
        });
      });
  } else if (type === "both") {
    Score.findOneAndUpdate({ email: email }, { $push: { typeBoth: score } })
      .exec()
      .then((result) => {
        res.json({
          state: true,
        });
      });
  }
});
```

Inside the end function I add the following code:

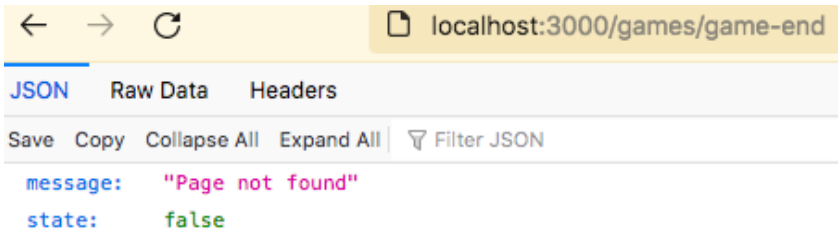
```
function end() {
  //Fired when the game ends
  const gameType = sessionStorage.getItem("type");
  const gameScore = sessionStorage.getItem("score");
  //Calls the endpoint /games/game-scores with the
  //method PUT
  fetch("http://localhost:3000/games/game-scores", {
    method: "PUT",
    body: JSON.stringify({
      //Using a manual email for now
      email: "doctor@who.com",
      type: gameType,
      score: gameScore,
    }),
    headers: {
      "Content-type": "application/json; charset=UTF-8",
    },
  }).then(() => {
    location.href = "http://localhost:3000/games/game-end";
  });
}
```

For now, I will use a manually entered email. Later I will change this.

To test, I will play a game and check the database to see if the score was stored:

Playing the game, I got a score of 3 on the game mode “both”.

I was redirected to /games/game-end, but I forgot to add that endpoint, so I got this:



It should have still worked though. Now looking inside the **scores** table for the 'doctor@who.com' account:

TEST SUCCESS! The score was correctly stored inside the database!

Here I make the /games/game-end endpoint and also the html and CSS for the game-end.html file:

The endpoint:

```
30  //Sets up an endpoint with the GET method on /games/game-end
31  route.get("/game-end", (req, res) => {
32    res.sendFile(path.join(__dirname, "/html_pages/game-end.html"));
33  });
```

The HTML:

```
1  <html lang="en">
2    <head>
3      <meta charset="UTF-8" />
4      <meta http-equiv="X-UA-Compatible" content="IE=edge" />
5      <meta name="viewport" content="width=device-width, initial-scale=1.0" />
6      <title>Alien Arithmetic | Alien Approach</title>
7    </head>
8    <body>
9
10     <div class="game-box" id="game-box">
11       <div id="main">
12         <h1 id="title">High Scores</h1>
```

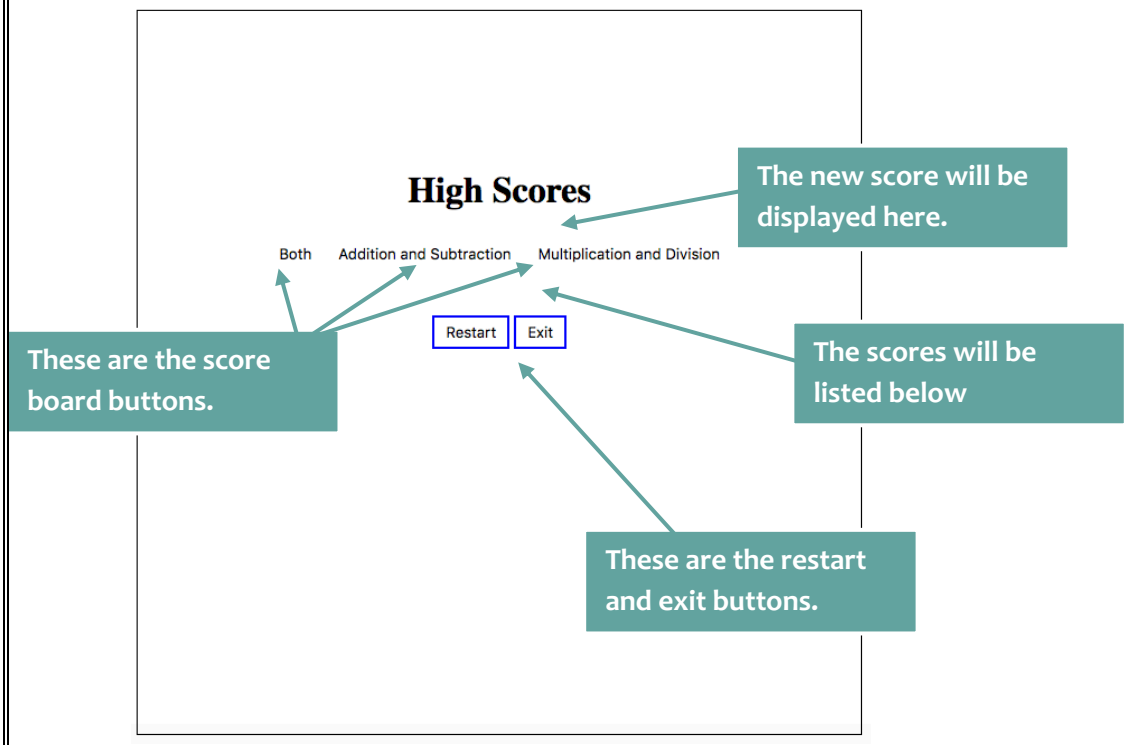
```
        <!-- This h3 tag shows the score just achieved -->
        <h3 id="your-score"></h3>
        <!-- This div tag contains the score board-->
        <div class="types">
          <!-- These buttons allow the user to choose which
            game mode scores they want to see  -->
          <button class="type both" onclick="">Both</button>
          <button class="type as" onclick="">Addition and Subtraction</button>
          <button class="type md" onclick="">
            Multiplication and Division
          </button>
        </div>
        <ol id="scores"></ol>
        <!-- This button takes the user to the game menu page -->
        <button class="action restart" onclick="">Restart</button>
        <!-- This button takes the user to the games page -->
        <button class="action exit" onclick="">Exit</button>
      </div>
    </div>
  </body>
</html>
```

The CSS:

```
<style>
      body {
        margin: 0;
        padding: 0;
      }

      .game-box {
        /* Styles the box*/
        border: 1px solid black;
        width: 90vh;
        height: 90vh;
        margin-left: 50vw;
        margin-top: 5vh;
        transform: translateX(-50%);
      }

      a,p,h1,h2,h3,ol,li,ul {
        /* Defaults the styles of tags */
        padding: 0;
        margin: 0;
      }
      #main {
        margin: 0 auto;
        text-align: center;
        width: 70%;
      }
      #title {
        margin-top: 20vh;
      }

      #your-score {
        /*Styles the score just achieved*/
        margin-top: 10px;
      }
```

```
      button.type {
        /*Styles the game mode buttons*/
        border: 2px solid white;
        background-color: #ffffff;
        padding: 5px 10px;
      }
      button:hover {
        background-color: #d7d7d7;
      }

      .types {
        /*Styles the box containing the score board*/
        margin: 20px auto;
        display: flex;
        align-items: center;
        justify-content: center;
      }
      li {
        /*Styles the score items  */
        margin: 5px auto;
        width: 50%;
      }
      ol {
        text-align: center;
      }
      .action {
        /*Styles the restart and exit buttons  */
        border: 2px solid blue;
        background-color: white;
        margin-top: 20px;
        padding: 5px 10px;
      }
    </style>
```

Calling the endpoint this is what it looks like:



To get the scores, I need to make an endpoint that gets the scores from the database. This is the endpoint code in the games.js file:

```
66  //Sets up an endpoint with the POST method on /games/game-score
67  route.post("/game-score", (req, res) => {
68    //This endpoint gets the Scores from the database
69    //depending of the game mode given
70    const { type, email } = req.body;
71    Score.findOne({ email: email })
72      .exec()
73      .then((data) => {
74        const content = data;
75        let scores = "";
76        if (type === "typeAS") {
77          scores = content.typeAS;
78        } else if (type === "typeMD") {
79          scores = content.typeMD;
80        } else {
81          scores = content.typeBoth;
82        }
83        res.json({
84          message: scores,
85        });
86      });
87  });
```

Now making the function inside the game-end page to call this endpoint and send the type and email:

```
167   function getScores(type) {
168           //Sends a POST request to the server in order to get the array of scores
169           fetch("http://localhost:3000/games/game-score", {
170             method: "POST",
171             body: JSON.stringify({
172               email: "doctor@who.com",
173               type: type,
174             }),
175             headers: {
176               "Content-type": "application/json; charset=UTF-8",
177             },
178           })
179             .then((res) => {
180               return res.json();
181             })
182             .then((data) => {
183               displayScores(data.message);
184             });
185         }
```

Now making the displayScores function:

```
function displayScores(array) {
        //Displays the scores in the DOM
        const list = document.getElementById("scores");
        // Sorts the array
        array = array.sort(function (a, b) {
          return b - a;
        });
        if (!array.length) {
          //If the array given is empty add the p tag below
          //to the list of scores
          list.innerHTML = "<p>There are no scores here</p>";
        } else {
          //If the array given is not empty add the array
          //into the DOM
          list.innerHTML = "";
          for (let i = 0; i < array.length; i++) {
            if (i >= 10) {
              //This only allows the first top ten
              //scores to be displayed
              break;
            }
            list.innerHTML += `<li>${array[i]}</li>`;
          }
        }
      }
```

The code below gets the score and game mode from the session storage:

```
//Gets the score and game mode from the session storage
      const currentType = sessionStorage.getItem("type");
      const score = sessionStorage.getItem("score");
```

Here I add the getScores function into the html with the correct parameters:

```
<!-- These buttons allow the user to choose which
            game mode scores they want to see  -->
          <button class="type both" onclick="getScores('Both')">Both</button>
          <button class="type as" onclick="getScores('typeAS')">Addition and Subtraction</button>
          <button class="type md" onclick="getScores('typeMD')">
            Multiplication and Division
          </button>
```

When the user selects a score board, I want the score board button to be highlighted. The code below does this:

```
114  const selectedTypes = document.querySelectorAll(".type");
115        selectedTypes.forEach((ele) => {
116          //For each score board button selected, do the following
117          ele.addEventListener("click", (res) => {
118            selectedTypes.forEach((e) => {
119              //When the button is clicked make all
120              //the buttons have white borders
121              e.style.borderColor = "white";
122            });
123            //The button clicked will get a red border
124            ele.style.borderColor = "red";
125          });
126        });
```

The score board to be shown first, will be the for the game mode just played. The code below does this:

```
129  //Depending on the game mode previously played
130        //the corresponding score board button is highlighted and displayed
131        if (currentType === "both") {
132          document.querySelector(".both").style.borderColor = "red";
133          getScores("Both");
134        } else if (currentType === "as") {
135          document.querySelector(".as").style.borderColor = "red";
136          getScores("typeAS");
137        } else {
138          document.querySelector(".md").style.borderColor = "red";
139          getScores("typeMD");
140        }
```

Also, here is the code that displays the score just attained:

```
112   const yourScore = document.getElementById("your-score");
113         yourScore.innerText = `Your Score: ${score}`;
```

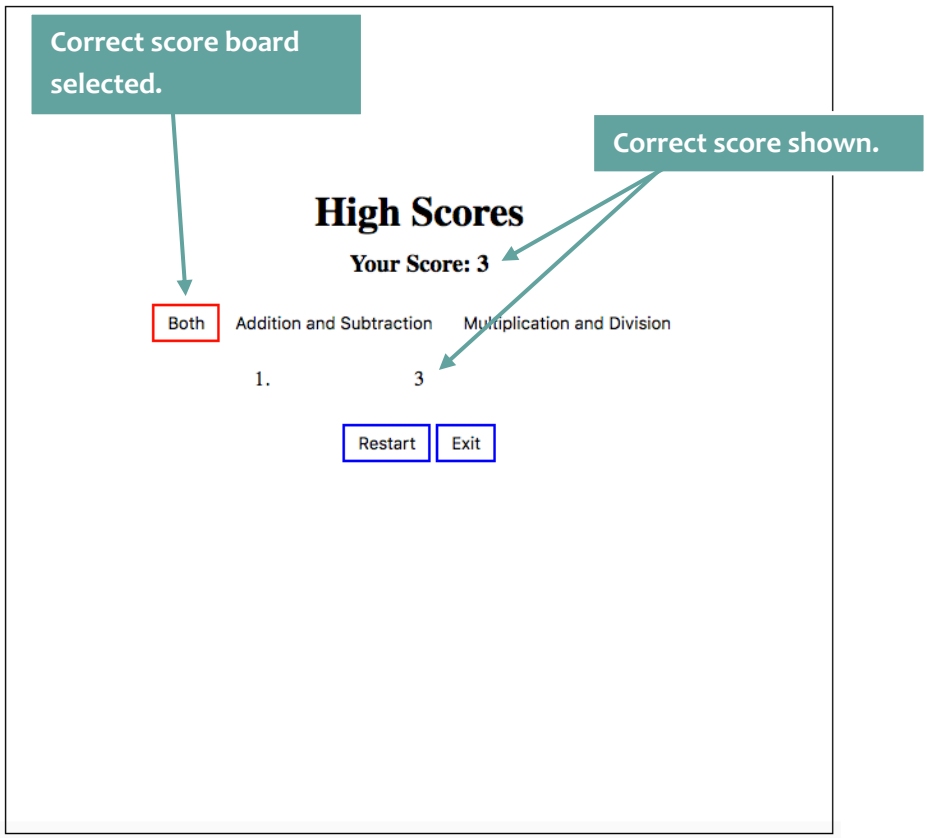
Now testing whether the scores in the tables are shown correctly with all the other features being implemented correctly:

In the session storage:

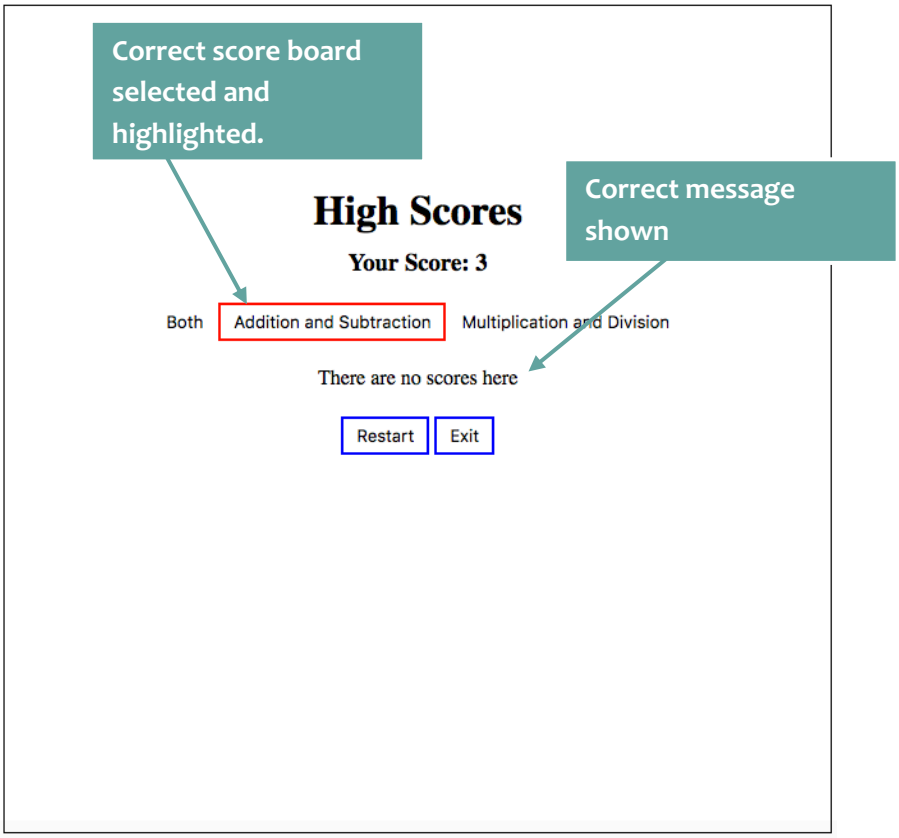
Key	Value
score	3
type	both

Therefore, the 'both' game mode score board should be shown by default and the score 3 is on the board. The button should also be highlighted:

On the screen after refreshing the page:



Clicking the other score board buttons:



TEST SUCCESS! Everything works as intended.

Now I make the Restart and exit buttons work:

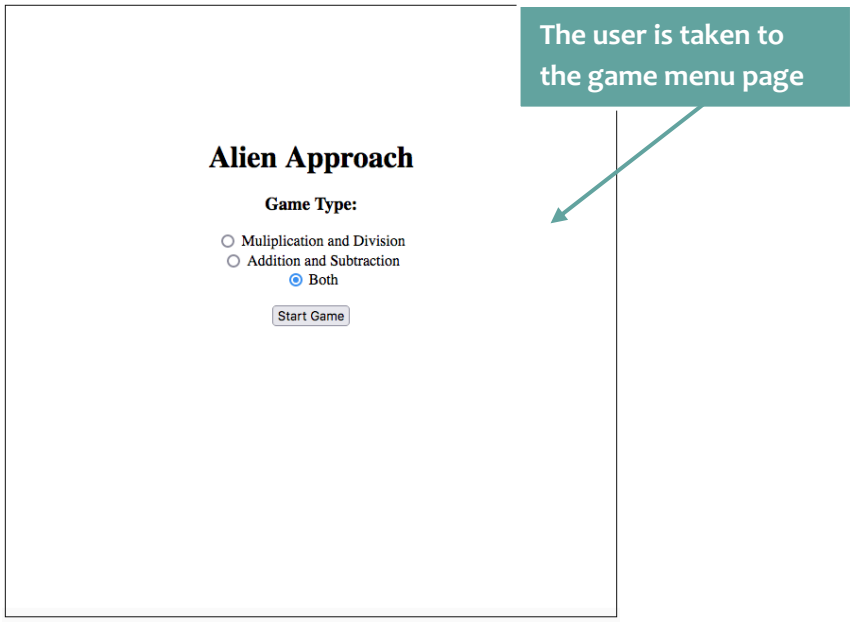
In the JavaScript:

```
142  function nextLocation(loc) {
143          //Calls an endpoint when called
144          location.href = "http://localhost:3000/games/" + loc;
145        }
```

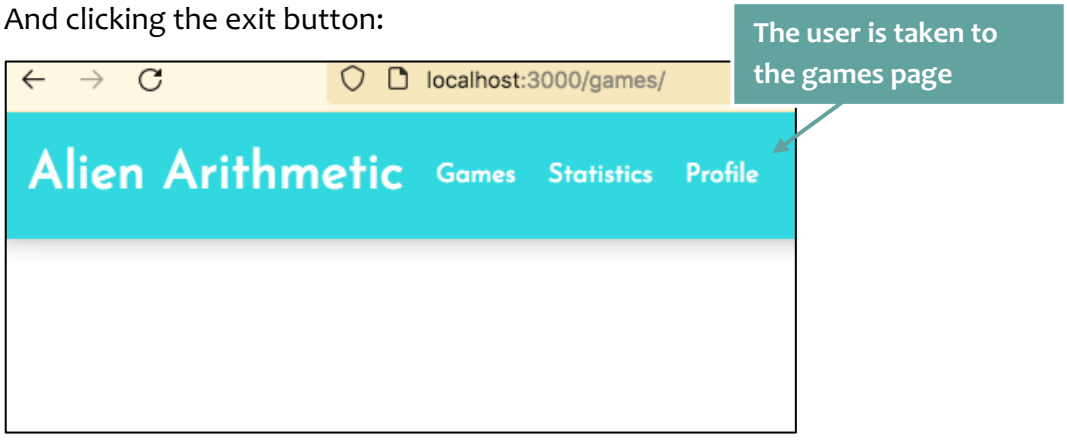
In the HTML:

```
101  <!-- This button takes the user to the game menu page -->
102          <button class="action restart" onclick="nextLocation('game-menu')">
103            Restart
104          </button>
105          <!-- This button takes the user to the games page -->
106          <button class="action exit" onclick="nextLocation('')
```

Now clicking the Restart button, the following is shown:



And clicking the exit button:



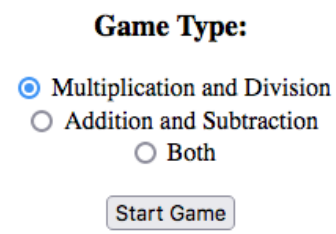
TEST SUCCESS! The user is directed to the correct locations!

Testing the Game Menu

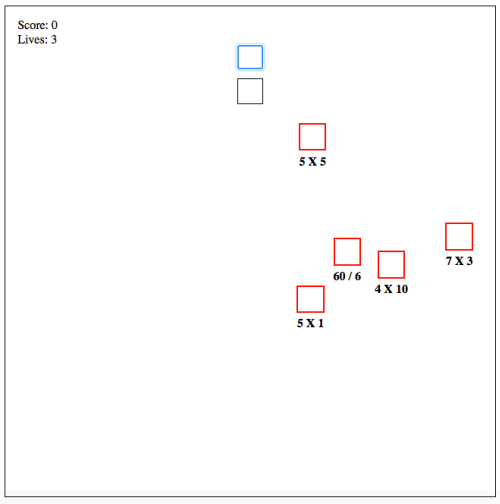
Test ID	Test Input	Expected Outcome
GM1	Select each radio button one at a time and play the game.	The questions are correct for the game mode selected.
GM2	Observe the game mode selected after restarting.	The game mode should be the same as the game mode previously selected.

TEST GM1

Input:

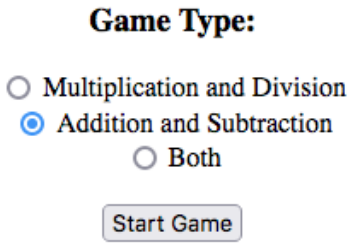


Outcome:

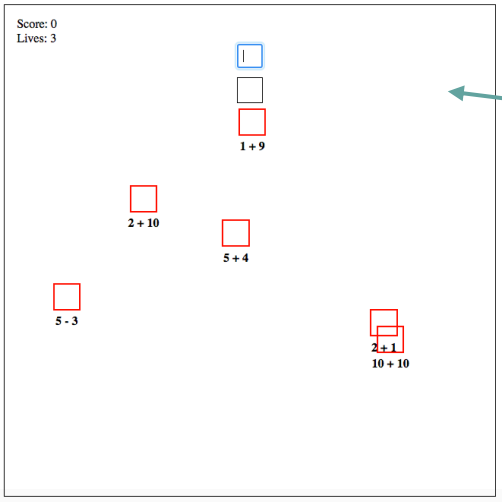


After start game is clicked with 'multiplication and division' game mode selected, the enemies have the correct questions.

Input:

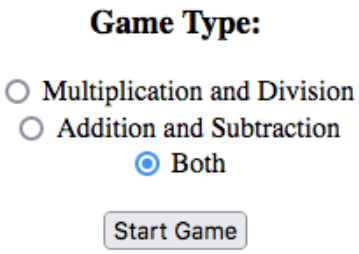


Outcome:

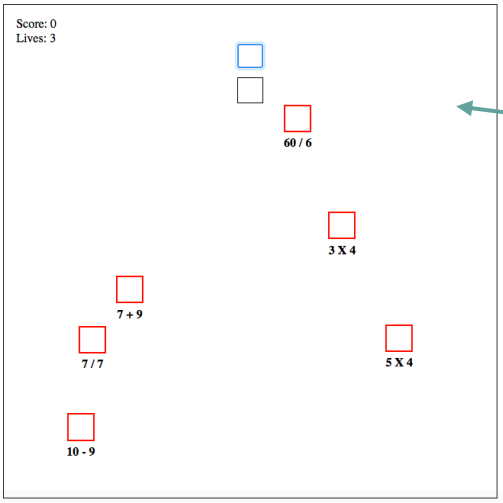


After start game is clicked with 'addition and subtraction' game mode selected, the enemies have the correct questions.

Input:



Outcome:

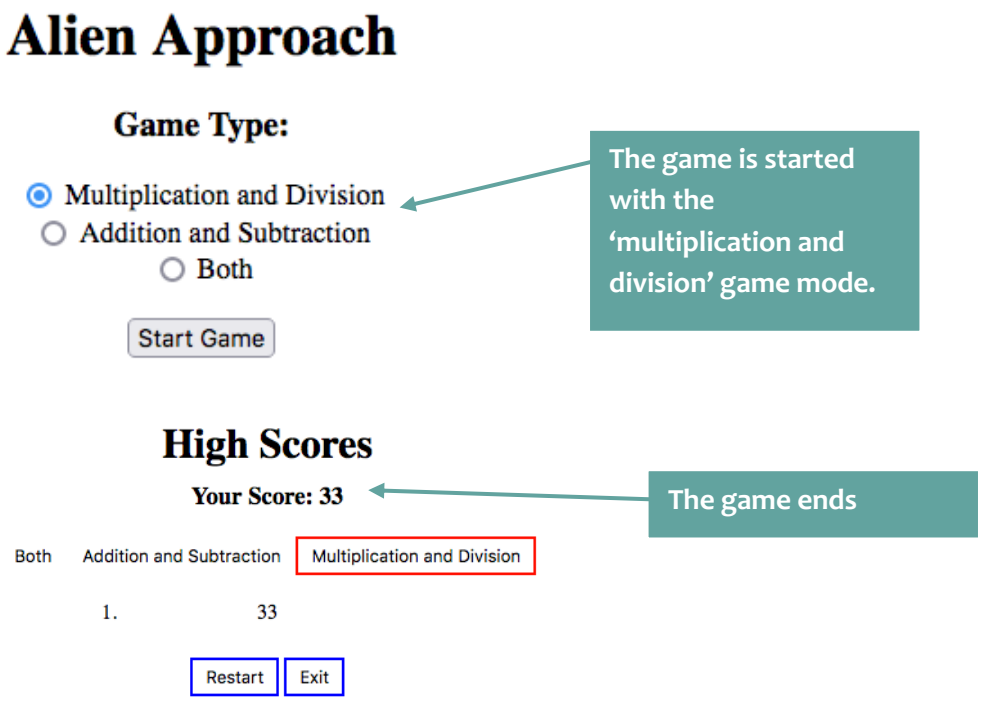


After start game is clicked with 'both' game mode selected, the enemies have the correct questions.

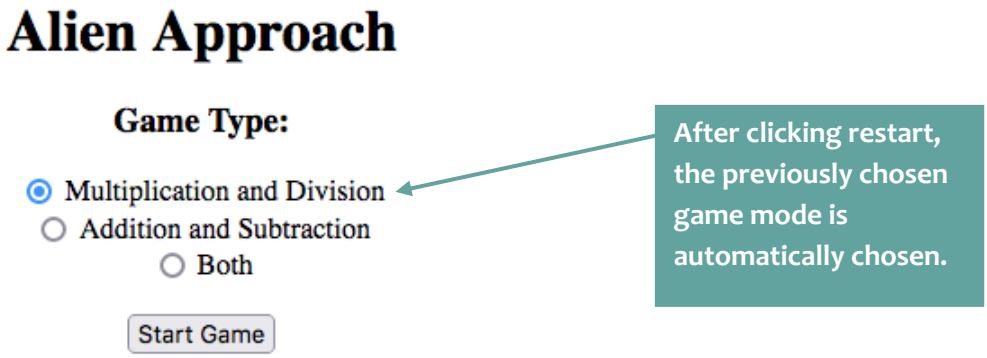
Test Success!

TEST GM2

Input:



Outcome:

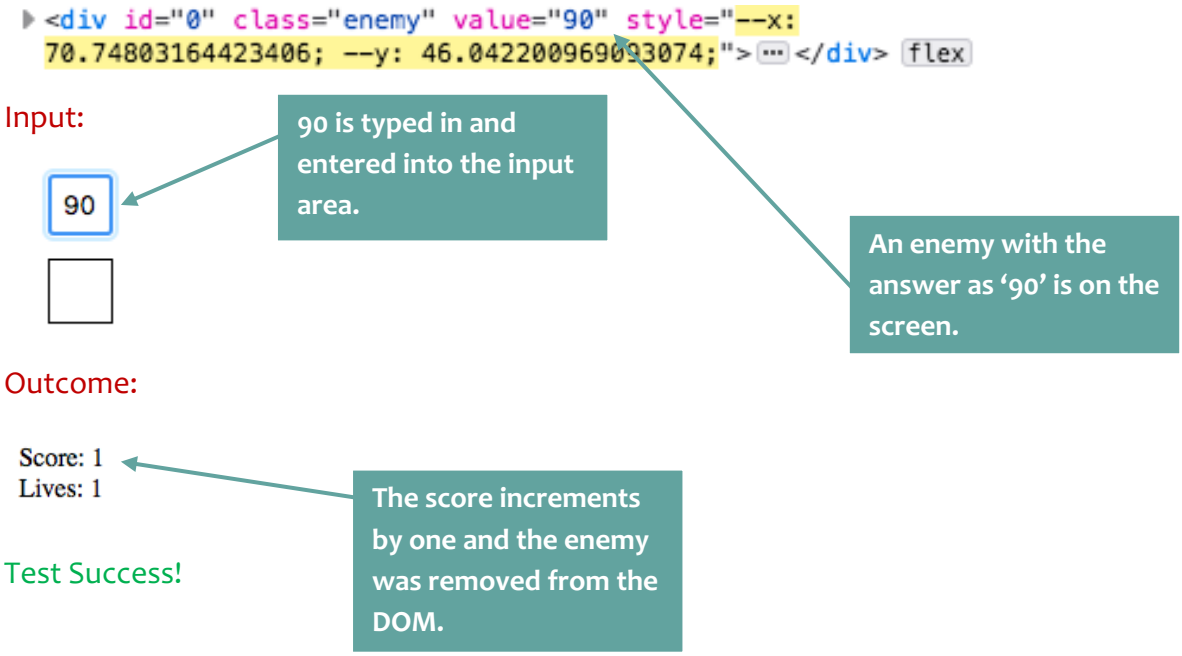


Test Success!

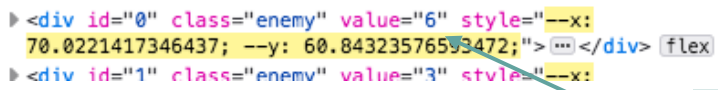
Testing the Actual Game

Test ID	Test Input	Expected Outcome
AG1	Enter an answer into the input area.	The enemy with the questions to that answer should disappear from the DOM.
AG2	Enter a wrong answer into the input area.	Nothing should happen.
AG3	Observe the lives and score counters.	Score should increase by 1 when answer is correct, and lives should decrease by one when an enemy reaches the player.
AG4	Play the game.	As the game progresses the enemies should start to move faster, and the questions should become more challenging. All the other outcomes expected from playing the game.
AG5	Observe when lives go down to 0.	User redirected to Game End page.

TEST AG1



TEST AG2

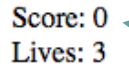


An enemy with the answer as '6' is on the screen.

Input:

15 is entered.

Outcome:

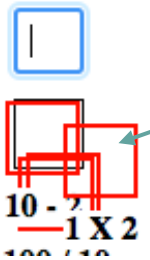


The enemy remains in the DOM and the score does not increment.

Test Success!

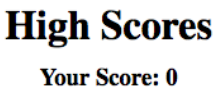
TEST AG3

Input:



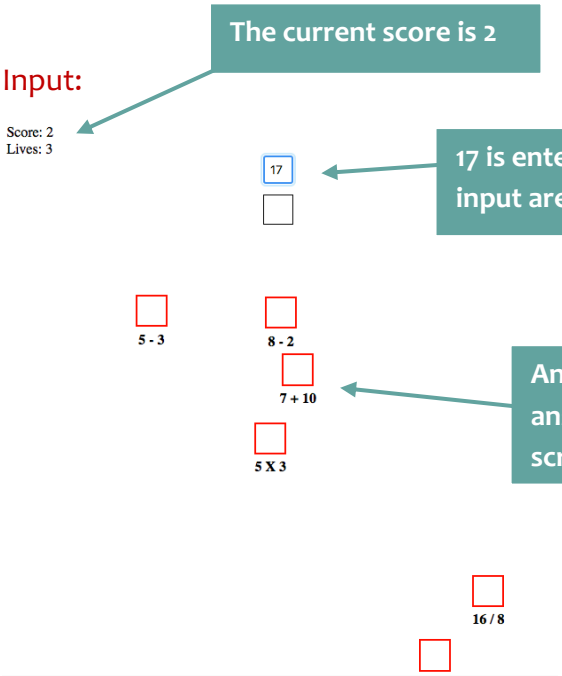
Here the player is overwhelmed by more than 3 enemies.

Outcome:



The game ends

Input:

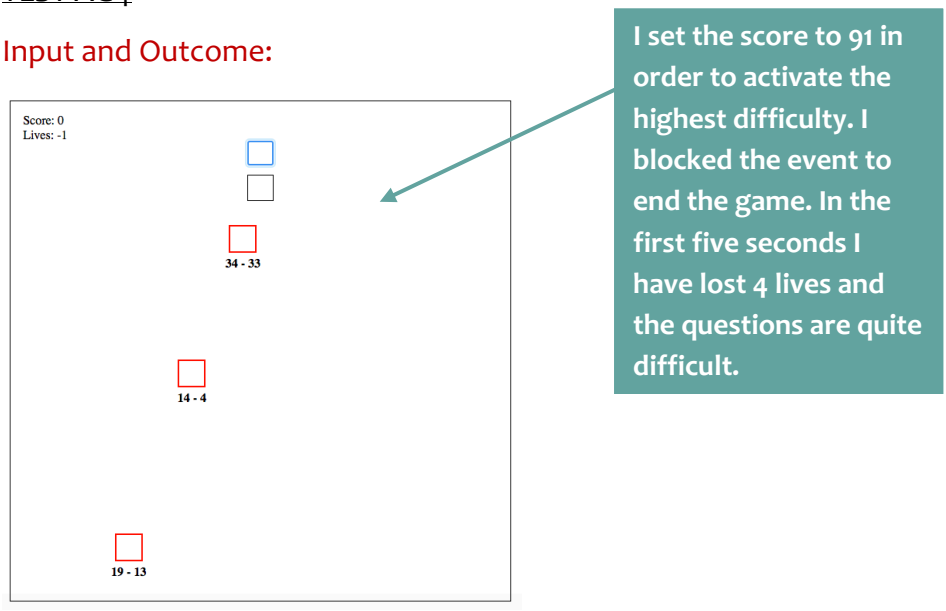


Outcome:

Test Success!

TEST AG4

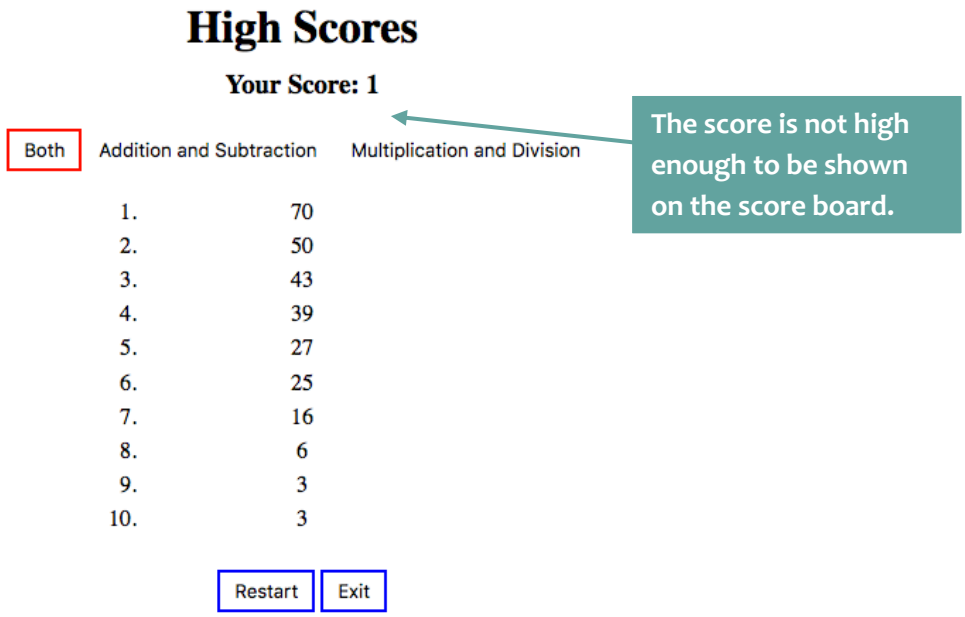
Input and Outcome:



Test Success!

TEST AG5

Input and Outcome:



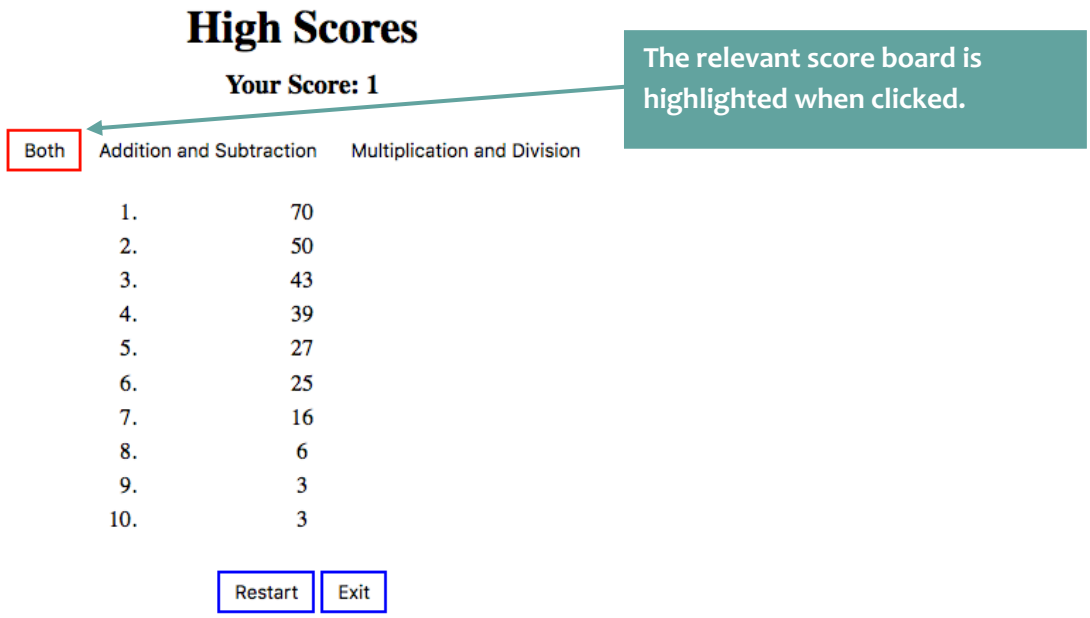
Test Success!

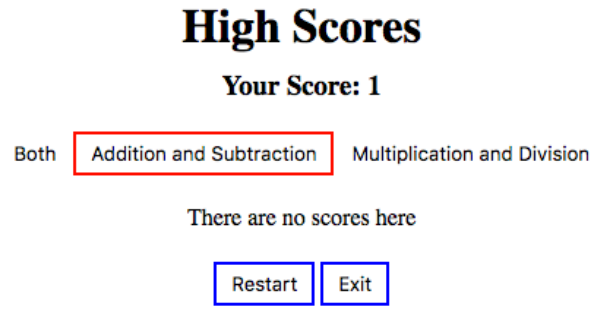
Testing the Game End

Test ID	Test Input	Expected Outcome
GE1	Click all the game mode buttons.	A top 15 list in descending order should appear for the relevant game mode.
GE2	Observe the current game mode score board	The score just attained should be on the score board if it is great enough.
GE3	Click the 'Restart' and 'Exit' buttons.	The restart button should take the user to the game menu, and the exit button takes you to the games page.

TEST GE1

Inputs and Outcomes:





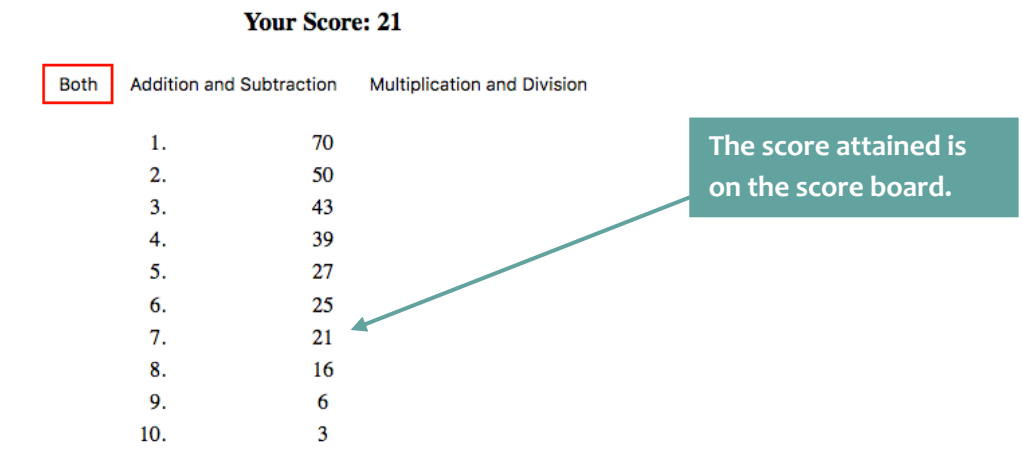
Test Success!

TEST GE2

Input:

Score: 21
Lives: 3

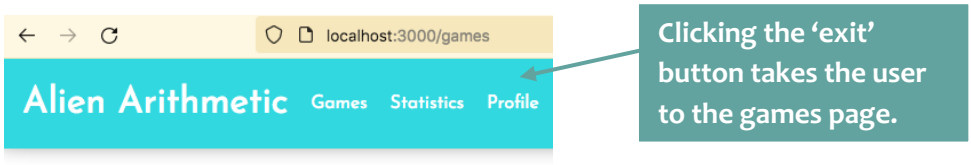
Outcome:

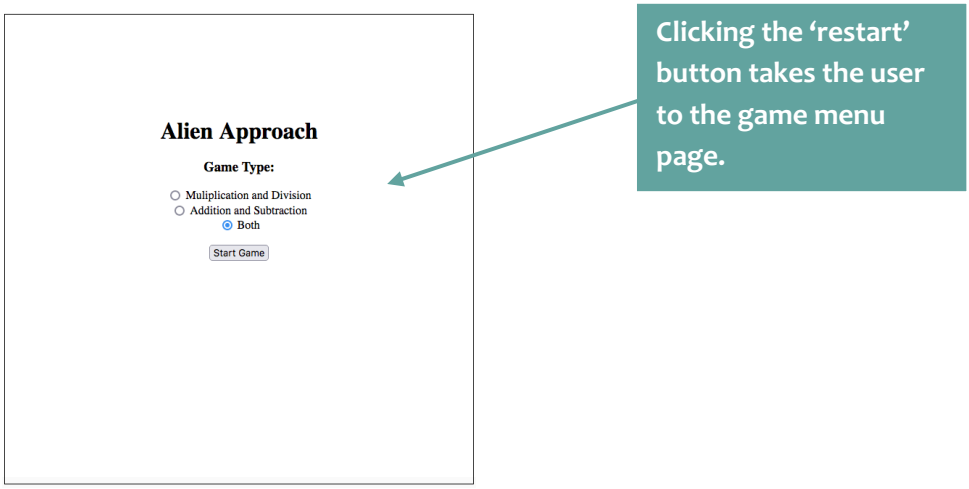


Test Success!

TEST GE3

Inputs and Outcomes:





Test Success!

Stakeholder Evaluation

The student liked the simplicity of the game menu. The game modes were a very reasonable addition that they were happy about. In fact, the idea of game modes was given to me by a student during the play testing of an early version of the game. That earlier though was too much altered for me to include in this development documentation.

The game itself had outstanding positive feedback. No one was disappointed, and it quickly became popular while they were playtesting. Although, the most common feedback I got back, after its praise, was that the game itself is extremely bland and sort boring to look at. Some also questioned the name of the game being called 'Alien Approach' since there didn't seem to be any aliens. This Is an easy thing to address for the most part. Some CSS just needs to be changed. They sort the ramp in difficulty was reasonable and that the questions were not too far-fetched. They had a little issue with the input system, but they soon came to accept that this game will also test your typing speed.

The game menu was liked, especially for the game scores being displayed for each game mode.

Overall Evaluation: Extremely Positive. The game CSS needs to be changed.

Responding to Feedback

I need to make the game look more appropriate for its name: "Alien Approach". I will do this by changing the CSS in the game.css file to the following:

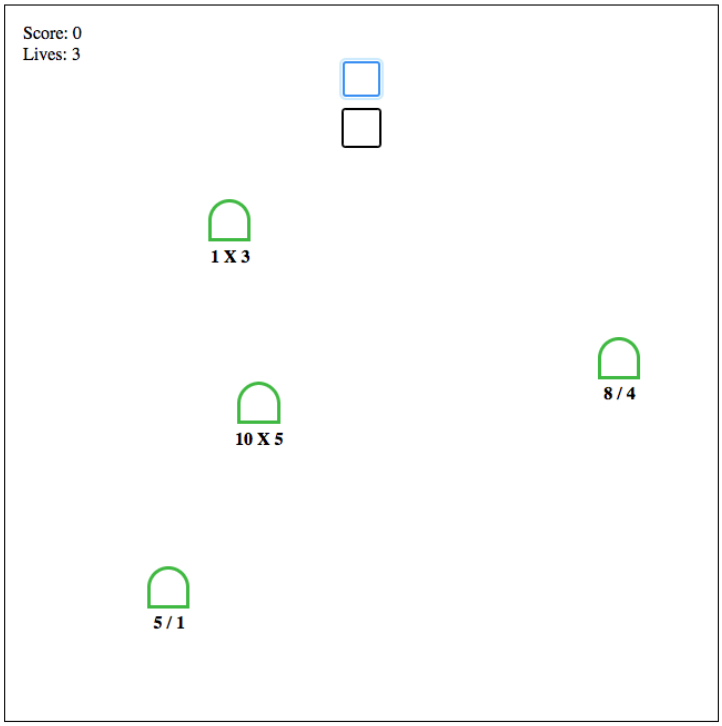
In the enemy CSS:

```
57  border: 3px solid #42bc42;
58  border-radius: 50% 50% 0 0;
```

In the player CSS:

```
33  border: 2px solid black;
34  border-radius: 10%;
```

Now the game looks like this:



More alien like if you ask me.

SETTING UP THE PAGES

In this section I will go through each page of the website, and set each one up, making them functional and also styling them.

The Home Page

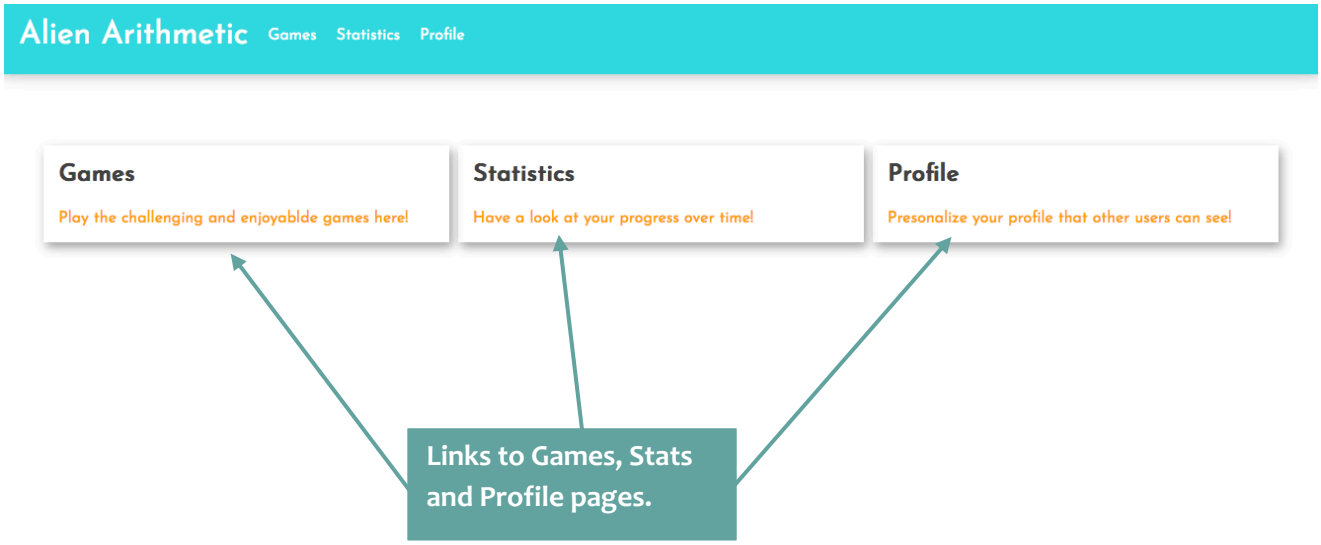
For the home page I added the following HTML under the nav bar:

```
<div class="main">
      <div class="block game">
        <!-- Container holding the link to the Games page
        also with a short description -->
        <h2>Games</h2>
        <p class="descr">
          <a href="/games" class="home-link"
            >Play the challenging and enjoyablde games here!</a
          >
        </p>
      </div>
      <div class="block stats">
        <!-- Container holding the link to the Stats page
        also with a short description -->
        <h2>Statistics</h2>
        <p class="descr">
          <a href="/stats" class="home-link"
            >Have a look at your progress over time!</a
          >
        </p>
      </div>
      <div class="block pfp">
        <!-- Container holding the link to the Profile page
        also with a short description -->
        <h2>Profile</h2>
        <p class="descr">
          <a href="/pfp" class="home-link"
            >Presonalize your profile that other users can see!</a
          >
        </p>
      </div>
    </div>
```

Then at the top of the body I add the following CSS:

```
<style>
      .main {
        /* Styles the container to
        arrange the other element on the
        screen*/
        margin: 5vh auto;
        padding: 5vh;
        padding-top: 5vh;
        display: flex;
        align-items: center;
        justify-content: space-evenly;
      }
      .block {
        /* Styles the class block div tags*/
        width: 30%;
        max-height: 5%;
        padding: 15px;
        box-shadow: 3px 5px 10px rgb(163, 163, 163);
      }

      .descr {
        margin-top: 20px;
      }
      .home-link {
        /* Turn the link tags orange to indicate they
        are clickable*/
        color: orange;
        cursor: pointer;
      }
      .home-link:hover {
        /* The links when hovered over, decrease in opacity*/
        opacity: 0.75;
      }
    </style>
```

Now going to the home page, the following is seen:

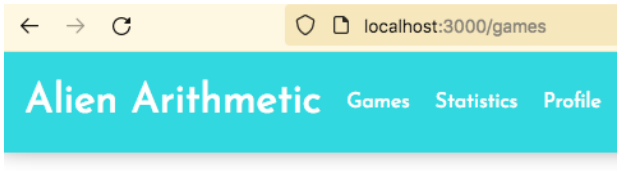


Testing the Home Page

Test ID	Test Input	Expected Outcome
H1	Click the button for Game on the nav bar and underneath the 'Games' title	User redirected to games page.
H2	Click the button for Statistics on the nav bar and underneath the 'Statistics' title	User redirected to stats page.
H3	Click the button for Profile on the nav bar and underneath the 'Profile' title	User redirected to profile page.

TEST H1

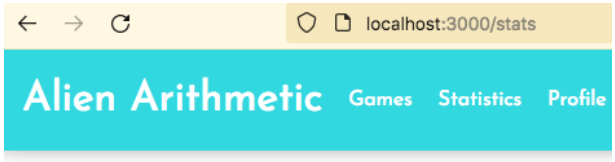
Input and Outcome:



Test Success!

TEST H2

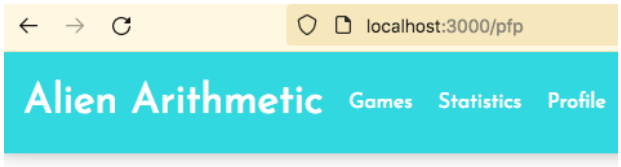
Input and Outcome:



Test Success!

TEST H3

Input and Outcome:



Test Success!

Stakeholder Evaluation

The students were alright with this page. They said it was bland and sort of useless. This criticism I take fully. On this page should have a world leader-board for all the students. This however was left out of the design and development plan. Therefore, this feature with have to wait until post development.

Overall Evaluation: Alright. No further changes needed.

The Games Page

In the game-page.html file I added the following HTML and CSS:

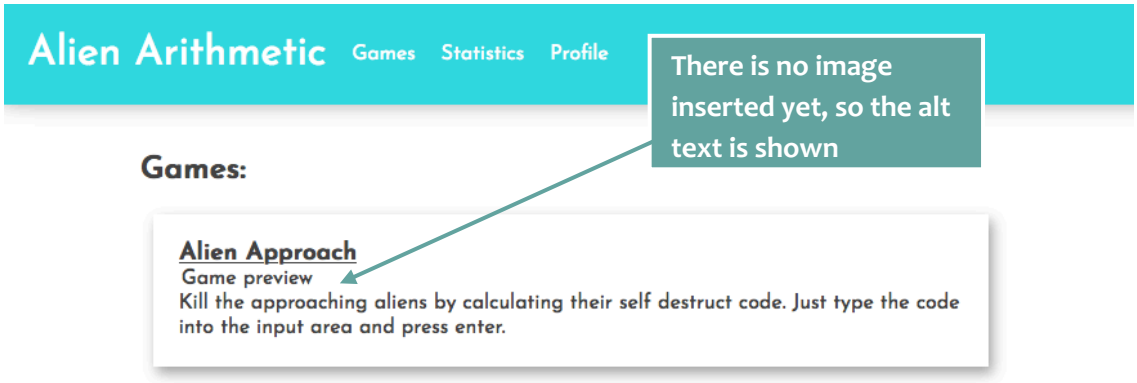
The HTML after the nav bar HTML:

```
<div class="main">
      <h2>Games:</h2>
      <div class="game-list">
        <h3 class="game-title">Alien Approach</h3>
        <!-- Shows a snippet of the game game-play. This also acts as the button to go to the game -->
        <img
          src="#"
          alt="Game preview"
          class="game-preview"
          onclick="toGame()"
        />
        <!-- Gives a short description of how to play the game -->
        <p class="game-description">
          Kill the approaching aliens by calculating their self destruct code.
          Just type the code into the input area and press enter.
        </p>
      </div>
    </div>
```

The CSS at the top of the body:

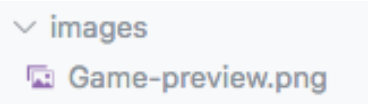
```
<style>
  .main {
    margin: 0 auto;
    padding: 15vh;
    padding-top: 5vh;
  }
  .game-list {
    /* Styles the container holding the list of games*/
    margin: 20px 0;
    margin-left: 10px;
    padding: 20px;
    width: 50vw;
    box-shadow: 3px 5px 10px rgb(163, 163, 163);
  }
  .game-preview {
    /* Styles the game play snippet*/
    width: 300px;
    margin: 20px auto;
    cursor: pointer;
  }
  .game-title {
    text-decoration: underline;
  }

  img {
    border: 2px solid white;
  }
  img:hover {
    /*When you hover over the image, the border goes orange */
    border-color: orange;
  }
</style>
```

The page now looks like such:



Now to insert the link the image in the source tag:

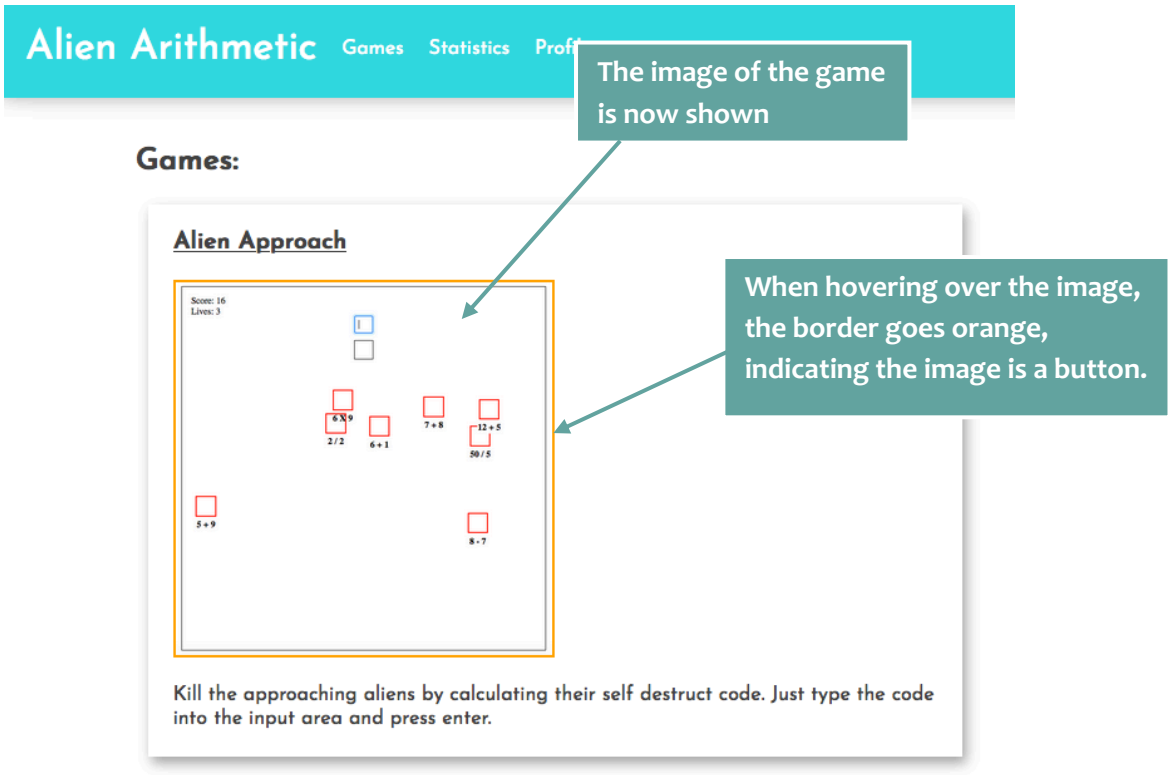
To do this, the image must be stored on the server. Therefore, in the public folder I added a new folder called images with a screen shot of the game being played:



Inside the HTML image tag:

```
56   src="http://localhost:3000/images/Game-preview.png"
```

Now refreshing the page, you can see the following:



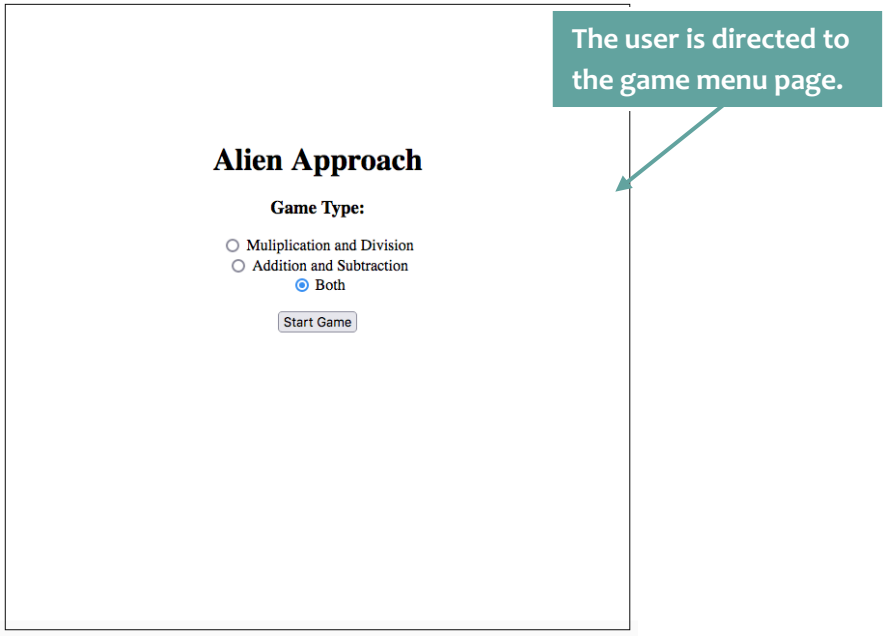
To make the image a button, simply include the onclick event listener to the image tag in the HTML:

```
59  onclick="toGame()"
```

In the script tags is the toGame function:

```
68  <script>
69        function toGame() {
70          //Directs the user to the game menu
71          location.href = "/games/game-menu";
72        }
73      </script>
```

Refreshing the page and testing by clicking the image:



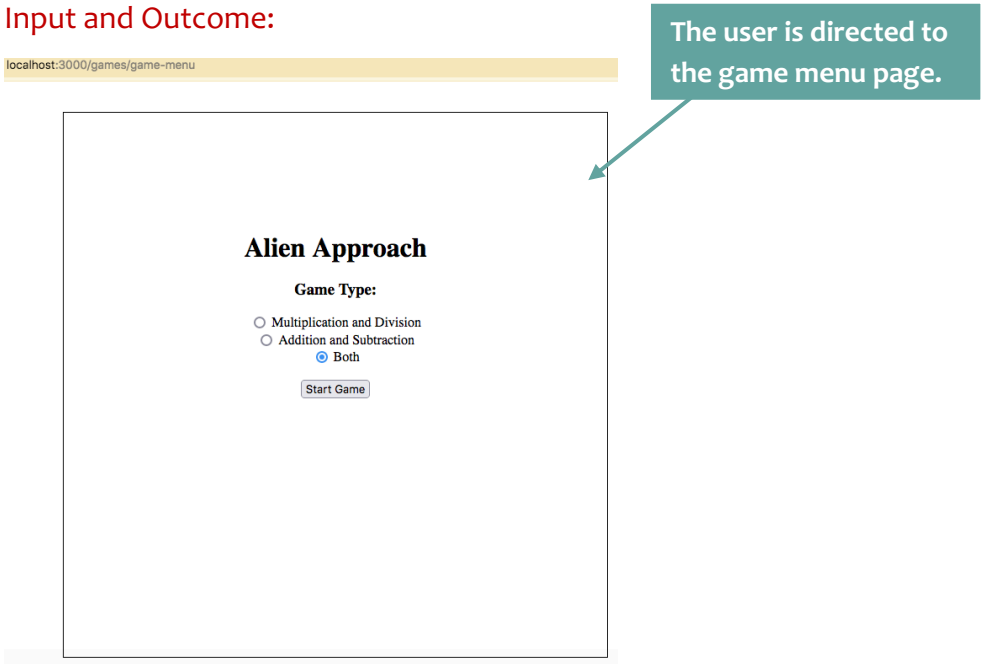
TEST SUCCESS! The games page is now working and fully functional.

Testing the Games Page

Test ID	Test Input	Expected Outcome
G1	Click the game preview picture.	User redirected to game menu page.
G2	Hover cursor over the game preview.	The border changes colour to orange.

TEST G1

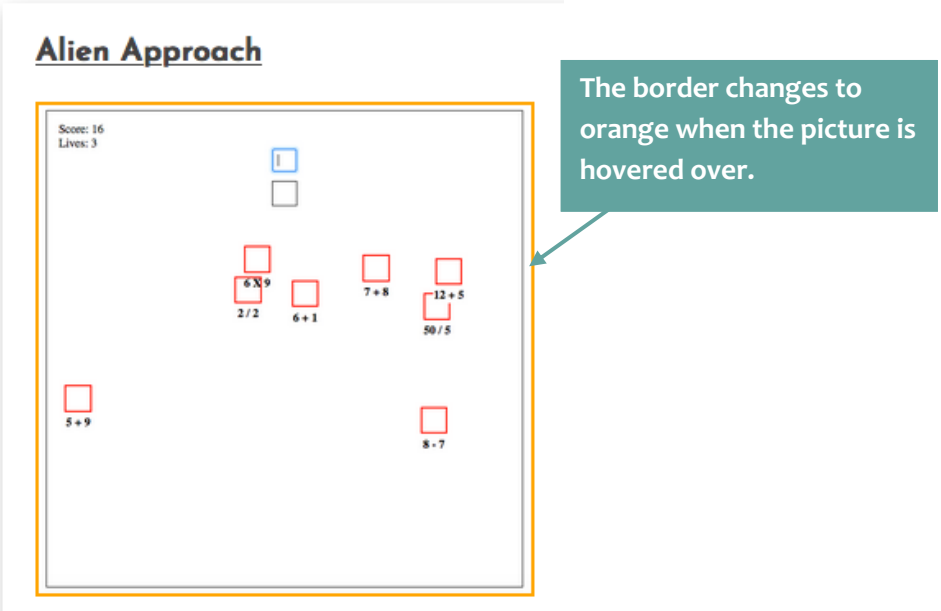
Input and Outcome:



Test Success!

TEST G2

Input and Outcome:



Test Success!

Stakeholder Evaluation

The students liked the page. They really look forward to seeing a longer list of games with more variety and different challenges displayed and listed, but they fully understand the challenge of creating a game and didn't push for another one to be made.

Overall Evaluation: Positive. No further changes needed.

The Stats Page

Inside the stats page I put the following HTML under the nav bar:

```
<div class="main">
      <!-- Creates the div to hold everything -->
      <div class="block">
        <h2>Both</h2>
        <!-- Creates the container to hold the graph -->
        <div class="container">
          <!-- y axis label-->
          <p class="y-axis">Score</p>
          <!-- This is where the graph is displayed -->
          <canvas id="Both" class="graph"></canvas>
        </div>
        <br />
        <div class="attempt">
          <!-- x axis label -->
          <p class="x-axis">Attempt Number</p>
        </div>
      </div>
      <!-- All the comments above apply exactly below -->
      <div class="block">
        <h2>Multiplication and Division</h2>
        <div class="container">
          <p class="y-axis">Score</p>
          <canvas id="MD" class="graph"></canvas>
        </div>
        <br />
        <div class="attempt">
          <p class="x-axis">Attempt Number</p>
        </div>
      </div>
      <div class="block">
        <h2>Addition and Subtraction</h2>
        <div class="container">
          <p class="y-axis">Score</p>
          <canvas id="AS" class="graph"></canvas>
        </div>
        <br />
        <div class="attempt">
          <p class="x-axis">Attempt Number</p>
        </div>
      </div>
    </div>
```

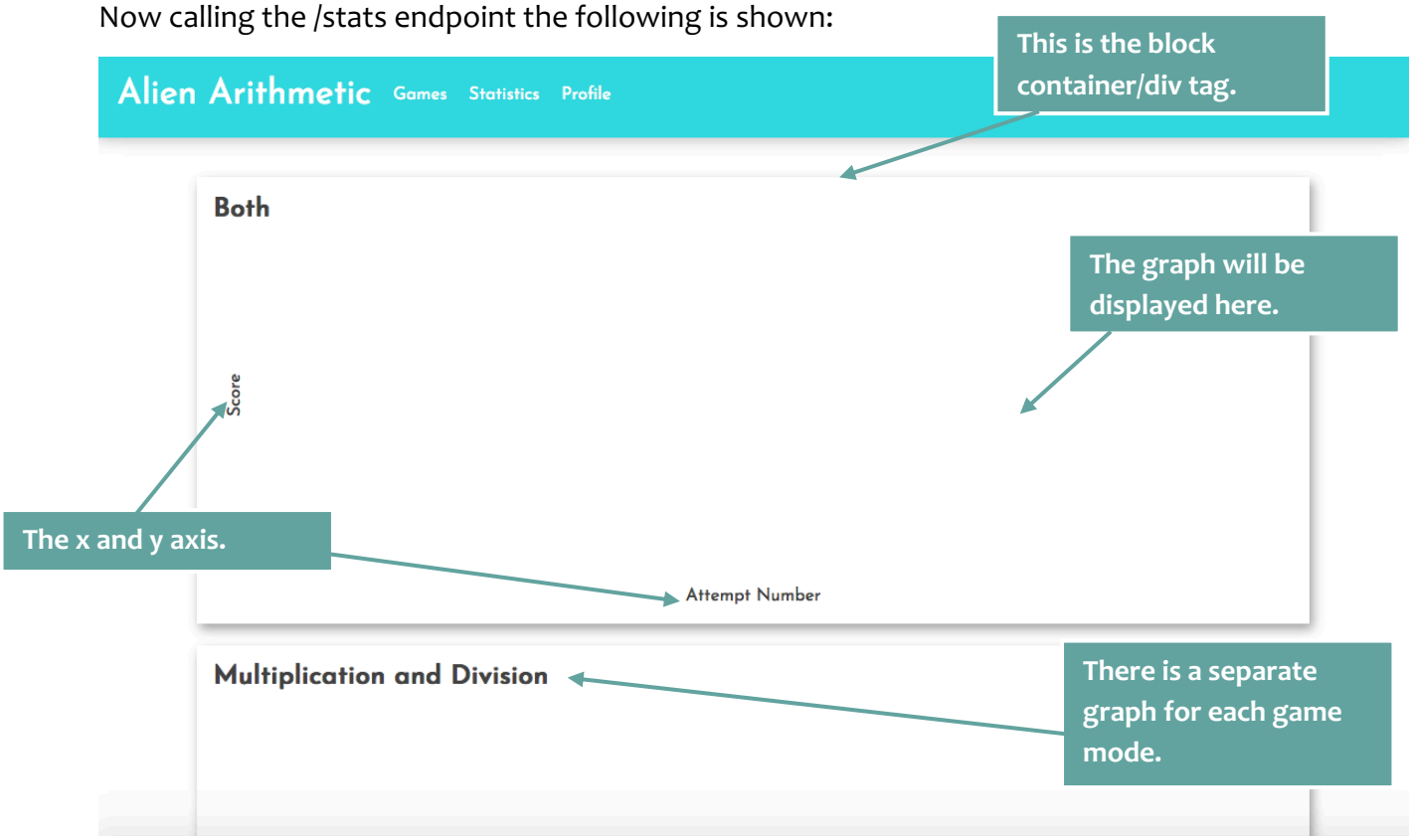
Then I added the following CSS in the style tags at the top of the body:

```
<style>
      .main {
        /* Styles the container to align all the
        children elements */
        margin: 0 auto;
        padding: 15vh;
        padding-top: 5vh;
        display: flex;
        flex-direction: column;
        align-items: center;
      }
```

CSS continues:

```
21        .block {
22          /* Styles the container to contain the graph
23          and everything else*/
24          width: 100%;
25          padding: 15px;
26          margin-bottom: 20px;
27          box-shadow: 3px 5px 10px rgb(163, 163, 163);
28        }
29        .graph {
30          /* Styles the graph*/
31          margin-top: 10px;
32          width: 100%;
33          color: rgb(255, 165, 0);
34          height: 100%;
35          max-height: 300px;
36        }
37        .container {
38          /* This aligns the graphs to the center*/
39          display: flex;
40          align-items: center;
41          width: 90%;
42        }
43        .y-axis {
44          /* Styles the y axis */
45          transform-origin: 50%;
46          transform: rotate(-90deg);
47        }
48        .attempt {
49          /* Styles the x axis*/
50          text-align: center;
51        }
52      </style>
```

Now calling the /stats endpoint the following is shown:



Now I add the JavaScript to display the graphs. At the bottom of the body, add the following:

```
<!-- Imports the graph libary -->
    <script src="https://cdnjs.cloudflare.com/ajax/libs/Chart.js/3.1.1/chart.min.js"></script>
    <script>

      function makeChart(data, canvas, title) {
        //Creates a graph with the given data and title
        //in the specified canvas
        let xValues = [];
        if (data[0] !== 0) {
          //Adds 0 to the front of the data if 0 is not already present
          data.unshift(0);
        }
        const yValues = data;
        for (let i = 0; i < yValues.length; i++) {
          //Makes the xValues list
          xValues.push(i);
        }

        const chart = new Chart(canvas, {
          //Creates the chart
          type: "line",
          data: {
            labels: xValues,
            datasets: [
              {
                //Styles the chart and inputs the data
                label: title,
                pointRadius: 4,
                pointBackgroundColor: "rgb(255, 165, 0)",
                backgroundColor: "rgb(50 216 223)",
                borderColor: "rgb(50 216 223)",
                data: yValues,
              },
            ],
          },
        });
      }
    </script>
```

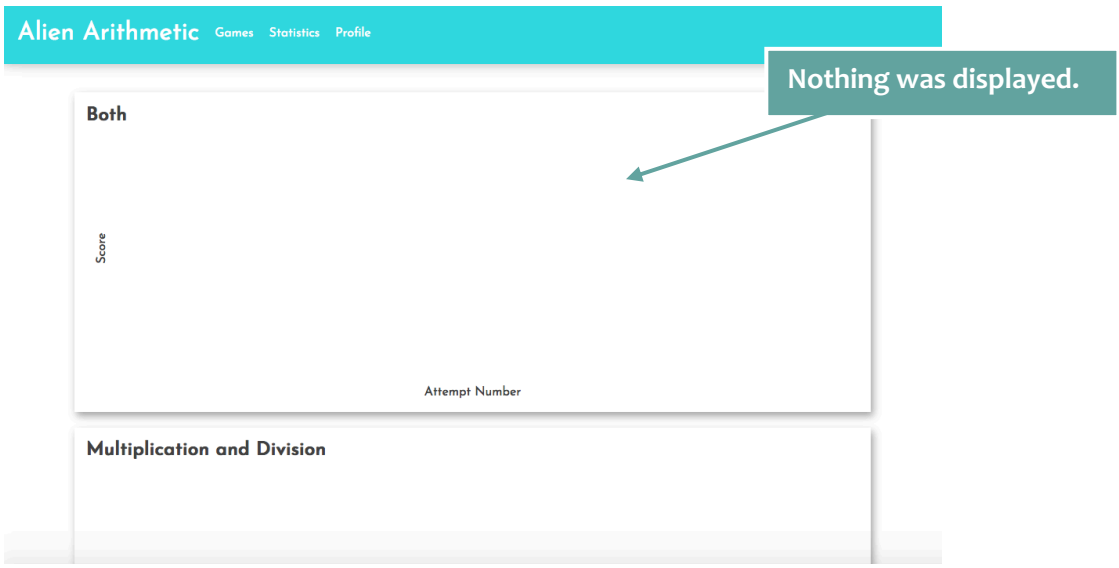
To get the data I must access the database. So, here I make the endpoint in the stats.js file to do just that:

```
//Sets up an endpoint with the GET method on /stats/game-scores
route.get("/game-scores", (req, res) => {
  const email = req.body.email;
  //Returns the scores of the user
  Score.findOne({ email: email })
    .exec()
    .then((data) => {
      res.json({
        message: data,
      });
    });
});
```

Now in the stats.html file I add the JavaScript to call this endpoint and display the data:

```
105  //Sends a GET request to the server in order to get back the scores for the user
106        fetch("http://localhost:3000/stats/game-scores", {
107          body: JSON.stringify({
108            email: "doctor@who.com"
109          })
110        })
111          .then((res) => {
112            return res.json();
113          })
114          .then((data) => {
115            const content = data.message;
116            //Displays the data
117            makeChart(content.typeBoth, "Both", "Both types");
118            makeChart(content.typeMD, "MD", "Multiplication and Division Data");
119            makeChart(content.typeAS, "AS", "Addition and Subtraction Data");
120          });
```

Now to test by saving and refreshing the /stats endpoint:



TEST FAILED!

In the Firefox console the following error message was shown:

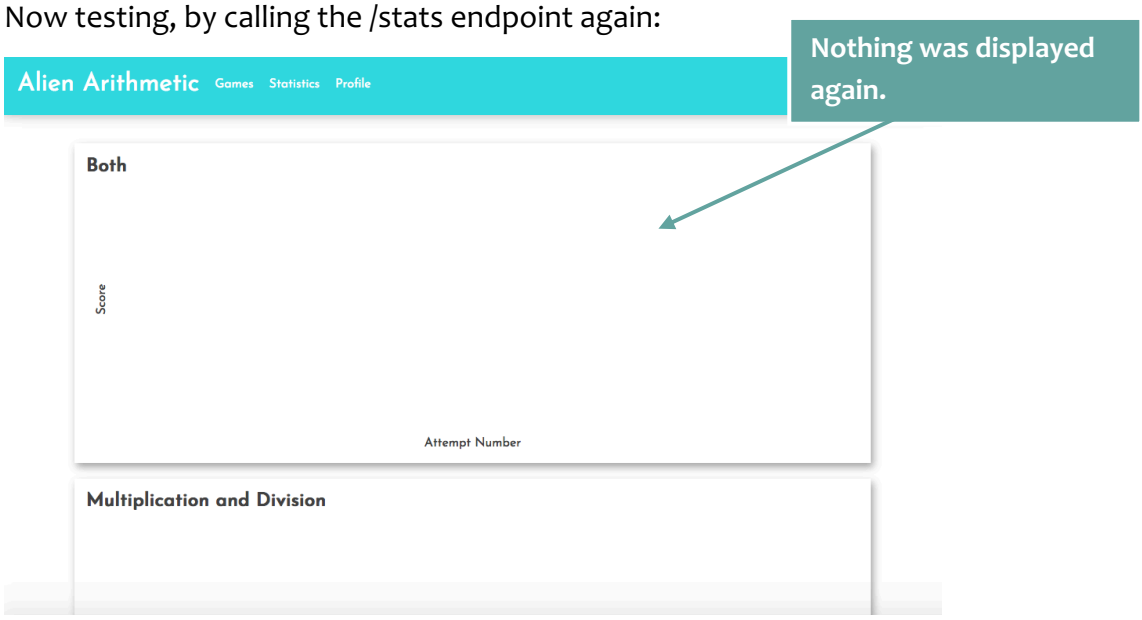
```
Uncaught (in promise) TypeError: Window.fetch: HEAD or GET Request stats:106:12
cannot have a body.
    <anonymous> http://localhost:3000/stats:106
```

This is saying that a GET request cannot have a body. At the moment, sending the email manually is temporary. I will, at some point, be able to do this automatically. While this is not the case, I will change the method on the endpoint and in the fetch to be POST as follows:

```
106  fetch("http://localhost:3000/stats/game-scores", {
107          method: "POST",
```

```
21  route.post("/game-scores", (req, res) => {
```

Now testing, by calling the /stats endpoint again:

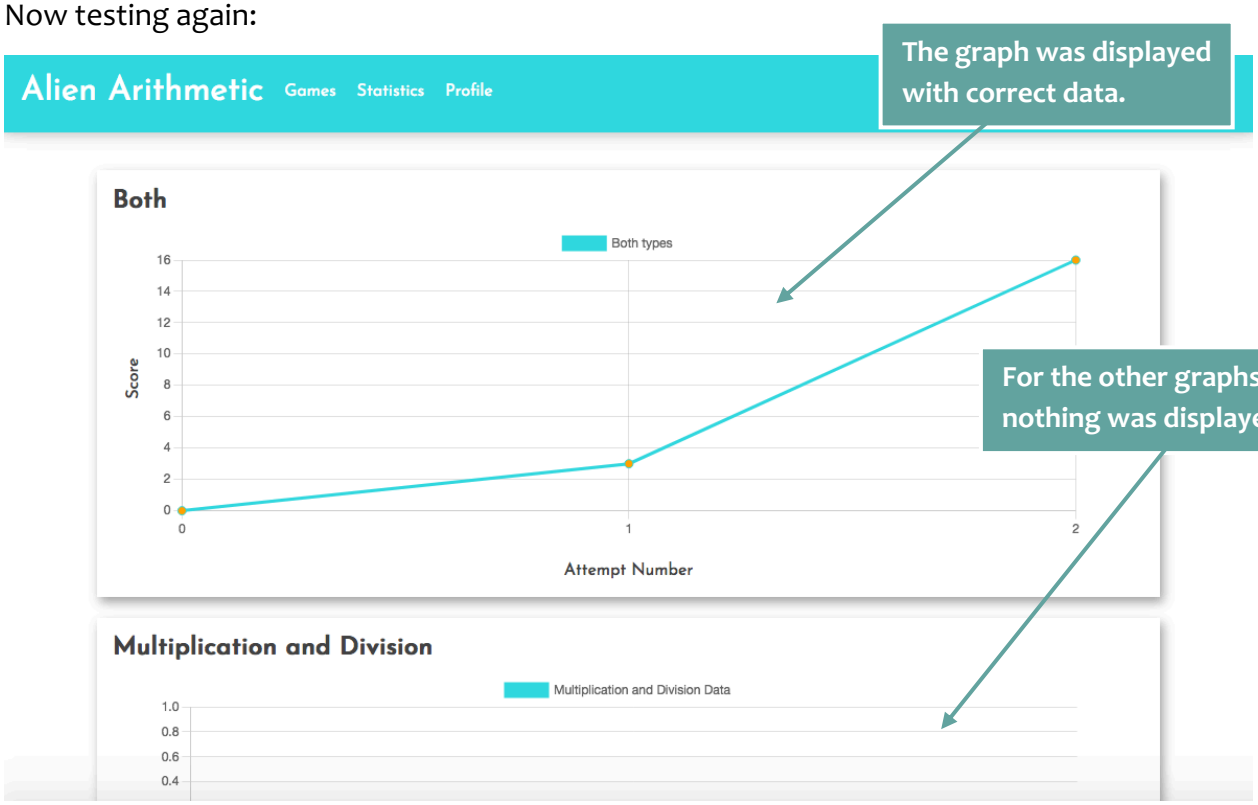


TEST FAILED!

I forgot to define the headers in the fetch algorithm:

```
111  headers: {
112              "Content-type": "application/json; charset=UTF-8",
113          }
```

Now testing again:



TEST SUCCESS!

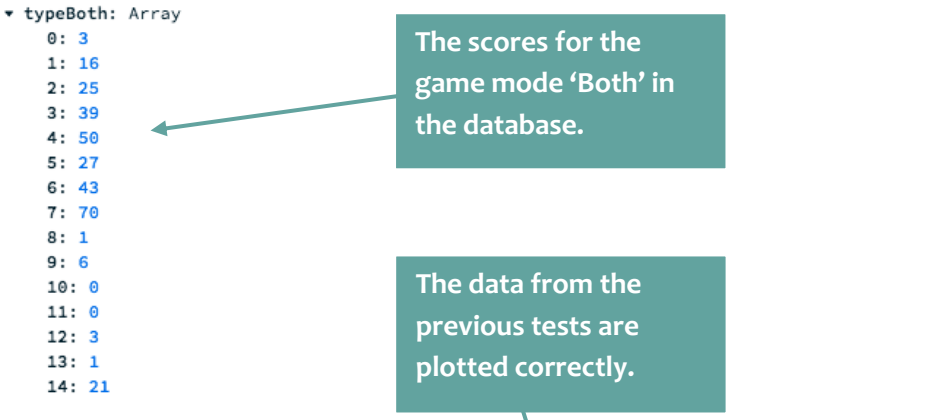
Everything is working fine.

Testing the Stats Page

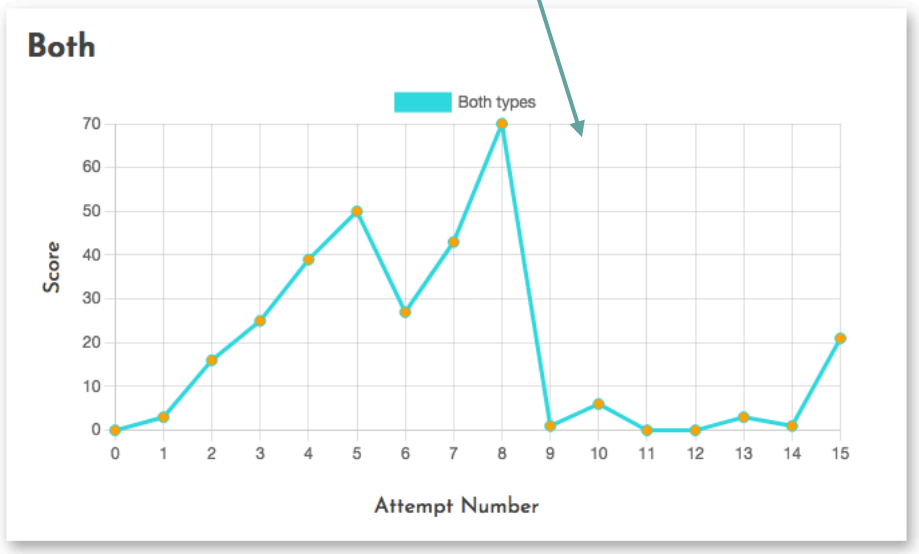
Test ID	Test Input	Expected Outcome
S1	Play a game and come back to the stats page.	The new data point with the correct score and attempt number should appear.
S2	Observe the graphs.	The data should all be correctly placed.

TEST S1 and S2:

Input:



Outcome:



Test Success!

Stakeholder Evaluation

The students liked the graphs very much. As more scores are plotted, the trends that students see can really feel motivating as they start to improve their metal math skills. They said other graphs would be nice, but they are not needed.

Overall Evaluation: Positive. No further changes needed.

The Profile Page

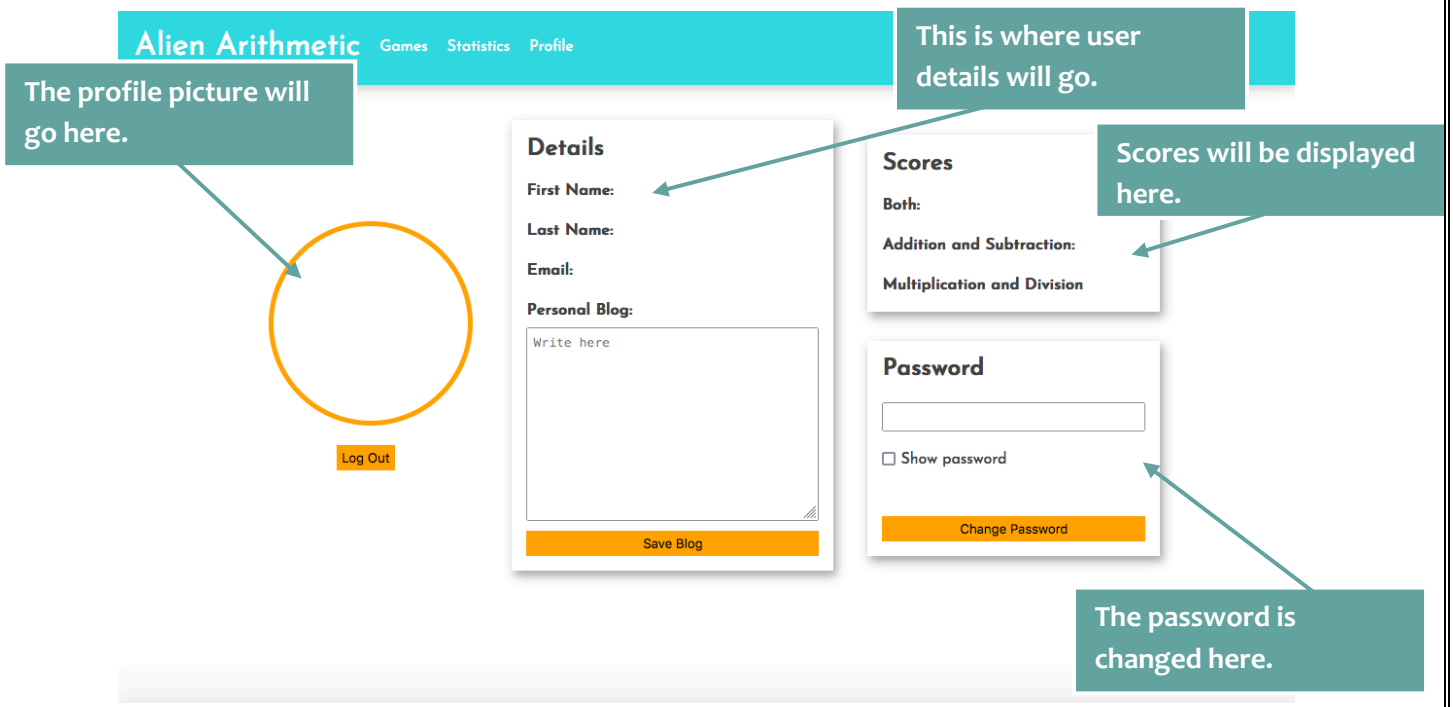
In the pfp.html file I added the following html under the nav bar:

```
<div class="main">
      <!-- Div that contains everything -->
      <div class="block image">
        <!-- Div that contains the profile picture -->
        <div class="img">
          <p id="initials"></p>
        </div>
        <!-- Button to log out -->
          <button class="log-out" onclick="">Log Out</button>
      </div>
      <div class="column one">
        <!-- This div contains the details of the user -->
        <h2>Details</h2>
        <br>
        <h4 >First Name:</h4>
        <p class="firstname detail"></p>
        <br>
        <h4 >Last Name:</h4>
        <p class="lastname detail"></p>
        <br>
        <h4 >Email:</h4>
        <p class="email detail"></p>
        <br>
        <h4>Personal Blog:</h4>
        <textarea class="blog" cols="30" rows="10" placeholder="Write here"></textarea>
        <!-- Button to save the blog to the database -->
        <button class="save-blog" onclick="">Save Blog</button>
      </div>

      <div class="column two">
        <div class="block scores">
          <!-- This div shows the scores of the user -->
        <h2>Scores</h2>
        <br>
        <h4 >Both:</h4>
        <p class="typeBoth detail score"></p>
        <br>
        <h4 >Addition and Subtraction:</h4>
        <p class="typeAS detail score"></p>
        <br>
        <h4 >Multiplication and Division</h4>
        <p class="typeMD detail score"></p>
      </div>

      <div class="block password-block">
        <!-- This div contains the area to change the user's password -->
        <h2>Password</h2>
        <br>
        <input type="password" class="password">
        <br><br>
        <!-- Check box to toggle password visibilty -->
        <input type="checkbox" onclick="" id="show">
        <label for="show">Show password</label>
        <br><br>
        <p id="passwordError" class="error"></p>
        <br>
        <!-- Button to save the new password to the database -->
        <button class="change-password" onclick="">Change Password</button>
      </div>
      </div>
      </div>
    </div>
```

At the top of the body, I add the following CSS:

```
<style>
      .main {
        /* Arranges the elements on the page */
        margin: 0 auto;
        padding: 15vh;
        padding-top: 5vh;
        display: flex;
        align-items: center;
        justify-content: space-evenly;
      }
      .column {
        /* Styles the column classes */
        width: 30%;
      }
      .password {
        margin-top: 10%
      }
      .one {
        /* Styles column one*/
        box-shadow: 3px 5px 10px rgb(163, 163, 163);
        padding: 15px;
      }
      .block {
        /* Styles the block classes */
        padding: 15px;
        box-shadow: 3px 5px 10px rgb(163, 163, 163);
      }
      .image {
        box-shadow: 0px 0px;
        width: 20%;
      }
      .img {
        /* Styles the image */
        margin: 0 auto;
        border: 5px solid orange;
        border-radius: 50%;
        width: 200px;
        height: 200px;
        display: flex;
        align-items: center;
        justify-content: center;
      }
      #initials {
        /* Increases the font size of the initials */
        font-size: 50px;
      }
      button {
        /* Sets the general css for buttons*/
        border: 0;
        background-color: orange;
        padding: 5px;
      }
      .save-blog, .change-password {
        margin-top: 10px;
        width: 100%
      }
      .log-out {
        /* Styles the log out button */
        margin-top: 20px;
        position: relative;
        left: 50%;
        transform: translateX(-50%);
      }
      button:hover {
        /* When buttons are hovered over
        they decrease in opacity*/
        opacity: 0.75;
      }
      #blog {
        width: 100%;
        padding: 5px;
        margin-top: 5px;
      }
      h4 {
        margin-bottom: 2.5px;
      }
      #password {
        width: 100%;
        padding: 5px
      }
      .error {
        /* Styles the error message from
        the change password field */
        color: red;
      }
    </style>
```

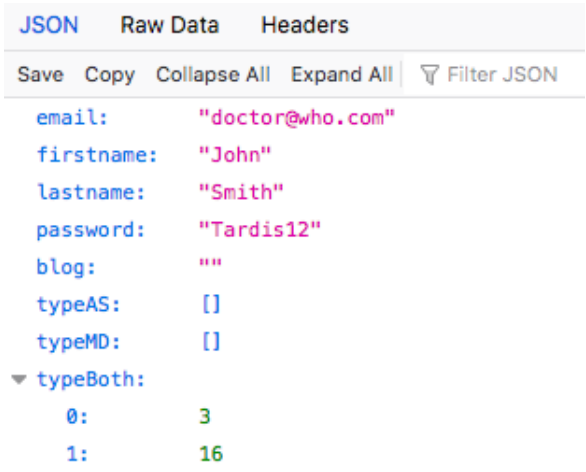
Now calling the /pfp endpoint the following is shown:



Now to write the code to display all of the details of the user. To do this I must make an endpoint to get the data from the database. In the pfp.js file:

```
21  //Sets up an endpoint with the GET method on /pfp/info
22  route.get("/info", (req, res) => {
23    //For now the email is given in manually
24    let email = "doctor@who.com";
25    User.findOne({ email: email })
26      .exec()
27      .then((userData) => {
28        Score.findOne({ email: email })
29          .exec()
30          .then((scoreData) => {
31            //Get the necessary data from the return data
32            //from the database
33            const { email, blog, firstname, lastname, password} = userData;
34            const { typeBoth, typeMD, typeAS } = scoreData;
35            res.json({
36              email: email,
37              firstname: firstname,
38              lastname: lastname,
39              password: password,
40              blog: blog,
41              typeAS: typeAS,
42              typeMD: typeMD,
43              typeBoth: typeBoth,
44            });
45          });
46      });
47  });
```

Testing this endpoint by calling it in the URL:



TEST SUCCESS! The correct data is given back.

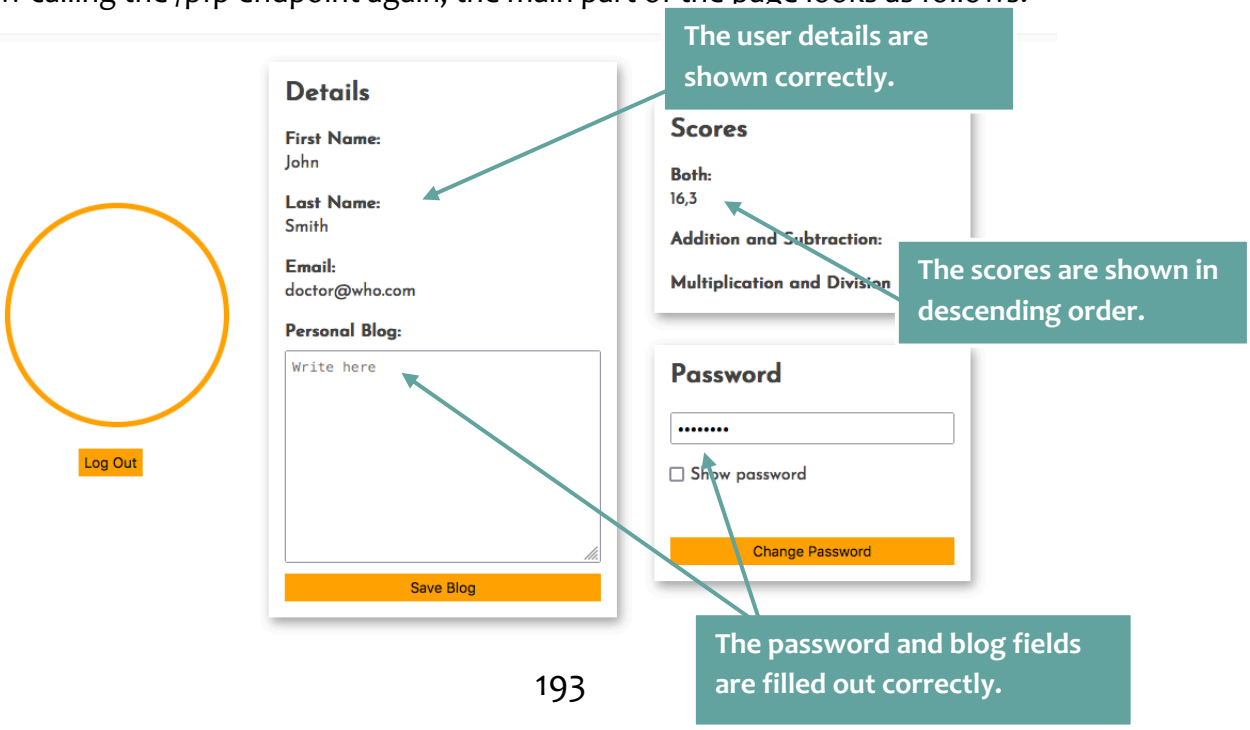
Now in the pfp.html file, I use fetch to get this data and display it on the screen:

```
//Sends a GET request to the server in order to display
      //the users details on the screen
      fetch("http://localhost:3000/pfp/info")
      .then((res) => {
        return res.json()
      })
      .then((data) => {
        for (let key in data) {
          //For each detail of the user, do the below
          const detail = document.querySelector(`.${key}`)
          if( key == "password" || key == "blog") {
            //For the password and blog add it to the
            //display this way
            detail.value = data[key]
          } else {
            //For all the other details do show it this way
            if (key.slice(0, 4) == "type") {
              //If the data is a score type then sort the scores
              //and then add it to the DOM
              let sortedScores = sortScores(data[key])
              detail.textContent = sortedScores
            } else {
             detail.textContent = data[key]
            }
          }
        }
      })
```

The sortScores function:

```
function sortScores(array) {
        //Sorts an array of integer values into descending
        //order
        const sorted = array.sort(function (a, b) {
        return b - a;
        });
        return sorted
      }
```

Now calling the /pfp endpoint again, the main part of the page looks as follows:



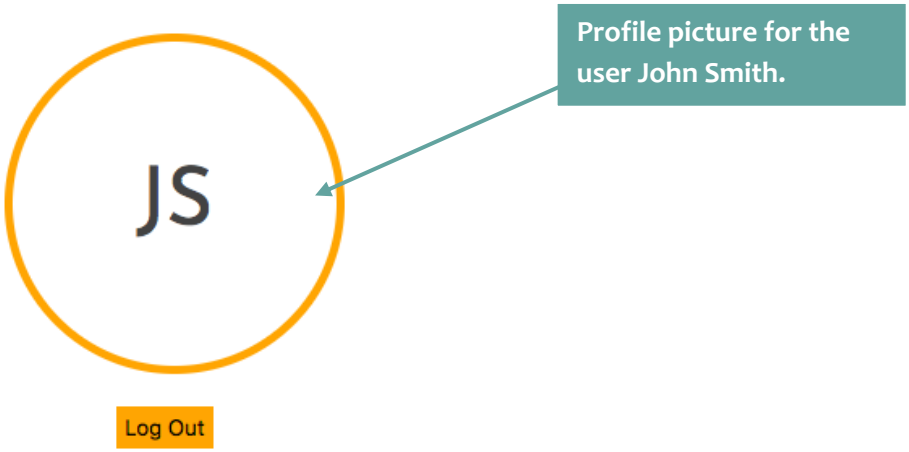
Now to make the profile picture. Since accessing a file from the local computer of the user is a complicated ordeal, the profile picture will automatically be made as being the initials of the user. Here is the function to do that:

```
function makePFP(obj) {
        //Makes the profile picture as the initials
        //of the user
        const firstInitial = obj.firstname.slice(0,1)
        const lastInitial = obj.lastname.slice(0,1)
        const pfp = document.getElementById("initials")
        pfp.textContent = firstInitial + lastInitial
      }
```

Inside the fetch algorithm, I added this function in as follows:

```
.then((data) => {
          makePFP(data)
          for (let key in data) {
```

Now refreshing the stats page and observing the profile picture area:



TEST SUCCESS! This worked.

Here I make the changing password and saving blog functions work. To change the password, I must make a separate endpoint to update the data in the database. Here is the endpoint in the pfp.js file:

```
//Sets up an endpoint with the PUT method on /pfp/change-password
route.put("/change-password", (req, res) => {
  //For now the email is given in manually
  let email = "doctor@who.com";
  const { password } = req.body;
  //Updates the password for the given user
  User.findOneAndUpdate(
    { email: email },
    { password: password },

  )
    .exec()
    .then(() => {
      res.json({
        //If the validation is a success, do the
        //send back a positive response
        message: "Password Changed",
        state: true,
      });
    })
    .catch((err) => {
      //If the validation is a failure, do the
      //send back a negative response with the error
      //message
      res.json({
        message: err.message,
        state: false,
      });
    });
});
```

In the pfp.html file I made a function that is called when the 'change password' button is clicked:

```
function changePassword() {
        //Sends a PUT request to the server in order to change the password
        const password = document.querySelector(".password")
        fetch("http://localhost:3000/pfp/change-password", {
          method: "PUT",
          body: JSON.stringify({
            password: password.value
          }),
          headers: {"Content-type": "application/json; charset=UTF-8"}
        })
        .then((res) => {
          return res.json()
        })
        .then((data) => {
          const errorMsg = document.getElementById("passwordError")
          const changeButton = document.querySelector(".change-password")
          if (data.state) {
            //IF the response is positive, then confirm the password
            //change on the DOM
            changeButton.textContent = data.message
          } else {
            //Else, put display the error message
            errorMsg.innerText = data.message
          }
        })
      }
```

In the HTML:

```
161   <button class="change-password" onclick="changePassword()">Change Password</button>
```

I also made the 'Show password' checkbox work with the following changes:

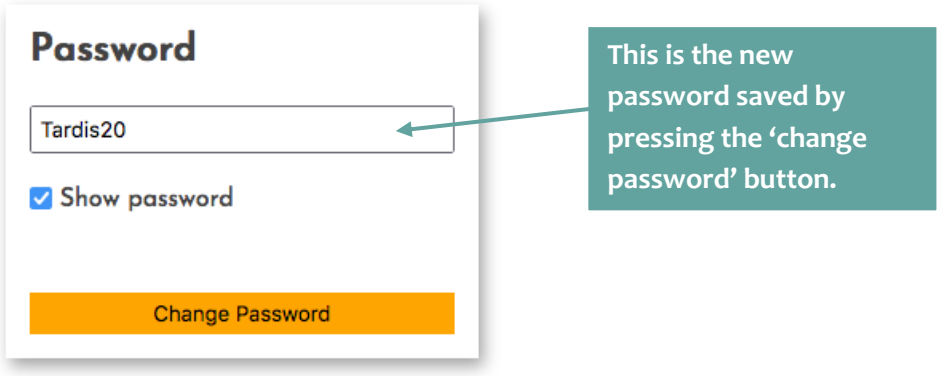
In JavaScript:

```
167   function showPassword() {
168           //Function to toggle passsword visability
169           const password = document.querySelector(".password")
170           if (password.type === "password") {
171             password.type = "text"
172           } else {
173             password.type = "password"
174           }
175         }
```

In HTML:

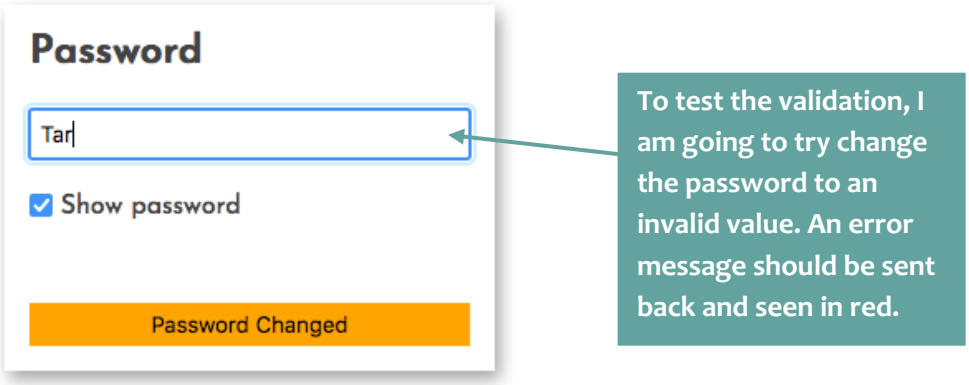
```
155   <input type="checkbox" onclick="showPassword()" id="show">
```

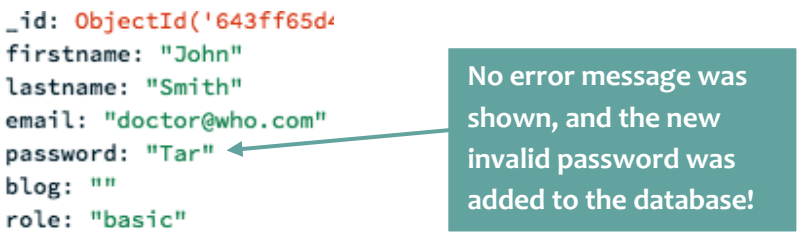
Now testing the changing password function:




```
_id: ObjectId('643ff65d4
firstname: "John"
lastname: "Smith"
email: "doctor@who.com"
password: "Tardis20"
blog: ""
role: "basic"
```

Now in the database, the password has changed to the new password.





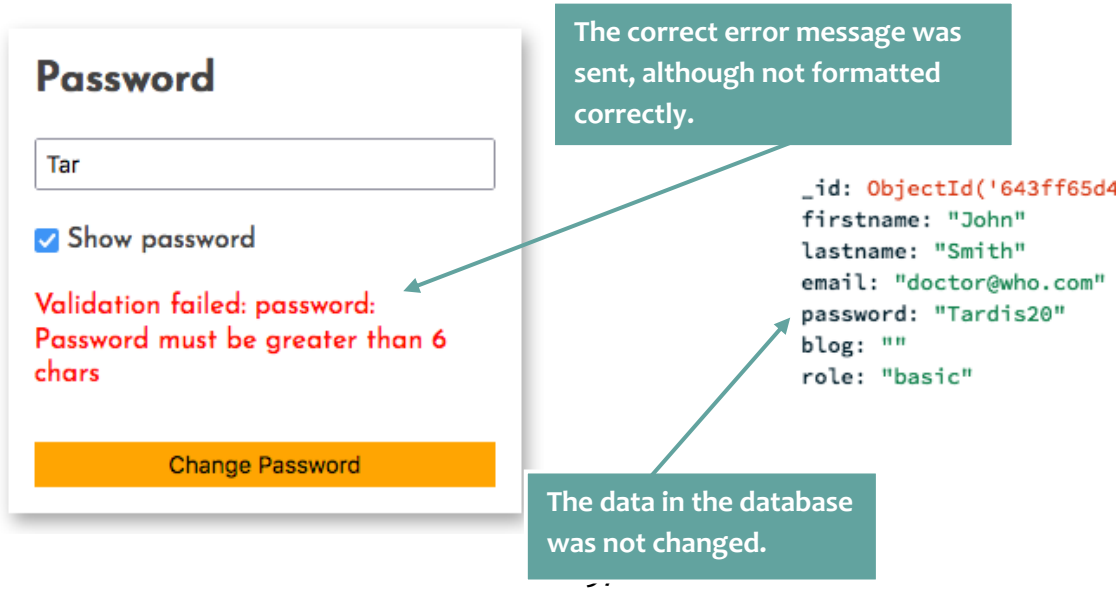
TEST FAILED!

Here was the problem:

When updating the database, no validation is done on the new value as a default response. Therefore, invalid values can put in the database. To solve this, I looked online and found the parameter to enforce validation on updates. I changed the /pfp/change-password endpoint as follows:

```
55  User.findOneAndUpdate(
56      { email: email },
57      { password: password },
58      { runValidators: true }
59    )
```

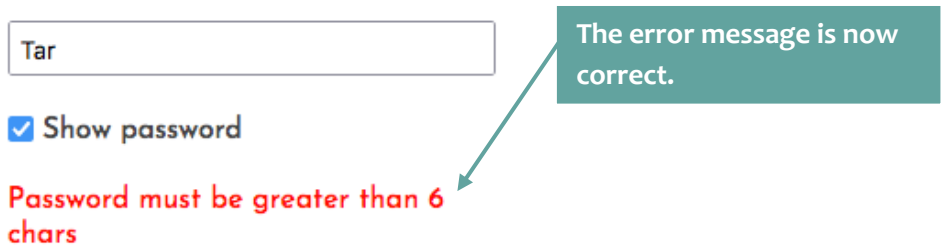
Now trying to input the same data after resetting to the previous database data:



TEST SUCCESS!

Here is what I added in the fetch algorithm to display the error message correctly:

```
270   //Else, display the error message
271   errorMsg.innerText = data.message.slice(29)
```



Now to tackle the saving blog function.

Here is the endpoint in the pfp.js file to update the blog upon being called:

```
80  //Sets up an endpoint with the PUT method on /pfp/save-blog
81  route.put("/save-blog", (req, res) => {
82    //For now the email is given in manually
83    let email = "doctor@who.com";
84    const { blog } = req.body;
85    //Updates the password for the given user
86    User.findOneAndUpdate(
87      { email: email },
88      { blog: blog },
89      { runValidators: true }
90    )
91      .exec()
92      .then(() => {
93        res.json({
94          //If the validation is a success, do the
95          //send back a positive response
96          message: "Blog saved",
97          state: true,
98        });
99      })
100     .catch((err) => {
101       //If the validation is a failure, do the
102       //send back a negative response with the error
103       //message
104       res.json({
105         message: err.message,
106         state: false,
107       });
108     });
109 });
```

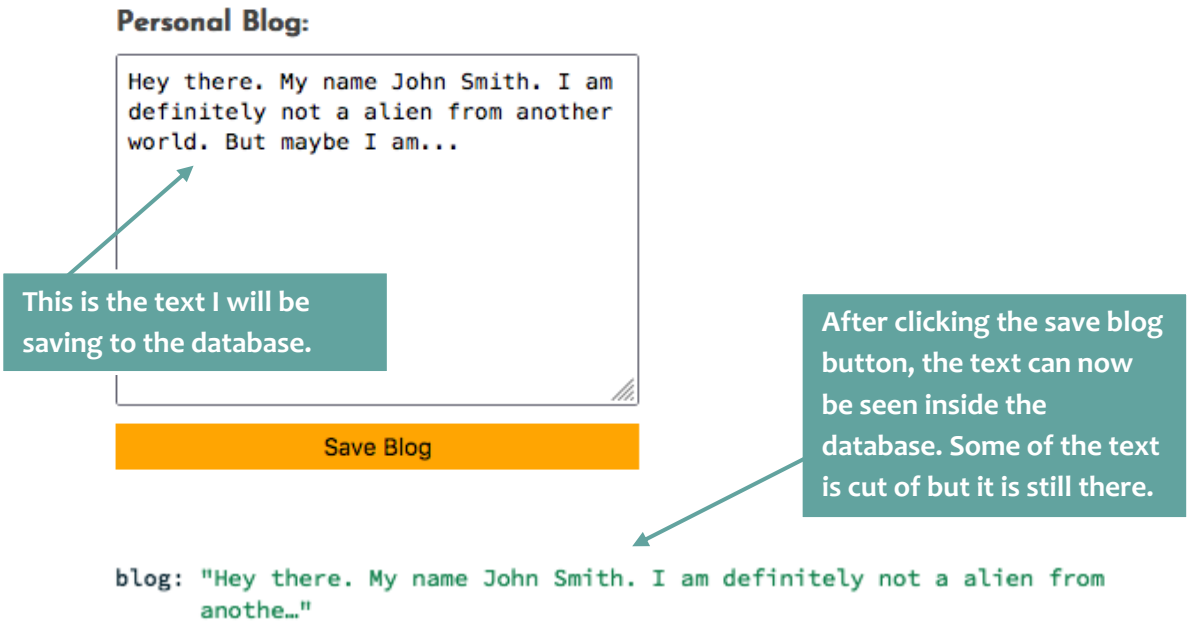
Here is the function to call this endpoint hence, saving the blog:

```
function saveBlog() {
        //Function that saves the blog text to the database
        const blog = document.querySelector(".blog")
        const blogText = blog.value
        //Sends a PUT request to the server in order to save the blog
        fetch("http://localhost:3000/pfp/save-blog", {
          method: "PUT",
          body: JSON.stringify({
            blog: blogText
          }),
          headers: {"Content-type": "application/json; charset=UTF-8"}
        })
        .then((res) => {
          return res.json()
        })
        .then((data) => {
          //Changes the 'Save blog' button to 'Blog Saved' after
          //the blog is successfully saved
          const errorMsg = document.getElementById("blogError")
          const saveButton = document.querySelector(".save-blog")
          if (data.state) {
            //If the response is positive, then confirm the password
            //change on the DOM
            errorMsg.textContent = ""
            saveButton.textContent = data.message
          } else {
            //Else, display the error message
            errorMsg.innerText = data.message.slice(25)
          }
        })
      }
```

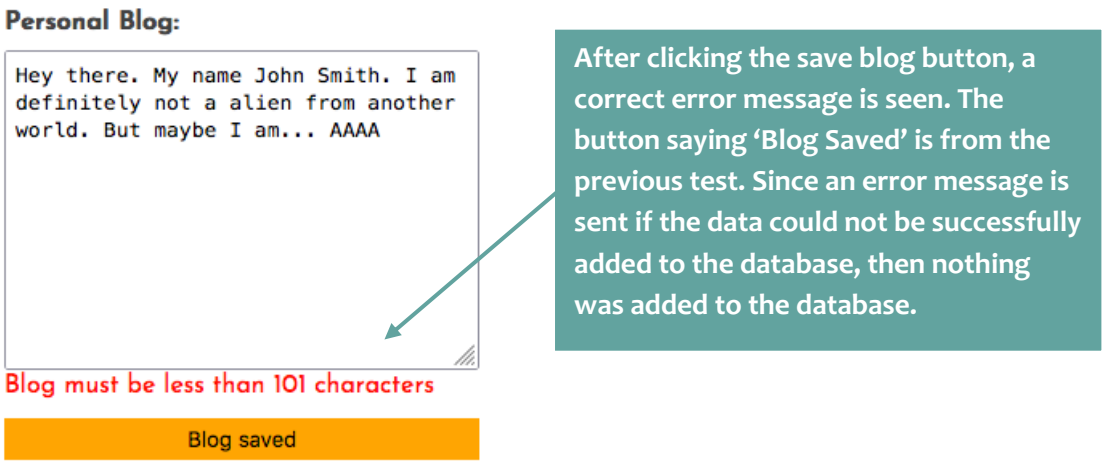
In HTML:

```
<button class="save-blog" onclick="saveBlog()">Save Blog</button>
```

Now to test if it works:



And now testing putting in text with more that 100 characters:



TEST SUCCESS!

The logout button can only be properly done later on so I will leave it at here for now.

Testing the Profile Page

Test ID	Test Input	Expected Outcome
P1	Observe the data on screen.	All the relevant data should match the data in the database. For instance: First Name should match, the list of scores should match and so on.
P2	Type in a password with less than 6 chars into the new password field and click 'Save Password'.	Error message should be return saying: "Password must be greater than 6 chars".
P3	Type in a password with greater than 6 chars into the new password field and click 'Save Password'.	The text on the button should change to "Password Saved". The new password should appear in the database.
P4	Type less than or equal to 100 characters into the blog text area and click 'Save blog'.	The text of the button should change to "Blog Saved". The blog should appear inside of the database.
P5	Type greater than 100 characters into the blog text area and click 'Save blog'.	The text of the button should not change, and the text should not be saved the database. Error msg sent.

Test P1

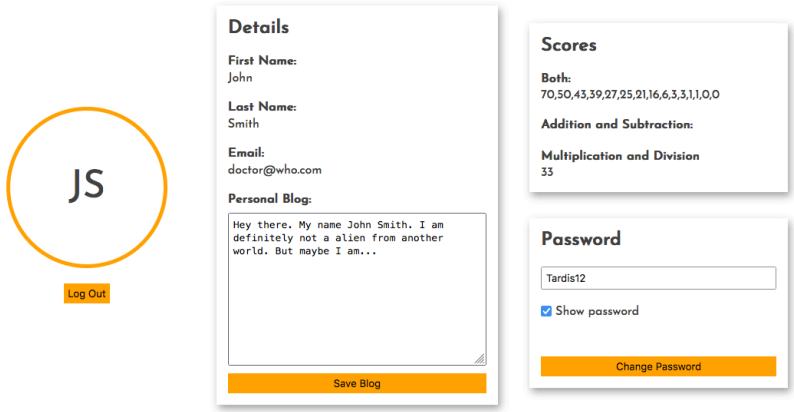
Input:


```
firstname: "John"
lastname: "Smith"
email: "doctor@who.com"
password: "Tardis12"
blog: "Hey there. My name John Smith. I am definitely not a alien from anothe…"
role: "basic"
```

```
email: "doctor@who.com"
▾ typeBoth: Array
    0: 3
    1: 16
    2: 25
    3: 39
    4: 50
    5: 27
    6: 43
    7: 70
    8: 1
    9: 6
    10: 0
    11: 0
    12: 3
    13: 1
    14: 21
  ▾ typeMD: Array
    0: 33
  ▸ typeAS: Array
  role: "basic"
```


This is the data of the user John Smith, in the users and scores tables.

Outcome:

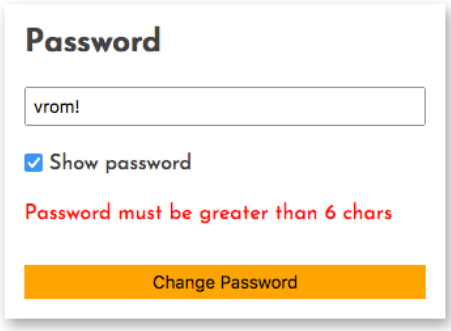


The information shown is all correct.

Test Success!

Test P2

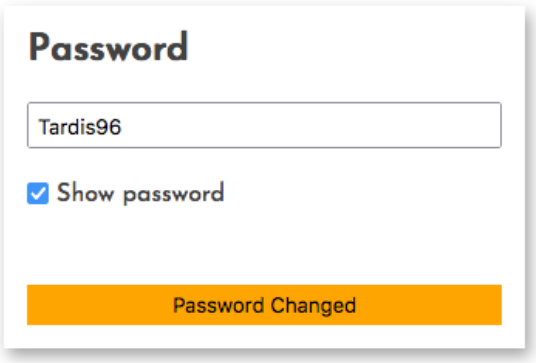
Input and Outcome:



Test Success!

Test P3

Input:



Outcome:

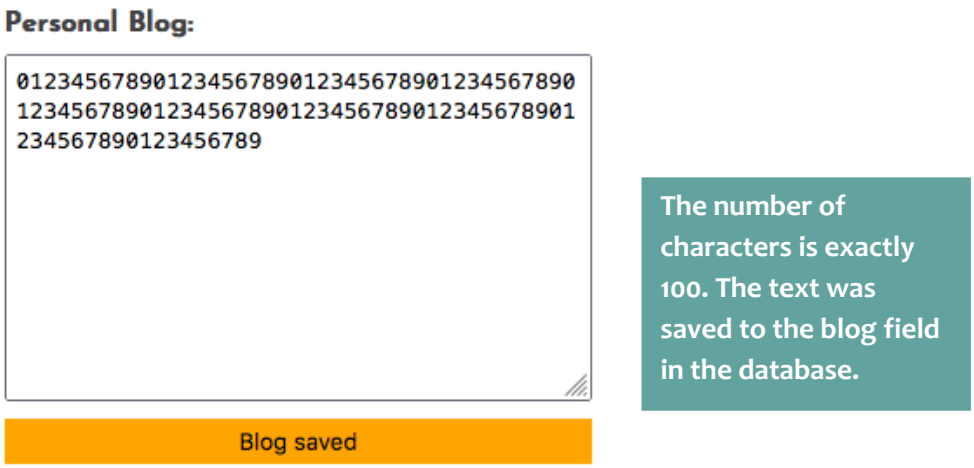
```
password: "Tardis96"
```

The old password was updated.

Test Success!

Test P4

Input:



The number of characters is exactly 100. The text was saved to the blog field in the database.

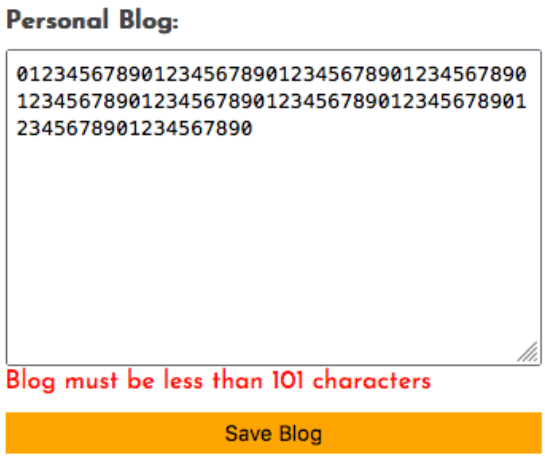
Outcome:

```
blog: "0123456789012345678901234567890123456789012345678901234567890123456789…"
```

Test Success!

Test P5

Input and Outcome:



There are 101 characters in the text area. Nothing was saved to the database and an error message was sent back. Also, the button text did not change.

Test Success!

Stakeholder Evaluation

The majority of the students were happy with the profile page. The liked the change password and saving a blog feature. The one thing they would have preferred was the ability to add their own profile picture. Otherwise, they were quite happy with everything else. The custom profile pictures would require I a lot more learning and is far out of the scope of what I am capable of doing in this period of time. Therefore, this feature will have to be left until further development after this website is done.

Overall Evaluation: Positive. No further changes needed.

The Admin Page

I added a new html file called admin.html in the html_pages folder and in the routes folder I added the file admin.js:

To access the file, I put the following in the admin.js file:

```
//Imports
const express = require("express");
const route = express.Router();
const path = require("path");
const Score = require("../models/Scores");
const User = require("../models/Users");


//Sets up an endpoint with the GET method on /admin
route.get("/", (req, res, next) => {
  //Sends the admin.html file to the display
  res.sendFile(path.join(__dirname, "/html_pages/admin.html"));
});


module.exports = route;
```

I also had to add the route to my server in the server.js file like such:

```
const adminPage = require("./api/routes/admin");
```

```
app.use("/admin", adminPage);
```

In the admin.html file I added the following HTML:

```
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta http-equiv="X-UA-Compatible" content="IE=edge" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Alien Arithmetic | Admin Page</title>
    <link rel="stylesheet" href="http://localhost:3000/styles/styles.css" />
  </head>
  <body>


    <div class="main">
      <h2>Users</h2>
      <div class="user-records">
        <div class="columns">
          <!-- Here are the field names for the user details -->
          <h4 style="width: 10%">ID</h4>
          <h4 style="width: 40%">Details</h4>
          <h4 style="width: 40%">Scores</h4>
          <h4 style="width: 10%">High-Scores</h4>
        </div>
        <!-- Over here is where the user records will go -->
      </div>
      <!-- Button to log out -->
      <button class="logout" onclick="logOut()">Log out</button>
    </div>

  </body>
</html>
```

Here I add the CSS at the top of the body:

```
<style>
      html {
        /* Makes the background this colour*/
        background-color: rgb(50 216 223);
      }
      .main {
        /* The main container */
        margin: 2.5% auto;
        width: 90%;
        padding: 20px;
        border: 0;
        background-color: white;
        box-shadow: 3px 5px 10px rgb(92, 92, 92);
      }
      .user-record {
        /* Styles the user record */
        display: flex;
        flex-direction: column;
        align-items: center;
        padding: 10px;
        width: 100%;
      }
      .user-records {
        /* Styles the box containing the user records */
        display: flex;
        flex-direction: column;
        align-items: center;
        width: 100%;
      }
      .columns {
        /* Styles the box with the field names */
        display: flex;
        justify-content: space-between;
        width: 100%;
        padding: 5px;
      }
      .record {
        /* Arranges the boxes of data for each record */
        display: flex;
        justify-content: space-between;
        width: 100%;
        margin-top: 10px;
        border: 2px solid orange;
        padding: 5px;
      }
      .show-more {
        /* Makes the p tag seem like a button */
        cursor: pointer;
      }

      .scores h5 {
        width: fit-content;
      }

      span {
        margin-left: 10px;
      }

      .detail {
        margin-bottom: 10px;
      }
      .drop {
        display: none;
        position: absolute;
      }

      ol {
        /* Styles and overflow area and makes it invisible */
        background-color: white;
        color: black;
        text-align: center;
        width: fit-content;
      }
      li {
        /* Styles the scores displayed in the overflow list */
        margin: 0;
        padding: 0 10px;
        text-align: center;
      }
      .del-button {
        position: absolute;
        left: 90%;
        display: none;
      }
      .record:hover .del-button {
        /* Makes the delete button appear when
        hovering over over a record */
        display: block;
      }

      .logout,
      .del-button button {
        /* Styles the del and log out buttons */
        border: 0;
        background-color: orange;
        padding: 5px;
        cursor: pointer;
      }
      .logout {
        margin-top: 5%;
      }
    </style>
```

Now it looks like so:

To get the information, I added the /admin/all-info endpoint into the admin.js file like so:

```
//Sets up an endpoint with the GET method on /admin/all-info
route.get("/all-info", (req, res) => {
  //Returns all the basic user info
  let allData = {};
  User.find({ role: "basic" })
    .exec()
    .then((userData) => {
      Score.find({ role: "basic" })
        .exec()
        .then((scoreData) => {
          for (let user in userData) {
            const { email, firstname, lastname, blog } = userData[user];
            const { typeBoth, typeMD, typeAS } = scoreData[user];
            const highBoth = maxNumber(typeBoth);
            const highMD = maxNumber(typeMD);
            const highAS = maxNumber(typeAS);
            allData[user] = {
              email: email,
              firstname: firstname,
              lastname: lastname,
              blog: blog,
              typeAS: typeAS,
              typeMD: typeMD,
              typeBoth: typeBoth,
              highBoth: highBoth,
              highMD: highMD,
              highAS: highAS,
            };
          }
          res.json({
            data: allData,
          });
        });
    });
});
```

Here is the maxNumber function:

```
function maxNumber(array) {
    //Finds the biggest number out of
    //an array
  const max = array.reduce((a, b) => {
    return Math.max(a, b);
  }, 0);
  return max;
}
```

Testing if the information is correct by calling the endpoint:

TEST SUCCESS!

In the admin.html file, here is the fetch algorithm to call the endpoint and get the user data:

```
//   Calls the /admin/all-info endpoint with the GET method
      //   in order to get all basic user info
      fetch("http://localhost:3000/admin/all-info")
        .then((res) => {
          return res.json();
        })
        .then((data) => {
          const userData = data.data;
          for (let user in userData) {
            //Each user is added to the DOM here
            addRecord(user);
            addData(userData[user], user);
          }
        });
```

Here is the addRecord function:

```
function addRecord(id) {
        //Adds the template for a user record into the user records box/section
        const userRecords = document.querySelector(".user-records");
        const record = `
        <div class="record" id="${id}">
            <div class="del-button">
              <button onclick="deleteUser(${id})">Delete</button>
            </div>
            <div class="id" style="width: 10%">
              <p class="id-number">${id}</p>
            </div>
            <div class="details" style="width: 40%">
              <h5>Firstname:</h5>
              <p class="detail" id="firstname${id}"></p>
              <h5>Lastname:</h5>
              <p class="detail" id="lastname${id}"></p>
              <h5>Email:</h5>
              <p class="detail" id="email${id}"></p>
              <h5>Personal Blog:</h5>
              <p class="detail" id="blog${id}"></p>
            </div>
            <div class="scores" style="width: 40%">

                <h5>Both: <span class="both" id="typeBoth${id}"></span></h5>
                <p class="show-more" onclick="showMore('typeBoth${id}', event)">Show more ▸</p>
                <ol class="drop" id="overflowtypeBoth${id}" style="display: none"></ol>
                <br />

                <h5>
                    Multiplication and Division: <span class="md" id="typeMD${id}"></span>
                </h5>
                <p class="show-more" onclick="showMore('typeMD${id}', event)">Show more ▸</p>
                <ol class="drop" id="overflowtypeMD${id}" style="display: none"></ol>
                <br />

                <h5>Addition and Subtraction: <span class="as" id="typeAS${id}"></span></h5>
                <p class="show-more" onclick="showMore('typeAS${id}', event)">Show more ▸</p>
                <ol class="drop" id="overflowtypeAS${id}" style="display: none"></ol>
                <br />

            </div>
            <div class="high-scores" style="width: 10%">
              <p class="high-both" id="highBoth${id}"></p>
              <br />
              <p class="high-as" id="highAS${id}"></p>
              <br />
              <p class="high-md" id="highMD${id}"></p>
            </div>
          </div>`;
          userRecords.innerHTML += record;
        }
```

Here is the addData function:

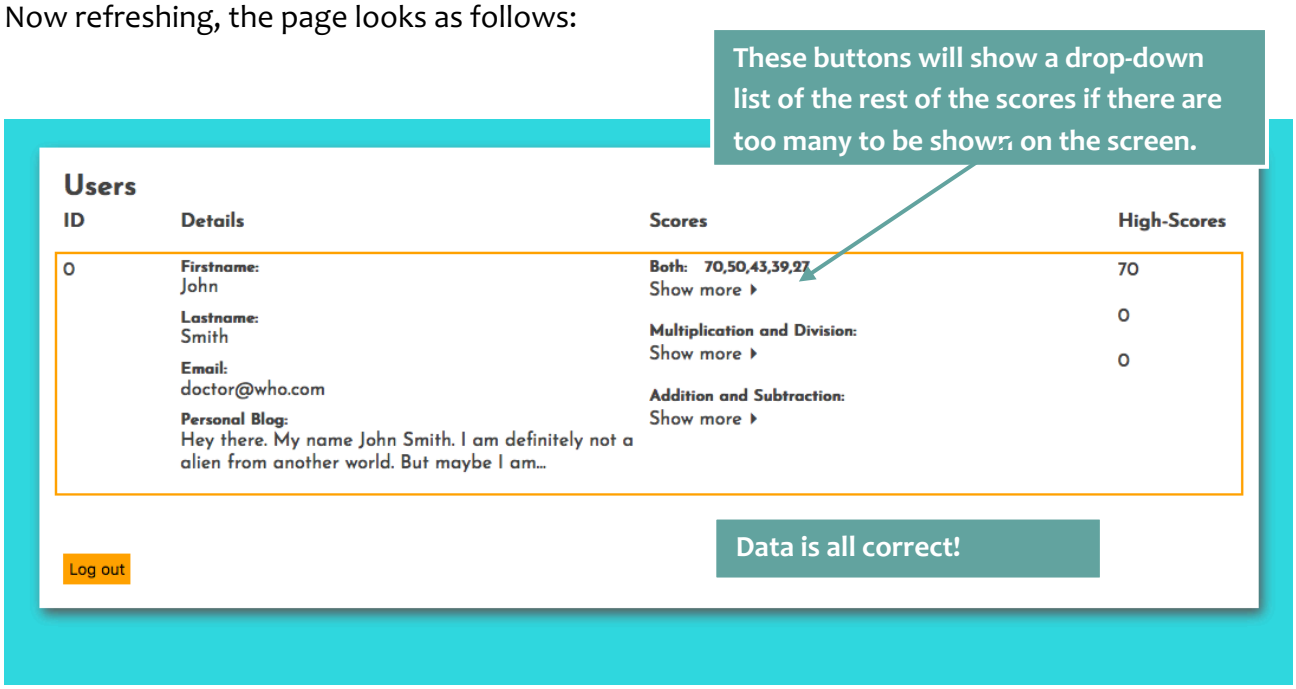
```
239  function addData(data, id) {
240          //Adds the data to the user records on screen
241          for (let detail in data) {
242            const ele = document.getElementById(detail + id);
243            if (detail.slice(0, 4) === "type") {
244              //If the user detail is a score, then do the below
245              const { topFive, overFlow } = sortScores(data[detail]);
246              ele.innerText = topFive;
247              addOverflow(detail, id, overFlow);
248            } else {
249              ele.innerText = data[detail];
250            }
251          }
252        }
```

Here is the addOverflow function and sortScores function:

```
231  function addOverflow(detail, id, overflow) {
232          //Adds the overflow scores onto the overflow list
233          const ele = document.getElementById("overflow" + detail + id);
234          for (let i in overflow) {
235            ele.innerHTML += `<li>${overflow[i]}</li>`;
236          }
237        }
```

```
214  function sortScores(array) {
215          //Returns the sorted array and the overflow scores
216          //that are not displayed on the screen
217          const sorted = array.sort(function (a, b) {
218            return b - a;
219          });
220          let topFive = [];
221          for (let i in sorted) {
222            if (i > 4) {
223              break;
224            }
225            topFive.push(sorted[i]);
226          }
227          const overFlow = sorted.slice(5);
228          return { topFive, overFlow };
229        }
```

Now refreshing, the page looks as follows:

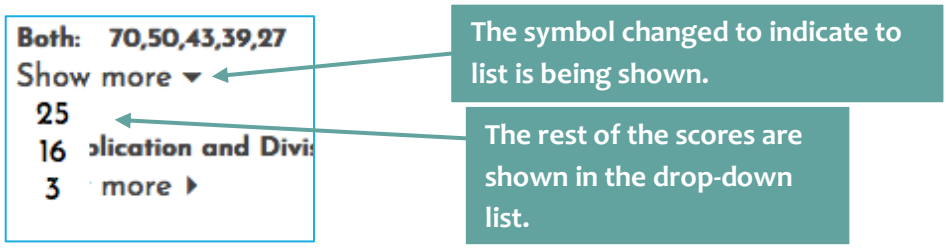


The 'Show more' buttons are not yet functional although the overflow data is invisible within the DOM. This is shown when inspecting the user record using the Firefox dev tools:

I made it functional with the following function which is called upon the button being clicked:

```
function showMore(type, event) {
        //Makes the overflow list visible
        const dropDown = document.getElementById("overflow" + type);
        const element = event.target;
        if (dropDown.style.display == "none") {
          dropDown.style.display = "block";
          element.innerText = "Show more ▾";
        } else {
          dropDown.style.display = "none";
          element.innerText = "Show more ▸";
        }
      }
```

Now when clicking the ‘Show more’ button the following is seen:



And clicking the button again:

TEST SUCCESS!

The ‘Delete’ buttons are also not functional. Here I made the endpoint to delete the given user in the admin.js file:

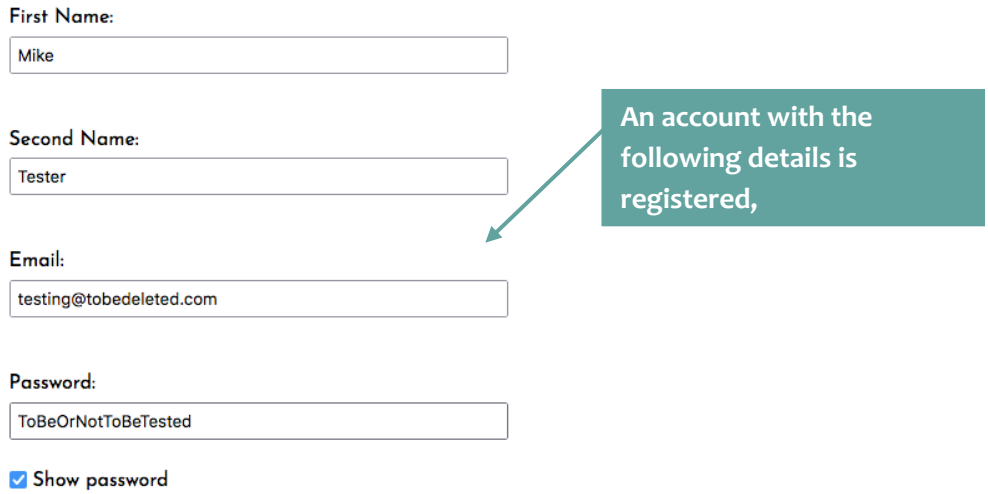
```
//Sets up an endpoint with the DELETE method on /admin/{param}
route.delete("/:userEmail", (req, res) => {
  //Deletes the record with the given in corresponding
  //email address in the database
  User.deleteOne({ email: req.params.userEmail })
    .exec()
    .then((result) => {
      Score.deleteOne({ email: req.params.userEmail })
        .exec()
        .then(() => {
          res.status(201).json({
            message: "User deleted",
          });
        });
    })
    .catch((err) => {
      next(err);
    });
});
```

In the admin.html file I added the function to call this endpoint with the fetch algorithm:

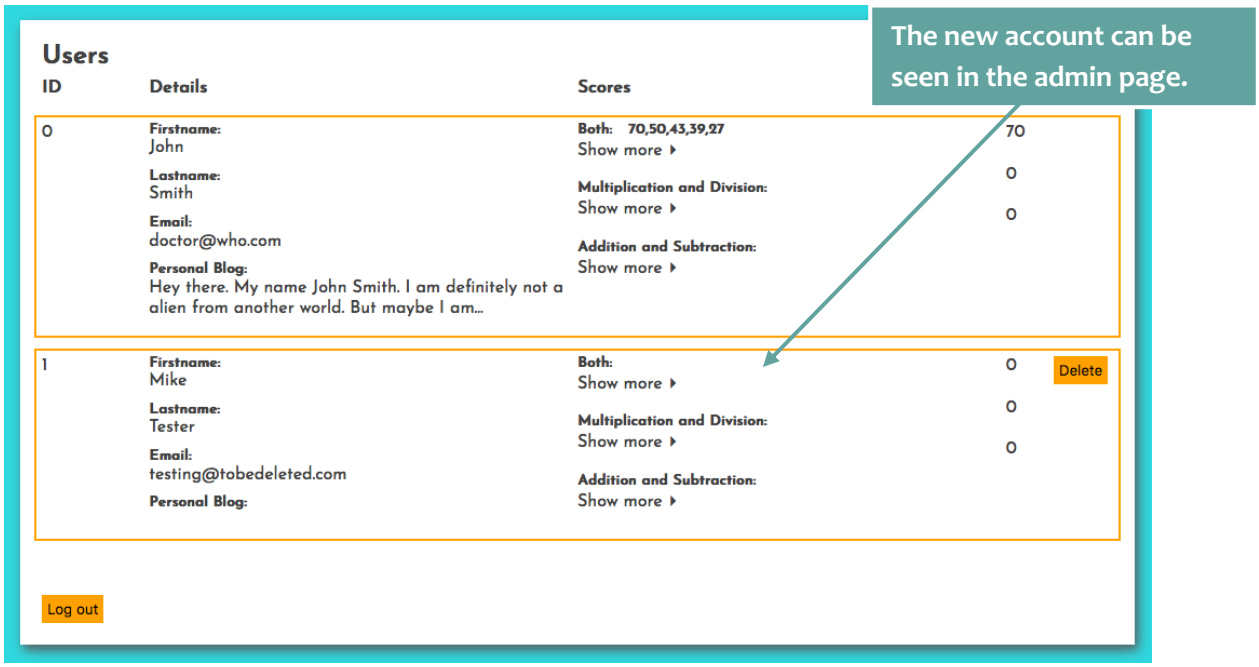
```
function deleteUser(id) {
        //Deletes the specified user from the database
        const email = document.getElementById("email" + id).textContent;
        fetch(`http://localhost:3000/admin/${email}`, {
          method: "DELETE",
        }).then(() => {
          const record = document.getElementById(id);
          record.remove();
        });
      }
```

This function is called when clicking a delete button.

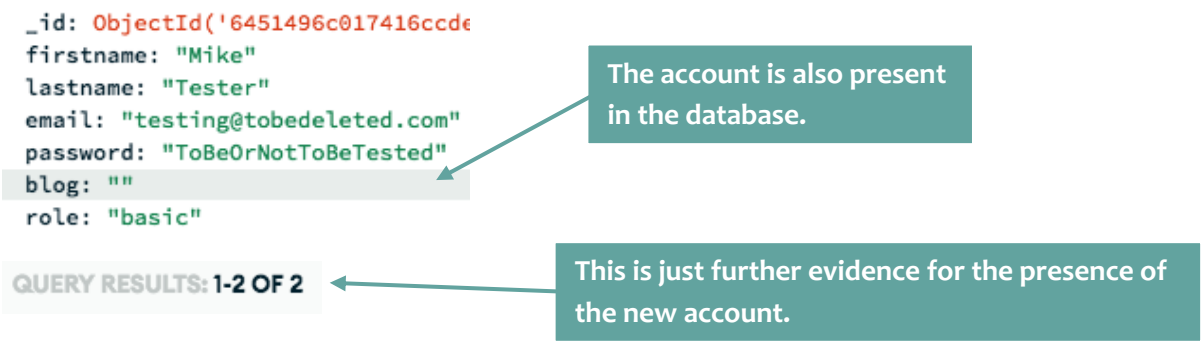
To test this, I will make a test account by registering another person with the register page with the following details entered:



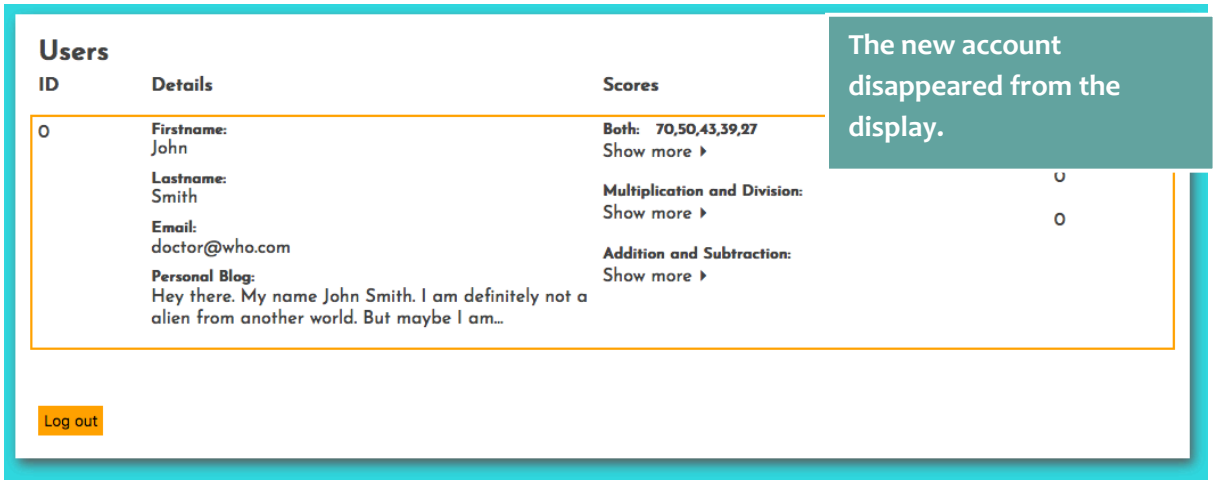
Now going to the admin page:



And in the database:



After clicking the delete button:



In the database:

TEST SUCCESS!

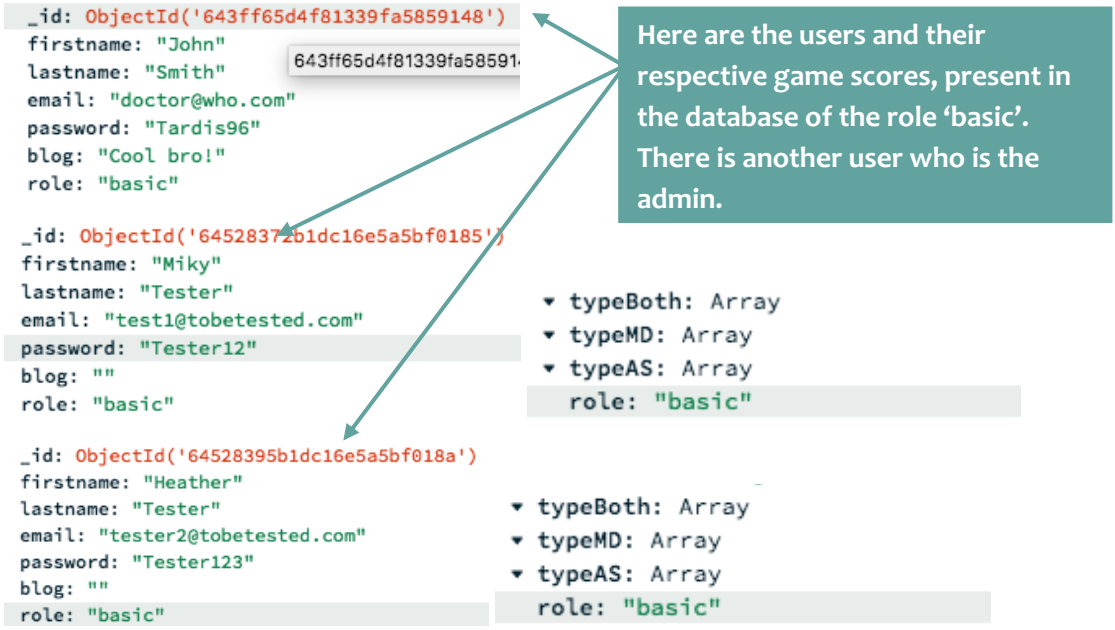
The logout button can only be properly done later on so I will leave it at here for now.

Testing the Admin Interface

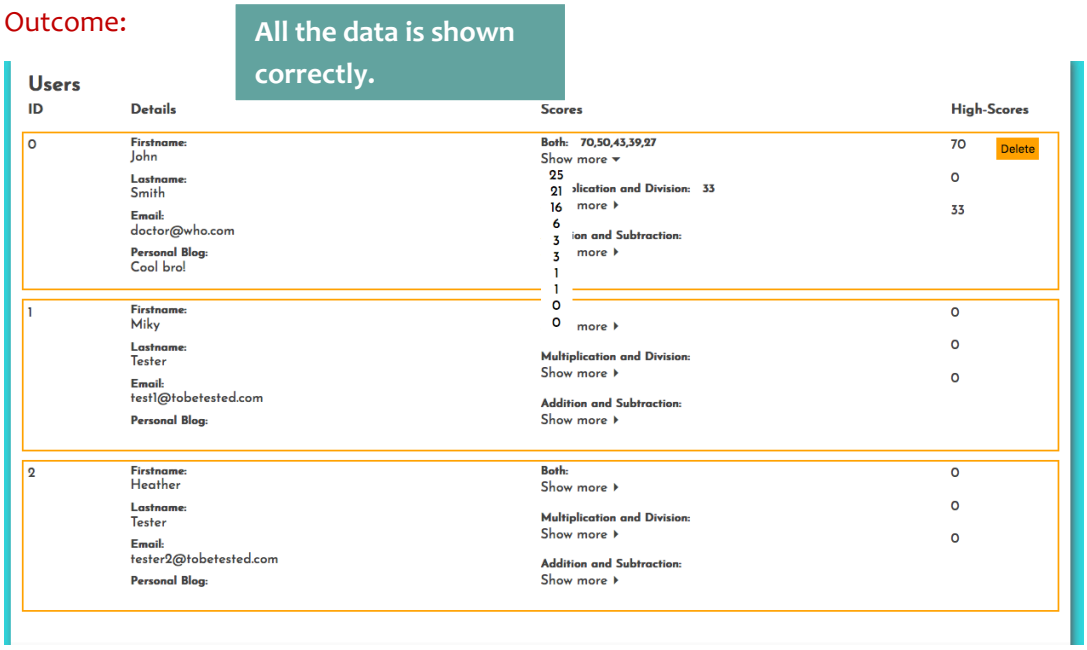
Test ID	Test Input	Expected Outcome
A1	Observe the data for each record on the screen.	All the relevant data should match the data in the database. For instance: First Name should match, the list of scores should match and so on.
A2	Click delete button for a record.	The record should be removed from the DOM. The record should be removed from the database.

Test A1:

Input:



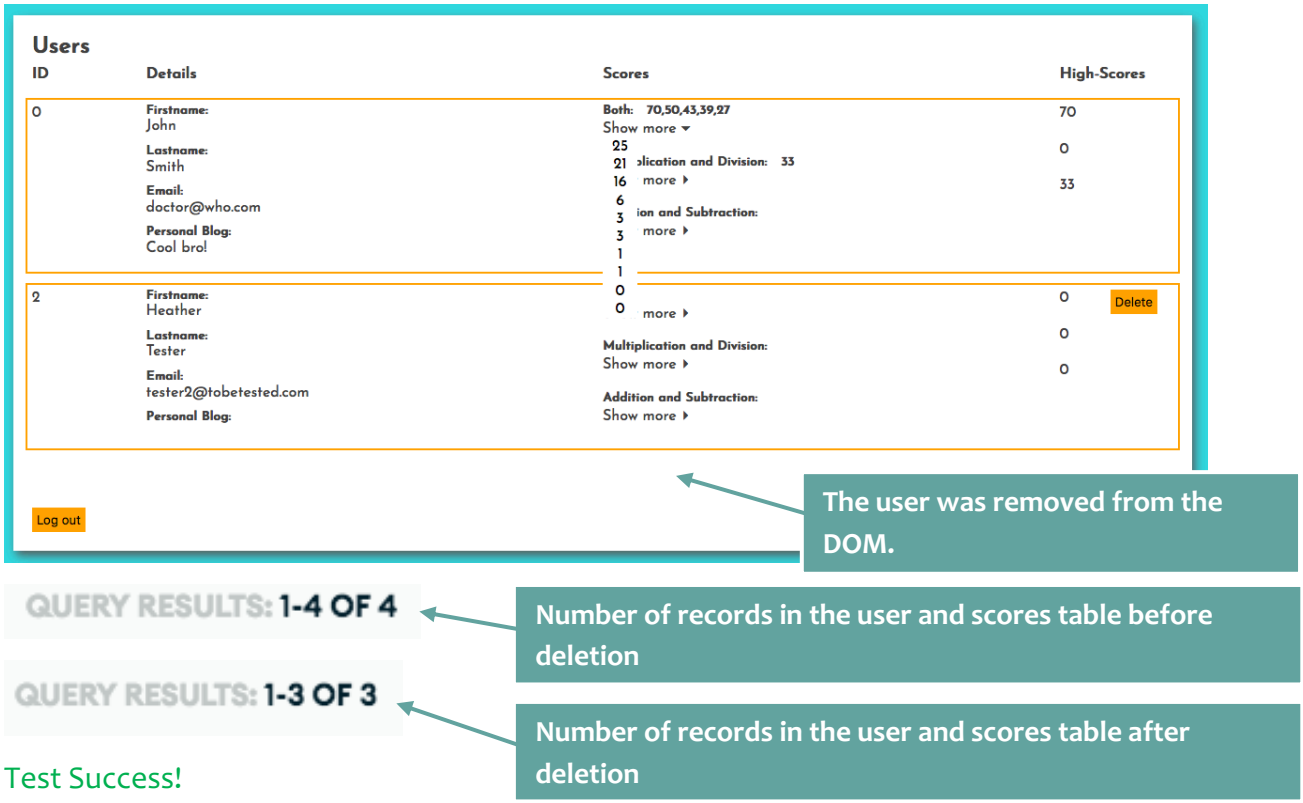
Outcome:



Test Success!

Test A2:

Input and Outcome:



Test Success!

Stakeholder Evaluation

My client was sufficiently happy with the page as a whole. He said he would like the logout button to work, and I reassured him that this function will be implemented in the future. He also liked the detailed shown to him about the user and would have liked to see a graph for each user to gather their progress on improving their mental math. This feature would be very time consuming and convoluted to add, so I have decided to add it in as a later improvement to the website after this development.

Overall Evaluation: Positive. No further changes needed.

PROTECTING THE ROUTES

In this section I will add the following features to the website:

- Users who are not logged in will not be able to use the website.
- The logout buttons will be given their function.
- Non-admin users cannot access the admin page or the admin related endpoints.

To stop unauthorized access to the website, the server needs to know whether some is logged in or not. To make this possible, I will use cookies. In the login.js file I added the following code:

```
5   const jwt = require("jsonwebtoken")
```

This imports a package called 'jsonwebtoken'. It is used for authentication purposes.

Inside the /login endpoint after the given email and password are validated positively:

```
43  if (userContent.password === req.body.password) {
44              // Creates a token that stores the email, userId and role of the
45              //user.
46              const maxAge = 1 * 60 * 60;
47              //This is the code to be used to undecipher the token
48              const jwtSecret =
50                //This defines the token
51              const token = jwt.sign(
52                {
53                  email: userContent.email,
54                  userID: userContent._id,
55                  role: userContent.role,
56                },
57                jwtSecret,
58                {
59                  expiresIn: maxAge,
60                }
61              );
62              //Adds a cookie with the authentication token
63              res.cookie("token", token, {
64                maxAge: maxAge * 1000,
65              });
66
67              res.status(200).json({
68                message: "You logged in",
69                state: true,
70              });
```

Now when logging in and looking in the storage on the website using the Firefox dev tools:

Now I need to make a function that is called on every endpoint call that checks whether the user is authorized or not. To do this I added to the middleware folder a file called userAuth.js and put the following code into it:

```
1  //Imports
2  const jwt = require("jsonwebtoken");
3  const path = require("path");
4  //This code is used to decipher the token
5  const jwtSecret =
8  //This exports the function called userAuth
9  exports.userAuth = (req, res, next) => {
10   //This get the token from the cookies from the request body
11   const token = req.cookies.token;
12   try {
13     //If the token is able to be deciphered, allow access to the user
14     const { email, userID, role } = jwt.verify(token, jwtSecret);
15     //Sends the role, email and userId as arguments to next function called.
16     next({ role: role, id: userID, email: email });
17   } catch {
18     //If the token is NOT able to be deciphered, deny access to the user
19     //and send them to the login page
20     res.sendFile(path.join(__dirname, "../routes/html_pages/login.html"));
21   }
22 };
```

Now to use this function, I must import it into the wanted file. Since I want all the routes that are related to the website to be protected, I imported it into all the route files, expect in register.js and login.js, files like so:

```
7  const {userAuth} = require("../middleware/userAuth")
```

Then to make this act on an endpoint, it is implemented like so:

```
19  route.get("/", userAuth, (auth, req, res, next)
```

NOTE:

The endpoint can be of any method. The new argument ‘auth’ is the object passed in from the next function in the userAuth function. Also, the ‘next’ argument must be present otherwise the ‘res’ argument is confused to be the ‘next’ argument.

Using the ‘auth’ argument, which contains the id, email, and role of the current user, many endpoints can now be made properly functional.

First, I populate all the endpoints as I did above:

For example, in the pfp.js file, here are the new endpoint setups:

```
17  //Sets up an endpoint with the GET method on /pfp
18  route.get("/", userAuth, (auth, req, res, next)
```

```
22  //Sets up an endpoint with the GET method on /pfp/info
23  route.get("/info", userAuth, (auth, req, res, next) => {
24    //The email is gotten from the return obj of the userAuth function
25    let email = auth.email;
```

```
50  //Sets up an endpoint with the PUT method on /pfp/change-password
51  route.put("/change-password", userAuth, (auth, req, res, next) => {
52    //The email is gotten from the return obj of the userAuth function
53    let email = auth.email;
```

```
81  //Sets up an endpoint with the PUT method on /pfp/save-blog
82  route.put("/save-blog", userAuth, (auth, req, res, next) => {
83    //The email is gotten from the return obj of the userAuth function
84    let email = auth.email;
```

Everywhere I put in a manual email, I can now change to get the email from the userAuth return object. In a special case, the /stats/game-scores endpoint was changed to the below:

```
21  //Sets up an endpoint with the GET method on /stats/game-scores
22  route.get("/game-scores", userAuth, (auth, req, res, next) => {
23    const email = auth.email
```

It used to be a post method. Therefore, fetch algorithm in stats.html file was changed:

```
105  //Sends a GET request to the server in order to get back the scores for the user
106        fetch("http://localhost:3000/stats/game-scores")
```

For the /home endpoint in the home.js file, if the logged user is an admin, they should be routed to the admin page. The endpoint was changed as follows:

```
15  //Sets up an endpoint with the GET method on /home
16  route.get("/", userAuth, (auth, req, res, next) => {
17    if ((auth.role = "admin")) {
18      //If the user is an admin, go to the admin page
19      res.sendFile(path.join(__dirname, "/html_pages/admin.html"));
20    } else {
21      res.sendFile(path.join(__dirname, "/html_pages/home.html"));
22    }
23  });
```

Also, the routes in the admin.js file should all stop access for anyone who is not an admin. I changed to routes to the following:

```
18  //Sets up an endpoint with the GET method on /admin
19  route.get("/", userAuth, (auth, req, res, next) => {
20    if (auth.role === "admin") {
21      //Sends the admin.html file to the display
22    res.sendFile(path.join(__dirname, "/html_pages/admin.html"));
23    } else {
24      res.json({
25        message: "Access Denied"
26      })
27    }
28  });
```

And:

```
30  //Sets up an endpoint with the GET method on /admin/all-info
31  route.get("/all-info", (req, res) => {
32    //Returns all the basic user info
33    if (auth.role === "admin") {
```

In the if block is where everything happens, and the info of all users are sent. In the else block:

```
66  } else {
67      res.json({
68        message: "Access Denied"
69      })
70    }
```

And the last endpoint:

```
74  //Sets up an endpoint with the DELETE method on /admin/{param}
75  route.delete("/:userEmail", userAuth, (auth, req, res, next) => {
76    //Deletes the record with the given in corresponding
77    //email address in the database
78    if (auth.role === "admin") {
```

In the if block is where everything happens, and the selected user is deleted. In the else block:

```
66  } else {
67      res.json({
68        message: "Access Denied"
69      })
70    }
```

Now to make the logout endpoint in the login.js file:

```
23  //Sets up an endpoint with the GET method on /login/logout
24  route.get("/logout", (req, res) => {
25    //This endpoint clears the authentication cookie
26    //and reroutes the user to the login page
27    res.clearCookie("token", undefined);
28    res.sendFile(path.join(__dirname, "/html_pages/login.html"));
29  });
```

Now making the logout buttons functional in the admin.html and pfp.html files like so:

```
179  function logOut() {
180          //Logouts the user and take them to the login page
181          location.replace("http://localhost:3000/login/logout")
182        }
```

In the HTML:

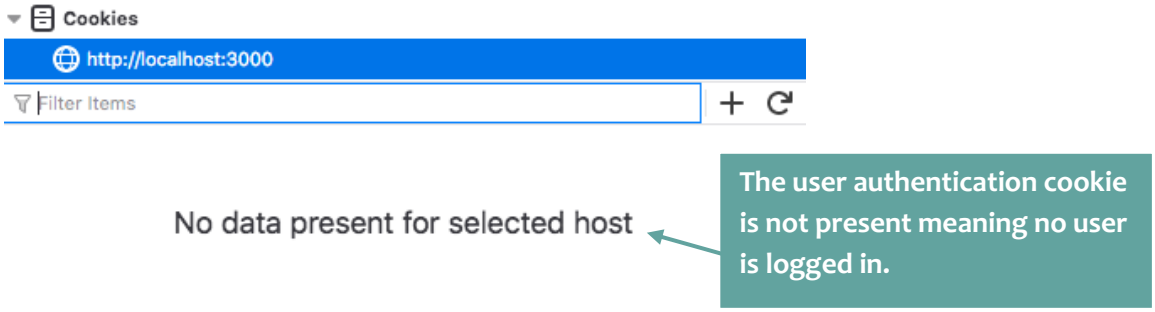
```
111  <!-- Button to log out -->
112            <button class="log-out" onclick="">Log Out</button>
```

Testing the Website

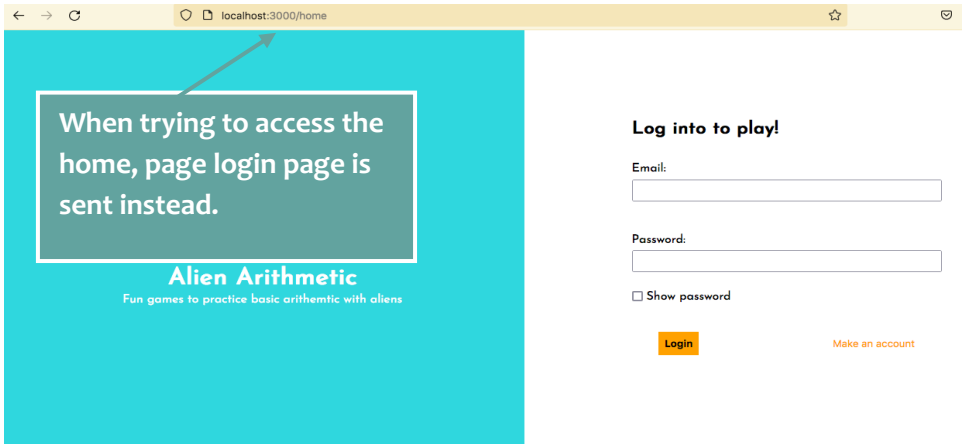
Test ID	Test Input	Expected Outcome
W1	Try accessing the website when the user is not logged in.	The user should be rerouted to the login page.
W2	A logged in tries to access an endpoint that doesn't exit.	The user is sent to the 404 page.
W3	A logged in user tries to access the admin page or admin page related endpoints.	The user should be sent an access denied message

Test W1

Input:



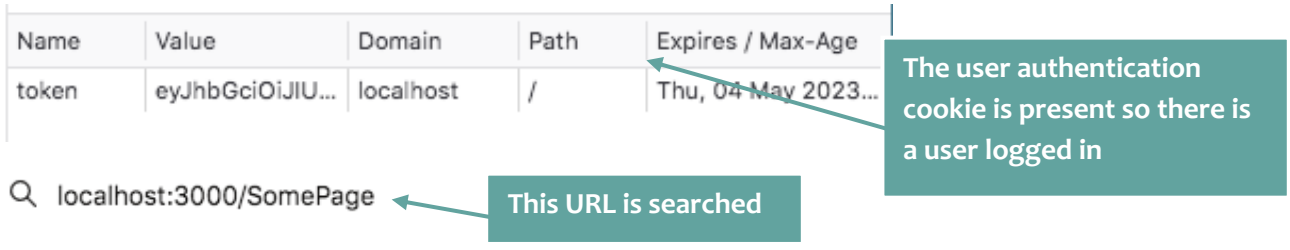
Outcome:



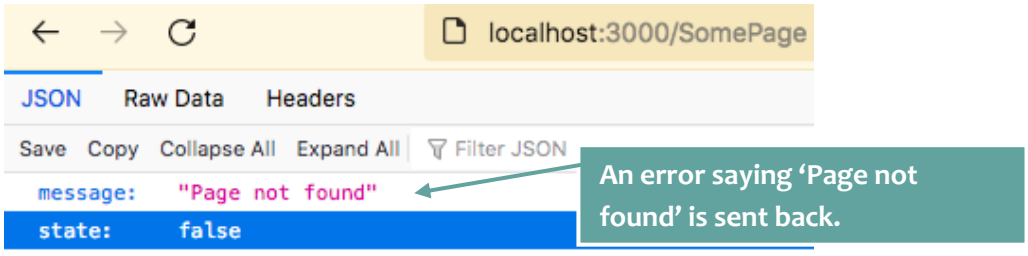
Test Success!

Test W2

Input:



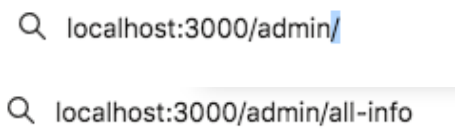
Outcome:



Test Success!

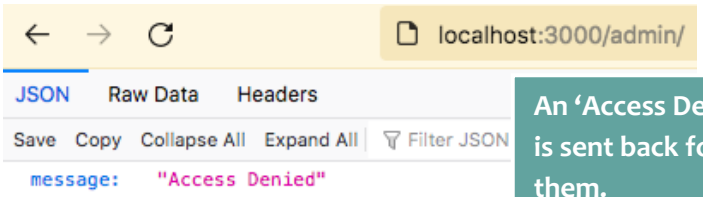
Test W3

Inputs:



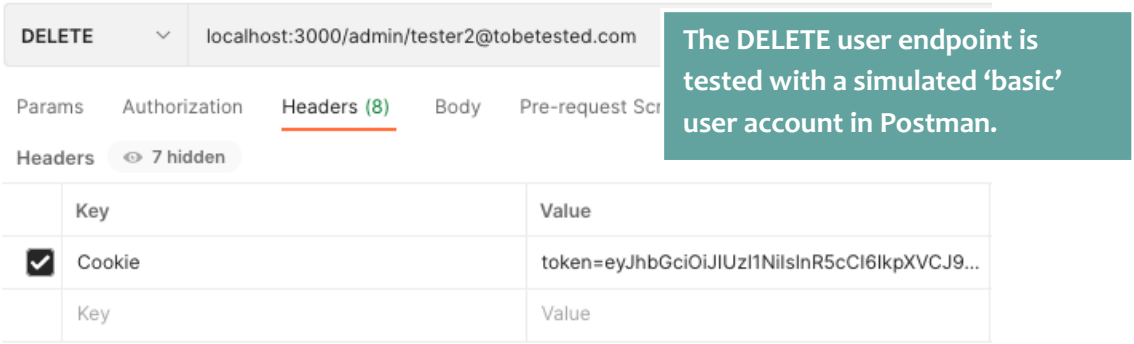
The logged in user tries the /admin related endpoints.

Outcomes:



An 'Access Denied' error is sent back for both of them.

Input:



The DELETE user endpoint is tested with a simulated 'basic' user account in Postman.

Outcome:

Access is denied.

Test Success!

POST DEVELOPMENT

In this section I will do the following list of tasks:

- I will go through the Post Development Test Plan created in the Design section. For each test, I will say whether the test was a failure or a success and give evidence from the testing I did in the Development stage. I will also confirm my verdict with my Stakeholders. For the tests relying solely on user opinions, I will consult my Stakeholders. For all their feedback on tests that they thought failed, I will condense it into a summary of my own words.

- I will go through the Success Criteria that were failed or were partially met in the Post Development test. I will comment on their reasons for not being finished or even started, and how to address them.

- I will comment on the robustness of certain features and possible inputs that could break them.

- I will comment on the useability features for each page of the website and how successful or unsuccessful they were.

- I will talk about further Maintenance on the website. This includes fixing bugs, making minor updates and so on.

- I will talk about the limitations to improving on the project. I will comment on the potential ways around such limitations.

- I will talk about the potential improvements to the project. These include major updates and the overhauling of website and coding design and how these can be used to tackle the limitations.

POST DEVELOPMENT TEST PLAN

Test ID	Feature	Success Criteria	The Function in Question	Expected Outcome	Success? Yes/No/Partially. If Yes, Evidence:	User Opinion Feature successful: **Yes/No. If No, why?**
1	Navigation bar	Navigation bar that allows quick and easy access to different parts of the website	Nav bar buttons	Users quickly able to navigate the website without getting confused. The nav buttons work as intended.	Yes. Evidence: Test on page 121	Yes
2	Home page	Home page that welcomes the user and gives directories to different pages on the website.	Home page navigation buttons	The home buttons take the user to the correct pages.	Yes. Evidence: Test H1,2,3	Yes
3	Stats page	Stats page that shows all scores on a line graph and so on.	Line graphs	The correct data is shown on the line graphs.	Yes. Evidence: Test S1,2	Yes
4	Game page	Game page that shows a list of the games on the website	Game preview	Clicking the game preview take the users to the game menu.	Yes. Evidence: Test G2	Yes
5	Profile page	Profile page that displays all of the user's details as well as the user's top ten scores.	User details and scores	Correct user details and scores displayed.	Yes. Evidence: Test P1	Yes

Test ID	Feature	Success Criteria	The Function in Question	Expected Outcome	Success? Yes/No/Partially. If Yes, Evidence:	User Opinion Feature successful: **Yes/No. If No, why?**
		A field that allows users to write a personal blog.	Blog area	Validation on blog works. Saving the blog works.	Yes. Evidence: Test: P4,5	Yes
		Area for user to change their password.	Change password area	Validation on new password works. Saving the new password works.	Yes. Evidence: Test P2,3	Yes
6	Admin interface	Admin page for teacher to see all the details of students.	All user details	All correct data displayed.	Yes. Evidence: Test A1	Yes
7	Login and sign-up window	Login and sign-up page that is simple and allows users to navigate between the login, student and admin register pages.	The Student, Admin, to login and to register buttons.	They work as intended.	Yes. Evidence: Test R10, L10	Yes
		Each input should be fully validated.	Login Button	If the user input is correct, they are taken to the home page. If not, then an error message is shown.	Yes. Evidence: Test L1 – L8	Yes

Test ID	Feature	Success Criteria	The Function in Question	Expected Outcome	Success? Yes/No/Partially. If Yes, Evidence:	User Opinion Feature successful: **Yes/No. If No, why?**
			Register Button	If the user input is correct, they are taken to the login page and their details are added to the database. If not, then an error message is shown.	Yes. Evidence: Test R1 – R9	
		Admin register should have a field for a code that allows for an admin account to be created.	Admin code field	Only users with the special key can become an admin.	Yes. Evidence: Test R12, 13	Yes
8	**Game** {One or two math games	One or two math games. Minimum of one.	The game being present	A playable game.	Yes. Evidence: Test AG1 - 4	Yes
9	Increasing difficulty	Game/games that in increase in difficulty as your score increases	Increasing difficulty through progression	Questions getting harder and enemy speed increases.	Yes. Evidence: Test AG4	Yes
10	Different Game Types	Players have the option to choose between having only addition and subtraction questions, multiplication or division questions or a mix of both types	Game mode options	The different game modes work as intended.	Yes. Evidence: Test GM1	Yes

Test ID	Feature	Success Criteria	The Function in Question	Expected Outcome	Success? Yes/No/Partially. If Yes, Evidence:	User Opinion Feature successful: **Yes/No. If No, why?**
11	Heavily math orientated	There will be little extra thinking with mechanics not involved with the math part of the game.	The game functionality	Up to user opinion.	N/A	No. There is some barrier to entry for people who can't type fast.
12	Practices all parts of mental math	Addition, subtraction, dividing, multiplying, fractions, algebra. The last two may be limited by time	The math questions	Questions of algebra, addition, subtraction, multiplication and division.	Partially	Yes
13	Appropriate aesthetic	Satisfying animations and bold colours.	Game styles	Up to user opinion.	N/A	Yes
14	Repeatable and fun}	Game is easy to play, easy to access and repeat many times over.	Game styles	Up to user opinion.	N/A	Yes
15	Change password option	Users have the ability to change their password when-ever they want through the profile page	Change password area	Validation on new password works. Saving the new password works.	Yes. Evidence: Test P3	Yes
16	Aqua, white accents with orange buttons	Aqua, white accents with orange buttons for the whole webpage.	Website styles	Up to user opinion.	N/A	Yes

Test ID	Feature	Success Criteria	The Function in Question	Expected Outcome	Success? Yes/No/Partially. If Yes, Evidence:	User Opinion Feature successful: **Yes/No. If No, why?**
17	Appropriate font and style	Appropriate font and style for the website.	Website styles	Up to user opinion.	N/A	Yes
18	Option to toggle light and dark mode	Option to toggle light and dark mode that changes the styling depending on the mode selected.	Dark Light mode toggle buttons	The styles of the website change depending on the mode.	No	No. We can't see or use this feature.
19	Automated email capabilities	Automated email capabilities that will send reminder emails to the users email for them to spend such and such time on one of the math games. This feature can be turned off.	Automated Emails	Automated emails are sent to the user at relevant times. They can be turned off.	No	No. We don't get emails ever and there is no option to turn notifications off.

COMMENTING ON MY SUCCESS

Overall, many of my Success Criteria were met with exceptional results. In some cases, I even added some extra features, such as the "Show password" check box where-ever a password was involved, or "Delete" button for each record on the Admin Interface Page.

Despite the success, some features were either partially done or not done at all. And some features my Stakeholders were not happy about. Here I will list and go through each feature and explain my reasons:

Heavily Math Orientated Game

The users, which include my stakeholders and my client, said there was some bar to entry to people who could not type fast. I realised this problem very early on into people playtesting my game. Some players would have the math brain capable of answering the questions but could not type fast enough to accommodate for the speed at which their brain was going. And, as it turns out, the majority of players could not type fast. They had the common problem that they did not know where the numbers were on the keyboard, and they would have to constantly look down to type the correct number they wanted. There was also the odd time when the numbers on the keyboard were very far away from each other, like between 1 and 0, making it difficult to type the answer to the question.

In response to this issue, I say there is very little that can be done. When using certain keyboards which have a keypad, this problem can be counteracted, and implementing an automatic enter algorithm would invite a whole slew of problems for the player. For instance, if the player wants to type 42, after typing 4, an enemy on the screen with the answer of 4 will be killed rather the intended enemy with answer 42 potentially leading to the game ending.

However, there is a solution, but it involves using another device. Since quick access to the keys is ideal, then this game on iPad or on mobile would be very optimal. That although is a thing of the future and out of my field of knowledge at the moment.

Practices All Parts of Mental Math

I simply did not have the time to include questions that involved algebra. This is why I considered this feature to be partially met. This, however, can be added during further maintenance on the website, as well as adding fraction questions as well.

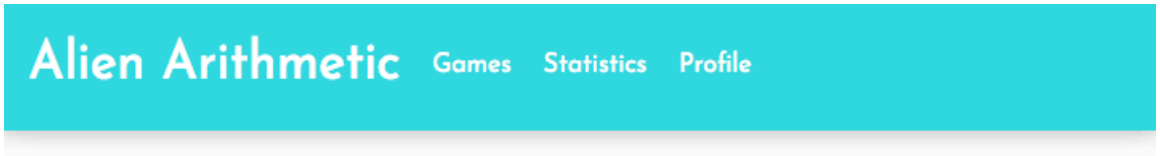
Option to Toggle Light and Dark Mode

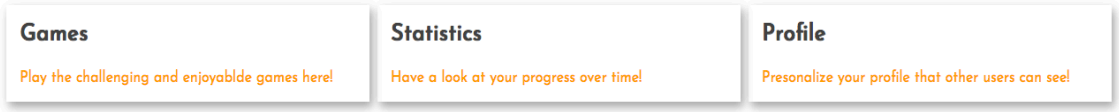
In truth, this feature was added in as a whim, something I was not to worried about not completing. I had seen this feature on many modern websites and software and thought it to be a suitable addition. But, after finishing the project and looking over the CSS I had down on each page of the website, I knew that it would take up too much of my time to implement. I would have to make all the dark mode CSS and make the CSS change depending on the condition of a toggle button. It is for this main reason I left this feature out. This feature is not likely to be added in further improvements to the website since it is quite useless to the users.

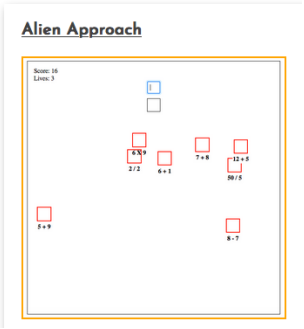
Automated Email Capabilities

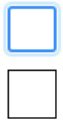
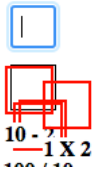
This feature was something I often saw on other website of my type. They would spam your email inbox with loads of useless reminders and random junk. I spite of this, I thought it might be a good idea to send reminders to students who check their email every day. Sadly, due to lack of time, I was unable to acquire the knowledge necessary to implement this feature into the website. This feature will definitely be added in future improvements to the website.

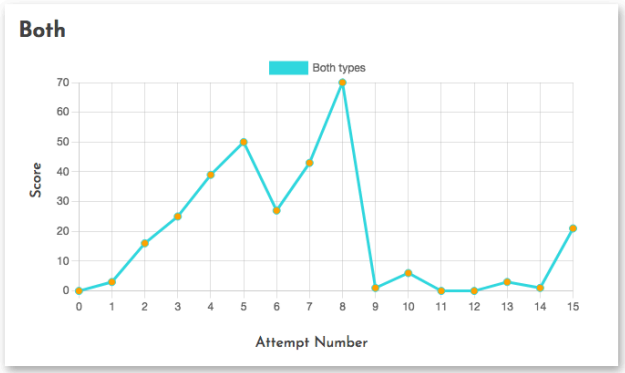
USABILITY FEATURES TESTING

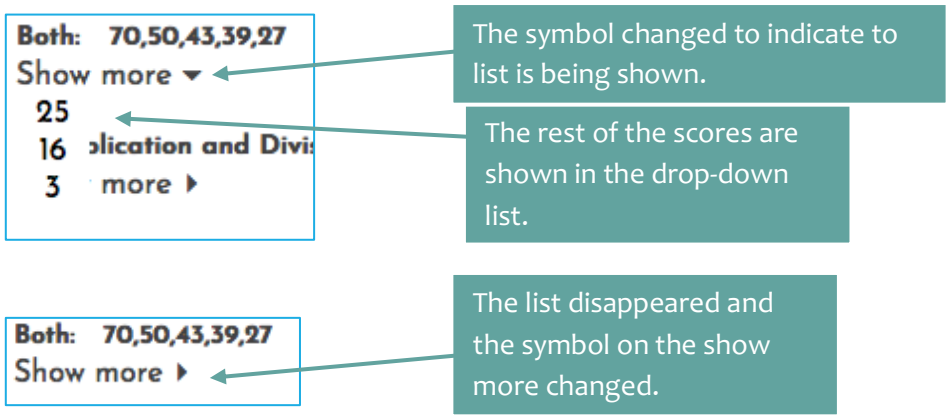
Website page	Usability Feature	How successful were these features?
The Website	All buttons are the colour orange. Buttons will decrease in opacity when hovered over and the cursor will become a pointer. When on the website the navigation bar will be above every page apart from in the actual games. The pages are simple with little text and clear appropriate titles. Below any password field there is a check box called 'Show password' that toggles password visibility.	Throughout the website, the buttons were consistently orange, and they would decrease in opacity when hovered over, apart from the "Show more" on the admin page which looked like: Here is the nav bar on each page:  The titles on the website were simple and there was little to no filler text anywhere on the website. The show password check box works as seen in **Test R11**, however there is a little problem with it. If you refresh the page while it is checked, it will stay checked and the password will convert to being non-visible. This is a very minor bug that can be fixed during the maintenance of the project.

Website page	Usability Feature	How successful were these features?
Student/Admin Register and Login	When validation error happens, the error appears in red underneath the appropriate input field. The errors appear systematically if there are multiple errors starting from the first input field.	These features were very successful as a whole and I think they served their purposes. The evidence is in **Test R9**.
Home	There is a short description of the function of each page of the website.	The short descriptions were there. Whether they served the ultimate purpose or not I can't say for sure. Here are the descriptions: 

Website page	Usability Feature	How successful were these features?
Games	The game preview is a screen shot of actual game gameplay The game preview acts as the button to enter the game.	Yes, these features did indeed work as intended. The game preview looked like this:  Since it goes orange when hovered over it signifies it is a button. It does act as a button to enter the game as shown in **Test G1.**
Game Menu	When entering the game menu, the game mode you previously played is automatically selected. The game selection is made using radio buttons.	This all works and has been tested and they work. The test for the automatic selection is Test **GM1.**

Website page	Usability Feature	How successful were these features?
Actual Game	When starting the game, the input area is auto selected. The enemies are a different colour from the player The questions have a white background.	The input area is indeed automatically selected when starting the game. This is shown by entering game the input area looks as follows:  The colour of the enemies are red and the player is black and questions are still somewhat readable when overlapped. This is shown in this image: 
Game end	The relevant score board is shown for the game mode just played. The selected score board changes boarder colour.	This all works. The evidence being **Test GE1.**

Website page	Usability Feature	How successful were these features?
Stats	The data points are plotted in a different colour than the line.	This is indeed the case as seen in this image: 
Profile	The error message for the password appears below the input field in red. The button to save blog and save password word change their text when clicked from "Save blog" to "Blog saved", and "Change Password" to "Password Changed".	The error messages are in red and the buttons do change their text when clicked as shown in **Tests P2 to P5**.

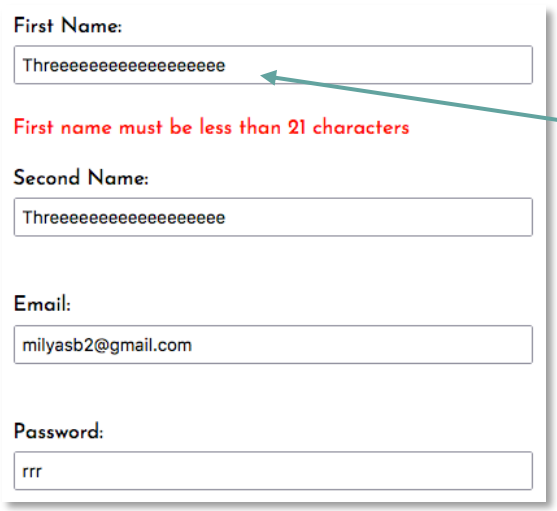
Website page	Usability Feature	How successful were these features?
Admin	If the list of scores is greater than five, clicking a button below the list called 'Show more' displays a drop-down list that sits on top of everything else. The button to delete users only appears when hovering over that particular record.	The show more button works as shown below:  The delete button also works in **Test A2.**

ROBUSTNESS TESTING

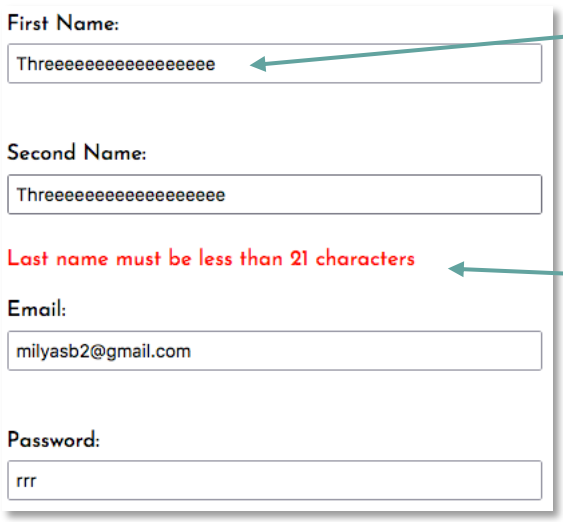
In this section I will carry out a few robustness tests on my Register and Login inputs. Some of the redundancy tests I have already done in the register and login page tests but here I will try to input, in particular, erroneous, boundary and extreme data into the input field to see whether the response is correct or not.

The Register Page

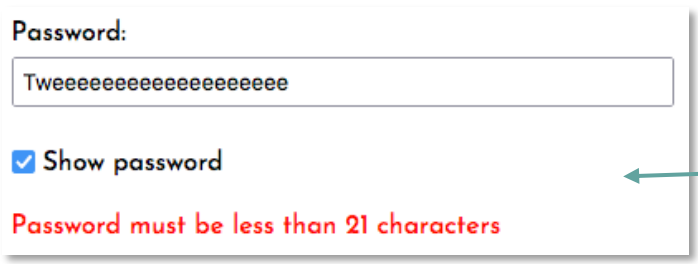
Boundary Data



Here I have entered exactly 21 characters into the First Name and Last Name fields and the response back is correct.

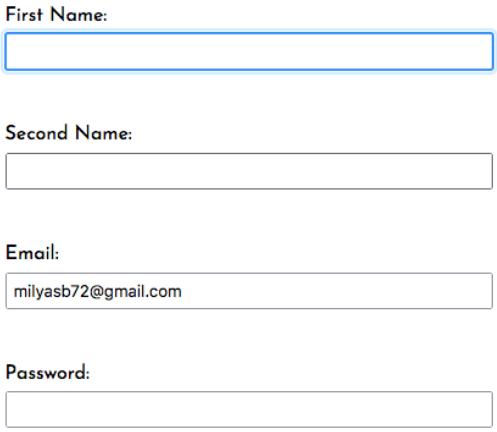


I reduced the number of characters in the First Name field exactly 20 characters, and no error was shown, which is correct. It then moved done to the Second Name field and displayed the correct error message.



The same applies to the Password field. Exactly 21 characters and the correct error message was shown. There is no need to test the email field since that went through extensive tests during the Register Tests Section.

Erroneous Data



The erroneous data I used was spaces. I filled all the fields, apart from the email field because there is no need to, with spaces in the boundary conditions.

And the data was accepted, which is very wrong. In the database we can see that it indeed was saved.

In order to stop this from happening I added the following code in the Users.js file as I have done in this example for each field:

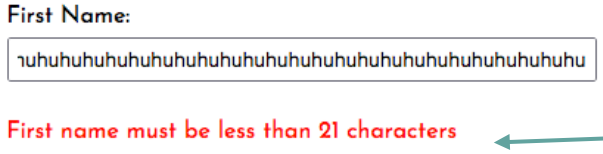
```
//Inside the object is where the field is validated
type: String,
trim: true,
```

Now when submitting the same erroneous data this is the response:

I gave the correct response which is to not treat spaces as characters.

Since it works for one field, and each field has the same exact code associated with it, it is safe to assume it is the same for the rest of them.

Extreme Data

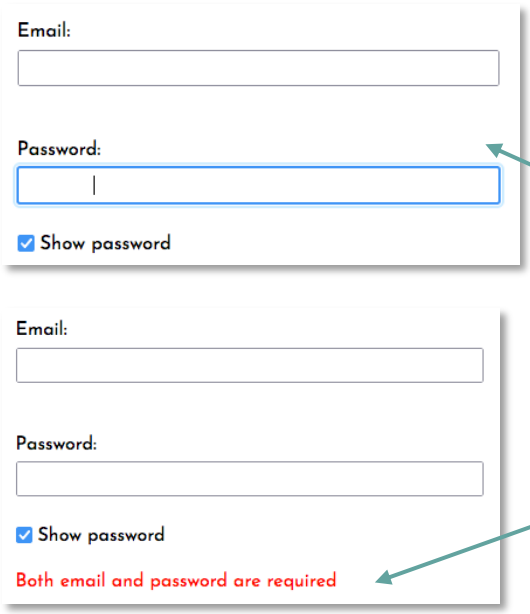


Here I inputted over 100 characters into the field, and it gave back the correct response.

The Login Page

The code validating the login has no real validation of the fields. Rather, it simply compares the inputted values to the data in the database. Therefore, if the register page is correct and doesn't allow the wrong data to be added to the database, the login doesn't need to do much at all. Therefore, I will test erroneous and extreme data to see if the code breaks, but there is no need to test boundary data.

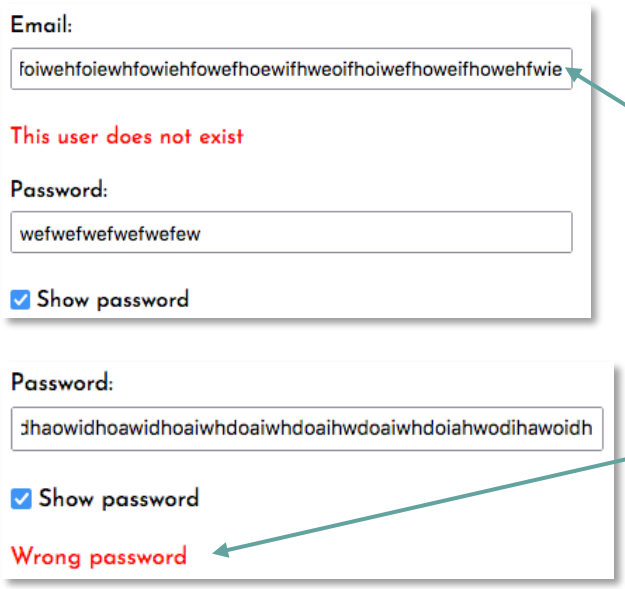
Erroneous Data



I used spaces as erroneous data again in the email and password fields.

This time the erroneous data was correctly dismissed, and the correct response was sent back.

Extreme Data



Here I inputted over 100 characters into the email field to see if the code could cope and it sent back the correct response.

For this test, I did the same in the password field, but this time with a valid email in the email field. The correct response was sent back.

LIMITATIONS

Below I list limitations I faced that impeded the improvement of my project. I will also comment on the potential ways around such limitations.

The greatest limitation of this project was **time**. Throughout the project many features and improvements were missed out due to the time limitations. Some of these I have already talked about like the automated emails and dark and light mode options. But there are many others, like for instance, the animations of the player character and of the enemies as they crawl. I also couldn't add more statistical representations of data and the biggest of all, another game. I think the best way around time limitations is to know exactly how long each feature takes to implement, although this is extremely hard due to the common problem of bugs.

The next limitation was **understanding how things worked on the web**. During development, I ran into this error that kept on popping up called "Cross Origin Resource Sharing Denied". This denial was stopping my HTML pages from being shown on the server. After doing some research on what Cross Origin Resource Sharing was (or CORS for short) I finally was able to understand it. With this understanding I felt satisfied with finding a solution to the error on Stack Overflow and on YouTube. Following the instructions, I was able to remove the error. Throughout development there were many on these moments where I was too stubborn to let go the understanding of how some aspect of code or the web worked. This led me onto long hauls of reading and listening to explanations of such things, which, in the end, sucked up my very valuable time. The best way around this is to sometimes accept that understanding this aspect is not necessary.

Another limitation I faced was the **complexity of some libraries**. After doing the project in a certain way and then completely scrapping it, I tried a different way using a framework called **React**. Using **React** allows for great levels of user interaction and would have allowed for many more features to be added, for instance, loading screens. But after learning about **React** more, it became very complex and, in my eyes, would have taken too long to learn and understand for me to use it effectively. A way around this limitation was to look for less complex solutions even if it did not give the best result which turned out to be the fetch API algorithm.

One other limitation was the fact that my **website can only be run locally**. This means that's use case is extremely limited. Because of this, my stakeholders can only use the website if the code is downloaded on their device. I will address this in Potential Improvements.

There are also limitations with **cross device compatibility**. This website would function terribly on mobile and would not be very effective on tablet either. I will address this in Potential Improvements.

The last limitation I faced was **learning something for the first time.** During the first rendition of the website, I jumped in straight away into new waters. Because of this many of the skills I learned at the start were quite useless and often a waste of time. This even led me to starting all over again with a more refined eye with respect to what I was looking for.

POTENTIAL IMPROVEMENTS

Here I will list the many potential improvements to the site that could have been made or that might be made in the future.

I could change the CSS and certain features on the website to **accommodate for cross device compatibility.** On tablets and on mobile, I could add a number pad for users to play the games with more ease. I could change the CSS measurements to use EM units which are based on the device, making the website fit on various screen sizes.

I could **host the website on the internet** to the public to allow people to actually use it. This would require some money to do though.

If I had the time to learn automated emails, I could have implemented a **Forgot Password system** that allows users to reset their password using a link sent to the email inbox. The system could be accessed on the login and register pages as a simple button saying: "Forgot password". Of course, I could also finish the automated email capabilities.

I could have **hashed the passwords inside of the database** to make the passwords more secure. I contemplated this feature during development of the register and login pages, but I pushed it to the side due to the time constraints.

I could have added a **world leader board on the home page** displaying the scores in descending order for each game and game mode, with links to the score owner's profile allowing others to see their blog and other scores.

I could have added **more data representations of user data** on the stats page. For instance, there could have been a pie chart showing the distribution of time on the separate game modes.

I could have made **another game**, with a different premise and with different questions.

I could have made **animations for the player and enemies** crawling towards the player. There could also have been animations for the enemies dying and the player taking damage.

I could have added a **level system to the game**, allowing for players to skip easier stages of the game to start on harder ones.

MAINTENANCE

In this section, I will detail the features on the website that need a little more work.

If a user wants to change the code or make new additions to the website, they could do so **quite easily** if they are familiar with using Git Hub. On Git Hub, people are able to make a request to the developers of the code to change it in such and such way. Since this website is up on Git Hub, by giving the link to my users, anyone can make a request to me asking to make a change.

My **client has full access to the MongoDB database**. Therefore, he can do all CRUD operations on the database using the MongoDB Compass website. Therefore, if a score was not saved to the database for a user, my client can add it in for them. If a user wants to change their email, my client can change it manually for them, until a feature that allows this is implemented.

Adding new pages and features to the website will **not be very easy**. Since the database is stored on the internet, interaction with it is only limited by internet connection, but the website at the current moment is being run locally. Therefore, to make a change for every user, that change would have to be made on each individual user's device.

The **Show password check boxes** below the Password fields **break** when the page is refreshed. **This is an easy fix**. All I need to do is something that notices when a page is refreshed. When it is refreshed, the inputs should be reset to their default states.

The **rest position of the line graphs without any data look very weird**, showing the positive and negative values on the y axis. I could solve this by placing a default value of zero on the graph.

The **"Page not found" and "Access Denied" messages should have actual pages** to display their message. This can be done by making two new html files called 404.html and AccessDenied.html, and then displaying these pages instead of the messages.

After logging out, the user is still able to access the website, to a certain extent using the back buttons. This could be stopped but at the moment I don't know how.

There is a potential that **the user Authentication token expires while a user is playing a game**. This means after finishing the game, the game score will not be saved to the database. To stop this, I could make it so the user Auth token does not expire at all and can only be removed be logging out or by closing the browser.

The questions of the enemies should include algebraic questions. This can be added quite easily by added a new function that creates an algebra question at random.